MA0501 - Análisis Numérico I

Notas del curso

Juan Gabriel Calvo

1 de agosto de 2023

Índice general

1. Error en cálculos numéricos **13**

1.1. Introducción 13
1.2. Representación de números en computadoras 14
1.2.1. Aritmética de punto fijo 15
1.2.2. Aritmética de punto flotante 16
1.3. Reglas de la IEEE 18
1.3.1. Precisión simple de la IEEE 18
1.3.2. Precisión doble de la IEEE 19
1.3.3. Otros formatos utilizados para almacenar datos 21
1.4. Exactitud y operaciones en punto flotante 22
1.5. Estabilidad 25
1.6. Algunos ejemplos *curiosos* 26
1.6.1. Calculando derivadas por definición 26
1.6.2. Resolviendo sistemas de ecuaciones lineales 27
1.6.3. Aproximando valores propios 28
1.6.4. Errores de cancelación 28
1.7. Ejercicios Adicionales 29

2. Ecuaciones no lineales **33**

2.1. Introducción 33
2.2. Método de Bisección 34
2.2.1. Descripción del algoritmo 34
2.2.2. ¿Cuándo detener la iteración? 36
2.3. Método de aproximaciones sucesivas 37
2.4. Orden de convergencia 42
2.5. Método de Newton 45
2.6. Método de la Secante 48
2.7. Reseña histórica 50
2.8. Ejercicios Adicionales 51

3. Interpolación de funciones **65**
3.1. Introducción . . . 65
3.2. Interpolación de Lagrange . . . 66
3.3. Interpolación de Hermite . . . 69
3.4. Aproximando derivadas de funciones . . . 72
3.5. Interpolación lineal a trozos . . . 74
3.6. Trazadores cúbicos . . . 75
3.7. Polinomios de Chebyshev . . . 78
3.8. Coordenadas baricéntricas . . . 81
3.9. ¿Cómo compartir un secreto? . . . 84
3.10. Ejercicios Adicionales . . . 86

4. Integración Numérica **97**
4.1. Introducción . . . 97
4.2. Fórmulas de Newton-Cotes . . . 98
4.2.1. Fórmula del trapecio . . . 98
4.2.2. Fórmula de Simpson . . . 101
4.3. Fórmulas compuestas . . . 102
4.4. Método adaptivo de Simpson . . . 104
4.5. Polinomios ortogonales . . . 107
4.6. Cuadraturas de Gauss . . . 111
4.7. Otras cuadraturas . . . 115
4.8. Métodos de Monte Carlo para integración . . . 116
4.9. Ejercicios . . . 117

5. Factorización de matrices **133**
5.1. Introducción . . . 133
5.2. Descomposición en Valores Singulares . . . 139
5.3. Factorización QR . . . 145
5.3.1. Ortogonalización de Gram-Schmidt . . . 147
5.3.2. Triangularización de Householder . . . 149
5.4. Factorización LU . . . 152
5.4.1. Eliminación Gaussiana sin pivoteo . . . 152
5.4.2. Eliminación Gaussiana con pivoteo parcial . . . 157
5.4.3. Pivoteo completo . . . 160
5.5. Factorización de Cholesky . . . 161
5.6. Resolución de sistemas de ecuaciones . . . 163
5.7. Ejercicios Adicionales . . . 164

6. Condición y estabilidad **175**
6.1. Introducción . . . 175
6.2. Número de condición . . . 175
6.3. Estabilidad . . . 178
6.4. Error al resolver sistemas de ecuaciones . . . 182
6.5. Ejercicios . . . 184

7. Aplicaciones de sistemas de ecuaciones **187**
7.1. Problemas de mínimos cuadrados . . . 187
7.2. Mejor aproximación en la 2-norma . . . 191
7.3. Sistemas de ecuaciones no lineales . . . 193
7.4. Trazadores cúbicos naturales . . . 196
7.5. Ejercicios . . . 199

8. Ecuaciones Diferenciales Ordinarias **207**
8.1. Introducción . . . 207
8.2. Método de Euler . . . 208
8.3. Método del trapecio . . . 210
8.4. Métodos de Runge-Kuta Explícitos . . . 213
8.5. Métodos de Runge-Kuta Implícitos . . . 216
8.6. Ecuaciones Diferenciales de tipo "stiff" . . . 218
8.7. Resolviendo ecuaciones diferenciales ordinarias en `MATLAB` . . . 219
8.8. Método de Diferencias Finitas . . . 222
8.9. A-estabilidad . . . 226
8.10. Ejercicios Adicionales . . . 227

9. Valores y vectores propios **237**
9.1. Introducción . . . 237
9.2. Reducción a una matriz de Hessenberg . . . 240
9.3. Secuencias de Sturm . . . 241
9.4. Método de Jacobi . . . 244
9.5. Iteración de potencias . . . 247
9.5.1. Cociente de Rayleigh . . . 248
9.5.2. Iteración de Potencias . . . 248
9.5.3. Iteración Inversa de Potencias . . . 250
9.5.4. Iteración del cociente de Rayleigh . . . 250
9.6. Algoritmo QR . . . 251
9.7. Ejercicios Adicionales . . . 252

10. Ejercicios finales **257**

Índice de figuras

1.1. Números normales y subnormales de un ejemplo de juguete . 17
1.2. Ejemplos de error numérico. 27

2.1. Interpretación geométrica del Método de Bisección. 35
2.2. Orden de convergencia 44
2.3. Interpretación geométrica del Método de Newton 46
2.4. Error relativo para diferentes algoritmos 48

3.1. Fenómeno de Runge con polinomios de Lagrange 69
3.2. Fenómeno de Runge con polinomios de Hermite 72
3.3. Error en las aproximaciones para la derivada de una función . 74
3.4. Trazador lineal para $f(x) = 1/(1 + 25x^2)$, $|x| \leqslant 1$, con $n = 6$. 76
3.5. Error relativo con polinomios interpolantes a trozos. 78
3.6. Puntos de Chebyshev 80
3.7. Polinomios de interpolación con puntos de Chebyshev 81

4.1. Interpretación geométrica de la fórmula del trapecio. 99

5.1. Reflexión de Householder. 151

7.1. Polinomio de mínimos cuadrados para $f(x) = 1/(1 + 25x^2)$. 191

8.1. Solución numérica para la ED de van der Pol 221
8.2. Solución numérica para le ED implícita 222
8.3. Péndulo simple. 231

Índice de cuadros

1.1. Precisión simple de la IEEE 18
1.2. Precisión doble de la IEEE 19
1.3. Clases de enteros con signo y sin signo 22

3.1. Cálculo de diferencias divididas 90
3.2. Cálculo de diferencias divididas para el polinomio de Hermite 91

8.1. Códigos para resolver EDOs en `MATLAB` 220

Índice de algoritmos

1. Método de bisección 36
2. Método de Newton 46
3. Método de la Secante 49

4. Método adaptivo de Simpson 106

5. Ortogonalización de Gram-Schmidt (inestable) 148
6. Ortogonalización de Gram-Schmidt (estable) 149
7. Triangularización de Householder 152
8. Cálculo implícito de $Q^*\boldsymbol{b}$ 152
9. Cálculo implícito de $Q\boldsymbol{x}$ 152
10. Eliminación Gaussiana sin pivoteo 156
11. Eliminación Gaussiana con pivoteo parcial 160
12. Factorización de Cholesky 162
13. Sustitución hacia atrás 163
14. Sustitución hacia adelante 164

15. Mínimos cuadrados con ecuaciones normales 189
16. Mínimos cuadrados con factorización QR 189
17. Mínimos cuadrados con valores singulares 189
18. Método de Newton en varias variables 195

19. Forma de Hessenberg 240
20. Método de Jacobi 246
21. Método de Jacobi modificado 247
22. Método de Iteración de Potencias 249
23. Método de Iteración Inversa de Potencias 250
24. Método del cociente de Rayleigh 251
25. Método QR 251
26. Método QR práctico 252

27. Método QR 256

Capítulo 1

El error en las aproximaciones numéricas

1.1. ¿Por qué es importante hablar del error al realizar cálculos numéricos?

Realizar cálculos numéricos de manera acertada es parte esencial en diversas disciplinas y áreas de la ciencia, en donde utilizamos computadoras y programas para obtener resultados que resuelvan un problema específico. Debido a que las computadoras utilizan un número finito de memoria para guardar información, solo es posible almacenar un conjunto finito de números racionales, lo cual conlleva ciertas restricciones. Por ejemplo, los números que se representan no pueden ser ni arbitrariamente grandes ni arbitrariamente cercanos a cero. Además, al ser un sistema discreto, se hace necesario utilizar técnicas de redondeo para representar cualquier número real, introduciendo así errores en nuestros cálculos.

En este capítulo, describiremos la aritmética de punto fijo y de punto flotante (Sección 1.2), y mencionaremos generalidades básicas de cómo guardar números en memoria mediante estándares de la IEEE (*Institute of Electrical and Electronics Engineers*, Sección 1.3). Como veremos en la Sección 1.4, algo tan sencillo como sumar tres números reales en una computadora puede producir un resultado incorrecto. Por ello, es siempre necesario verificar que nuestros resultados numéricos sean válidos. Debemos preguntarnos, por ejemplo, si el resultado obtenido aproxima al valor real (*exactitud*), y si el resultado obtenido es cercano al valor real si los datos de entrada son ligeramente perturbados (*estabilidad*). Exploraremos estas dos definiciones con más detalle en la Sección 1.5. En la Sección 1.6 discutiremos algunos

ejemplos para clarificar el porqué debemos ser cuidadosos al realizar cálculos numéricos por la presencia de errores numéricos.

1.2. Representación de números en computadoras

En esta sección describimos dos métodos clásicos para representar números en un sistema finito, siguiendo las ideas de [20]. Al almacenar datos en una computadora utilizamos *dígitos binarios* (o *bits* en inglés). Cada bit es capaz de guardar uno de dos estados (0 o 1), por lo que es natural considerar números reales desarrollados en su expansión binaria. Asumimos que se está familiarizado con conversiones entre base dos y base diez, y denotaremos por $(x)_b$ al número x representado en base b.

Llamamos *palabra* a una cadena finita de bits que son manejados como una unidad por la máquina. Supongamos por ejemplo que trabajamos en un sistema donde cada palabra tiene 32 bits. Si se desea almacenar un número entero en una palabra, la manera más sencilla sería usar el primer bit para indicar el signo (una posible convención es utilizar 0 para números positivos y 1 para negativos) y utilizar los 31 bits restantes para guardar el valor absoluto del número. Por ejemplo, el número $(-65)_{10} = -(100\ 0001)_2$ correspondería a la palabra

1	000 0000 0000 0000 0000 0000 0100 0001

.

Note que en este caso particular, el número cero se puede representar de dos maneras diferentes (con el bit de signo positivo y negativo). Además, con palabras de 32 bits es posible representar cualquier número entero cuyo valor absoluto sea menor o igual a $2^{31} - 1$.

Existen diversos formatos para guardar números. Por ejemplo, el formato *Complemento 2* es más conveniente para realizar sumas y restas con números enteros, el cual ganó popularidad por su facilidad de implementación en *hardware*; referimos a [20, Capítulo 3] si es de interés profundizar en estos temas. No obstante, nuestro interés será conocer la aritmética de punto flotante y las reglas establecidas por la IEEE, las cuales son las que se utilizan actualmente en la mayoría de sistemas. El primer conjunto de normas se estableció en el año 1985 [13] (IEEE 754-1985) y fue renovado en el 2008 [14] (IEEE 754-2008). Este conjunto de reglas brinda consistencia entre diferentes computadoras que utilicen los mismos estándares, normaliza reglas de redondeo y trata de manera similar situaciones especiales (como la división por cero). Es importante mencionar que en dicho sistema se incluye una palabra para hacer referencia a *infinito* y otra para indicar que el valor

guardado no es un número (*NaN*, que en inglés significa *Not a Number*); ver Ejercicio 1.18.

A continuación describimos los dos métodos básicos para almacenar números reales en una computadora; si se desea conocer más en estos temas se puede iniciar con [10].

1.2.1. Aritmética de punto fijo

En este caso, la palabra se divide en tres campos. El primer bit indica el signo, mientras que la parte entera y fraccionaria del valor absoluto del número se guardan en un segundo y tercer campo, respectivamente. Por ejemplo, imaginemos que utilizamos palabras de 32 bits, de los cuales 15 bits guardan la parte entera y 16 bits guardan la parte fraccionaria. En este caso, el número $(-11/2)_{10} = (-101.1)_2$ se representa (de manera exacta) como

1	000 0000 0000 0101	1000 0000 0000 0000

El número $(1/10)_{10} = (0.000\overline{1100})_2$ se expresa (de manera aproximada) como

(1.1)

0	000 0000 0000 0000	0001 1001 1001 1001

Note que en este caso es necesario truncar o redondear la expansión fraccionaria en base dos, pues solo podemos guardar 16 de sus bits. No entraremos en detalles sobre cómo redondear números, pero referimos a [20, Capítulo 5] para más detalles. Esto propone una primera complicación. Imagine que al escribir un programa ocupa declarar la constante $c = 1/10$. El valor que se guarda en la computadora y que será utilizado posteriormente en las operaciones no es exactamente 1/10. Si se utiliza la representación dada en (1.1), el valor exacto guardado corresponde a $6553/65536 \approx 0.099990$. Surge la pregunta: ¿tal redondeo puede provocar errores significativos en el resultado final del programa? Si tal es el caso, diremos que el algoritmo es susceptible a *errores de redondeo*. Además, dicho error se transmite a los cálculos que se efectúen posteriormente. En este caso, los *errores de propagación* pueden afectar el resultado final.

La representación en punto fijo tiene ciertas desventajas. Si intentamos guardar un número pequeño entre cero y uno, los bits que corresponden a la parte entera son todos cero e implican un gasto innecesario en memoria, tal como ocurre en (1.1). Sería mejor si esos bits se utilizaran para guardar más bits de su parte fraccionaria, con el fin de mejorar la precisión del número guardado. Esta es la idea detrás de la aritmética de punto flotante.

1.2.2. Aritmética de punto flotante

La representación en punto flotante se basa en la notación científica. Cualquier número real x distinto de cero puede ser escrito de la forma

$$x = \pm S \times 10^E, \text{ donde } 1 \leqslant S < 10, \ E \in \mathbb{Z}.$$

Los números S y E se llaman *mantisa* y *orden de magnitud*, respectivamente. Al trabajar en base dos, para $x \neq 0$ podemos escribir

$$x = \pm S \times 2^E, \text{ donde } 1 \leqslant S < 2, \ E \in \mathbb{Z}. \tag{1.2}$$

En este caso,

$$S = (1, b_1 b_2 b_3 \ldots)_2$$

y diremos que el número está escrito en *forma normalizada* (note que el primer bit siempre es un 1). Por ejemplo,

$$11/2 = (1.011)_2 \times 2^2.$$

Debido a limitaciones de memoria, para guardar el valor x expresado en (1.2) se utilizan $p-1$ bits para representar su parte fraccionaria $b_1 b_2 b_3 \ldots b_{p-1}$ (note nuevamente que debemos redondear o truncar si la expansión fraccionaria tiene más de $p-1$ números). El exponente también se guarda con cierta cantidad predeterminada de bits; dichas reglas las mencionaremos en la siguiente sección. Así, un número real x se llamará *número en punto flotante* de un sistema, si puede ser guardado de manera exacta en dicho sistema. Si x no es un número en punto flotante, debe ser redondeado antes de ser guardado. Denotamos también por ϵ_m al *épsilon de la máquina*, el cual se define como la distancia entre 1 y el siguiente número mayor que 1 que podemos representar en el sistema de punto flotante. La *precisión* p se refiere al número de bits que se utiliza para guardar la mantisa S.

Ejercicio 1.1. Verifique que en un sistema de precisión p se cumple que $\epsilon_m = 2^{-(p-1)}$.

Ejemplo 1.2. Consideremos un ejemplo pequeño de *juguete* para familiarizarnos con algunas propiedades que discutiremos más adelante. Suponga que en nuestra computadora (muy limitada) solo tenemos espacio para guardar números de la forma

$$\pm(b_0.b_1 b_2)_2 \times 2^E, \ E \in \{-1, 0, 1\},$$

donde todos los números (distintos de cero) son normalizados; esto es, $b_0 = 0$ solo si $b_1 = b_2 = 0$ (y el número representado es entonces 0). Se puede verificar que los números que podemos guardar pertenecen al conjunto

$$\left\{0, \pm\frac{1}{2}, \pm\frac{5}{8}, \pm\frac{3}{4}, \pm\frac{7}{8}, \pm 1, \pm\frac{5}{4}, \pm\frac{3}{2}, \pm\frac{7}{4}, \pm 2, \pm\frac{5}{2}, \pm 3, \pm\frac{7}{2}\right\};$$

ver Figura 1.1. Note que conforme la magnitud de los números aumenta, la distancia entre ellos también. Por ejemplo, en los intervalos $[1, 2]$ y $[2, 3]$ la distancia entre números en punto flotante es $1/4$ y $1/2$, respectivamente. Además, la precisión en este caso es $p = 3$ y el epsilon de la máquina es $\epsilon_m = 5/4 - 1 = 2^{-2}$.

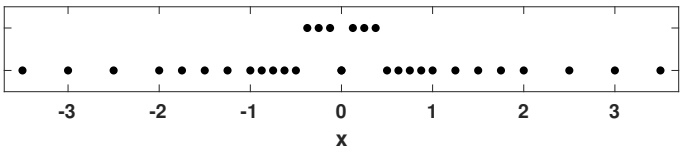


Figura 1.1: Números normalizados (abajo) del *ejemplo de juguete* (Ejemplo 1.2) junto a los números subnormales (arriba).

Además, en el intervalo $[0, 1]$ se podrían incluir más números en el sistema si eliminamos la condición $b_0 = 1$. A estos nuevos números los llamaremos *subnormales*. Así, al tomar $b_0 = 0$, podemos incluir los números subnormales $\{\pm 1/8, \pm 1/4, \pm 3/8\}$; ver Figura 1.1. De esta forma, en el intervalo $[0, 1]$ todos los saltos serían de $1/8$. Note además que $(0.11)_2 \times 2^1 = (1.10)_2 \times 2^0$ realmente es un número en punto flotante normal, por lo que primero se debe normalizar cada número para luego ser guardado en memoria.

¿Cómo representamos números en este sistema? Cada vez que desee guardar en memoria un número real x, la computadora deberá buscar en el sistema de punto flotante un valor apropiado $\text{fl}(x)$ siguiendo alguna regla de redondeo apropiada. Considere por ejemplo $x = 19/10$ (el cual no es un número en punto flotante). Podemos redondear *para arriba* o *al valor más cercano*, en cuyo caso x es guardado como $\text{fl}(x) = 2$. Por el contrario, si se redondea *para abajo* entonces $\text{fl}(x) = 1.75$. □

Ejercicio 1.3. En el Ejemplo 1.2, suponga que utilizamos redondeo al valor más cercano. Determine el valor que se guarda en memoria para los números $x = 2^{-5}$, $y = 21/10$. ¿Qué cree que ocurra para $z = 5$?

1.3. Reglas de la IEEE

En esta sección describimos de manera breve las normas establecidas en el 2008 por la IEEE 754-2008 según [14].

1.3.1. Precisión simple de la IEEE

El sistema de precisión simple de la IEEE emplea palabras de 32 bits para guardar números reales en aritmética de punto flotante. Del conjunto de bits, el primer bit indica el signo, los siguientes 8 bits representa el exponente E y los 23 bits restantes guardan la parte fraccionaria $b_1b_2b_3\ldots b_{23}$ de la mantisa para el caso de números normales; ver Cuadro 1.1. De los $2^8 = 256$ diferentes valores posibles para el exponente, uno es reservado para números en modo subnormal y otro para los casos especiales ∞ y NaN. Así, este sistema permite guardar valores normales en un rango de exponentes $-126 \leqslant E \leqslant 127$. Note que el valor de E que se guarda en los bits $a_1a_2\ldots a_8$ corresponde a $E + 127$, y que el épsilon de la máquina es $\epsilon_m = 2^{-23}$.

Cuadro 1.1: Precisión simple de la IEEE (tomado de [20])

$\pm$	$a_1a_2a_3\ldots a_8$	$b_1b_2b_3\ldots b_{23}$

Si $a_1a_2a_3\ldots a_8$ es	Entonces el valor representado es
$(0000\ 0000)_2 = (0)_{10}$	$\pm(0.b_1b_2\ldots b_{23})_2 \times 2^{-126}$
$(0000\ 0001)_2 = (1)_{10}$	$\pm(1.b_1b_2\ldots b_{23})_2 \times 2^{-126}$
$(0000\ 0010)_2 = (2)_{10}$	$\pm(1.b_1b_2\ldots b_{23})_2 \times 2^{-125}$
$\vdots$	$\vdots$
$(0111\ 1111)_2 = (127)_{10}$	$\pm(1.b_1b_2\ldots b_{23})_2 \times 2^{0}$
$(1000\ 0000)_2 = (128)_{10}$	$\pm(1.b_1b_2\ldots b_{23})_2 \times 2^{1}$
$\vdots$	$\vdots$
$(1111\ 1101)_2 = (253)_{10}$	$\pm(1.b_1b_2\ldots b_{23})_2 \times 2^{126}$
$(1111\ 1110)_2 = (254)_{10}$	$\pm(1.b_1b_2\ldots b_{23})_2 \times 2^{127}$
$(1111\ 1111)_2 = (255)_{10}$	$\pm\infty$ si $b_1 = b_2 = \ldots = b_{23} = 0$, NaN en otro caso

Por ejemplo, los números $(11/2)_{10} = (1,011)_2 \times 2^2$ y $(1)_{10} = (1)_2 \times 2^0$ se guardan como

$(11/2)_{10}$	0	1000 0001	011 0000 0000 0000 0000 0000
$(1)_{10}$	0	0111 1111	000 0000 0000 0000 0000 0000

Note que en ambos casos el bit inicial b_0 no se representa pues se sabe que siempre es 1 para el caso de números normales.

Se debe prestar especial atención a los números subnormales (que corresponden a la primera fila del Cuadro 1.1). En este caso, asumimos que el primer bit es $b_0 = 0$. De esta manera, el menor número positivo que se puede representar es 2^{-149}. A diferencia de los números normales, en donde se tienen 24 bits de precisión, en los números subnormales el número de bits significativos es menor. Por ejemplo, el número $2^{-135} = 2^{-126} \cdot 2^{-9}$ se representa como

$(2^{-135})_{10}$	0	0000 0000	000 0000 0100 0000 0000 0000

el cual tiene solamente 15 cifras significativas.

1.3.2. Precisión doble de la IEEE

En este caso la palabra tiene 64 bits, de los cuales se utilizan 11 bits para guardar el exponente y 52 bits para la parte fraccionaria; ver Cuadro 1.2. En este caso, $-1022 \leqslant E \leqslant 1023$ para números en modo normal y el valor de E que se guarda en los bits $a_1a_2a_3 \ldots a_{11}$ corresponde a $E + 1023$. Además, la precisión es $p = 53$ y el épsilon de la máquina es $\epsilon_m = 2^{-52}$.

Cuadro 1.2: Precisión doble de la IEEE (tomado de [20])

$\pm$	$a_1a_2a_3 \ldots a_{11}$	$b_1b_2b_3 \ldots b_{52}$

Si $a_1a_2a_3 \ldots a_{11}$ es	Entonces el valor representado es
$(000\ 0000\ 0000)_2 = (0)_{10}$	$\pm(0.b_1b_2 \ldots b_{52})_2 \times 2^{-1022}$
$(000\ 0000\ 0001)_2 = (1)_{10}$	$\pm(1.b_1b_2 \ldots b_{52})_2 \times 2^{-1022}$
$(000\ 0000\ 0010)_2 = (2)_{10}$	$\pm(1.b_1b_2 \ldots b_{52})_2 \times 2^{-1021}$
$\vdots$	$\vdots$
$(011\ 1111\ 1111)_2 = (1023)_{10}$	$\pm(1.b_1b_2 \ldots b_{52})_2 \times 2^{0}$
$(100\ 0000\ 0000)_2 = (1024)_{10}$	$\pm(1.b_1b_2 \ldots b_{52})_2 \times 2^{1}$
$\vdots$	$\vdots$
$(111\ 1111\ 1101)_2 = (2045)_{10}$	$\pm(1.b_1b_2 \ldots b_{52})_2 \times 2^{1022}$
$(111\ 1111\ 1110)_2 = (2046)_{10}$	$\pm(1.b_1b_2 \ldots b_{52})_2 \times 2^{1023}$
$(111\ 1111\ 1111)_2 = (2047)_{10}$	$\pm\infty$ si $b_1 = b_2 = \ldots = b_{52} = 0$, NaN en otro caso

Ejercicio 1.4. ¿Cuántos números reales diferentes se pueden expresar de manera exacta en el formato de precisión simple de la IEEE? ¿Y en formato doble?

Ejercicio 1.5. ¿Cuál es el mayor número positivo que se puede guardar de manera exacta en formato simple de la IEEE? ¿Y en formato doble?

Ejercicio 1.6. ¿Cuál es el menor número entero positivo que no puede ser guardado de manera exacta en formato simple de la IEEE? ¿Y en formato doble?

Ejercicio 1.7. [`MATLAB`] En `MATLAB`, el comando `format hex` permite visualizar los números y resultados en base hexadecimal. Por defecto, los números son guardados en formato doble, pero el comando `single(x)` permite utilizar precisión simple. Verifique las representaciones de los números 11/2, 1 y 2^{-135} en precisión simple presentadas anteriormente. ¿Cuál es su representación en precisión doble? Nota: ejecute el comando `help format` para conocer diferentes opciones de formato que puede utilizar en `MATLAB`.

Ejercicio 1.8. [`MATLAB`] Utilizando `format hex` visualice las representaciones de los números $x_1 = 0$, $x_2 = -0$. Concluya que el número real cero tiene dos representaciones diferentes. ¿Es necesario tener dos representaciones diferentes? ¿Cuál es el resultado al operar $1/x_1$ y $1/x_2$?

Ejercicio 1.9. [MATLAB] Del Cuadro 1.2 se deduce que el mayor número real en el sistema de punto flotante de precisión doble es

$$x_{\text{máx}} = (2 - 2^{-52}) \times 2^{1023}.$$

¿Qué cree que ocurra si intentamos guardar $x_{\text{máx}} + 2^k$ con $k \in \{0, 10, 100, 1000\}$? Visualice dichos números. ¿Cuándo el resultado es infinito?

1.3.3. Otros formatos utilizados para almacenar datos

En diversas aplicaciones es de interés guardar exclusivamente números enteros en cierto rango específico. En estos casos, utilizar aritmética de punto flotante no es la manera óptima de utilizar la memoria. Para realizar mediciones, es usual utilizar el término *byte*, el cual corresponde a una agrupación de 8 bits. Así, un número real en precisión simple ocupa 4 bytes en memoria, mientras que uno en precisión doble ocupa 8 bytes.

Suponga que ocupa guardar un índice (entero no negativo) k en el intervalo $[0, 255]$. En este caso es posible expresar

$$(k)_{10} = (b_1 b_2 b_3 b_4 b_5 b_6 b_7 b_8)_2$$

en una palabra de tan solo 1 byte, por lo que utilizar el formato de precisión doble implica un gasto de memoria ocho veces mayor. Por tal razón, en diferentes lenguajes de programación es natural encontrar clases de datos para almacenar números enteros, como las que se muestran en el Cuadro 1.3.

Ejercicio 1.10. [MATLAB] Defina $n = 2018$. Para convertir un valor y a cada clase descrita en el Cuadro 1.3 se utilizan los comandos `uintx(y)` o `intx(y)`(donde se reemplaza `x` por el número de bits correspondiente). Ejecute `uint8(n)`, `uint16(n)`, `uint32(n)`, `uint64(n)`. Comente sus resultados.

Cuadro 1.3: Clases de enteros con signo y sin signo

Clase	Rango	Tipo	Bytes
uint8	$[0, 2^8 - 1]$	Entero sin signo de 8 bits	1
uint16	$[0, 2^{16} - 1]$	Entero sin signo de 16 bits	2
uint32	$[0, 2^{32} - 1]$	Entero sin signo de 32 bits	3
uint64	$[0, 2^{64} - 1]$	Entero sin signo de 64 bits	4
int8	$[-2^7, 2^7 - 1]$	Entero con signo de 8 bits	1
int16	$[-2^{15}, 2^{15} - 1]$	Entero con signo de 16 bits	2
int32	$[-2^{31}, 2^{31} - 1]$	Entero con signo de 32 bits	3
int64	$[-2^{63}, 2^{63} - 1]$	Entero con signo de 64 bits	4

Ejercicio 1.11. [`MATLAB`] Fije $n = 100$. Para construir una matriz $A_{n\times n}$ con entradas enteras no negativas, aleatorias (con distribución uniforme) y menores que 255, basta ejecutar el comando `A1=randi(255,n)`. Para convertir dicha matriz a la clase uint8, se utiliza el comando `A2 = uint8(A1)`. Utilice el comando `whos A1 A2` para comparar el tamaño en memoria de cada matriz.

1.4. Exactitud y operaciones en punto flotante

Por simplicidad, imaginemos que estamos utilizando el sistema de precisión doble de la IEEE, el cual es usualmente el sistema predeterminado en las computadoras actuales. Defina $\mathbb{F}$ como el conjunto (discreto) de números en punto flotante del sistema de precisión doble; esto es, el subconjunto de los números reales descritos en el Cuadro 1.2. Considere el mapeo $\mathrm{fl} : \mathbb{R} \to \mathbb{F}$, donde a cada número real x se le asigna el correspondiente número en punto flotante $\mathrm{fl}(x)$, utilizando redondeo al valor más cercano. Esto es, si $x_- \leqslant x < x_+$, donde $x_-, x_+ \in \mathbb{F}$ son dos números en punto flotante consecutivos, se define $\mathrm{fl}(x) := x_-$ si $|x - x_-| \leqslant |x - x_+|$, o $\mathrm{fl}(x) := x_+$ en caso contrario[1]. De esta manera, tenemos el siguiente resultado:

Teorema 1.12. *Sea $x \in \mathbb{R}$ un número que puede ser representado de manera normal en un sistema de punto flotante con precisión p. Entonces, existe*

[1] ¿Para cuáles valores de x cree que se guarde $\mathrm{fl}(x) = \mathrm{Inf}$?

$\delta \in \mathbb{R}$ tal que

$$\mathrm{fl}(x) = x(1+\delta),$$

donde $|\delta| < \epsilon_m/2$.

Demostración. Primero, si $x \in \mathbb{F}$ es claro que la igualdad se cumple con $\delta = 0$. Asumamos que $x \notin \mathbb{F}$. Al ser x un número normal, podemos escribir $x = S \times 2^E$, $(1 \leqslant S < 2)$. Así, existen dos números en punto flotante consecutivos $x_-, x_+ \in \mathbb{F}$ tales que $x_- < x < x_+$. El *error absoluto*[2] se puede acotar entonces por

$$|x - \mathrm{fl}(x)| \leqslant \frac{1}{2} \cdot |x_+ - x_-| = \frac{1}{2} \cdot 2^{-(p-1)} \cdot 2^E.$$

Como $|x| > 2^E$, el *error relativo* se puede acotar por

$$\frac{|x - \mathrm{fl}(x)|}{|x|} < \frac{1}{2} \cdot \frac{2^{-(p-1)} \cdot 2^E}{2^E} = \frac{1}{2} \cdot 2^{-(p-1)} = \frac{\epsilon_m}{2},$$

de donde concluimos el resultado. □

De esta manera, el valor guardado es exacto hasta un factor de $1 + \epsilon_m/2$. Por ejemplo, al guardar un número x en precisión doble, su representación $\mathrm{fl}(x)$ es exacta hasta un factor de orden $1 + 2^{-53} \approx 1 + 10^{-16}$, lo que significa que tiene aproximadamente 16 decimales correctos. En el caso de precisión simple, son aproximadamente 7 dígitos de exactitud.

Ejercicio 1.13. [`MATLAB`] El comando `pi` en `MATLAB` aproxima el valor de $\pi = 3.14159265358979323846\ldots$ en precisión doble. Determine el número de dígitos correctos que brindan `pi` y `single(pi)`. ¿Cuál es el valor de `double(single(pi))-pi`? Comente su resultado (los comandos `single` y `double` se encargan de convertir números a precisión simple y doble, respectivamente).

Considere ahora dos números en punto flotante $x, y \in \mathbb{F}$. Para las cuatro operaciones aritméticas básicas $+, -, \times, /$, denote por $\oplus, \ominus, \otimes, \oslash$ las operaciones correspondientes tal como son implementadas en la computadora. Imagine que deseamos calcular $x + y$. Es claro que $x + y$ no necesariamente

[2] Dado un valor exacto x y una medición o aproximación $\tilde{x}$ de x, el error absoluto se define como $|x - \tilde{x}|$; el error relativo se define como $|x - \tilde{x}|/|x|$.

pertenece al sistema de punto flotante, por lo que el resultado debe ser redondeado. Así, $x \oplus y$ representa el número en punto flotante que es guardado en la computadora como resultado de la operación $x + y$. Del Teorema 1.12 se concluye que

$$x \oplus y = (x + y)(1 + \delta),$$

con $|\delta| < \epsilon_m/2$. Esto es, el resultado tiene un error relativo δ acotado por la precisión de la máquina. De esta manera, si utilizamos precisión doble, el resultado tendrá aproximadamente 16 dígitos de precisión. El mismo análisis aplica para el caso de las operaciones $\ominus, \otimes, \oslash$.

¿Qué ocurre si realizamos más de dos operaciones? Veamos un ejemplo sencillo. Considere $x = 1$, $y = 2^{-25}$, $z = -1$, los cuales pertenecen al sistema de punto flotante de precisión simple; i.e., $\text{fl}(x) = x$, $\text{fl}(y) = y$, $\text{fl}(z) = z$. Deseamos calcular $x+y+z = 2^{-25}$. Al calcular $x \oplus y$, debido a que en formato simple solo hay 23 bits en la parte fraccionaria, obtenemos $x \oplus y = 1$. Luego, $(x \oplus y) \oplus z = 0$. De esta manera, el error relativo es

$$\frac{|x + y + z - ((x \oplus y) \oplus z)|}{|x + y + z|} = 1,$$

¡el cual es 2^{23} veces la precisión de la máquina!

Ejercicio 1.14. Para los valores utilizados anteriormente, calcule

$$x \oplus (y \oplus z) \quad \text{y} \quad (x \oplus z) \oplus y$$

en precisión simple. ¿Algún resultado es correcto? ¿Qué ocurre en precisión doble?

Ejercicio 1.15. ¿Es $\oplus$ conmutativa? ¿Asociativa?

Recomendamos leer [20, Capítulo 6] para ampliar estos temas. Nuestra idea es ilustrar posibles complicaciones que pueden surgir en algo tan sencillo como sumar tres números en una computadora. No obstante, la implementación de redondeo y aritmética escapa a nuestros objetivos.

1.5. Estabilidad

En general, no es posible encontrar la solución exacta de un problema numérico, debido a la necesidad de aproximar dicho problema continuo en un espacio discreto. La noción de estabilidad es la manera estándar de caracterizar la solución de un problema como *correcta*, a pesar de no ser exacta. Referimos a [29, Secciones 14, 15] para más detalles.

Considere un *problema* matemático como una función $f : X \to Y$, de un espacio vectorial X a un espacio vectorial de soluciones Y. Considere fijos una computadora con sistema de punto flotante que satisface el Teorema 1.12, un algoritmo para f y la implementación del algoritmo como código en la computadora. Al correr el programa, el resultado es una colección de números en punto flotante que pertenecen a Y. Denote como $\tilde{f}(x)$ este resultado.

Es claro que $\tilde{f}(x)$ será afectado por errores de redondeo que son inevitables (por ejemplo, al ingresar datos de entrada). Incluso si todas los cálculos son efectuados sin error, estos redondeos podrían cambiar el resultado final. También, dicho resultado podría ser afectado por tolerancias de convergencia. Incluso puede ser que el algoritmo sea multivaluado o estocástico, por lo que tome distintos valores $\tilde{f}(x)$ después de diferentes ejecuciones.

De esta manera, obtener el resultado exacto puede ser difícil. En lugar de buscar exactitud en la respuesta, buscamos determinar si el algoritmo es estable. En palabras sencillas, un algoritmo es estable si da una respuesta casi correcta a casi la misma pregunta. Matemáticamente, el algoritmo $\tilde{f}$ para el problema f es estable si para todo $x \in X$, existe $\tilde{x}$ con

$$\frac{\|\tilde{x} - x\|}{\|x\|} = \mathcal{O}(\epsilon_m),$$

tal que

$$\frac{\|\tilde{f}(x) - f(\tilde{x})\|}{\|f(\tilde{x})\|} = \mathcal{O}(\epsilon_m),$$

donde utilizamos la notación usual $\phi(x, \tilde{x}) = \mathcal{O}(\epsilon_m)$ para indicar que existe una constante $C > 0$ tal que

$$|\phi(x, \tilde{x})| \leqslant C\epsilon_m \quad (\epsilon_m \to 0)$$

para todo $x \in X$. En el Capítulo 6 retomaremos esta definición y su significado en casos concretos; por el momento estudiaremos algunos ejemplos interesantes en la siguiente sección.

1.6. Algunos ejemplos *curiosos*

En esta sección mencionamos, de manera introductoria y a modo de motivación, algunos ejemplos básicos que muestran cuidados que se deben tener al realizar cálculos numéricos. En los siguientes capítulos retomaremos estos temas.

1.6.1. Calculando derivadas por definición

Considere la función $f : \mathbb{R} \to \mathbb{R}$ dada por $f(x) = \operatorname{sen} x + e^x \cos x$. Deseamos aproximar numéricamente la derivada en $x_0 = 1$ utilizando las fórmulas

$$D_1 f(h) := \frac{f(x_0 + h) - f(x_0)}{h}, \quad D_2 f(h) := \frac{f(x_0 + h) - f(x_0 - h)}{2h},$$

que sabemos convergen a $f'(x_0)$ cuando h tiende a cero.

Ejercicio 1.16. Utilizando polinomios de Taylor, demuestre que

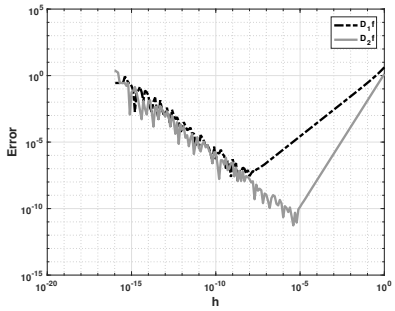
$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} + \mathcal{O}(h), \text{ y que}$$

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} + \mathcal{O}(h^2).$$

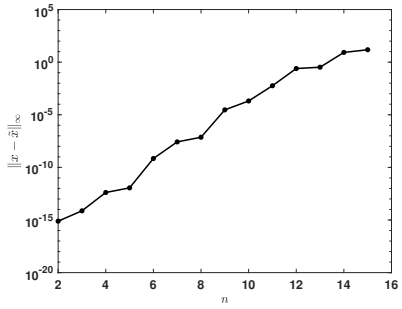
¿Cuáles son las condiciones mínimas que debe cumplir f para que dichas igualdades tengan validez?

En la Figura 1.2a se muestra la diferencia entre el valor exacto de la derivada y la aproximación, $\mathcal{E}_h^i := |f'(x_0) - D_i f(h)|$, $i \in \{1, 2\}$, utilizando precisión doble. El mínimo valor del error se alcanza para $h \approx 10^{-5}$ para $D_2 f$. De la Figura 1.2a podemos observar dos comportamientos diferentes:

- Cuando h se acerca a cero sabemos que $D_i f(h)$ debe converger a $f'(x_0)$. No obstante, el error en la aproximación crece. Esto se debe a errores de aproximación, pues para $h \approx 0^+$, al calcular $x_0 + h$ obtenemos x_0; esto es, $x_0 \oplus h = x_0$ y así $D_i f(x_0) = 0$ (error por redondeo).
- Cuando h se acerca a uno, el error en la aproximación crece. Esto se debe a errores de truncamiento. Si utilizamos la expansión de Taylor de $f(x)$ centrada en x_0 podemos deducir que las aproximaciones $D_1 f(h)$ y $D_2 f(h)$ tienen sentido solo si h es suficientemente pequeño.



(a) Error en la aproximación de la derivada, $\mathcal{E}_h^i$, en función de h; ver Sección 1.6.1.



(b) Error $\|\boldsymbol{x}-\tilde{\boldsymbol{x}}\|_\infty$ para diferentes valores de n; ver Sección 1.6.2.

Figura 1.2: Ejemplos de error numérico.

1.6.2. Resolviendo sistemas de ecuaciones lineales

Sea $n \in \mathbb{N}$ y defina H_n como la matriz $n \times n$ de Hilbert, cuyas entradas vienen dadas por $h_{ij} = 1/(i + j - 1)$ para $1 \leqslant i, j \leqslant n$. Consideremos un experimento sencillo utilizando precisión doble. Sea $\boldsymbol{x} = [1, \ldots, 1]^T \in \mathbb{R}^n$. Primero, calculamos el producto $\boldsymbol{b} := H_n\boldsymbol{x}$, el cual corresponde a un vector donde cada entrada b_i es la suma de los elementos de la fila i de A. Luego, resolvemos el sistema lineal $\tilde{\boldsymbol{x}} = H_n^{-1}b$ numéricamente (esto lo estudiaremos en el Apartado 5.6). Es claro que esperamos que $\tilde{\boldsymbol{x}}$ sea un vector que aproxima a la solución exacta $\boldsymbol{x}$. Para $n = 6$, el quinto componente de $\tilde{\boldsymbol{x}}$ es

$$\tilde{x}_5 = 0.999999999303.$$

En este caso, observamos que la respuesta es acertada hasta 9 decimales (y no 16, como esperaríamos en precisión doble). No obstante, cuando aumentamos el valor de n, pronto obtendremos soluciones totalmente alejadas del valor real. Incluso para $n = 14$ algunas entradas de $\tilde{\boldsymbol{x}}$ son negativas. En este caso, el *número de condición* de la matriz es muy alto, lo que implica que perdemos exactitud en nuestros cálculos. En la Figura 1.2b mostramos el error relativo $\|\boldsymbol{x} - \tilde{\boldsymbol{x}}\|_\infty$ para diferentes valores de n. En el Apartado 6.2 definiremos formalmente el número de condición y su importancia en cálculos numéricos. En palabras sencillas, es una medida de cuánto se amplifican los errores de entrada al calcular la salida del algoritmo. De esta manera, no podemos confiar en el resultado obtenido $\tilde{\boldsymbol{x}}$ para valores de $n \geqslant 10$.

Ejercicio 1.17. [MATLAB] En MATLAB, el comando `H=hilb(n)` genera la matriz de Hilbert H de tamaño $n \times n$ y el comando `b = ones(n,1)` crea un vector de tamaño n donde todas las entradas son 1. Para resolver cualquier sistema $H\boldsymbol{x} = \boldsymbol{b}$ se utiliza el comando $\boldsymbol{x} = H\backslash\boldsymbol{b}$. Además, `norm(x,'inf')` calcula la norma $\|\boldsymbol{x}\|_\infty$. Con dichas instrucciones, escriba un programa que reproduzca la Figura 1.2b.

1.6.3. Aproximando valores propios

Considere las matrices

$$A_1 = \begin{bmatrix} 1 & 1/\epsilon \\ 0 & 1 \end{bmatrix}, \quad A_2 = \begin{bmatrix} 1 & 1/\epsilon \\ \epsilon & 1 \end{bmatrix}.$$

Note que para $\epsilon \approx 0$, A_2 es muy cercana a A_1 (en el Capítulo 5 definiremos diferentes normas para matrices; en este caso, $\|A_1 - A_2\|_\infty = \epsilon$). Por ello, es de esperar que los valores propios de A_1 y A_2 estén próximos. No obstante, si calculamos el polinomio característico y encontramos las raíces obtendremos que los valores propios de A_1 y A_2 son $\{1, 1\}$ y $\{0, 2\}$, respectivamente, los cuales no están a una distancia de orden ϵ. De esta manera, se concluye que utilizar polinomios característicos para aproximar valores propios es un algoritmo inestable, pues a pesar de tener dos datos de entrada *cercanos*, las salidas del algoritmo no lo son.

1.6.4. Errores de cancelación

El fenómeno de *cancelación* se refiere a la dificultad de restar dos números que son casi iguales. Por ejemplo, considere $x = 3.141592653589793$ y $y = 3.141592653585682$. Su diferencia es $z = x - y \approx 4.1 \times 10^{-12}$. Note que z puede ser representado en el rango normalizado de precisión simple. Sin embargo, al guardar x y y en dicho formato para luego realizar la resta, el resultado será $\tilde{z} = 0$, debido a que $\text{fl}(x) = \text{fl}(y)$.

Consideremos un ejemplo sencillo. Deseamos calcular una aproximación para e^{-10} utilizando la serie de Taylor centrada en $x_0 = 0$,

$$e^x = \sum_{n=0}^{\infty} b_n, \quad b_n = \frac{x^n}{n!},$$

la cual sabemos tiene radio de convergencia infinito. Una manera consiste en escribir un programa que calcule cada término b_n a partir del anterior mediante la fórmula

$$b_n = b_{n-1} \cdot \frac{x}{n} \quad (n > 0), \quad b_0 = 1,$$

e ir calculando sumas parciales. En el momento en que $|b_n|$ es suficientemente pequeño se detiene el proceso. Supongamos que se utiliza precisión simple y detenemos el cálculo de la suma cuando $|b_n| < \epsilon_m$. En este caso obtenemos el valor -6.256183×10^{-5} el cual no tiene sentido: ¡la exponencial es una función positiva!

¿Qué está pasando? Prestemos atención a los sumandos. Por ejemplo $b_9 \approx -2.755732 \times 10^3$, $b_{10} \approx 2.755732 \times 10^3$. Estos términos tienen un error absoluto por redondeo de al menos 10^{-4} y este error se transmite a la suma lo que ocasiona el resultado incorrecto. La dificultad reside así en el hecho que términos consecutivos alternan de signo y errores de cancelación se hacen presentes. Este problema se puede solucionar de manera sencilla escribiendo $e^{-10} = 1/e^{10}$ y calculando con el método descrito el valor de e^{10}.

1.7. Ejercicios Adicionales

Ejercicio 1.18. [`MATLAB`] (`Inf` y `NaN`) En `MATLAB`, calcule las siguientes operaciones: 1/0, 0/0, 0/`Inf`, `Inf`/0, `Inf`/`Inf`, 1/`NaN`. ¿Son los resultados los "esperados"? ¿Cuál es el valor que se obtiene al realizar cualquier operación entre `NaN` y un número real cualquiera?

Ejercicio 1.19. [`MATLAB`] Considere la ecuación $f(x) := x^2 + bx + 1 = 0$, donde $b \in \mathbb{R}$. Es sabido que el mayor cero de dicha ecuación es

$$x_1 = \frac{-b + \sqrt{b^2 - 4}}{2}.$$

(a) Calcule numéricamente la aproximación x_1 mediante la fórmula anterior para $b = 1000$. Evalúe $f(x_1)$.

(a) Es fácil verificar que $x_1 = \dfrac{2}{-b - \sqrt{b^2 - 4}}$. Calcule nuevamente la aproximación x_1 para $b = 1000$ con esta fórmula y evalúe $f(x_1)$. Compare los resultados y justifique la diferencia encontrada.

Ejercicio 1.20. [`MATLAB`] Dado un vector v de tamaño n, el comando

```
c = poly(v)
```

construye un vector $\boldsymbol{c}$ de tamaño $n+1$, con coeficientes del polinomio

$$p(x) = \sum_{k=1}^{n+1} c(k)x^{n+1-k}$$

cuyos ceros son los elementos de v. Si resolvemos en aritmética exacta la ecuación $p(x) = 0$, es claro que debemos obtener como soluciones a los elementos de v. En `MATLAB`, podemos utilizar la función `roots` para encontrar las raíces de un polinomio. Ejecute el siguiente programa, que grafica la norma infinita del error relativo en función de n para $v = [1, 2, 3, \ldots, n+1]$.

```
n_max = 24;
e = zeros(n_max,1);
for n = 1:n_max
    e(n) = norm(abs((n+1:-1:1)'-roots(poly(1:n+1))),'inf')...
    ./norm(abs((n+1:-1:1)'),'inf');
end
semilogy(2:n_max+1, e, '.-')
xlabel('n')
ylabel('inf-norm error')
```

Justifique el comportamiento del gráfico.

Ejercicio 1.21. [`MATLAB`] (Calculando $\ln(1+x)$ para $x \ll 1$) Al calcular $\ln(1+x)$ para $0 < x \ll 1$ cercano a la precisión de la máquina, el argumento $1+x$ tiene un error grande de redondeo y el error relativo crece. La función `log1p` evita este problema y da un resultado preciso.

(a) Calcule $\ln(1+x)$ directamente usando `MATLAB`, para x un vector de valores pequeños de x logarítimicamente espaciados. Grafique el error relativo (utilizando `log1p` como valor real).

(b) Compare `log1p` con la serie de Taylor de $\ln(1+x)$ para x pequeño. Utilice los primero cinco términos. Experimentalmente, determine un óptimo valor x_0 para el cual esta aproximación es válida si $x < x_0$ (con un error relativo de orden precisión de la máquina).

(c) Considere la aproximación

$$\ln(1+x) = \begin{cases} x & \text{si } x < \varepsilon_m \\ \dfrac{x \ln(1+x)}{(1+x)-1} & \text{en caso contrario.} \end{cases}$$

Verifique que esta aproximación da el valor acertado (hasta precisión de la máquina) para $0 \leqslant x < 3/4$.

(d) Un comportamiento similar ocurre al calcular $e^x - 1$ para valores pequeños de x, pues por errores de redondeo $e^x - 1$ puede ser cero. Estudie la función `expm1` de `MATLAB` y realice un estudio similar al anterior.

Ejercicio 1.22. [`MATLAB`] Considere una inversión de C_0 colones a una tasa de interés compuesto del $x\,\%$ anual, donde se capitaliza el interés n veces por año. El valor final de la inversión al primer año viene dado por la ecuación

$$C = C_0 \left(1 + \frac{x}{n}\right)^n.$$

Note que conforme n aumenta, C es creciente y $C \to C_0 e^x$.

(a) Considere $x = 5\,\%$ y por simplicidad haga $C_0 = 1$. Utilizando precisión simple, escriba una función que calcule C de manera directa para cada entrada n. Grafique C en función de n para $n \in [1, 10^6]$. Compare sus resultados con el valor límite $C_0 e^x$. ¿Qué ocurre con la gráfica?

(b) Note que podemos calcular C mediante la fórmula equivalente

$$C = C_0 e^{n \ln\left(1 + \frac{x}{n}\right)}.$$

Repita el punto anterior utilizando esta nueva fórmula. ¿Se observa alguna mejora en los resultados?

(c) Programe la fórmula del apartado (b) utilizando el comando `log1p` del Ejercicio 1.21. Comente sus resultados.

(d) Repita sus experimentos en precisión doble. Comente sus resultados. ¿Existe algún n a partir del cual se observan resultados no esperados?

Ejercicio 1.23. [`MATLAB`] Considere el siguiente código en `MATLAB`:

```
f(1)=0;
f(2)=1;
for i=2:(n-1)
    f(i+1)=f(i)+f(i-1);
end

g(n)=f(n);
g(n-1)=f(n-1);
for i=(n-1):-1:2
    g(i-1)=g(i+1)-g(i);
end
```

Para cualquier entero positivo n de entrada, el programa calcula los primeros n términos de la sucesión de Fibonacci en el orden usual. Dicha secuencia es guardada en el vector f. Luego, en g se guarda la misma secuencia pero calculando los números en orden inverso, empezando por $g(n), g(n-1), \ldots, g(1)$.

(a) Determine el mínimo valor de n para el cual $f(n)$ sobrepasa el valor máximo que se puede guardar en precisión doble. Sugerencia: recuerde que el término n de la sucesión de Fibonacci puede expresarse como

$$f(n) = \frac{1}{\sqrt{5}} \left(\phi^n - \frac{(-1)^n}{\phi^n} \right),$$

donde $\phi = (1 + \sqrt{5})/2$.

(b) Estime (a mano) el el mínimo valor de n para el cual $f(1) \neq g(1)$. Explique por qué ocurre este fenómeno.

(c) Verifique sus estimaciones de los incisos (a) y (b) en `MATLAB`.

Capítulo 2

Solución numérica de ecuaciones no lineales

2.1. Introducción

Suponga que desea resolver la ecuación

$$e^{2x} + e^x + x = 0.$$

En cursos introductorios de cálculo se demuestra la existencia de al menos una solución en cierto intervalo a partir del Teorema del Valor Intermedio (TVI). Por ejemplo, si definimos $f(x) := e^{2x} + e^x + x$, $x \in \mathbb{R}$, se tiene que $f(0) > 0$ y $f(-1) < 0$. Al ser f continua en el intervalo $I = [-1, 0]$, se concluye que existe al menos un número real $c \in (-1, 0)$ que satisface $f(c) = 0$. Esto es, c es una *raíz* o *solución* de la ecuación inicial.

Con las herramientas conocidas, en este caso particular se puede demostrar incluso que existe una única solución real. Para tal efecto, si existieran dos raíces diferentes $a < b$, al aplicar el Teorema de Rolle en el intervalo $[a, b]$ se obtendría la existencia de un número real $d \in (a, b)$ tal que $f'(d) = 0$. Sin embargo, es fácil verificar que f' no tiene ceros reales. De esta manera, se concluye que la única solución real se encuentra en el intervalo $(-1, 0)$. Note además que no es posible calcular una fórmula cerrada para c.

La pregunta natural que surge es: ¿cómo se puede aproximar el valor de dicha solución? En este capítulo estudiaremos cómo aproximar soluciones de ecuaciones no lineales. En general, la idea será construir sucesiones de números reales que converjan a la solución. Note que inicialmente calculamos $f(0)$ y $f(-1)$ de manera arbitraria. Pudimos haber sustituido valores diferentes que también cumplieran con las hipótesis del TVI. Por ello, una primer

noción será tratar de disminuir de manera paulatina el intervalo I hasta obtener un intervalo lo suficientemente pequeño que contenga la solución. Esto derivará el famoso *Método de Bisección*. Posteriormente, deduciremos el *Método de Newton* como un caso particular del *Método de Aproximaciones Sucesivas*, basado en la Teoría de Punto Fijo. Finalmente, discutiremos el *Método de la Secante* como una variación del Método de Newton.

Anted de iniciar, es importante aclarar que se debe tener ciertas precauciones al diseñar cualquier algoritmo. En el ejemplo anterior fue sencillo demostrar que la raíz era única, pero este no siempre será el caso. Por ello, será importante primero conocer si existe alguna solución. Si este es el caso, ¿podemos garantizar que es única? En caso negativo, ¿cómo se comporta la sucesión que se obtiene al aplicar el algoritmo escogido? ¿Existen cuidados computacionales que deben ser considerados al resolver dichas ecuaciones? ¿Cuál método converge a la solución de manera más rápida? En las siguientes secciones contestaremos estas preguntas.

2.2. Método de Bisección

2.2.1. Descripción del algoritmo

El algoritmo más sencillo se basa en una aplicación directa del Teorema del Valor Intermedio. Considere una función continua $f : [a_0, b_0] \to \mathbb{R}$ con $f(a_0)f(b_0) < 0$, i.e., los valores de $f(a_0)$ y $f(b_0)$ tienen signos opuestos. Así, sabemos que al menos existe una solución $c \in I_0 := (a_0, b_0)$ de la ecuación $f(c) = 0$. Por simplicidad, asumamos primero que dicha raíz es única.

Luego calculamos el punto medio del intervalo $c_0 := (a_0+b_0)/2$ y verificamos si c_0 es una raíz. En dicho caso detenemos el algoritmo pues hemos obtenido nuestra solución. En caso contrario nos preguntamos: ¿la raíz pertenece al intervalo de la izquierda $L_0 := (a_0, c_0)$ o al de la derecha $R_0 := (c_0, b_0)$? Si está en L_0, definimos $a_1 := a_0$ y $b_1 := c_0$. Si está en R_0, definimos $a_1 := c_0$ y $b_1 := b_0$. De esta manera, garantizamos que $c \in I_1 := (a_1, b_1)$. Note que la longitud de I_1 es la mitad de I_0, por lo que el tamaño de nuestro intervalo de búsqueda se redujo a la mitad; ver Figura 2.1.

Luego repetimos el proceso con el intervalo I_1, obteniendo su punto medio c_1 y definiendo un subintervalo $I_2 := (a_2, b_2)$ al cual la solución pertenece. Después de k pasos obtenemos una sucesión de intervalos anidados $I_k \subset I_{k-1} \subset \ldots \subset I_1 \subset I_0$, con

$$|I_k| = \frac{1}{2^k}|I_0| = \frac{1}{2^k}|b_0 - a_0|,$$

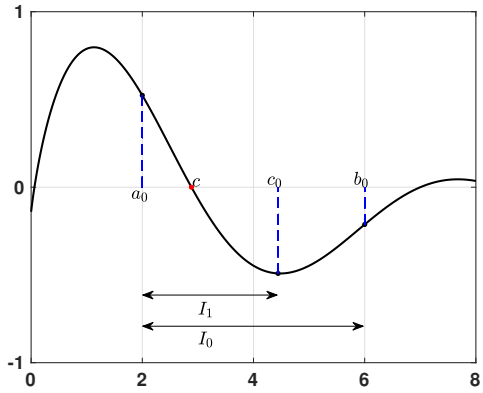


Figura 2.1: Interpretación geométrica del Método de Bisección.

siempre y cuando $f(c_j) \neq 0$ para $j \in \{0, 1, \ldots, k\}$ (en cuyo caso el algoritmo se detiene al encontrar un cero). Es posible demostrar que la sucesión de puntos medios $\{c_k\}$ converge a c:

Teorema 2.1. *Suponga que la sucesión definida anteriormente satisface que* $c_k \neq c \ \forall \ k \in \mathbb{N}$. *Entonces se tiene que* $\{c_k\}$ *converge a* c*; esto es, para todo* $\varepsilon > 0$ *existe* $N_\varepsilon \in \mathbb{N}$ *tal que si* $k > N_\varepsilon$ *entonces* $|c_k - c| < \varepsilon$.

Demostración. Sea $\varepsilon > 0$. Tome $N_\varepsilon = \left\lceil \log_2 \left(\dfrac{b_0 - a_0}{\varepsilon} \right) - 1 \right\rceil$. De esta manera, para $k > N_\varepsilon$ se tiene que

$$|c_k - c| < \frac{1}{2}|I_k| = \frac{1}{2^{k+1}}|b_0 - a_0| < \varepsilon,$$

por lo que la suceción converge a c. □

En el Algoritmo 1 se presenta el pseudocódigo del método de bisección. Los criterios para determinar que la iteración debe detenerse pueden variar dependiendo del problema, los cuales discutimos en la siguiente sección.

Ahora, ¿qué ocurre si la raíz no es única? El método convergerá a una solución de la ecuación, pero dependerá del intervalo inicial y de la ubicación de las raíces; ver Ejercicio 2.2. Es importante mencionar que no hemos asumido ninguna hipótesis sobre la diferenciabilidad de la función: solo se necesita que f sea continua para establecer la convergencia de la sucesión a una raíz de f.

Data: Funcion continua f, intervalo inicial $I_0 := (a_0, b_0)$ con $f(a_0)f(b_0) < 0$
Result: Aproximación c_k de una raíz c de f en el intervalo I_0
$k := 0$;
while *la iteración no haya convergido* **do**
  $c_k := (a_k + b_k)/2$;
  if c_k *es una raíz* **then**
    Retorne c_k;
  end
  if $f(c_k)f(b_k) < 0$ **then**
    $a_{k+1} := c_k$, $b_{k+1} := b_k$;
  else
    $a_{k+1} := a_k$, $b_{k+1} := c_k$;
  end
  $k := k + 1$;
end

Algoritmo 1: Método de bisección

Ejercicio 2.2. [MATLAB] Considere $p(x) = (3x - 1)(4x - 5)(3x - 7)$. Es claro que todas sus raíces se encuentran en el intervalo $(0, 3)$. ¿Cuál es el valor de convergencia del método de bisección si el intervalo inicial es $I_0 = (0, 3)$? ¿Y si $I_0 = (-3, 5/2)$? Note que en este caso es sencillo tener una idea de dónde buscar las raíces, pero este no siempre será el caso.

2.2.2. ¿Cuándo detener la iteración?

Existen diferentes posibilidades para determinar cuándo detener la iteración del ciclo *while* expuesto en el Algoritmo 1. En cualquier caso, se define un parámetro que llamaremos *tolerancia* y denotaremos por ϵ, que permitirá determinar cuando el error en el paso k es aceptable para nuestros cálculos.

Si deseamos que nuestra aproximación c_k esté cerca de c, del Teorema 2.1 se deduce que en el paso k el error se puede acotar por

$$e_k := |c - c_k| < \frac{1}{2^{k+1}}|b_0 - a_0|.$$

Por lo tanto, si realizamos $k_{\text{mín}}$ iteraciones, con

$$k_{\text{mín}} > \log_2\left(\frac{b_0 - a_0}{\epsilon}\right) - 1,$$

se cumplirá que $e_k < \epsilon$. No obstante, si la función crece o decrece de manera rápida cerca de c, podriar ocurrir que $f(c_k)$ no sea lo suficientemente pequeño. Por ello, se podría pensar también detener la operación si $|f(c_k)| < \epsilon$. Sin embargo, esto podría ser un problema si la función crece o decrece de manera muy lenta en un vecindario de c. Por ello, la decisión debe tomarse según de las características del problema, dependiendo si necesitamos que $|f(c_k)|$ sea lo suficientemente pequeño o si requerimos que c_k esté suficientemente cerca de c; ver Ejercicio 2.3.

Ejercicio 2.3. [`MATLAB`] Escriba una función en `MATLAB` cuyas entradas sean la función f, el intervalo inicial (a_0, b_0) y la tolerancia ϵ. Realice dos versiones diferentes:

(a) `Biseccion1.m`, en donde la iteración se detiene si $|f(c_k)| < \epsilon$.

(b) `Biseccion2.m`, en donde el número de iteraciones que se realizan es exactamente

$$k = \left\lceil \log_2 \left(\frac{b_0 - a_0}{\epsilon} \right) - 1 \right\rceil,$$

donde $\lceil \cdot \rceil$ es la función techo.

Compare el comportamiento de los dos códigos para aproximar el cero de la función $f(x) = \sqrt[1009]{x - 2018}$ con $I_0 = (2000, 3000)$ y $\epsilon = 10^{-6}$. Indique el número de iteraciones que se efectuaron en cada caso y compare los errores relativos. Repita el mismo ejercicio para la función $f(x) = (x - 2018)^{1009}$. ¿Se puede deducir que algún criterio para detener la iteración es superior a otro?

2.3. Método de aproximaciones sucesivas

En esta sección estudiaremos el teorema de punto fijo de Brouwer y de Banach. Antes de enunciar dichos teoremas, necesitamos la siguiente definición:

Definición 2.4. *Sea (X, d) un espacio métrico*[1]*. Se dice que $x \in X$ es*

[1]Si no se está familiarizado con el concepto de espacio métrico, esta sección puede simplificarse tomando $X = [a, b]$ y $d(x_1, x_2) = |x_1 - x_2|$.

un punto fijo de un mapeo $g : X \to X$ *si* $g(x) = x$. *Además,* g *se llama contracción si existe* $L \in (0,1)$ *tal que* $d(g(x), g(y)) \leqslant L\ d(x,y), \forall\ x, y \in X$. □

Iniciamos con un teorema abstracto de existencia, enunciado en su forma más general:

Teorema 2.5 (T. Punto Fijo de Brouwer)**.** *Sea* K *un subconjunto convexo, compacto y no vacío de un espacio euclídeo. Entonces, toda función continua* $g : K \to K$ *posee al menos un punto fijo.*

Demostración. La demostración general del teorema escapa a nuestros objetivos. En nuestro caso, presentamos una prueba sencilla para el caso unidimensional. Considere $g : [a,b] \to [a,b]$ continua. Si $g(a) = a$ o $g(b) = b$ el teorema se cumple. En caso contrario, como $g([a,b]) \subset [a,b]$, tenemos que $g(a) > a$ y $g(b) < b$. Defina $f(x) := g(x) - x$. Así, $f(a) > 0$, $f(b) < 0$ y por el TVI se concluye que existe $c \in (a,b)$ tal que $f(c) = 0$; es decir, c es un punto fijo de g. □

Comentario 2.6. En una forma un poco más general, podemos reemplazar *espacio euclídeo* en el teorema anterior por un *espacio de Banach*; ver [25]. □

Si bien es cierto el teorema anterior garantiza la existencia de un punto fijo, no nos brinda un algoritmo para aproximarlo. La siguiente prueba es constructiva y permitirá así aproximar numéricamente puntos fijos de contracciones.

Teorema 2.7 (T. Punto Fijo de Banach)**.** *Sea* (X,d) *un espacio métrico completo y considere una contracción* $g : X \to X$ *donde* X *es cerrado. Entonces,* g *tiene exactamente un punto fijo.*

Demostración. Sea $c_0 \in X$ arbitrario. Defina la sucesión $\{c_k\}_{k=0}^{\infty}$ dada por

$$c_k := g(c_{k-1})\ \forall k \geqslant 1.$$

Como g es una contracción, se tiene que

$$d(c_{k+1}, c_k) = d(g(c_k), g(c_{k-1})) \leqslant L\ d(c_k, c_{k-1})$$

y aplicando el mismo procedimiento k veces concluimos que

$$d(c_{k+1}, c_k) \leqslant L^k d(c_1, c_0)\ \forall\ k \in \mathbb{N}.$$

Luego, para $n, m \in \mathbb{N}$, $m > n$, al aplicar la desigualdad triangular y la desigualdad anterior, obtenemos que

$$\begin{aligned} d(c_m, c_n) &\leqslant d(c_m, c_{m-1}) + d(c_{m-1}, c_{m-2}) + \ldots + d(c_{n+1}, c_n) \\ &\leqslant \left(L^{m-1} + L^{m-2} + \ldots + L^n\right) d(c_1, c_0) \\ &< L^n \; d(c_1, c_0) \sum_{k=0}^{\infty} L^k = \frac{L^n}{1-L} d(c_1, c_0). \end{aligned}$$

Sea $\varepsilon > 0$. Si $d(c_1, c_0) = 0$, entonces c_0 es un punto fijo. En caso contrario, como $0 < L < 1$, tome $N \in \mathbb{N}$ suficientemente grande tal que

$$L^N < \frac{1-L}{d(c_1, c_0)} \; \varepsilon.$$

Entonces, para $m > n > N$ se tiene que $d(c_m, c_n) < \varepsilon$ y $\{c_k\}$ es una sucesión de Cauchy. Por ser un espacio completo y al ser X cerrado, sabemos que existe $c \in X$ tal que $\lim_{k\to\infty} c_k = c$.

Veamos que c es un punto fijo de g. Al ser g continua, se tiene que

$$\lim_{k\to\infty} g(c_{k-1}) = g(c),$$

y como

$$\lim_{k\to\infty} g(c_{k-1}) = \lim_{k\to\infty} c_k = c,$$

concluimos que $g(c) = c$.

La unicidad se obtiene del hecho que g es una contracción. En efecto, si c y $\tilde{c}$ son dos puntos fijos, entonces

$$d(c, \tilde{c}) = d(g(c), g(\tilde{c})) \leqslant L \; d(c, \tilde{c}),$$

por lo que $d(c, \tilde{c}) = 0$, y así $c = \tilde{c}$. □

Con respecto a la construcción del teorema anterior, tenemos la siguiente definición:

Definición 2.8. *Sea g una función continua $g : [a, b] \to [a, b]$. Dado un valor inicial $c_0 \in [a, b]$, la recursión $c_{k+1} = g(c_k)$, $k \geqslant 1$, es llamada iteración simple.* □

Si bien es cierto el Teorema 2.7 es el caso más general que podemos considerar, si asumimos que g es derivable y su derivada es acotada por una constante menor que 1, podemos concluir que g es una contracción:

Ejercicio 2.9. Suponga que $g : [a, b] \to [a, b]$ es diferenciable y que existe $L \in (0, 1)$ tal que $|g'(x)| \leqslant L \ \forall \ x \in (a, b)$. Utilice el Teorema del Valor Medio para demostrar que g es una contracción.

De esta manera, tenemos el siguiente teorema con hipótesis un poco más restrictivas:

Teorema 2.10. *Sea $g : [a, b] \to [a, b]$ una función continua. Suponga además que g es diferenciable con*

$$|g'(x)| \leqslant L < 1, \ \forall x \in (a, b).$$

Entonces g tiene un único punto fijo $c \in [a, b]$ y el proceso de iteración simple converge a c.

Demostración. Este teorema es consecuencia directa del Teorema 2.7 y del Ejercicio 2.9. □

Ejemplo 2.11. Al inicio de este capítulo hablamos de cómo obtener la solución de la ecuación $e^{2x} + e^x + x = 0$, y descubrimos que su único cero pertenece al intervalo $I = [-1, 0]$. Note que dicha ecuación puede reescribirse como

$$-e^{2x} - e^x = x,$$

por lo que la solución que buscamos es un punto fijo de la función

$$g(x) = -e^{2x} - e^x.$$

Sin embargo,

$$g(I) = [-2, g(-1)] \nsubseteq I,$$

por lo que g no satisface las condiciones del Teorema 2.7. Si reemplazamos el intervalo por $I = [-1, -1/2]$ se puede verificar que en este caso que $g(I) \subset I$. Sin embargo, g no es una contracción pues, por ejemplo, para $x = -1/2$, $y = -3/4$, se tiene que

$$\frac{|g(x) - g(y)|}{|x - y|} > 1.$$

No obstante, si tomamos $I = [-1, -7/10]$, se puede verificar que

$$|g'(x)| < 1 \ \forall x \in I.$$

De esta manera, el Teorema 2.10 permite garantizar la existencia de un punto fijo para g en el intervalo $[-1, -7/10]$. Si tomamos $c_0 = -1$, después de 100 iteraciones simples obtenemos que $c_{100} \approx -0.7249$, el cual aproxima con dos decimales la solución correcta. □

Comentario 2.12. Surge la pregunta: ¿cuál debe ser el mínimo número de iteraciones que debemos realizar para satisfacer una tolerancia ϵ dada? De la prueba del Teorema 2.7, por inducción se tiene que

$$|c - c_k| = |g(c) - g(c_{k-1})| \leqslant L|c - c_{k-1}| \leqslant \ldots \leqslant L^k|c - c_0|.$$

En particular, note que

$$|c - c_0| \leqslant |c - c_1| + |c_1 - c_0| \leqslant L|c - c_0| + |c_1 - c_0|,$$

por lo que

$$|c - c_0| \leqslant \frac{1}{1 - L}\ |c_1 - c_0|.$$

Combinando las dos desigualdades anteriores concluimos que

$$|c - c_k| \leqslant \frac{L^k}{1 - L}\ |c_1 - c_0|.$$

Por lo tanto, podemos garantizar que el error en la iteración k satisface $|c - c_k| < \epsilon$ si tomamos $k > k_{\text{mín}}$, con

$$k_{\text{mín}} = \left\lceil \frac{\ln |c_1 - c_0| - \ln \epsilon(1 - L)}{\ln(1/L)} \right\rceil. \tag{2.1}$$

Note que esta es una cota para el número de iteraciones, la cual aumenta conforme L se acerca a 1. Como veremos más adelante, no es óptima y solo garantiza que el error es menor que la tolerancia definida. Usualmente, un número de iteraciones mucho menor es suficiente para obtener aproximaciones razonables. □

Ejemplo 2.13. Considere la ecuación $f(x) = 0$, con

$$f(x) = e^x - 2x - 1,\ x \in \mathbb{R}.$$

Es fácil ver que f tiene dos ceros reales ($x = 0$ y una raíz en el intervalo $I := (1, 2)$). Intentemos reescribir la ecuación como un problema de punto fijo $g(x) = x$. Si definimos

$$g(x) := \frac{e^x - 1}{2},$$

se puede comprobar que $g(I) \not\subset I$. Sin embargo, si tomamos

$$g(x) := \ln(2x + 1),$$

se puede verificar que $g([1, 2]) \subset [1, 2]$. En este último caso, podemos garantizar la existencia de al menos un punto fijo para g. Además, se tiene que

$$|g'(x)| = \frac{2}{2x + 1} \leqslant \frac{2}{3}$$

para $x \in I$. Por lo tanto, podemos garantizar que dicho punto fijo es único (ver Ejercicio 2.10). Si realizamos el proceso de iteración simple con $c_0 = 3/2$, obtenemos la siguiente sucesión (donde redondeamos a 4 dígitos):

$$(1,5; 1,3863; 1,3278; 1,2962; 1,2691; 1,2636; 1,2605; 1,2588; 1,2578; \ldots).$$

Dicha sucesión parece indicar que la solución (con dos dígitos de exactitud al menos) es $1,25$. Si deseamos encontrar k tal que $|c_k - c| < 0,5 \cdot 10^{-15}$, la cota encontrada en la Ecuación 2.1 implica que $k > 84$. No obstante, experimentalmente obtenemos que $c_{60} = 1,256431208626170$ es el valor aproximado con 15 decimales de exactitud. □

Ejercicio 2.14. Sea $g : I \to I$ una función continua en el intervalo cerrado $I := [a, b]$. Sea $c \in (a, b)$ un punto fijo de g, y suponga además que g es derivable en un vecindario de c con $|g'(c)| < 1$. Demuestre que la sucesión definida por $c_k := g(c_{k-1}),\ k \geqslant 1$, converge a c si c_0 es un valor cercano a c (desde luego, al ser c desconocido, en práctica es difícil determinar cuán cerca debemos tomar c_0; sin embargo, se puede utilizar por ejemplo el método de bisección para reducir el intervalo de búsqueda).

2.4. Orden de convergencia

Empezamos esta sección con la definición de *orden de convergencia* de una sucesión:

Definición 2.15. *Considere una sucesión* $\{c_k\}_{k=0}^{\infty}$ *tal que* $\lim_{k\to\infty} c_k = c$ *con* $c_k \neq c\ \forall\ k \in \mathbb{N}$. *Suponga que*

$$\lim_{k\to\infty} \frac{|c_{k+1} - c|}{|c_k - c|} = \mu \geqslant 0.$$

Entonces se dice que c_k converge a c

(a) sublinealmente si $\mu = 1$.

(b) superlinealmente si $\mu = 0$.

(c) linealmente si $\mu \in (0, 1)$, y $\rho := -\log \mu$ se define como la razón de convergencia asintótica.

En el caso de convergencia superlineal, se dice que c_k converge a c con orden $q > 1$ si

$$\lim_{k\to\infty} \frac{|c_{k+1} - c|}{|c_k - c|^q}$$

existe. En particular, la convergencia se llama cuadrática si $q = 2$.

Ejercicio 2.16. Considere las sucesiones dadas por

$$a_k = \frac{1}{k+1}, \quad b_k = \frac{1}{2^k}, \quad c_k = \frac{1}{3^{2^k}}.$$

Demuestre que:

(a) a_k converge sublinealmente a 0.

(b) b_k converge linealmente a 0.

(c) c_k converge superlinealmente a 0, y en particular, que la convergencia es cuadrática.

El comportamiento de estas sucesiones se observa en la Figura 2.2.

Comentario 2.17. Para el caso del método de bisección, si bien es cierto el error $e_k := |c - c_k|$ no decrece en cada iteración, es usual decir que la convergencia es lineal pues

$$\frac{e_{k+1}}{e_k} \approx \frac{1}{2}.$$

□

Ejemplo 2.18. Con base en la Definición 2.15, es fácil ver que si g es derivable y satisface las hipótesis del Teorema 2.10, el método de iteración

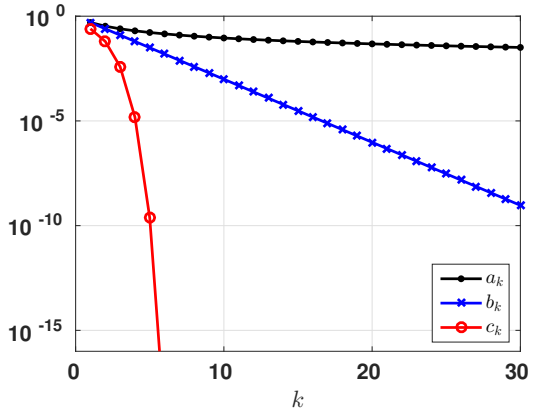


Figura 2.2: Convergencia de las sucesiones a_k, b_k y c_k del Ejercicio 2.16, las cuales convergen sublineal, lineal y superlinealmente, respectivamente. En este caso, la convergencia de c_k es cuadrática.

simple converge al menos linealmente pues

$$\lim_{k\to\infty} \frac{|c_{k+1} - c|}{|c_k - c|} = \lim_{k\to\infty} \left| \frac{g(c_k) - g(c)}{c_k - c} \right| = |g'(c)| < 1.$$

Note que si $g'(c) = 0$, entonces la convergencia es superlineal. En particular, es posible demostrar que la convergencia es cuadrática:

Teorema 2.19. *Suponga que $c \in (a, b)$ es un punto fijo de una función derivable $g : [a, b] \to [a, b]$, tal que $g'(c) = 0$. Suponga además que g'' es continua en un intervalo abierto $I := (c - h, c + h) \subset [a, b]$ y*

$$\sup_{x\in I} |g''(x)| \leqslant M$$

para algún $M > 0$. Entonces existe $\delta > 0$ tal que, si $c_0 \in (c - \delta, c + \delta)$, la sucesión definida por

$$c_k = g(c_{k-1}) \ (k \geqslant 1)$$

converge (al menos) cuadráticamente a c.

Demostración. El polinomio de Taylor de $g(x)$ centrado en $x = c$ viene dado por

$$g(x) = g(c) + g'(c)(x - c) + \frac{g''(\xi_x)}{2}(x - c)^2,$$

para algún ξ_x entre x y c. Sustituyendo $x = c_k$ obtenemos que

$$c_{k+1} = c + \frac{g''(\xi_k)}{2}(c_k - c)^2,$$

para algún ξ_k entre c y c_k. Así,

$$|c_{k+1} - c| = \left|\frac{g''(\xi_k)}{2}\right| |c_k - c|^2. \tag{2.2}$$

Tome

$$\delta := \text{mín}\{1/M, h\}.$$

Note que si $c_0 \in (c - \delta, c + \delta) =: I_\delta$, entonces de (2.2) se deduce que

$$|c_1 - c| \leqslant \frac{M}{2} \cdot \frac{1}{M}|c_0 - c| < \frac{\delta}{2},$$

y así $c_1 \in I_\delta$. Por inducción se deduce que

$$|c_{k+1} - c| < \frac{1}{2}|c_k - c|,$$

por lo que $c_k \in I_\delta$ para todo k. Luego, la convergencia se deduce del hecho que

$$|c_k - c| < 2^{-k}|c_0 - c|,$$

y como

$$\lim_{k\to\infty} \frac{|c_{k+1} - c|}{|c_k - c|^2} = \lim_{k\to\infty} \left|\frac{f''(\xi_k)}{2}\right| = \frac{|f''(c)|}{2},$$

se deduce que el método converge al menos cuadráticamente. □

2.5. Método de Newton

Imagine que deseamos resolver la ecuación $f(x) = 0$. Suponga que $\lambda(x)$ es una función sin ceros reales, y defina $g(x) := x - \lambda(x)f(x)$. Note que

$$g(c) = c \Leftrightarrow f(c) = 0.$$

Además, como $g'(c) = 1 - \lambda(c)f'(c)$, si tomamos

$$\lambda(x) = \frac{1}{f'(x)},$$

entonces la función g tendrá un cero de multiplicidad dos. Por el Teorema 2.19, si tomamos c_0 cercano a c la iteración simple convergerá al punto fijo de manera cuadrática. Esto motiva el famoso *Método de Newton*:

Data: Función continua y derivable f, valor inicial c_0 (cercano a la raíz).
Result: Aproximación c_k de la raíz c de f
for $k = 1, 2, \ldots$ **do**
$\quad c_k := c_{k-1} - \dfrac{f(c_{k-1})}{f'(c_{k-1})}$;
end

Algoritmo 2: Método de Newton

Ejercicio 2.20. El Algoritmo 2 tiene un simple sentido geométrico: dado un punto $(c_k, f(c_k))$ en la gráfica de f, si se traza la recta tangente sobre tal punto, la intersección de dicha recta con el eje x es el punto $(c_{k+1}, 0)$; ver Figura 2.3. Verifique dicha afirmación.

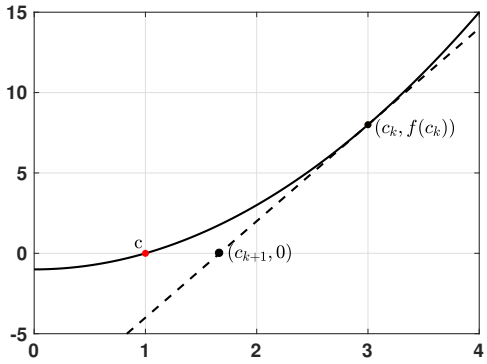


Figura 2.3: Interpretación geométrica del Método de Newton; ver Ejercicio 2.20.

Para el método de Newton tenemos el siguiente resultado[2]:

Teorema 2.21. *Suponga que $f \in C^2(I_\delta)$, donde $I_\delta := [c-\delta, c+\delta]$, $\delta > 0$, tal que $f(c) = 0$, $f'(c) \neq 0$. Entonces, si tomamos c_0 lo suficientemente cerca*

[2]Si $f \in C^3(I_\delta)$, es posible aplicar el Teorema 2.19 para concluir de forma directa; no obstante, presentamos una prueba donde solo se requiere tener dos derivadas continuas.

de c, la sucesión $\{c_k\}$ definida en el Algoritmo 2 converge a c de manera cuadrática.

Demostración. SPG podemos asumir que $f'(x) \neq 0\ \forall x \in I_\delta$, pues al ser f' continua, existe un vecindario de c donde la derivada no se anula, y bastaría disminuir el valor de δ inicial. Utilizando el desarrollo de Taylor alrededor de c_k, se tiene que

$$0 = f(c) = f(c_k) + (c - c_k)f'(c_k) + (c - c_k)^2 f''(\xi_k)/2,$$

para algún ξ_k entre c y c_k. Utilizando la iteración de Newton para c_{k+1}, es posible escribir

$$c_{k+1} - c = \frac{(c_k - c)^2 f''(\xi_k)}{2f'(c_k)}. \tag{2.3}$$

Tome $\tilde{\delta} := \text{mín}\{\delta, 1/A\}$. Note que si $c_0 \in (c - \tilde{\delta}, c + \tilde{\delta}) =: I_{\tilde{\delta}}$, entonces de (2.3) se deduce que

$$|c_1 - c| \leqslant \frac{A}{2} \cdot \frac{1}{A}|c_0 - c|,$$

donde

$$A := \max_{(x,y)\in \overline{I}_{\tilde{\delta}}^2} \left|\frac{f''(x)}{f'(y)}\right|$$

(el cual está bien definido pues $f''(x)/f'(y)$ es continua en el compacto $[c - \tilde{\delta}, c + \tilde{\delta}]^2$). Por inducción, si $c_0 \in (c - \tilde{\delta}, c + \tilde{\delta})$, es fácil deducir que para todo k se tiene que

$$|c_{k+1} - c| \leqslant \frac{1}{2}|c_k - c|,$$

por lo que en particular $c_k \in I_{\tilde{\delta}}$. La convergencia de la sucesión se deduce del hecho que

$$|c_{k+1} - c| \leqslant 2^{-k}\,\text{mín}\{\delta, 1/A\} \to 0$$

cuando $k \to \infty$. Por otra parte, el método de Newton converge de manera cuadrática pues de la continuidad de f' y f'' se deduce que

$$\lim_{k\to\infty} \frac{|c_{k+1} - c|}{|c_k - c|^2} = \left|\frac{f''(c)}{2f'(c)}\right|.$$

□

Ejemplo 2.22. Considere las funciones definidas en el Ejemplo 2.13. Si aplicamos los diferentes métodos estudiados, podemos graficar el error relativo en función del número iteraciones (donde cada método se detiene hasta garantizar un error relativo menor a $\epsilon = 10^{-6}$). En todos los casos, tomamos como valor exacto al término de la iteración c_k para el cual se tiene convergencia hasta la precisión de la máquina. Podemos verificar que la razón de convergencia del método de Newton es cuadrática, y que las convergencias asintóticas para el método de bisección y de iteración simple son, respectivamente, log(2) y log(3/2) (al menos); ver Figura 2.4. □

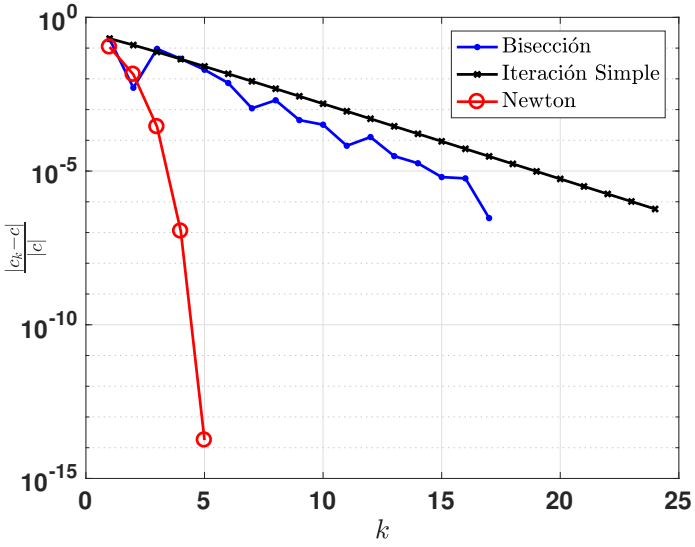


Figura 2.4: Error relativo para diferentes algoritmos en función del número de iteraciones k.

2.6. Método de la Secante

El método de Newton tiene la ventaja de converger de manera cuadrática, pero asume que podemos evaluar la derivada de la función f en cada iteración. En la práctica, puede ser posible no tener conocimiento a priori de f', o podría ser costoso evaluarla. Como sabemos que c_k converge a c, tiene sentido aproximar

$$f'(c_k) \approx \frac{f(c_k) - f(c_{k-1})}{c_k - c_{k-1}}.$$

De esta manera, se deduce el Método de la Secante, cuyo pseudocódigo se muestra en el Algoritmo 3.

Data: Función continua f, valores iniciales c_0, c_1 (cercanos a la raíz).
Result: Aproximación c_k de la raíz c de f
for $k = 1, 2, \ldots$ **do**
  $c_{k+1} = c_k - \dfrac{c_k - c_{k-1}}{f(c_k) - f(c_{k-1})}\, f(c_k)$;
end

Algoritmo 3: Método de la Secante

Ejercicio 2.23. El Algoritmo 3 tiene un simple sentido geométrico. Considere la recta secante que pasa por los puntos $(c_0, f(c_0))$ y $(c_1, f(c_1))$, con $c_0 \neq c_1$, $f(c_0) \neq f(c_1)$. Entonces, el punto de intersección de dicha recta con el eje x es el punto $(c_2, 0)$. Verifique dicha afirmación.

Con respecto a la convergencia, tenemos el siguiente resultado:

Teorema 2.24. *Sea $f \in C^1([c-h, c+h])$, para algún $h > 0$. Suponga que $f(c) = 0$ y $f'(c) \neq 0$. Entonces, la secuencia definida en el Algoritmo 3 converge a c superlinealmente si c_0 y c_1 se toman suficientemente cerca de c.*

Demostración. De la iteración del Método de la Secante se obtiene que

$$c_{k+1} - c = (c_k - c)\left(1 - \frac{c_k - c_{k-1}}{f(c_k) - f(c_{k-1})} \cdot \frac{f(c_k) - f(c)}{c_k - c}\right)$$

y por el Teorema del Valor Medio podemos escribir

$$c_{k+1} - c = (c_k - c)\left(1 - \frac{f'(\mu_1)}{f'(\mu_2)}\right),$$

donde μ_1 se encuentra entre c y c_k, y μ_2 entre c_{k-1} y c_k. De esta forma, debemos buscar acotar el cociente de las derivadas, para lo cual utilizamos la hipótesis de continuidad sobre f'. Suponga primero que $f'(c) = \alpha > 0$. El caso $\alpha < 0$ es análogo. Si tomamos $\varepsilon = \alpha/4 > 0$, sabemos que existe $\delta > 0$ tal que $|f'(x) - \alpha| < \alpha/4$ si $|x - c| < \delta$. Es fácil ver que para esta escogencia de ε se cumple que

$$\left|1 - \frac{f'(\mu_1)}{f'(\mu_2)}\right| < \frac{2}{3}$$

si $\mu_1, \mu_2 \in (c-\delta, c+\delta)$. De esta manera, considere $c_0, c_1 \in (c-\delta, c+\delta)$. De la última igualdad deducimos que

$$|c_{k+1} - c| \leqslant \frac{2}{3}|c_k - c|,$$

por lo que por inducción $c_k \in (c-\delta, c+\delta)\ \forall k$. La convergencia se deduce del hecho que

$$|c_{k+1} - c| \leqslant \left(\frac{2}{3}\right)^k |c_0 - c|,$$

y como

$$\frac{|c_{k+1} - c|}{|c_k - c|} \to 0,$$

la convergencia es superlineal. En el Ejercicio 2.27 veremos que la razón de convergencia es $q = \frac{1}{2}(1+\sqrt{5})$, por lo que el orden de convergencia se encuentra entre lineal y cuadrático. □

2.7. Reseña histórica

El *Método de la Regla Falsa* es uno de los más antiguos; ver su descripción en el Ejercicio 2.31. Dicha idea se ha encontrado en tablas cuneiformes de la antigua Babilonia, en papiros del antiguo Egipto y de la antigua China, y corresponde a una combinación del Método de Bisección y de la Secante. Similar al Método de la Bisección estudiado en la Sección 2.2, el Método de Regla Falsa genera una sucesión de intervalos anidados que contiene un cero de la ecuación. Sin embargo, es posible que la longitud de dicho intervalo no tienda a cero, como ocurre al encontrar un cero de la función $f(x) = 2x^3 - 4x^2 + 3x$ en el intervalo inicial $[-1, 1]$; ver Ejercicio 2.31.

El Método de Newton (también llamado Método de Newton-Raphson) surgió a partir de un ejemplo por Isaac Newton en 1664 que describimos a continuación; referimos a [6] para una historia más completa y su relación con el trabajo de Francois Vieta, Joseph Raphson y Thomas Simpson. Considere la ecuación

$$x^3 - 2x - 5 = 0.$$

Se puede verificar que la solución c pertenece al intervalo $(2, 3)$, por lo que su parte entera es $c_0 = 2$. Si se sustituye $x = 2 + p$, la ecuación se transforma en

$$p^3 + 6p^2 + 10p - 1 = 0.$$

Como $p \in (0,1)$, si despreciamos los términos de grado dos y tres, tenemos que $p \approx 1/10$, y una mejor aproximación es $c_1 = c_0 + p = 2.1$. De manera similar, sustituya $p = 0.1 + q$, para obtener la nueva ecuación

$$q^3 + 6.3q^2 + 11.23q + 0.061 = 0.$$

De nuevo, al despreciar los términos de grado dos y tres, se piensa en sustituir $q = -0.0054 + r$, y análogamente se tiene que $r \approx -0.00004853$. De esta manera,

$$c \approx c_3 = c_0 + p + q + r = 2.09455147.$$

Se puede ver que en general, el número de dígitos de precisión se duplica en cada iteración. Además, se puede verificar que

$$p = c_1 - c_0 = -\frac{f'(x_0)}{f(x_0)},$$

y

$$q = c_2 - c_1 = -\frac{f'(x_1)}{f(x_1)},$$

el cual coincide con nuestra definición del Método de Newton.

Existen otros métodos de igual relevancia que el de Newton, los cuales se describen en la Sección 2.8. Por ejemplo, el *Método de Halley* se remonta al año 1650, y es una generalización del Método de Newton con orden de convergencia cúbica; ver Ejercicio 2.32. Una descripción geométrica del Método de Halley se encuentra en [24].

El *Método de Muller*, presentado en 1956 por David E. Muller en [19], se basa en el Método de la Secante, pero utiliza tres puntos para construir una parábola en lugar de una recta. El *Método de Steffensen* descrito en [16] es una simplificación del de Newton, al utilizar una aproximación para la derivada; ver Ejercicio 2.34. Posteriormente, en 1971 se presenta el *Método de Illinois* [7], el cual mejora el Método de Regla Falsa; ver Ejercicio 2.33. Existen algoritmos más elaborados, tales como el de Brent [2], el de Ridders [22] y el de Sidi [26], por mencionar algunos. Referimos al lector interesado a cada una de las referencias para estudiar la descripción de cada método.

2.8. Ejercicios Adicionales

Ejercicio 2.25. Sea $c > 0$. Demuestre que la iteración

$$c_{k+1} = \frac{1}{2}\left(c_k + \frac{c}{c_k}\right)$$

converge cuadráticamente a $\sqrt{c}$ para cualquier $c_0 > 0$. Sugerencia: identifique dicha sucesión como la iteración del Método de Newton para una función específica.

Ejercicio 2.26. [MATLAB] Escriba una función en MATLAB que implemente el método de la secante, dada una función f, y los dos valores iniciales x_0, x_1. Pruebe su programa con la función $f(x) = e^x - 2x - 1$, $x_0 = 1$, $x_1 = 2$. Compare su desempeño con el método de bisección. Dibuje en un mismo gráfico el error relativo en función del número de iteraciones, para ambos métodos. Utilice como valor exacto $c = 1,256431208626170$.

Ejercicio 2.27. [27, Ejercicio 1.10] La iteración del método de la secante puede ser escrito como

$$x_{k+1} := \frac{x_k f(x_{k-1}) - x_{k-1} f(x_k)}{f(x_{k-1}) - f(x_k)}. \tag{2.4}$$

Suponga que f tiene segunda derivada continua en un vecindario de un cero c de f, con $f'(c) > 0$, $f''(c) > 0$. Defina

$$\phi(x_k, x_{k-1}) := \frac{x_{k+1} - c}{(x_k - c)(x_{k-1} - c)},$$

donde x_{k+1} viene dado por la Ecuación 2.4.

(a) Calcule $\varphi(x_{k-1}) := \lim_{x_k \to c} \phi(x_k, x_{k-1})$.

(b) Calcule $\lim_{x_{k-1} \to c} \varphi(x_{k-1})$.

(c) Deduzca que $\lim_{x_{k-1}, x_k \to c} \phi(x_k, x_{k-1}) = \dfrac{f''(c)}{2f'(c)}$.

(d) Asuma que $\lim_{k \to \infty} \dfrac{|x_{k+1} - c|}{|x_k - c|^q} = A$. Demuestre que $q - 1 - 1/q = 0$ por lo que $q = \dfrac{1}{2}(1 + \sqrt{5})$.

(e) Deduzca finalmente que $\lim_{k \to \infty} \dfrac{|x_{k+1} - c|}{|x_k - c|^q} = \left(\dfrac{f''(c)}{2f'(c)} \right)^{q/(1+q)}$.

Ejercicio 2.28. (Método de Newton para raíces de multiplicidad m) Considere una función $f \in C^{m+1}([a, b])$ con una raíz $c \in (a, b)$ de multiplicidad m; i.e.,

$$f(c) = f'(c) = \ldots f^{(m-1)}(c) = 0 \text{ y } f^{(m)}(c) \neq 0.$$

(a) Explique porqué razón el método de Newton no converge cuadráticamente en este caso.

(b) Dado x_0 cercano a c, demuestre que si realizamos la iteración

$$x_{k+1} := x_k - m\frac{f(x_k)}{f'(x_k)}$$

recuperamos la convergencia cuadrática (note que debemos saber a priori el orden del cero).

(c) [`MATLAB`] Implemente una función en `MATLAB` que incluya como entrada un entero m que represente el orden del cero. Verifique el comportamiento de la iteración para $g(x) := x^3 - 3x^2 + 3x - 1$ con $x_0 = 1/2$ y $m \in \{1, 2, 3, 4\}$. Comente sus resultados.

Ejercicio 2.29. [`MATLAB`] (Intervalos de convergencia del método de Newton) Considere la función

$$f(x) = 816x^3 + 3835x^2 + 6000x - 3125,$$

la cual tiene tres raíces reales diferentes contenidas en el intervalo $[1.4, 1.7]$.

(a) Utilice la función `fzero` de `MATLAB` para aproximar las tres soluciones. (Para esta función es posible encontrar las tres raíces de manera exacta pues son números racionales).

(b) Defina un vector de valores iniciales x_0 = `linspace(1.4,1.7)`. Para cada elemento de x_0 obtenga el valor de convergencia del método de Newton. Grafique el valor de convergencia en función del valor inicial. Utilice la derivada exacta al calcular la iteración.

(c) La *base de atracción* se refiere al mayor intervalo I_c que contiene a un cero c, que garantiza que el método convergerá a c si tomamos cualquier elemento del intervalo como valor inicial. Teóricamente, para la convergencia es necesario que $|x_0 - c| \leqslant \left|\dfrac{f''(c)}{2f'(c)}\right|^{-1}$. Compare este estimado con los resultados numéricos de la sección anterior. ¿Hay algún estimado particularmente *malo*? En caso afirmativo, ¿puede explicar el porqué?

Ejercicio 2.30. Considere un polinomio $p(x) = a_n x^n + \ldots + a_1 x + a_0$.

(a) Dado x_0 real, ¿cuántas sumas y multiplicaciones se requieren para evaluar $p(x_0)$ de manera directa?

(b) Considere la sucesión dada por

$$\begin{aligned} b_n &= a_n, \\ b_{n-1} &= a_{n-1} + b_n x_0, \\ &\vdots \\ b_1 &= a_1 + b_2 x_0, \\ b_0 &= a_0 + b_1 x_0. \end{aligned}$$

Verifique que $b_0 = p(x_0)$. ¿Cuántas sumas y multiplicaciones se requieren para calcular b_0? Compare su resultado con el del inciso (a).

(c) Escriba una función `evaluaPolis` en `MATLAB` que reciba de entrada el vector con los coeficientes del polinomio $c = [a_n, a_{n-1}, \ldots, a_1, a_0]$ y el valor x_0. La salida es la evaluación $p(x_0)$ utilizando el algoritmo del inciso (b).

(d) Para n entero positivo, defina el vector de coeficientes como

$$\texttt{c} = \texttt{poly(1:n)},$$

el cual corresponde a los coeficientes del único polinomio mónico con raíces $\{1, 2, \ldots, n\}$. Para $n = 15$, verifique que $p(1) = \ldots = p(15) = 0$. ¿Ocurre lo mismo para $n = 19$? ¿Qué sucede en este último caso?

(e) Escriba una función `derivaPolis` que reciba como entrada el vector de coeficientes c de p y devuelva el vector de coeficientes de p'.

(f) Escriba una función `newtonPolis` que realice las iteraciones del método de Newton en el caso de polinomios. Para ello, utilice las funciones `evaluaPolis` y `derivaPolis` definidas anteriormente (de esta forma, no estamos introduciendo errores de truncamiento al aproximar la derivada). Las entradas deben ser el vector de coeficientes c, el valor inicial x_0 y la tolerancia deseada ϵ. Utilice $|x_k - x_{k-1}| < \epsilon$ como condición para detener la iteración.

(g) Verifique el comportamiento de la función `newtonPolis` para $p(x) = 8x^3 - 60x^2 + 142x - 105$. Defina un vector de valores iniciales $x_0 =$ `linspace(0,4,1e3)` y tome $\epsilon = 10^{-8}$. Para cada elemento de x_0 obtenga el valor de convergencia del método de Newton. Grafique el valor de convergencia en función del valor inicial.

(h) Compare los estimados teóricos de las bases de atracción que garantizan convergencia, con los resultados numéricos de la sección anterior. ¿Hay algún estimado particularmente *malo*? En caso afirmativo, ¿puede explicar el porqué?

Ejercicio 2.31. *(Método de la Regla Falsa)* [MATLAB] A continuación describimos uno de los métodos más antiguos para resolver ecuaciones no lineales. Considere una función continua f definida en el intervalo $I_0 = [a_0, b_0]$, donde $f(a_0)f(b_0) < 0$. En la iteración k, dado el intervalo $I_k = [a_k, b_k]$, calcule

$$c_k = \frac{a_k f(b_k) - b_k f(a_k)}{f(b_k) - f(a_k)}.$$

Luego evalúe $f(c_k)$. Si $f(a_k)f(c_k) < 0$, tome $I_{k+1} = [a_k, c_k]$; en caso contrario $I_{k+1} = [c_k, b_k]$.

(a) Explique el significado geométrico del método, así como su conexión con el Método de la Secante y de Bisección.

(b) Escriba una función en MATLAB que implemente el Método de la Regla Falsa. Debe tener como entradas la función f, el intervalo inicial $I_0 = [a_0, b_0]$ y la tolerancia ϵ. Considere $|c_k - c_{k-1}| < \epsilon$ como condición para detener la iteración.

(c) Compruebe el comportamiento de su código con la función $f(x) = 1/x - \sin(x) + 1$, con $a_0 = -1.3$, $b_0 = -0.5$. Numéricamente, ¿cuál es el orden de convergencia?

(d) Compare la razón de convergencia para la función del inciso anterior al utilizar los Métodos de Bisección y Secante. En este caso particular, ¿el método de Regla Falsa tiene alguna ventaja sobre los otros?

(e) Considere ahora $f(x) = 2x^3 - 4x^2 + 3x$ con intervalo inicial $[-1, 1]$. ¿Qué ocurre con los intervalos I_k en este caso?

Ejercicio 2.32. *(Método de Halley)* Dada una función $f \in C^2([a, b])$ y un valor inicial x_0, defina la sucesión $\{x_k\}_{k=0}^{\infty}$ dada por

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)} \left[1 - \frac{f(x_k)}{f'(x_k)} \cdot \frac{f''(x_k)}{2f'(x_k)} \right]^{-1}.$$

(a) Sea $f \in C^3([a, b])$ tal que $f(c) = 0$, donde c es un cero simple. Demuestre que existe $\delta > 0$ tal que si $x_0 \in (c - \delta, c + \delta)$, entonces x_k converge a c. ¿Qué hipótesis adicionales se necesitan?

(b) Sea $f \in C^3([a,b])$ tal que $f(c) = 0$, donde c es un cero simple. Demuestre que la velocidad de convergencia es cúbica.

(c) [MATLAB] Escriba una función `suc = iter(f,x0,tol)` en MATLAB que calcule la sucesión definida anteriormente, donde las entradas son la función f, el valor inicial x_0 y cierta tolerancia `tol`. Por simplicidad en la implementación, asumiremos que f es un polinomio, por lo que al calcular sus derivadas podemos hacerlo de manera exacta (a precisión de la máquina).

(d) [MATLAB] Verifique el comportamiento de su función para $f(x) = 816x^3 - 3835x^2 + 6000x - 3125$. Utilice $x_0 = 1.6$, `tol` $= 10^{-6}$. Compare el comportamiento de la sucesión obtenida con la que se obtiene al aplicar el Método de Newton. Dibuje en un mismo gráfico el error $|x_k - c|$ para ambos casos. Comente sus resultados.

(e) [MATLAB] Grafique los intervalos de atracción del Método de Halley. Para ello, considere el vector de valores iniciales `xx = linspace(1.4, 1.7)`. Grafique el valor de convergencia del método en función de `xx`. ¿Existe alguna mejoría con respecto al Método de Newton (ver Ejercicio 2.29)?

Ejercicio 2.33. [MATLAB] (*Método de Illinois*) Escriba una función en MATLAB que implemente el método que se describe a continuación:

Sea f una función continua y considere dados x_0, x_1 con $f(x_0)f(x_1) < 0$. Asuma que para un k, $f(x_{k-1})f(x_k) < 0$. En este caso, x_{k+1} es definida por el método de la secante.

Si $f(x_k)f(x_{k+1}) < 0$, también se utiliza el método de la secante en la siguiente iteración. En caso contrario, hay un cambio de signo entre x_{k-1} y x_{k+1} (pues k, $f(x_{k-1})f(x_k) < 0$ y $f(x_k)f(x_{k+1}) > 0$). Luego, encuentre la intersección de la recta que pasa por $(x_{k+1}, f(x_{k+1}))$ y $(x_{k-1}, f(x_{k-1})/2)$ con el eje x. La abscisa de este punto se selecciona como x_{k+2} si hay un cambio de signo entre su imagen y la imagen de x_{k+1}. En caso contrario, encuentre la intersección de la recta que pasa por $(x_{k+1}, f(x_{k+1}))$ y $(x_{k-1}, f(x_{k-1})/4)$ con el eje x y pruebe si hay cambio de signo. De ser necesario, se deben calcular y probar puntos adicionales utilizando $f(x_{k-1})/2^n$, $n \in \{1, 2, 3, \ldots\}$ (es posible probar que eventualmente un cambio de signo ocurrirá). De ser necesario, se continua con la iteración. Note que $f(x_{k+1})f(x_{k+2}) < 0$ y podemos utilizar el mismo procedimiento.

Verifique el comportamiento de su función para $f(x) = e^x - 2x - 1$, $x_0 = 1$, $x_1 = 2$ y compare con los otros métodos estudiados.

Ejercicio 2.34. *(Método de Steffensen)* Dado x_0, considere la iteración dada por

$$x_{k+1} := x_k - \frac{[f(x_k)]^2}{f(x_k + f(x_k)) - f(x_k)},\ k = 0, 1, 2 \ldots,$$

para aproximar numéricamente un cero c de la ecuación $f(x) = 0$.

(a) Explique su conexión con el método de Newton.

(b) Demuestre que la convergencia es cuadrática si x_0 está suficientemente cerca de la solución c.

Ejercicio 2.35. [MATLAB] Al inicio de cada año un banco deposita v colones en un fondo de inversión y retira, al final del año n, un capital de M colones. Sabemos que

$$M = v \sum_{k=1}^{n} (1 + r)^k,$$

donde r es la razón promedio anual de interés del fondo. Determine el valor de r si se desea duplicar el capital en 4 años.

Ejercicio 2.36. *(Enfriamiento de un cuerpo)* La Ley de Enfriamiento de Newton dice que la razón de cambio de temperatura en el tiempo t de un objeto es proporcional a la diferencia entre la temperatura ambiente T_a y la temperatura T del objeto; i.e.,

$$\frac{dT}{dt} = k(T_a - T),$$

donde k es la constante de proporcionalidad.

(a) Asuma que la temperatura ambiente es constante. Verifique que la temperatura del objeto en el tiempo t viene entonces dada por

$$T(t) = T_a + (T_0 - T_a)e^{-kt},$$

donde $T_0 = T(0)$ es la temperatura inicial del objeto.

(b) [MATLAB] Un médico forense desea estimar el valor de k para poder determinar el tiempo de muerte de cadáveres. Para ello, realiza diferentes mediciones, obteniendo los siguientes datos:

t_i (horas)	T_i ($^\circ C$)
0.0	37.00
0.2	36.72
0.4	36.41
0.6	36.12
0.8	35.90

Además, la temperatura ambiente de ese día es de $21^\circ C$. Para estimar k, el médico considera minimizar el funcional

$$f(k) := \sum_i \left(T(t_i) - T_i\right)^2 .$$

Aproxime el punto crítico de f y verifique que f alcanza un mínimo.

(c) [`MATLAB`] Un día caluroso en San Pedro a $31^\circ C$ una persona falleció. Al llegar al lugar de los hechos, determina que la temperatura del cuerpo es de $34^\circ C$. ¿Hace cuánto falleció la persona? (tome $37^\circ C$ como la temperatura promedio del cuerpo humano, y k como el valor estimado del inciso anterior).

Ejercicio 2.37. En este ejercicio se asume que en su computadora no existe la implementación de la operación *división*.

(a) Dado $b \in \mathbb{R}$, $b \neq 0$, escriba la fórmula de la iteración de Newton para $f(x) = 1/x - b$. Verifique las condiciones para garantizar que dicha sucesión converge a $1/b$. Recuerde que no deben aparecer divisiones en su algoritmo, pues la computadora no puede efectuarlas.

(b) [`MATLAB`] Implemente una función `myDivision` que implemente el algoritmo del inciso anterior. Como deseamos obtener $1/b$, tome la tolerancia igual a la precisión de la máquina y $|x_k - x_{k-1}| < \epsilon$ como condición para detener la iteración.

(c) [`MATLAB`] Fije $b = \pi$. Compruebe el comportamiento de su código para valores iniciales $x_0 \in$ `linspace(0,1,1e5)`. Grafique el valor de convergencia en función de x_0. ¿En qué intervalo debe estar x_0 para garantizar convergencia? Verifique su afirmación de manera teórica.

(d) Describa una posible estrategia (que no involucre realizar divisiones) para elegir x_0 para cualquier b dado, de forma tal que se pueda garantizar convergencia.

Ejercicio 2.38. Considere $N_c = 16$ colonias de insectos donde la población de machos y hembras se encuentra en el archivo `data.txt`. Sea Y_i una variable aleatoria que denota el número de hembras en cada colonia ($i \in \{1, \ldots, N_c\}$) y denote por n_i el total de insectos en cada grupo. Considere que las variables Y_i son independientes con distribución binomial y parámetro θ, dada por

$$f(y_i; \theta) = \binom{n_i}{y_i} \theta^{y_i} (1-\theta)^{n_i - y_i}.$$

El objetivo de este ejercicio es estimar el parámetro θ mediante el método de máxima verosimilitud.

(a) Defina la función de máxima verosimilitud

$$L(y_1, \ldots, y_{N_c}; \theta) = \prod_{i=1}^{N_c} f(y_i; \theta).$$

Calcule (a mano) el valor $\hat{\theta}$ que maximiza $l(\theta) := \log L(y_1, \ldots, y_{N_c}; \theta)$, donde los y_i son valores dados. Verifique que l alcanza un máximo en ese valor.

(b) [`MATLAB`] Utilice el comando

```
T = readtable('data.txt');
```

para cargar la tabla con la información. Puede visualizar el contenido de la tabla mediante el comando `disp(T)` o escribiendo de forma directa `T`. Note el nombre de los encabezados de cada columna. La información de los datos se muestra en la siguiente tabla:

Población	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Machos	11	22	27	29	24	29	25	26	38	14	23	31	20	6	34	12
Hembras	18	31	34	33	27	33	28	23	33	12	19	25	14	4	22	7

(c) [`MATLAB`] Para convertir el contenido de cada columna a un vector numérico ejecute los comandos

```
m = T.Machos;
h = T.Hembras;
```

De esta forma, tiene dos vectores $\boldsymbol{m}, \boldsymbol{h}$ con la población de machos y hembras en cada uno de las N_c colonias. Calcule el valor $\hat{\theta}$ que calculó en el inciso (a) para los datos suministrados. Este será el valor exacto.

(d) [`MATLAB`] Implemente en Matlab el método de la secante para aproximar una solución numérica de la ecuación

$$\frac{dl}{d\theta} = 0.$$

Utilice condiciones iniciales apropiadas. Compare su resultado con el valor exacto encontrado en el inciso anterior.

(e) [`MATLAB`] Compare además su resultado con la aproximación que brinda la función `fsolve` de `MATLAB`. Comente sobre la salida que brinda el programa al ejecutar

```
[x,fval,exitflag,output,jacob] = fsolve(...).
```

Indique el algoritmo que utilizó (que se guarda en la variable `output`).

Ejercicio 2.39. Considere el Método de Bisección con intervalo inicial $[0, 1]$. Determine el número de iteraciones para el cual el error es de orden *precisión de la máquina*. Esto implica que a partir de esa iteración, la máquina no puede obtener una mayor cantidad de cifras significativas correctas.

Ejercicio 2.40. Dado c_0, considere la iteración simple $c_k = g(c_{k-1})$ para $k \geqslant 1$.

(a) Para $g(x) = x - (x^2 - 3)/2$ definida en $[1, 2]$, verifique analíticamente que se cumplen las hipótesis para garantizar que (c_k) converge a su punto fijo en dicho intervalo.

(b) Escriba un código que implemente la iteración simple. Realice 80 iteraciones con $c_0 = 1.5$. Grafique el error relativo en función del número de iteraciones. Experimentalmente, ¿cuál es la pendiente de la recta en su gráfica? ¿Qué representa dicha pendiente?

(c) Considere la sucesión dada por

$$a_k = \frac{c_k c_{k+2} - c_{k+1}^2}{c_k + c_{k+2} - 2c_{k+1}}.$$

Es posible demostrar que (a_k) converge a c más rápido que (c_k), en el sentido que $\lim(a_k - c)/(c_k - c) = 0$. Escriba un código que calcule la

sucesión (a_k) para los valores de (c_k) del inciso anterior. Grafique el error relativo. ¿Se observa una convergencia más rápida? Explique el comportamiento del gráfico.

Ejercicio 2.41. En este ejercicio se espera verificar numéricamente el orden de convergencia del método de la secante, el cual es $q = \dfrac{1+\sqrt{5}}{2}$.

(a) Escriba una función

```
function ck = secante(f,c0,c1,tol,iterMax)
```

que realice la iteración del método de la secante, dada una función f, los valores iniciales c_0, c_1, la tolerancia deseada `tol` y un número máximo de iteraciones `iterMax`. La salida es la sucesión $\{c_k\}$ que genera el algoritmo, donde la iteración se detiene cuando $|f(c_k)| <$ `tol` o cuando se alcanza el número máximo de iteraciones (para unificar resultados).

(b) Compruebe el funcionamiento de su código para

```
ck = secante(@(x) x-cos(x),1/2,pi/4,1e-12,1e3);
```

Reporte el número de iteraciones realizadas y el valor de convergencia. Grafique el error $\|c-c_k\|$ en función de k. Tome $c = 0.7390851332151607$ como valor real.

(c) Sabemos que $\lim\limits_{k\to\infty} \dfrac{|c_{k+1}-c|}{|c_k-c|^q}$ existe. Por lo tanto, para k suficientemente grande, tiene sentido la aproximación

$$\frac{|c_{k+1}-c|}{|c_k-c|^q} \approx \frac{|c_k-c|}{|c_{k-1}-c|^q}.$$

Despejando q de la expresión anterior, se obtiene una aproximación para el orden de convergencia de la sucesión. Escriba un código que calcule q según la aproximación anterior, y verifique numéricamente que q se acerca al valor teórico $(1+\sqrt{5})/2$ conforme k aumenta. Para ello, muestre los valores de q en función de k.

Ejercicio 2.42. Sea $f : \mathbb{R} \to \mathbb{R}$ una función continua y estrictamente monótona (creciente o decreciente estrictamente). Se desea encontrar un cero $\xi \in \mathbb{R}$ de f, para lo cual se brinda un intervalo $[a_0, b_0]$ el cual no necesariamente satisface que $f(a_0)f(b_0) < 0$ (condición de cambio de signo).

Vamos a utilizar el siguiente método de localización para hallar un intervalo de $\mathbb{R}$ que cumpla esta condición para f. Definiendo $h := b_0 - a_0 + s$, se realiza:

$$m_k := \frac{f(b_k) - f(a_k)}{b_k - a_k}$$

y

$$[a_{k+1}, b_{k+1}] := \begin{cases} [a_k - h,\, a_k - s] & \text{si } m_k > 0 \wedge f(a_k) > 0, \\ [b_k + s,\, b_k + h] & \text{si } m_k > 0 \wedge f(a_k) < 0, \\ [b_k + s,\, b_k + h] & \text{si } m_k < 0 \wedge f(a_k) > 0, \\ [a_k - h,\, a_k - s] & \text{si } m_k < 0 \wedge f(a_k) < 0, \end{cases}$$

para todo $k = 0, 1, 2, \ldots$, donde $s \geqslant 0$ es un parámetro adicional. A este método de localización se le conocerá como *seguimiento de pendiente*, y se detiene cuando en el intervalo obtenido $[a_k, b_k]$ ocurra una de las siguientes dos situaciones:

- $f(a_k)f(b_k) \leqslant 0$, en cuyo caso el intervalo sería: $a = a_k$ y $b = b_k$.
- $f(a_k)f(a_{k-1}) \leqslant 0$, en cuyo caso el intervalo sería: $a = \text{mín}\{b_{k-1}, b_k\}$ y $b = \text{máx}\{a_{k-1}, a_k\}$.

(a) Explique en qué consiste el método de seguimiento de pendiente. Una descripción gráfica de su proceso también es aceptable.

(b) Considere la función: $f(x) := e^{x-5} + \frac{x}{3} - 2$, así como el intervalo $[0, 1]$ y $s = 0.5$. Implemente el método de seguimiento de pendiente para hallar un intervalo donde f cumpla la condición de cambio de signo.

(c) Aplique 5 iteraciones del método de bisección para aproximar un cero de f en el intervalo encontrado en b).

(d) Tomando $\xi = 4.382357533223939$ como valor exacto, determine el error absoluto de la aproximación obtenida en c).

(e) Mencione una situación que justifique por qué es necesario, para el método de seguimiento de pendiente, que f sea estrictamente monótona.

Ejercicio 2.43. [MATLAB] En este ejercicio se resolverá una ecuación por los tres métodos estudiados en clase. Considere la función de Bessel de primer tipo $J_\nu(x)$ (donde ν es el orden de la función), la cual se puede evaluar

en `MATLAB` mediante el comando `besselj(nu,x)`. Sabemos además que se cumple que

$$\frac{d}{dx}[x^{\nu} J_{\nu}(x)] = x^{\nu} J_{\nu-1}(x).$$

(a) Grafique $J_1(x)$ para $x \in [0, 10]$.

(b) Defina una función que represente $\frac{d}{dx} J_1(x)$ (la cual deberá utilizar para el método de Newton).

(c) Compare el comportamiento de los métodos de bisección, Newton y secante, para aproximar el primer cero positivo de J_1. Indique el valor de dicha solución con un error de orden precisión de la máquina. ¿Cuál es la velocidad de convergencia observado para cada método? Realice un gráfico que permita visualizar sus resultados. ¿Existe algún método que se comporte de manera distinta a las cotas teóricas?

Capítulo 3

Interpolación y aproximación de funciones

3.1. Introducción

Imagine que en cierta aplicación tiene un conjunto de datos conocido, producto de alguna medición de un fenómeno de interés. En este capítulo iniciamos estudiando el problema de *interpolar* dicho conjunto de datos mediante polinomios. Así, buscamos determinar un polinomio cuyo gráfico incluya los puntos dados y que aproxime de la mejor manera posible la función que desconocemos. Históricamente, este problema data hasta el año 300 A.C. en Babilonia y Grecia antigua, en donde se buscaba, por ejemplo, predecir la posición del sol, la luna y los planetas a partir de observaciones diarias, así como los agricultores que deseaban predecir tiempos óptimos para sus cosechas. Matemáticamente, los métodos más famosos fueron estudiados formalmente por Isaac Newton y Joseph-Louis Lagrange en los siglos XVII y XVIII, brindando los fundamentos de la teoría clásica de interpolación. Un dato interesante es que la fórmula de interpolación atribuida a Lagrange fue publicada 16 años antes por el matemático Edward Waring; parte de la historia sobre interpolación puede ser consultada en [18].

Como veremos, si utilizamos polinomios de un grado alto es posible obtener *interpoladores* con grandes oscilaciones que no aproximan de manera correcta el conjunto de datos en casos particulares. Este comportamiento es conocido como el *Fenómeno de Runge*. Por ello, es necesario también considerar el caso de interpoladores definidos a trozos, llamados *trazadores*.

Un enfoque alternativo para aproximar funciones consiste en utilizar polinomios de Chebyshev, los cuales permiten una implementación eficiente de

los métodos. Como veremos, la fórmula de interpolación baricéntrica permitirá evaluar de manera más eficiente los polinomios de interpolación en puntos de Chebyshev. Estudiaremos además reglas para aproximar derivadas y en en el Capítulo 4 utilizaremos fórmulas de interpolación para aproximar numéricamente integrales.

3.2. Interpolación de Lagrange

Sea n un entero no negativo. Denotaremos por $\mathbb{P}_n$ al conjunto de polinomios en una variable de grado a lo sumo n. Considere un conjunto dado de $n+1$ puntos $\{(x_i, y_i)\}_{i=0}^n$, con $x_i \neq x_j \ \forall \ i \neq j$. Deseamos calcular un polinomio de grado $p_n \in \mathbb{P}_n$ tal que

$$p_n(x_i) = y_i \text{ para } i = 0, 1, \ldots, n. \tag{3.1}$$

Como $\dim \mathbb{P}_n = n+1$, tiene sentido buscar un polinomio $p_n \in \mathbb{P}_n$, debido a que tenemos $n+1$ condiciones para determinar $n+1$ coeficientes. Veamos primero que dicho problema tiene a lo sumo una solución. Suponga que existen $p_n, q_n \in \mathbb{P}_n$ que satisfacen (3.1). Entonces, su diferencia es un polinomio de grado a lo sumo n con $n+1$ ceros. Del Teorema Fundamental del Álgebra se deduce que $p_n - q_n \equiv 0$, y así p_n es único.

Para demostrar la existencia de p_n, basta generar una base de polinomios $L_k \in \mathbb{P}_n$ ($k \in \{0, 1, \ldots, n\}$) tales que

$$L_k(x_i) = \delta_{ik}, \tag{3.2}$$

donde $\delta_{ik} = 1$ si $i = k$ y $\delta_{ik} = 0$ en caso contrario. Note que el conjunto de polinomios $\{L_k\}$ depende de n, pero omitimos dicha dependencia en la notación por simplicidad. De existir tales polinomios, bastaría definir

$$p_n(x) := \sum_{k=0}^{n} y_k L_k(x) \in \mathbb{P}_n.$$

Debe ser claro que p_n definido de esta manera satisface (3.1).

Para definir los polinomios L_k, utilizamos el hecho que conocemos sus n diferentes ceros[1]. De esta manera, podemos escribir

$$\begin{aligned} L_k(x) &= c_k(x - x_0) \ldots (x - x_{k-1})(x - x_{k+1}) \ldots (x - x_n) \\ &= c_k \prod_{i=0, i\neq k}^{n} (x - x_i). \end{aligned}$$

[1]Si $n = 0$, definimos $L_0(x) = 1$ y el polinomio es el polinomio constante $p_0(x) = y_0$

La constante c_k se determina con la condición $L_k(x_k) = 1$. Definimos entonces los polinomios

$$L_k(x) := \prod_{i=0, i\neq k}^{n} \frac{x - x_i}{x_k - x_i}, \tag{3.3}$$

los cuales satisfacen la Ecuación 3.2.

De esta manera, tenemos la siguiente definición:

Definición 3.1. *Sea f una función continua en un intervalo $[a, b]$. Dados $n+1$ valores distintos $x_i \in [a, b]$, $(i = 0, \ldots, n)$, el polinomio de interpolación de Lagrange de orden n de f se define como*

$$p_n(x) := \sum_{k=0}^{n} f(x_k) L_k(x) \in \mathbb{P}_n,$$

donde los polinomios $L_k(x)$ vienen dados por (3.3). □

Ejercicio 3.2. Estime el número de operaciones (sumas, restas, productos y divisiones) que se necesitan para evaluar L_k en un valor dado x. ¿Cuál es entonces el orden del número de operaciones requeridas para evaluar $p_n(x)$?

Con respecto al error entre f y p_n, tenemos el siguiente resultado:

Teorema 3.3. *Sea $f \in C^{n+1}([a, b])$. Entonces existe $\xi = \xi(x) \in (a, b)$ tal que*

$$f(x) - p_n(x) = \frac{f^{n+1}(\xi)}{(n+1)!} \pi_{n+1}(x) \ \forall \ x \in [a, b],$$

donde $\pi_{n+1}(x) := (x - x_0) \ldots (x - x_n) \in \mathbb{P}_{n+1}$.

Demostración. Si $x = x_i$ para algún $i \in \{0, \ldots, n\}$, es claro que la igualdad se cumple pues ambos lados se anulan. En caso contrario, considere la función auxiliar

$$\phi(t) := f(t) - p_n(t) - \frac{f(x) - p_n(x)}{\pi_{n+1}(x)} \pi_{n+1}(t). \tag{3.4}$$

Por simple evaluación se deduce que $\{x, x_0, x_1, \ldots, x_n\}$ son $n + 2$ ceros diferentes de $\phi(t)$. Si ordenamos dicho conjunto de ceros, entre dos raíces consecutivas podemos encontrar un valor en donde $\phi'(t)$ se anula (por el Teorema

de Rolle). De esta forma tenemos $n+1$ ceros diferentes de $\phi'(t) = 0$. Procediendo de manera similar, $\phi''(t)$ tiene n ceros diferentes. Inductivamente podemos concluir que $\phi^{(n+1)}(t)$ tiene un cero $\xi_x \in (a,b)$. Utilizando (3.4) para calcular $\phi^{(n+1)}$, al evaluar $t = \xi$ se obtiene

$$0 = f^{(n+1)}(\xi) - 0 - \frac{f(x) - p_n(x)}{\pi_{n+1}(x)}(n+1)!,$$

de donde se deduce el resultado. □

En general, el Teorema 3.3 no garantiza convergencia uniforme si se aumenta el grado del polinomio de interpolación. Para ello, se necesitaría que

$$\lim_{n\to\infty} \frac{1}{(n+1)!} \max_{\xi\in[a,b]} |f^{(n+1)}(\xi)| \max_{x\in[a,b]} |\pi_{n+1}(x)| = 0.$$

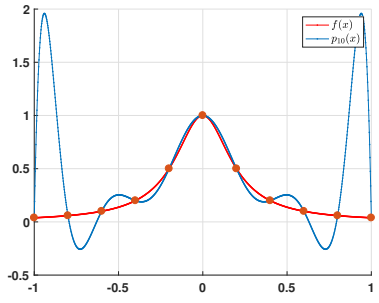
En el siguiente ejemplo veremos que en general no se cumple dicha condición, donde mostramos el famoso *fenómeno de Runge*:

Ejemplo 3.4. Considere la función $f(x) = \dfrac{1}{1+25x^2},\ x \in [-1,1]$. Es claro que $f \in C^\infty([-1,1])$. Considere $n+1$ nodos igualmente espaciados en el intervalo $[0,1]$, los cuales vienen dados por

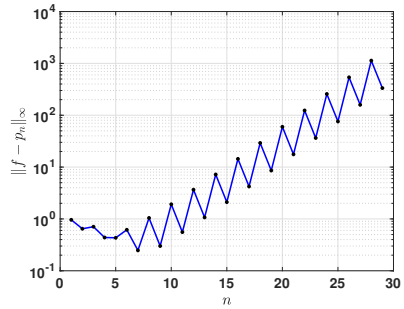
$$x_i = -1 + \frac{2}{n}i,\ i = 0,\dots,n.$$

Al graficar el polinomio de interpolación de Lagrange, conforme se aumenta el número de puntos (y por ende el grado del polinomio), se observa cómo se presentan mayores oscilaciones en los extremos del intervalo; ver Figura 3.1a. Además, en la Figura 3.1b se observa cómo la norma infinita de la diferencia entre la función y su polinomio interpolador crece conforme n aumenta. □

Comentario 3.5. En general, la interpolación con puntos equidistantes es exponencialmente mal condicionada: el polinomio p_n puede tener oscilaciones cerca de los extremos del intervalo cerca de 2^n veces mayor que la función f que es interpolada, incluso si $f \in C^\infty$ y aún si p_n es calculado de manera exacta sin errores de redondeo. Para profundizar en estos temas se recomienda leer [28, Capítulo 13]. Una alternativa consiste en encontrar una función que en lugar de interpolar los datos, minimice algún funcional de interés; ver Capítulo 7. □



(a) $f(x)$, $p_{10}(x)$ y nodos de interpolación.



(b) Error $\|f - p_n\|_\infty$ en función de n.

Figura 3.1: Polinomio de Lagrange y error para $f(x) = 1/(1+25x^2)$, $|x| \leqslant 1$; ver Ejemplo 3.4.

Comentario 3.6. `MATLAB` incluye la función `polyfit` para interpolar funciones en una variable. Dado un polinomio

$$p(x) = \sum_{k=0}^{n} c_k x^k,$$

basta considerar su vector de coeficientes

$$\boldsymbol{c} = [c_n, c_{n-1}, \ldots c_1, c_0]^T.$$

La instrucción `c = polyfit(x,y,n)` genera el vector de coeficientes $\boldsymbol{c}$ para el conjunto de datos dado en los vectores $\boldsymbol{x}, \boldsymbol{y}$. Note que se debe indicar el grado n para obtener el polinomio de interpolación; disminuyendo el valor de la tercer entrada permite obtener aproximaciones polinomiales de menor grado, relacionadas con el problema de mínimos cuadrados que se estudiará en el Capítulo 7. Para evaluar el vector de coeficientes del polinomio, se puede utilizar la función `polyval`. □

3.3. Interpolación de Hermite

Suponga ahora además que conocemos el valor de la derivada en los nodos de interpolación $\{x_i\}_{i=0}^n$. Esto es, dados (x_i, y_i, z_i) $(i \in \{0, 1, \ldots, n\})$ deseamos construir un polinomio h que satisfaga

$$h(x_i) = y_i, \quad h'(x_i) = z_i \quad \text{para } i \in \{0, 1, \ldots, n\}. \tag{3.5}$$

La primer pregunta que surge es: ¿cuál debe ser el grado del polinomio h? De las condiciones dadas en (3.5), tenemos $2n+2$ igualdades que satisfacer. Por lo tanto, es natural pensar que $h \in \mathbb{P}_{2n+1}$ pues $\dim \mathbb{P}_{2n+1} = 2n+2$. De manera análoga a la sección anterior, definiremos explícitamente una base de polinomios adecuada. Utilizando los polinomios L_k introducidos en (3.3), defina los polinomios $H_k, K_k \in \mathbb{P}_{2n+1}$ $(k \in \{0, 1, \ldots, n\})$ dados por

$$H_k(x) := L_k^2(x) \cdot \big(1 - 2L_k'(x_k)(x - x_k)\big),$$
$$K_k(x) := L_k^2(x) \cdot (x - x_k).$$

Calculando de manera directa podemos verificar que

$$H_k(x_i) = \delta_{ik}, \quad H_k'(x_i) = 0,$$

$$K_k(x_i) = 0, \quad K_k'(x_i) = \delta_{ik}.$$

Defina entonces

$$h_{2n+1}(x) := \sum_{k=0}^{n} y_i H_k(x) + z_i K_k(x). \tag{3.6}$$

Al evaluar es sencillo ver que h_{2n+1} satisface (3.5). Es posible además mostrar que dicho polinomio es único. Para tal efecto, suponga que h_{2n+1} y $\tilde{h}_{2n+1}$ satisfacen (3.5). En este caso, $h_{2n+1} - \tilde{h}_{2n+1}$ se anula en $n+1$ puntos $\{y_0, \ldots y_n\}$. Por el Teorema de Rolle, sabemos entonces que $h'_{2n+1} - \tilde{h}'_{2n+1}$ se anula en n puntos distintos (que se ubican entre dos ceros consecutivos de $h_{2n+1} - \tilde{h}_{2n+1}$). Por definición es claro que

$$(h'_{2n+1} - \tilde{h}'_{2n+1})(x_i) = 0 \ \forall \ i \in \{0, 1, \ldots, n\}.$$

De esta manera, $h'_{2n+1} - \tilde{h}'_{2n+1}$ es un polinomio de grado $2n$ con $2n+1$ ceros. Esto implica que $h'_{2n+1} - \tilde{h}'_{2n+1} \equiv 0$, por lo que $h_{2n+1} - \tilde{h}_{2n+1}$ es constante. Como ambas funciones coinciden en los x_i, entonces dicha constante es cero, y así $h_{2n+1} - \tilde{h}_{2n+1} \equiv 0$. Por lo tanto, concluimos que h_{2n+1} es único.

De esta manera, tenemos la siguiente definición:

Definición 3.7. *Sea $f \in C^1([a,b])$. Dados $n+1$ valores distintos $x_i \in [a,b]$, $(i = 0, \ldots, n)$, el polinomio de interpolación de Hermite de orden n de f se define como*

$$h_{2n+1}(x) := \sum_{k=0}^{n} f(x_k) H_k(x) + f'(x_k) K_k(x) \in \mathbb{P}_{2n+1}.$$

Con respecto al error, tenemos el siguiente resultado:

Teorema 3.8. *Sea $f \in C^{2n+2}([a,b])$. Entonces existe $\xi = \xi(x) \in (a,b)$ tal que*

$$f(x) - h_{2n+1}(x) = \frac{f^{2n+2}(\xi)}{(2n+2)!}\pi_{n+1}^2(x) \ \forall \ x \in [a,b],$$

donde $\pi_{n+1}(x) := (x-x_0)\dots(x-x_n) \in \mathbb{P}_{n+1}$.

Demostración. Si $x = x_i$ para algún $i \in \{0,\dots,n\}$, es claro que la igualdad se cumple pues ambos lados se anulan. En caso contrario, considere la función auxiliar

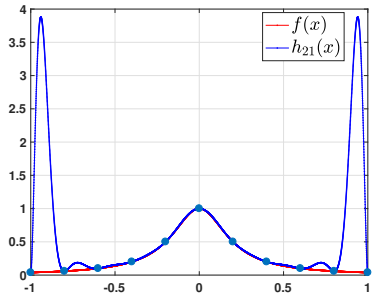
$$\phi(t) := f(t) - h_{2n+1}(t) - \frac{f(x) - h_{2n+1}(x)}{\pi_{n+1}^2(x)}\pi_{n+1}^2(t). \tag{3.7}$$

Similar a la prueba del Teorema 3.3, se tiene que $\{x, x_0, x_1, \dots, x_n\}$ son $n+2$ ceros diferentes de $\phi(t)$. Si ordenamos dicho conjunto de ceros, entre dos raíces consecutivas podemos encontrar un valor en donde $\phi'(t)$ se anula (por el Teorema de Rolle). De esta forma tenemos $n+1$ ceros diferentes de $\phi'(t) = 0$. Además, $\phi'(x_i) = 0 \ \forall i$ y en total ϕ' tiene $2n+2$ distintos ceros en $[a,b]$. Utilizando el teorema de Rolle, inductivamente podemos concluir que $\phi^{(2n+2)}(t)$ tiene un cero $\xi_x \in (a,b)$. Utilizando (3.7) para calcular $\phi^{(2n+2)}$, al evaluar $t = \xi$ se obtiene

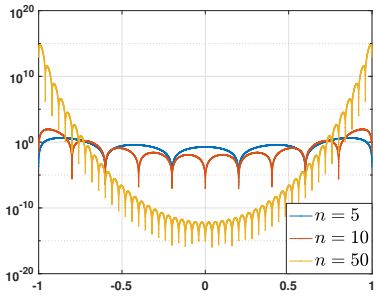
$$0 = f^{(2n+2)}(\xi) - 0 - \frac{f(x) - h_{2n+1}(x)}{\pi_{n+1}^2(x)}(2n+2)!,$$

de donde se concluye la igualdad del teorema. □

Ejemplo 3.9. Con la misma notación del Ejemplo 3.4, al calcular el polinomio de Hermite con $n = 10$ para la función $f(x) = 1/(1+25x^2)$ en el intervalo $[-1,1]$ se obtiene un comportamiento similar. Tal como se muestra en la Figura 3.2b, el error relativo alcanza un máximo del orden 10^{15} cuando $n = 50$. □



(a) $f(x)$ y h_{21}



(b) Error relativo

Figura 3.2: Polinomio de Hermite (izquierda) y error $|f(x) - h_{2n+1}(x)|$ (derecha) para $f(x) = 1/(1 + 25x^2)$, $|x| \leqslant 1$.

3.4. Aproximando derivadas de funciones

En la Sección 1.6.1 describimos un método para calcular la derivada de una función f dada en un punto x_0, utilizando fórmulas que se deducen al utilizar polinomios de Taylor. No obstante, si derivamos la fórmula obtenida en el Teorema 3.3 obtenemos que

$$f'(x) = p_n'(x) + \frac{1}{(n+1)!}\left(\pi_{n+1}(x)\frac{d}{dx}f^{n+1}(\xi) + f^{n+1}(\xi)\frac{d}{dx}\pi_{n+1}(x)\right).$$

Al sustituir $x = x_i$ y simplificar, se obtiene que

$$f'(x_i) = p_n'(x_i) + \frac{f^{n+1}(\xi_i)}{(n+1)!}\prod_{k=0,k\neq i}^{n}(x_i - x_k). \tag{3.8}$$

De esta manera, obtenemos el siguiente resultado:

Teorema 3.10. *Sea $f \in C^{n+2}([a,b])$ y considere un nodo de interpolación x_i, $i \in \{0,\ldots,n\}$. Entonces existe $\xi = \xi(x_i) \in (a,b)$ tal que*

$$f'(x_i) = \sum_{k=0}^{n} f(x_k)L_k'(x_i) + \frac{f^{n+1}(\xi)}{(n+1)!}\prod_{k=0,k\neq i}^{n}(x_i - x_k),$$

donde

$$L_k'(x) = \sum_{j\neq k}\frac{1}{x_k - x_j}\prod_{m\neq k,j}\frac{x - x_m}{x_k - x_m}.$$

Demostración. La prueba es un cálculo directo al utilizar (3.8) y derivar el polinomio de Lagrange dado en la Definición 3.1. □

De esta manera, tenemos la siguiente definición:

Definición 3.11. *Sea f una función derivable en un intervalo $[a, b]$ y considere una partición del intervalo $[a, b]$ de $n + 1$ puntos*

$$a = x_0 < x_1 < \ldots < x_{n+1} < x_n = b.$$

Se define la fórmula de $n + 1$ puntos para aproximar la derivada de f en un nodo x_i como la aproximación

$$f'(x_i) \approx \sum_{k=0}^{n} f(x_k) \sum_{j \neq k} \frac{1}{x_k - x_j} \prod_{m \neq k,j} \frac{x_i - x_m}{x_k - x_m}.$$

Ejemplo 3.12. Considere el caso de tres puntos equidistantes, dados por x_0, $x_1 = x_0 + h$ y $x_2 = x_1 + h$. Es fácil verificar que en este caso la fórmula del Teorema 3.10 se reduce a

$$f'(x_1) = \frac{f(x_1 + h) - f(x_1 - h)}{2h} + \frac{h^2}{6} f'''(\xi(x_1)),$$

la cual coincide con la aproximación de orden dos de la Sección 1.6.1. Similarmente, se puede obtener que

$$f'(x_0) = \frac{-3f(x_0) + 4f(x_1) - f(x_2)}{2h} + \frac{h^2}{3} f'''(\xi(x_0)),$$
$$f'(x_2) = \frac{f(x_0) - 4f(x_1) + 3f(x_2)}{2h} + \frac{h^2}{3} f'''(\xi(x_2)).$$

Debe ser claro que el comportamiento numérico de estas aproximaciones es similar al mencionado en la Sección 1.6.1. □

Ejemplo 3.13. Para el caso de cinco puntos equidistantes

$$\{x_0 - 2h, x_0 - h, x_0, x_0 + h, x_0 + 2h\},$$

el Teorema 3.10 se reduce a

$$f'(x_0) = \frac{f(x_0 - 2h) - 8f(x_0 - h) + 8f(x_0 + h) - f(x_0 + 2h)}{12h} + \\ + \frac{h^4}{30} f^{(5)}(\xi(x_0)),$$

la cual tiene el beneficio de tener un error de orden h^4. Si bien es cierto el problema de cancelación presentado en la Sección 1.6.1 es inevitable, en este caso el error decrece más rápido; ver Figura 3.3. De esta manera, si tomamos $h \approx 10^{-3}$, el error es del orden 10^{-12}. Si incluimos más nodos, el error decrecerá más rápido, pero normalmente utilizar cinco puntos en la práctica es suficiente. □

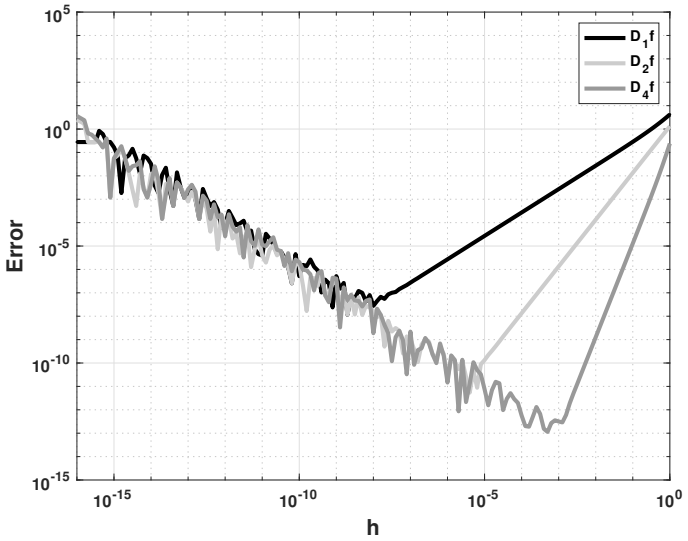


Figura 3.3: Error en las aproximaciones de orden 1, 2 y 4 para la derivada de la función del ejemplo de la Sección 1.6.1.

3.5. Interpolación lineal a trozos

Una manera de evitar posibles oscilaciones de los polinomios interpoladores conforme aumenta el grado, consiste en considerar funciones continuas a trozos. Asuma que tenemos una función continua $f \in C([a, b])$ y considere una partición del intervalo con $n + 1$ puntos

$$a = x_0 < x_1 < \ldots < x_{n-1} < x_n = b.$$

El caso más sencillo corresponde en considerar funciones lineales a trozos en dicha partición. Así, dado $x \in [x_{i-1}, x_i]$ $(i \in \{1, \ldots, n\})$, podemos interpolar

su valor mediante la recta que une los puntos $(x_{i-1}, f(x_{i-1}))$ y $(x_i, f(x_i))$. Esto es, en cada subintervalo definimos

$$s_L(x) := \frac{x_i - x}{x_i - x_{i-1}} f(x_{i-1}) + \frac{x - x_{i-1}}{x_i - x_{i-1}} f(x_i), \ x \in [x_{i-1}, x_i].$$

Si $f \in C^2([a,b])$, al sustituir $n = 1$ en el Teorema 3.3 se obtiene que para $x \in [x_{i-1}, x_i]$ el error en la aproximación viene dado por

$$f(x) - s_L(x) = \frac{f''(\xi)}{2}(x - x_{i-1})(x - x_i), \tag{3.9}$$

con ξ un valor entre x_{i-1} y x_i. Tomando valor absoluto, podemos acotar el error por

$$|f(x) - s_L(x)| \leqslant \frac{1}{8}\|f''\|_{L^\infty([x_{i-1},x_i])}(x_i - x_{i-1})^2, \ x \in [x_{i-1}, x_i],$$

al calcular el valor máximo de la función cuadrática de la derecha en (3.9). Tomando máximo sobre todos los intervalos, podemos concluir que

$$\|f - s_L\|_{L^\infty([a,b])} \leqslant \frac{h^2}{8}\|f''\|_{L^\infty([a,b])},$$

donde

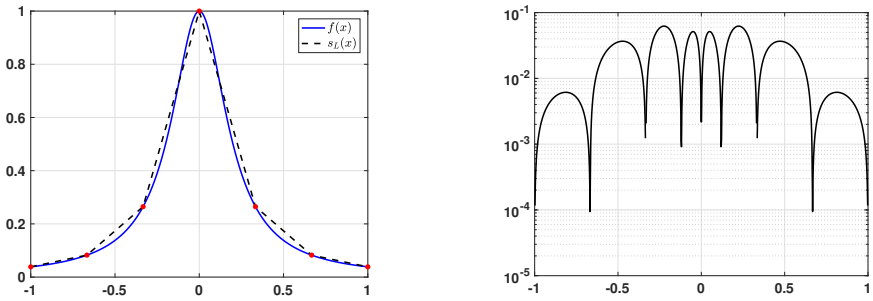
$$h := \max_{i \in \{1,\ldots,n\}} \{x_i - x_{i-1}\}.$$

De esta manera, en lugar de aumentar el grado de aproximación de los polinomios, estamos incrementando el número de nodos de interpolación, manteniendo polinomios lineales en cada subintervalo. Debe ser claro que en este caso no ocurre el fenómeno de Runge mencionado en el Ejemplo 3.4 debido al factor h^2; ver Figura 3.4.

3.6. Trazadores cúbicos

El interpolante lineal a trozos s_L definido en la sección anterior es una función continua, pero en cada nodo de la partición hay discontinuidades en su derivada. Si deseamos una función global con derivadas continuas, debemos aumentar el grado del polinomio en cada subintervalo. El caso más utilizado en práctica corresponde a los llamados *trazadores cúbicos*, en donde se define un polinomio de grado tres para cada subintervalo de la partición

$$a = x_0 < x_1 < \ldots < x_{n-1} < x_n = b.$$



(a) Trazador lineal (b) $|f(x) - s_L(x)|$

Figura 3.4: Trazador lineal para $f(x) = 1/(1 + 25x^2)$, $|x| \leqslant 1$, con $n = 6$.

Si tenemos conocimiento de $f(x_i)$ y $f'(x_i)$ $(i = 0, 1, \ldots, n)$, para cada subintervalo $[x_{i-1}, x_i]$ podemos construir el correspondiente polinomio de Hermite. Llamaremos *trazador cúbico de Hermite* a la función global s_3^H cuya restricción en cada intervalo $[x_{i-1}, x_i]$ es el interpolante de Hermite. En este caso es fácil ver que $s_3^H \in C^1([a, b])$ pues las derivadas coinciden en los puntos interiores de la partición. Es posible dar una fórmula sencilla para s_3^H. Escriba la restricción de s_3^H en el intervalo $[x_{i-1}, x_i]$ como

$$s_3^{H,i}(x) = c_{i,0} + c_{i,1}(x - x_{i-1}) + c_{i,2}(x - x_{i-1})^2 + c_{i,3}(x - x_{i-1})^3. \tag{3.10}$$

Dicho polinomio debe satisfacer las condiciones

$$s_3^{H,i}(x_{i-1}) = f(x_{i-1}), \qquad \frac{d}{dx} s_3^{H,i}(x_{i-1}) = f'(x_{i-1}),$$
$$s_3^{H,i}(x_i) = f(x_i), \qquad \frac{d}{dx} s_3^{H,i}(x_i) = f'(x_i).$$

Al sustituir (3.10) se obtiene que dichas constantes vienen dadas de manera explícita por

$$\begin{aligned} c_{i,0} &= f(x_{i-1}), \\ c_{i,1} &= f'(x_{i-1}), \\ c_{i,2} &= 3 \cdot \frac{f(x_i) - f(x_{i-1})}{(x_i - x_{i-1})^2} - \frac{f'(x_i) + 2f'(x_{i-1})}{x_i - x_{i-1}}, \\ c_{i,3} &= \frac{f'(x_i) + f'(x_{i-1})}{(x_i - x_{i-1})^2} - 2 \cdot \frac{f(x_i) - f(x_{i-1})}{(x_i - x_{i-1})^3}, \end{aligned} \tag{3.11}$$

por lo que tenemos de forma explícita la restricción de s_3^H sobre cada intervalo de la partición.

Ejercicio 3.14. Verifique que las constantes $c_{i,0}, c_{i,1}, c_{i,2}, c_{i,3}$ del polinomio (3.10) vienen dadas por (3.11).

Además, es posible deducir un estimado similar para el error en la norma infinita:

Ejercicio 3.15. Verifique que si $f \in C^4([a,b])$, entonces

$$\|f - s_3^H\|_{L^\infty([a,b])} \leqslant \frac{h^4}{384}\|f^{(4)}\|_{L^\infty([a,b])}.$$

Utilizando polinomios de grado tres podemos incluso definir un interpolante $s_3^N \in C^2([a,b])$ llamado *trazador cúbico natural*. Por el momento daremos la idea general de cómo se construye dicho interpolante; en el Capítulo 7 retomaremos esta construcción. Para definir completamente s_3^N debemos especificar un polinomio de grado tres en cada uno de los n subintervalos de la partición. De esta manera, ocupamos determinar $4n$ coeficientes en total. Como deseamos que $s_3^N \in C^2([a,b])$, tenemos las siguientes restricciones:

- $(n+1)$ condiciones de interpolación $s_3^N(x_i) = f(x_i),\ i = 0, 1, \ldots, n$.
- $(n-1)$ condiciones de continuidad para s_3^N (en los nodos internos de la partición).
- $(n-1)$ condiciones de continuidad para la primer derivada de s_3^N.
- $(n-1)$ condiciones de continuidad para la segunda derivada de s_3^N.

Así, tenemos $4n-2$ condiciones para encontrar $4n$ coeficientes. Por lo tanto, deben agregarse dos condiciones extra. Para los trazadores naturales se considera que

$$\left.\frac{d^2 s_3^N}{dx^2}\right|_{x=a} = \left.\frac{d^2 s_3^N}{dx^2}\right|_{x=b} = 0,$$

pero se pueden considerar variaciones. De esta manera se obtiene un sistema de $4n$ ecuaciones con $4n$ incógnitas que debe ser resuelto. En el Capítulo 7 demostraremos que dicho sistema tiene solución única.

En este caso se puede probar que

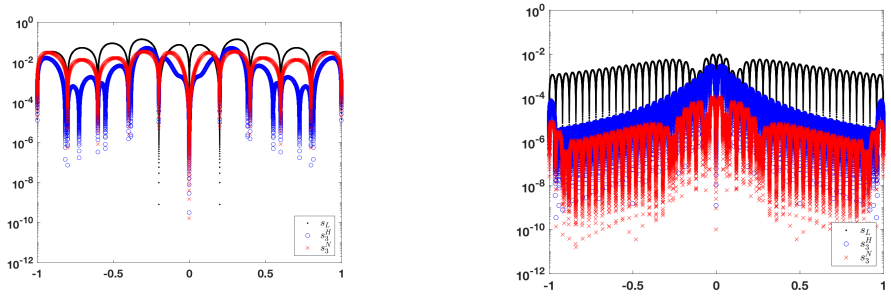
$$\|f - s_3^N\|_{L^\infty([a,b])} \leqslant \frac{5h^4}{384}\|f^{(4)}\|_{L^\infty([a,b])}.$$

Note que el error es cinco veces mayor al caso de splines cúbicos de Hermite, pero en el caso de los splines naturales no se utiliza información sobre la derivada de la función. Por esta razón es que es normal utilizar s_3^N para aproximar $f'(x_i)$. En este caso se puede demostrar que

$$\left\|\frac{d}{dx}f - \frac{d}{dx}s_3^N\right\|_{L^\infty([a,b])} \leqslant \frac{h^3}{24}\|f^{(4)}\|_{L^\infty([a,b])};$$

puede consultar [5] para la demostración de las últimas dos desigualdades.

Ejemplo 3.16. `MATLAB` incluye la función `interp1` para obtener diferentes interpolantes de funciones en una variable. En particular, incluye las opciones `linear` (interpolante lineal a trozos), `pchip` (trazador cúbico de Hermite) y `spline` (trazador cúbico natural). Utilizando la misma notación de los Ejemplos 3.4 y 3.9, al construir los diferentes interpoladores a trozos estudiados se puede observar que el error decrece conforme aumentamos el grado de interpolación; ver Figura 3.5. □



(a) $n = 10$ (b) $n = 50$

Figura 3.5: Error relativo con polinomios interpolantes a trozos.

3.7. Polinomios de Chebyshev

El fenómeno de Runge estudiado en los Ejemplos 3.4 y 3.9 puede evitarse si en lugar de utilizar puntos equidistantes en el intervalo $[-1, 1]$ utilizamos

los llamados *puntos de Chebyshev*. Como veremos más adelante, los *polinomios de Chebyshev* serán de gran utilidad.

Definición 3.17. *Sea $n \geqslant 0$. Para $x \in [-1,1]$, definimos el polinomio de Chebyshev (de primer tipo) de grado n como*

$$T_n(x) := \cos(n \arc\cos(x)).$$

Además, consideraremos los correspondientes puntos de Chebyshev (de segundo tipo), los cuales se definen como los ceros de T_n'. Dichos puntos vienen dados por

$$x_j = \cos\left(\frac{j\pi}{n}\right) \; j = 0, 1, \ldots, n. \tag{3.12}$$

Comentario 3.18. Es claro que $T_0(x) = 1$ y $T_1(x) = x$. De la identidad trigonométrica

$$\cos[(n+1)\theta] + \cos[(n-1)\theta] = 2\cos\theta\cos(n\theta),$$

al sustituir $\theta = \arc\cos x$ se deduce que

$$T_{n+1}(x) = 2xT_n(x) - T_{n-1}(x).$$

Como T_0 y T_1 son polinomios, se puede demostrar por inducción que

$$T_n \in \mathbb{P}_n \;\forall\; n \in \mathbb{N}.$$

Comentario 3.19. En la Figura 3.6 se muestran los puntos de Chebyshev para diferentes valores de n. Geométricamente considere $n+1$ puntos equidistantes en el semicírculo unitario superior, cuyos ángulos vienen dados por $\{j\pi/n\}_{j=0}^{n}$. Los puntos de Chebyshev corresponden a la proyección sobre el eje x de dichos puntos, por lo que se acumulan en los extremos del intervalo $[-1,1]$ conforme aumentamos n. Note además que $|T_n(x_j)| = 1$ pues son los puntos críticos del polinomio de Chebyshev. □

Ejercicio 3.20. Demuestre que:

1. el término principal de T_n es $2^{n-1}x^n \;\forall\; n \geqslant 1$.
2. si n es par entonces T_n es una función par.
3. si n es impar entonces T_n es una función impar.
4. $|T_n(x)| \leqslant 1 \;\forall x \in [-1,1],\; \forall\; n \in \mathbb{N}$.

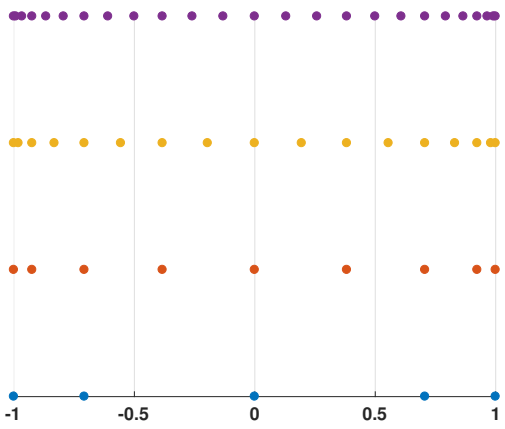


Figura 3.6: Puntos de Chebyshev para $n \in \{4, 8, 16, 24\}$.

Ejercicio 3.21. Dados dos enteros no negativos $m \neq n$, demuestre que

$$\int_{-1}^{1} T_m(x) T_n(x) (1 - x^2)^{-1/2} \, dx = 0.$$

Esta condición de *ortogonalidad* será retomada cuando hablemos de integración numérica.

Ejercicio 3.22. Sea $T_n(x)$ el polinomio de Chebyshev de grado n y considere los puntos de Chebyshev (3.12).

(a) Demuestre que $2^{-n}(T_{n+1}(x) - T_{n-1}(x))$ es el único polinomio mónico de grado $n+1$ cuyos $n+1$ zeros vienen dados por los puntos de Chebyshev (3.12).

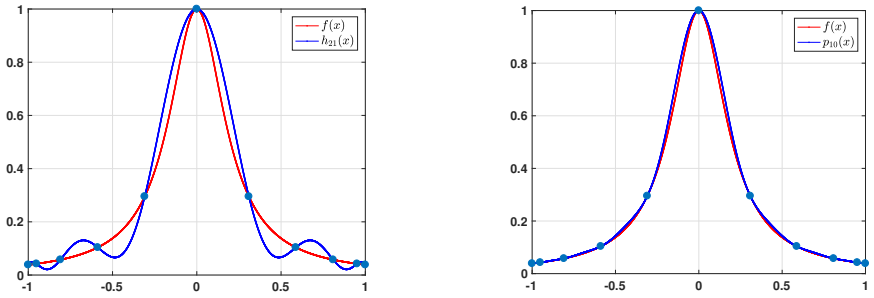
(b) Demuestre que

$$T'_{n+1}(x_j) - T'_{n-1}(x_j) = 2n(-1)^j, \ j \in \{1, \ldots, n-1\}$$

y que

$$T'_{n+1}(x_j) - T'_{n-1}(x_j) = 4n(-1)^j, \ j \in \{0, n\}.$$

Ejemplo 3.23. Retomemos el Ejemplo 3.4. Si en lugar de tomar $n+1$ puntos equidistantes en el intervalo $[-1, 1]$ utilizamos el polinomio de interpolación de Lagrange o de Hermite, con nodos de Chebyshev (3.12), se tiene que el fenómeno de Runge no ocurre; ver Figura 3.7. Intuitivamente, como los puntos de Chebyshev se acumulan en los extremos del intervalo, el polinomio de interpolación no puede presentar grandes oscilaciones en sus extremos. □



(a) p_{10} (b) h_{21}

Figura 3.7: Polinomios de interpolación de Lagrange y Hermite con puntos de Chebyshev.

3.8. Coordenadas baricéntricas

Para evaluar el polinomio de Lagrange p_n tal como se presentó en la Definición 3.1 se necesitan $\mathcal{O}(n^2)$ operaciones (ver Ejercicio 3.2). No obstante, si utilizamos interpolación con los puntos de Chebyshev es posible disminuir dicho número de operaciones.

Considere primero cualquier partición $\{x_i\}_{i=0}^n$. Defina

$$L(x) = \prod_{k=0}^{n}(x - x_k). \tag{3.13}$$

Mediante un cálculo directo podemos ver que

$$L'(x_k) = \prod_{i=0, i\neq k}^{n} (x - x_i),$$

por lo que

$$L_k(x) = \frac{L(x)}{L'(x_k)(x - x_k)}, \tag{3.14}$$

donde L_k son los polinomios de Lagrange definidos en (3.3). Si definimos

$$\lambda_k := \frac{1}{L'(x_k)},$$

podemos reescribir el polinomio de interpolación de Lagrange como

$$p_n(x) = L(x) \sum_{k=0}^{n} \frac{\lambda_k}{x - x_k} f(x_k).$$

Si los valores λ_k fueran conocidos, entonces solo necesitaríamos $\mathcal{O}(n)$ operaciones para evaluar $p_n(x)$.

Es claro que $\sum_{k=0}^{n} L_k(x) \in \mathbb{P}_n$. La imagen de cada uno de los $n+1$ nodos x_i $(i = 0, \ldots, n)$ es igual a 1, por lo que

$$\sum_{k=0}^{n} L_k(x) \equiv 1.$$

Por lo tanto, podemos dividir el lado derecho de (3.14) por $\sum_{k=0}^{n} L_k(x)$ para obtener que

$$L_k(x) = \frac{\lambda_k}{(x - x_k)} \cdot \frac{L(x)}{\sum_{k=0}^{n} L_k(x)} = \frac{\lambda_k}{(x - x_k)} \Bigg/ \sum_{j=0}^{n} \frac{\lambda_j}{x - x_j}. \tag{3.15}$$

De esta manera hemos probado el siguiente resultado:

Teorema 3.24 (Fórmula de interpolación baricéntrica)**.** *El polinomio de interpolación de Lagrange se puede escribir como*

$$p_n(x) = \sum_{j=0}^{n} \frac{\lambda_j f(x_j)}{x - x_j} \Bigg/ \sum_{j=0}^{n} \frac{\lambda_j}{x - x_j}, \text{ para } x \notin \{x_0, \ldots, x_n\}.$$

En los nodos de interpolación se entiende que $p_n(x_j) = f(x_j)$.

Demostración. Basta sustituir (3.15) en la definición del polinomio de Lagrange. □

Comentario 3.25. Note que si $x \to x_k$, el cociente tiende a $f(x_k)$, tal como se espera. A pesar de parecer una función racional, p definido de la forma anterior es efectivamente un polinomio. □

Si en particular tomamos los nodos $\{x_j\}$ como los puntos de Chebyshev (3.12), podemos obtener una fórmula explítica para los pesos λ_k:

Lema 3.26. *Para los puntos de Chebyshev (3.12) se tiene que*

$$\lambda_k = \frac{2^{n-1}}{n}(-1)^k,\ 1 \leqslant k \leqslant n-1,$$

y

$$\lambda_k = \frac{2^{n-2}}{n}(-1)^k,\ k \in \{0, n\}.$$

Demostración. De la parte (a) del Ejercicio 3.22 se tiene que

$$L(x) = 2^{-n}(T_{n+1}(x) - T_{n-1}(x)).$$

Utilizando la parte (b) del mismo ejercicio, para $1 \leqslant k \leqslant n-1$ obtenemos que

$$L'(x_k) = 2^{-n}(T'_{n+1}(x_k) - T'_{n-1}(x_k)) = n \cdot 2^{1-n}(-1)^k,$$

de donde se deduce la primer igualdad. De manera similar se comprueba la segunda igualdad. □

Teorema 3.27 (Fórmula baricéntrica en puntos de Chebyshev)**.** *El polinomio de interpolación de Lagrange en los puntos de Chebyshev (3.12) se puede escribir como*

$$p_n(x) = \sum_{j=0}^{n}{}' \frac{(-1)^j f(x_j)}{x - x_j} \Bigg/ \sum_{j=0}^{n}{}' \frac{(-1)^j}{x - x_j}, \text{ para } x \notin \{x_0, \ldots, x_n\},$$

donde el símbolo de prima en la sumatoria significa que el primer y último término se deben dividir por dos. Se entiende además que $p_n(x_j) = f(x_j)$ al ser los datos de interpolación.

Demostración. La prueba es consecuencia directa del lema anterior. □

Comentario 3.28. La ventaja de la fórmula baricéntrica para puntos de Chebyshev corresponde al hecho que los pesos λ_k son conocidos y fáciles de evaluar. De esta manera, para evaluar p en un valor x basta realizar $\mathcal{O}(n)$ operaciones. □

Ejercicio 3.29. Considere $n+1$ puntos equidistantes en el intervalo $[-1,1]$, los cuales vienen dados por

$$x_j = -1 + j \cdot \frac{2}{n},\ j = 0, 1, \ldots, n.$$

Demuestre que

$$\lambda_k = \left(\frac{n}{2}\right)^n \frac{(-1)^{n-k}}{k!(n-k)!}.$$

Concluya que en la fórmula baricéntrica podemos entonces utilizar

$$\lambda_k = (-1)^k \binom{n}{k}.$$

¿Cuál es el orden de operaciones que se debe realizar para calcular cada λ_k?

3.9. ¿Cómo compartir un secreto?

Concluimos el capítulo con una aplicación interesante de interpolación utilizada en cartografía, introducida en [23] por Adi Shamir en 1979. Considere que se tiene un dato secreto S (el cual asumimos es un número entero positivo) y fije k entero positivo. Se desea dividir S en n datos $S_1, \ldots, S_n$ (con $n \geqslant k$) de forma tal que:

- S se puede recuperar a partir de cualquier conjunto de k datos.
- S no se puede recuperar con menos de k datos.

Para tal efecto, considere un número primo $P > S$, $P > n$, y considere el conjunto de polinomios módulo P con coeficientes naturales. Seleccione $k-1$ enteros positivos aleatorios $\{a_i\}_{i=1}^{k-1}$ con $a_i < P\ \forall i$. Tome $a_0 = S$ y defina el polinomio

$$f(x) := a_0 + a_1 x + \ldots + a_{k-1} x^{k-1} \quad \text{mód } P.$$

Los n datos pueden tomarse como los puntos de interpolación

$$S_j = (j, f(j)) \quad j \in \{1, 2, \ldots, n\}.$$

De esta manera, dado cualquier conjunto de k datos distintos, podemos construir el polinomio de interpolación de grado $k-1$, y el dato secreto es

el coeficiente constante a_0. Debe ser claro que cualquier conjunto de datos con menos de k elementos es insuficiente para recuperar a_0.

Ejemplo 3.30. Considere $n = 6$, $k = 3$, $P = 1613$. El que desea compartir el secreto toma $S = 1234$ y considera el polinomio $f(x) = 1234 + 166x + 94x^2$ mód 1613. De esta manera comparte con seis personas los datos

$$(1, 1494), (2, 329), (3, 965), (4, 176), (5, 1188), (6, 775).$$

Veamos porqué no es posible recuperar S con menos de tres datos. Si un pirata informático conoce solo un dato, digamos $(1, 1494)$, lo único que sabe es que el polinomio de interpolación tiene la forma

$$f(x) = S + a_1x + a_2x^2 \quad \text{mód } 1613$$

y que se debe cumplir que

$$1494 = S + a_1 + a_2 \quad \text{mód } 1613, \tag{3.16}$$

y esta última ecuación tiene infinitas soluciones para S, a_1 y a_2. Si conoce otro dato, digamos $(2, 329)$, ahora conoce que se debe cumplir que

$$329 = S + 2a_1 + 4a_2 \quad \text{mód } 1613. \tag{3.17}$$

Al restar (3.17) y (3.16) se obtiene

$$448 = a_1 + 3a_2 \quad \text{mód } 1613, \tag{3.18}$$

y nuevamente dicha ecuación tiene infinitas soluciones. Si el pirata roba un nuevo dato, digamos $(3, 965)$, ahora sabe que

$$965 = S + 3a_1 + 9a_2 \quad \text{mód } 1613. \tag{3.19}$$

Al restar (3.19) y (3.17) se obtiene

$$636 = a_1 + 5a_2 \quad \text{mód } 1613. \tag{3.20}$$

Nuevamente, combinando (3.18) y (3.20) concluimos que

$$188 = 2a_2 \quad \text{mód } 1613,$$

de donde se deduce que $a_2 = 94$ (pues es sabido que $a_2 < P$). De manera similar se obtiene a_1 y el secreto S. □

3.10. Ejercicios Adicionales

Ejercicio 3.31. [MATLAB] Implemente un código en MATLAB que calcule el polinomio de interpolación de Chebyshev en los puntos (3.12) de una función dada f, mediante la fórmula baricéntrica

$$p_n(x) = {\sum_{j=0}^{n}}' \frac{(-1)^j f(x_j)}{x - x_j} \Big/ {\sum_{j=0}^{n}}' \frac{(-1)^j}{x - x_j}. \tag{3.21}$$

La función debe tener como entradas la función `f`, el grado del polinomio `n` y un vector de puntos `xx` en donde se desea evaluar p_n. La salida debe ser la imagen de cada elemento `xx`; esto es, p_n(`xx`).

(a) Para la función $f(x) = 1/(1 + 25x^2)$, $x \in [-1, 1]$, grafique f y p_n para el caso $n = 30$.

(b) Utilice las funciones de MATLAB dadas por `polyfit` y `polyval` para evaluar p_{30}. ¿Existe alguna ventaja o desventaja?

(c) Grafique el error relativo $\|f - p_n\|_{L^\infty([-1,1])}/\|f\|_{L^\infty([-1,1])}$ en función de n, $10 \leqslant n \leqslant 100$. Sugerencia: utilice la función `norm(f,'inf')`.

Ejercicio 3.32. En este ejercicio se busca construir una aproximación para la derivada de una función, a partir de su polinomio de interpolación de una manera diferente a las fórmulas estudiadas de $n + 1$ puntos.

(a) Demuestre que

$$L_k(x) = \frac{\lambda_k}{x - x_k} \Bigg/ \sum_{j=0}^{n} \frac{\lambda_j}{x - x_j}, \text{para } x \notin \{x_0, \ldots, x_n\},$$

donde $\{L_k\}$ es la base de polinomios de interpolación de Lagrange. Deduzca que

$$L_k(x)s(x) = \lambda_k \frac{x - x_i}{x - x_k},$$

donde $s(x) = \displaystyle\sum_{j=0}^{n} \lambda_j \frac{x - x_i}{x - x_j}$, $i \in \{0, 1, \ldots, n\}$.

(b) Del inciso (a), deduzca que

$$L_k'(x)s(x) + L_k(x)s'(x) = \lambda_k \left(\frac{x - x_i}{x - x_k} \right)'.$$

(c) Verifique que $s(x_i) = \lambda_i$ y deduzca que

$$L_k'(x_i) = \frac{\lambda_k}{\lambda_i}\frac{1}{x_i - x_k}(i \neq k).$$

(d) Demuestre que

$$L_k'(x_k) = -\sum_{i\neq k} L_i'(x_k).$$

(e) Denote la matriz $D \in \mathbb{R}^{n+1\times n+1}$ cuyas entradas vienen dadas por $D_{ik} = L_k'(x_i)$, para $i, j \in \{0, 1, \ldots, n\}$. ¿Qué representa el vector $Df(\hat{\boldsymbol{x}})$, donde $\hat{\boldsymbol{x}}$ es el vector de nodos de Chebyshev y $f(\hat{\boldsymbol{x}})$ el vector con las imágenes de dichos nodos?

(f) [MATLAB] Implemente una función que reciba n y calcule la matriz D del inciso anterior. Para $n = 20$, $f(x) = 1/(1+16x^2)$, calcule y grafique $f'(x)$, junto a los puntos $(\hat{\boldsymbol{x}}, Df(\hat{\boldsymbol{x}}))$. Cuantifique el error en los nodos de interpolación en esta gráfica.

(g) [MATLAB] Grafique $\|Df(\hat{\boldsymbol{x}}) - f'(\hat{\boldsymbol{x}})\|_\infty$ en función de n. ¿Cuál es el menor error posible?

Ejercicio 3.33. [27, Ejercicio 6.3] Considere $n+2$ puntos $\{(x_i, y_i)\}_{i=0}^{n+1}$ (donde los x_i son distintos). Sea q el polinomio de Lagrange de grado n que interpola los puntos $\{(x_i, y_i)\}_{i=0}^{n}$ y sea r el polinomio de Lagrange que interpola los puntos $\{(x_i, y_i)\}_{i=1}^{n+1}$. Defina

$$p(x) := \frac{(x - x_0)r(x) - (x - x_{n+1})q(x)}{x_{n+1} - x_0}.$$

Demuestre que p interpola todo el conjunto de puntos.

Ejercicio 3.34. [27, Ejercicio 6.10] Construya el polinomio de interpolación de Hermite para $f(x) = x^5$ usando $x_0 = 0$, $x_1 = a > 0$. Verifique el Teorema 3.8 y concluya que $\xi = (x + 2a)/5$.

Ejercicio 3.35. [27, Ejercicio 6.9] Sean $\epsilon > 0$, $n \geqslant 1$ y $f \in C^1([a, b])$. Suponga que $\{x_j\}_{j=0}^{2n-1}$ son $2n$ puntos distintos, donde $x_{n+j} = x_j + \epsilon$ para cada $j = 0, \ldots, n-1$. Denote por p_{2n-1} al polinomio de interpolación de Lagrange de f con nodos de interpolación $\{x_j\}_{j=0}^{2n-1}$.

(a) Demuestre que los coeficientes de p_{2n+1} que incluyen los términos $f(x_j)$ y $f(x_{n+j})$ vienen dados por

$$\frac{\phi_j(x)\phi_j(x-\epsilon)}{\epsilon\phi_j(x_j)}\left(\frac{x - x_j}{\phi_j(x_j + \epsilon)}f(x_j + \epsilon) - \frac{x - x_j - \epsilon}{\phi_j(x_j - \epsilon)}f(x_j)\right),$$

donde

$$\phi_j(x) := \prod_{i=0, i\neq j}^{n-1} (x - x_i).$$

(b) Encuentre el límite de la expresión anterior cuando $\epsilon \to 0$ y deduzca que $p_{2n-1} - h_{2n-1} \to 0$ cuando $\epsilon \to 0$, donde h_{2n-1} es el polinomio de Hermite de f con nodos $\{x_i\}_{i=0}^{n-1}$.

Ejercicio 3.36. [MATLAB] Considere la función $f(x) = \text{sgn}(x)$, $x \in [-1, 1]$.

(a) Construya el polinomio de Lagrange p_n con $n+1$ nodos equidistantes para $n \in \{10, 20, 30, 40, 50, 60, 70, 80\}$. Grafique el error $|p_n(x) - f(x)|$ en función de n.

(b) Repita el inciso anterior utilizando $n+1$ nodos de Chebyshev.

(c) ¿Qué observa cerca de $x = 0$ en cada caso?

Ejercicio 3.37. [Derivadas en coordenadas con polinomios de Chebyshev]

(a) Demuestre que la derivada del polinomio de Chevyshev $T_n(x)$ satisface la relación de recurrencia $(1 - x^2)T_n'(x) = n(T_{n-1}(x) - xT_n(x))$, para $n \geqslant 1$, $x \in [-1, 1]$.

(b) Suponga que $f(x) = \sum_{k=0}^{n} a_k T_k(x)$, $x \in [-1, 1]$. Su derivada puede expresarse en la misma base de polinomios $f'(x) = \sum_{k=0}^{n} b_k T_k(x)$. Obtenga una fórmula para los coeficientes $\{b_k\}$ en términos de los $\{a_k\}$. ¿Qué ventaja tiene este método?

(c) [MATLAB] Implemente una función que reciba como entrada a los coeficientes $\{a_k\}$ y calcule los coeficientes $\{b_k\}$. Para $a_k = k$ $(k = 0, 1, \ldots, 10)$, grafique $f(x)$ y $f'(x)$. Obtenga además una gráfica para el error en la aproximación de $f'(x)$.

Ejercicio 3.38. Dado n entero positivo, considere $\{(x_i, f(x_i))\}_{i=0}^{n}$ nodos de interpolación dados, donde $a = x_0 < x_1 < \ldots < x_n = b$. Sabemos que el polinomio de interpolación de grado (a lo sumo n) se puede escribir en su forma de Lagrange

$$p_n(x) = \sum_{k=0}^{n} f(x_k) L_k(x).$$

En este ejercicio, estudiaremos otra manera diferente de calcular dicho polinomio.

(a) Demuestre que es posible escribir

$$\begin{aligned} p_n(x) =& a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \\ & a_3(x - x_0)(x - x_1)(x - x_2) + \ldots + a_n(x - x_0)(x - x_1)\ldots(x - x_{n-1}). \end{aligned}$$

(b) Verifique que

$$a_0 = f(x_0),\ a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}.$$

(c) Determine el valor del coeficiente a_2 en términos de los nodos de interpolación.

(d) Concluya que si $n = 2$, podemos escribir

$$p_2(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1),$$

donde

$$f[x_0] = f(x_0),\ f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}, f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}$$

(e) Defina inductivamente

$$f[x_0, x_1, \ldots, x_k] = \frac{f[x_1, \ldots, x_k] - f[x_0, x_1, \ldots, x_{k-1}]}{x_k - x_0},$$

donde $f[x_k] = f(x_k)$. A este símbolo se le conoce como la *k-ésima diferencia dividida (hacia adelante)*. Demuestre que

$$\begin{aligned} p_n(x) =& f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \ldots + \\ & + f[x_0, x_1, \ldots x_n](x - x_0)(x - x_1)\ldots(x - x_{n-1}). \end{aligned} \tag{3.22}$$

(f) Numéricamente, si x_0 y x_1 son valores cercanos, ¿qué problema podría tener el cálculo de $f[x_0, x_1]$?

(g) A modo de ejemplo, considere el conjunto de puntos de interpolación

$$\{(1, -2), (2, 2), (3, 14), (4, 40)\}.$$

Calcule los valores de las diferencias divididas. Utilice como ejemplo el Cuadro 3.1 (el cual es la manera usual de tabular la información). Escriba explícitamente el polinomio interpolante de Lagrange $p_3(x)$ para este conjunto de puntos.

(h) Demuestre que existe $\xi \in [a,b]$ tal que $f[x_0, x_1, \ldots, x_n] = \dfrac{f^{(n)}(\xi)}{n!}$. Sugerencia: utilice (3.22) para definir el residuo $R(x) := f(x) - p_n(x)$ y considere el Teorema de Rolle.

(i) ¿Qué nos dice el resultado del inciso anterior si $n = 1$?

(j) [`MATLAB`] Escriba un código que calcule el polinomio de interpolación de Lagrange mediante el método de diferencias divididas. Como entrada debe recibir los nodos de interpolación y el valor de evaluación $\hat{x}$, y la salida es el valor $p_n(\hat{x})$ según la ecuación (3.22).

(k) [`MATLAB`] Utilice el código implementado en el inciso anterior para graficar el polinomio interpolador de Lagrange $p_{10}(x)$ para $f(x) = 1/(1+25x^2)$ con nodos equidistantes y nodos de Chebyshev. Para graficar, utilice el vector de puntos `xx = linspace(-1,1,1e3)`.

(l) Determine el número de operaciones elementales que se requieren para evaluar el polinomio de interpolación p_n en un valor $\tilde{x}$ dado (asumiendo guardamos todos los valores dados en el Cuadro 3.1).

x_0	$f(x_0)$			
		$f[x_0, x_1]$		
x_1	$f(x_1)$		$f[x_0, x_1, x_2]$	
		$f[x_1, x_2]$		$f[x_0, x_1, x_2, x_3]$
x_2	$f(x_2)$		$f[x_1, x_2, x_3]$	
		$f[x_2, x_3]$		
x_3	$f(x_3)$			

Cuadro 3.1: Cálculo de diferencias divididas

Ejercicio 3.39. En este ejercicio, se espera construir el polinomio de interpolación de Hermite con base en el método de diferencias divididas. Considere el polinomio de Hermite h_{2n+1} que interpola los datos $(x_i, f(x_i), f'(x_i))_{i=0}^n$.

(a) Demuestre que $f[x_0, x_1] = \int_0^1 f'((1-t)x_0 + tx_1)\, dt$. Concluya que $f[x_0, x_0] = f'(x_0)$.

(b) Considere la tabla de diferencias divididas dada en el Cuadro 3.2. Modifique el programa del Ejercicio 3.38 para calcular los coeficientes del polinomio de interpolación (según la asignación de nodos del Cuadro 3.2). Escriba una función

$$
\begin{array}{c|c|c|c|c}
x_0 = a & f(x_0) & & & \\
 & & f[x_0, x_1] & & \\
x_1 = a & f(x_1) & & f[x_0, x_1, x_2] & \\
 & & f[x_1, x_2] & & f[x_0, x_1, x_2, x_3] \\
x_2 = b & f(x_2) & & f[x_1, x_2, x_3] & \\
 & & f[x_2, x_3] & & \\
x_3 = b & f(x_3) & & &
\end{array}
$$

Cuadro 3.2: Cálculo de diferencias divididas para el polinomio de Hermite

```
function coeficientes = coefHermite(x,y,z)
```

que reciba los nodos x_k y los valores de interpolación $y_k = f(x_k)$, $z_k = f'(x_k)$. Sugerencia: al calcular $f[x_k, x_k]$, dicho valor debe remplazarse por $f'(x_k)$.

(c) Escriba una función

```
function yhat = evaluaHermite(coef,nodos,xhat)
```

que evalúe el polinomio de interpolación de Hermite dados los coeficientes de la diferencia dividida, los nodos de interpolación y el valor $\hat{x}$ de evaluación. La salida es el valor $\hat{y} = h_{2n+1}(\hat{x})$.

(d) Verifique su código para calcular el polinomio de interpolación de Hermite de $f(x) = x^6 - x - 1, x \in [-1, 1]$ en los nodos $\{-1, -1/3, 1/3, 1\}$. Grafique el error $|f(x) - h_{2n+1}|$ en función de x. Comente sus resultados.

Ejercicio 3.40. (Evaluando polinomios) Considere un polinomio de grado n escrito en la base usual

$$p(x) = \sum_{k=0}^{n} a_k x^k,$$

y defina el vector de coeficientes

$$\boldsymbol{a} = [a_0, \ldots, a_n]^T.$$

Para un valor x_0 dado, deseamos evaluar $p(x_0)$. Para ello, consideramos tres métodos diferentes:

Alg.1– Mediante evaluación directa, calculamos cada término $a_k x^k$ y luego sumamos todos los términos.

Alg.2– Defina la sucesión $\{b_k\}_{k=0}^n$ mediante la recursión hacia atrás

$$\begin{aligned} b_n &= a_n, \\ b_k &= a_k + b_{k+1}x_0 \quad \text{para } k = n-1, n-2, \ldots, 0. \end{aligned}$$

Entonces, $p(x_0) = b_0$.

Alg.3– Utilizar el comando `polyval()` de `MATLAB` para evaluar el polinomio.

(a) Determine el número de operaciones (sumas y multiplicaciones) que se deben realizar en el Algoritmo 1.

(b) Determine el número de operaciones (sumas y multiplicaciones) que se deben realizar en el Algoritmo 2.

(c) Para el Algoritmo 2, demuestre que $p(x_0) = b_0$.

(d) Compare el tiempo de ejecución de los Algoritmos 1, 2 y 3 en `MATLAB`. Para ello, genere un vector con entradas aleatorias de una distribución uniforme en $(0, 1)$ para los coeficientes mediante el comando

`a = rand(n,1).`

Utilice $n = 10^8$, $x_0 = 0.1$ y compare los tiempos de ejecución. (Sugerencia: los comandos `tic`, `toc`, `fliplr(a)`, `flipud(a)` pueden ser de utilidad)

(e) Considere ahora `a = ones(n,1)`, $n = 10^8$, $x_0 = 0.1$. ¿Cuál es el valor exacto de $p(x_0)$? Utilice el Algoritmo 1 y 2 para obtener aproximaciones $\widetilde{p}_1$ y $\widetilde{p}_2$, respectivamente. ¿Cuántas cifras significativas son correctas para $\widetilde{p}_1$ y $\widetilde{p}_2$?

Ejercicio 3.41. Suponga ahora que realizamos un cambio de base y expresamos el polinomio p en la base de los polinomios de Chebyshev,

$$p(x) = \sum_{k=0}^{n} c_k T_k(x), \quad |x| \leqslant 1,$$

donde $T_k(x) = \cos(k \arccos x)$. Sabemos que los polinomios de Chebyshev satisfacen la relación de recurrencia

$$T_{k+1}(x) = \alpha_k(x) T_k(x) + \beta_k(x) T_{k-1}(x), \tag{3.23}$$

para $\alpha_k(x) = 2x$ y $\beta_k(x) = -1$. Deseamos calcular $p(x_0)$. Para ello, considere el siguiente algoritmo:

Alg.4– Defina la sucesión $\{b_k(x_0)\}_{k=0}^{n+2}$ mediante la recurrencia hacia atrás:

$$b_{n+2}(x_0) = b_{n+1}(x_0) = 0,$$
$$b_k(x_0) = c_k + \alpha_k(x_0)b_{k+1}(x_0) + \beta_{k+1}(x_0)b_{k+2}(x_0) \quad \text{para } k = n, n-1, \ldots, 1.$$

Se tiene entonces que el valor deseado es

$$p(x_0) = T_0(x_0)c_0 + T_1(x_0)b_1(x_0) + \beta_1(x_0)T_0(x_0)b_2(x_0). \tag{3.24}$$

(a) Determine el número de operaciones (sumas y multiplicaciones) que se deben realizar en el Algoritmo 4.

(b) Verifique que la fórmula (3.24) es correcta.

(c) Escriba una función `y0 = evalCheb(c,x0)` en `MATLAB` que calcule $y_0 = p(x_0)$ mediante la fórmula (3.24), donde la entrada es el vector de coeficientes $\boldsymbol{c}$ y el valor x_0.

(d) Utilice `c = rand(n,1)`, $n = 10^8$, $x_0 = 0.1$ y compare el tiempo de ejecución con la pregunta (1d). Comente sus resultados.

(f) Considere el polinomio $p(x) = x^3 - 3x^2 + 1$. Exprese $p(x)$ como combinación lineal de polinomios de Chebyshev

$$p(x) = c_0T_0(x) + c_1T_1(x) + c_2T_2(x) + c_3T_3(x)$$

y verifique el comportamiento del Algoritmo 4 para $x_0 = 1$.

Ejercicio 3.42. Dado $n \in \mathbb{N}$, considere los $n+1$ puntos de Chebyshev dados por

$$x_j = \cos\left(\frac{j\pi}{n}\right) \; j = 0, 1, \ldots, n.$$

Dada una función f, es posible escribir el polinomio de interpolación de Lagrange con nodos $\{x_j\}_{j=0}^n$ en la base de polinomios de Chebysheb

$$p_n(x) = \sum_{k=0}^{n} f(x_k)L_k(x) = \sum_{k=0}^{n} c_kT_k(x).$$

Al conocer los valores $\{f(x_j)\}_{j=0}^n$ podemos calcular los coeficientes c_k de manera eficiente mediante la inversa de la transformada discreta de Fourier, la cual se implementa con el comando `ifft` en `MATLAB`. La siguiente función recibe como entrada los valores $\{f(x_j)\}_{j=0}^n$ en un vector columna y retorna los coeficientes en un vector $\boldsymbol{c}$:

```
function c = val2coef(values)
n = size(values, 1);
% Mirror the values
tmp = [values(n:-1:2,:) ; values(1:n-1,:)];
% apply ifft
c = real(ifft(tmp));
% Truncate
c = c(1:n,:);
% Scale the interior coefficients
c(2:n-1,:) = 2*c(2:n-1,:);
end
```

(a) Implemente una función `[coef, nodos] = interpCheb(n,f)` que reciba el grado n y la función f, que calcule los nodos de Chebyshev y los coeficientes $\{c_k\}$.

(b) Calcule los coeficientes `c = interpCheb(n,f)` para $f(x) = 1/(1 + 25x^2)$, $n = 10$. Defina `xx = linspace(-1,1,1e3)` y utilice el comando `evalCheb` para evaluar p_n en cada entrada de `xx`. Grafique f y p_n en un mismo gráfico, incluyendo los nodos de interpolación. Comente su resultado. ¿Ocurre el fenómeno de Runge?

(c) Grafique $\|f(\texttt{xx}) - p_n(\texttt{xx})\|_{\ell^\infty}$ en función de n para $n \in \{2, \ldots, 20\}$. ¿Qué velocidad de convergencia observa conforme n aumenta?

(d) Demuestre que para todo entero $n \in \mathbb{N}$, $n \neq 1$,

$$\int_{-1}^{1} T_n(x)\, dx = \frac{1 + (-1)^n}{1 - n^2}.$$

Deduzca que

$$\int_{-1}^{1} p_n(x)\, dx = \sum_{k=0,\ k \text{ par}}^{n} \frac{2c_k}{1 - k^2}. \tag{3.25}$$

(e) Programe una función `I = intCheb(c)`, cuya entrada es el vector de coeficientes

$$\boldsymbol{c} = [c_0, \ldots, c_n]^T,$$

que calcule el valor I de la integral (3.25).

(f) Para $f(x) = 1/(1 + 25x^2)$, $n = 10$, utilice la fórmula (3.25) para aproximar $\int_{-1}^{1} f(x)\, dx$. Grafique el error absoluto en función de n para $n \in \{2, \ldots, 20\}$. ¿Qué velocidad de convergencia observa conforme n aumenta?

(g) ¿Qué ventajas observa en interpolar en nodos de Chebyshev con expansiones en polinomios de Chebyshev para aproximar integrales?

Capítulo 4

Integración Numérica

4.1. Introducción

En este capítulo consideraremos el problema de calcular integrales definidas. Dada una función $f : (a, b) \to \mathbb{R}$ integrable, obtendremos una aproximación para

$$I_f := \int_a^b f(x)\ dx,$$

la cual denotaremos por $\widetilde{I}_f$. Asimismo, deduciremos expresiones para el error $|I_f - \widetilde{I}_f|$. Diremos que una fórmula de integración es de *orden* n si $I_p = \widetilde{I}_p$ para todo polinomio $p \in \mathbb{P}_n$; esto es, si la fórmula integra de manera exacta todo polinomio de grado a lo sumo n.

La primer idea será utilizar los polinomios de interpolación de Lagrange y sus estimados del error. Si en lugar de integrar f integramos su polinomio de Lagrange p_n, podemos considerar

$$\widetilde{I}_f := \int_a^b p_n(x)\ dx = \int_a^b \sum_{k=0}^{n} f(x_k) L_k(x)\ dx = \sum_{k=0}^{n} w_k f(x_k),$$

donde

$$w_k := \int_a^b L_k(x)\ dx. \tag{4.1}$$

Con esto deduciremos las conocidas fórmulas del trapecio ($n = 1$) y de Simpson ($n = 2$) para el caso de nodos de interpolación equidistantes. Luego definiremos sistemas de polinomios ortogonales que permitirán aumentar el

grado de exactitud de las aproximaciones, mediante el uso de los polinomios de Hermite. En general, buscaremos aproximaciones de la forma

$$\widetilde{I}_f = \sum_k w_k f(x_k),$$

donde $\{x_k\}$ y $\{w_k\}$ se llaman los *nodos* y los *pesos* de integración, respectivamente.

Históricamente, el problema de calcular áreas data de tiempos de la antigua Grecia en la escuela Pitagórica y en Babilonia, donde ya se usaba la regla del trapecio para resolver problemas relacionados con el movimiento de los planetas. Con el desarrollo del cálculo se desarollaron herramientas para determinar el valor de integrales definidas. No obstante, el término *Integración Numérica* se empieza a utilizar hasta 1915 en [9].

4.2. Fórmulas de Newton-Cotes

En esta sección seguimos el desarrollo de [27, Capítulo 7], con algunas modificaciones. Considere una partición del intervalo $[a, b]$ con $n+1$ puntos equidistantes, los cuales vienen dados por

$$x_j = a + j \cdot \frac{b-a}{n},\ j = 0, 1, \ldots, n. \tag{4.2}$$

Iniciamos con el estudio de dos de las fórmulas más básicas de integración:

4.2.1. Fórmula del trapecio

Dada una función f, su polinomio de interpolación de Lagrange de grado uno, con nodos $x_0 = a$ y $x_1 = b$, viene dado por

$$p_1(x) = f(a)\frac{x-b}{a-b} + f(b)\frac{x-a}{b-a};$$

ver Definición 3.1. Al integrar sobre el intervalo $[a, b]$ se obtiene que una aproximación para el valor de la integral es

$$\widetilde{I}_{\text{trap}} := \int_a^b p_1(x)\ dx = \frac{b-a}{2}\left(f(a) + f(b)\right).$$

Geométricamente, la fórmula anterior corresponde al área del trapecio que se forma entre los puntos $(a, 0)$, $(b, 0)$, $(a, f(a))$ y $(b, f(b))$; ver Figura 4.1. Para encontrar la fórmula del error entre el valor real de la integral y su

aproximación con p_1, se podría pensar en utilizar la cota obtenida en el Teorema 3.3, asumiendo que $f \in C^2([a,b])$. No obstante, si denotamos

$$c := (a+b)/2,$$

como $b-c = c-a = (b-a)/2$, utilizando la fórmula de integración por partes dos veces se deduce que

$$\begin{aligned} \mathcal{E}_1(f) :&= \int_a^b f(x)\ dx - \frac{b-a}{2}\left(f(a)+f(b)\right) \\ &= -\int_a^b (x-c)f'(x)\ dx \\ &= -\frac{1}{2}\int_a^b (x-a)(b-x)f''(x)\ dx. \end{aligned} \tag{4.3}$$

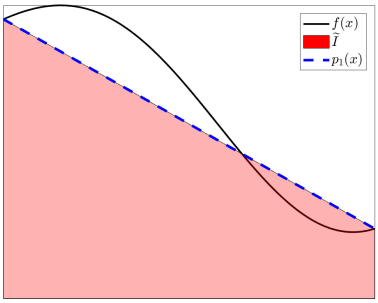


Figura 4.1: Interpretación geométrica de la fórmula del trapecio.

Para simplificar la expresión anterior utilizaremos el siguiente lema:

Lema 4.1 (Generalización del valor medio para integrales)**.** *Considere f continua en $[a,b]$ y g integrable en (a,b), tal que g no cambia de signo en $[a,b]$. Entonces, existe $\xi \in (a,b)$ tal que*

$$\int_a^b f(x)g(x)\ dx = f(\xi)\int_a^b g(x)\ dx.$$

Demostración. Suponga que $g(x) \geqslant 0$ en $[a,b]$ (el caso $g(x) \leqslant 0$ es análogo). Al ser f continua en $[a,b]$, alcanza un mínimo m y un máximo M. Por lo tanto,

$$m\int_a^b g(x)\ dx \leqslant \int_a^b f(x)g(x)\ dx \leqslant M\int_a^b g(x)\ dx.$$

Si $\int_a^b g(x)\ dx = 0$ no hay nada que probar. En caso contrario,

$$m \leqslant \frac{\int_a^b f(x)g(x)\ dx}{\int_a^b g(x)\ dx} \leqslant M.$$

Por el Teorema del Valor Intermedio aplicado a f concluimos el resultado. □

Utilizando el lema anterior, podemos garantizar que existe $\xi \in [a, b]$ tal que

$$\mathcal{E}_1(f) = -\frac{1}{2}f''(\xi)\int_a^b (x-a)(b-x)\ dx = -\frac{(b-a)^3}{12}f''(\xi).$$

De esta manera, hemos comprobado el siguiente resultado:

Teorema 4.2. *Sea $f \in C^2([a, b])$. Entonces, existe $\xi \in [a, b]$ tal que*

$$\int_a^b f(x)\ dx = \frac{b-a}{2}\left(f(a) + f(b)\right) - \frac{(b-a)^3}{12}f''(\xi).$$

Comentario 4.3. Del teorema anterior debe ser claro que la fórmula es exacta (el error es idénticamente cero) si f es una función lineal. Como veremos a continuación, para el caso del polinomio de Lagrange con $n = 2$ obtendremos una fórmula que es exacta para polinomios de grado 3, a pesar de estar integrando un polinomio cuadrático. □

Ejercicio 4.4. El comentario afirma que si $f \in \mathbb{P}_1$, entonces $\mathcal{E}_1(f) = 0$. ¿Es cierto el recíproco?

Ejercicio 4.5. Deduzca una cota superior para el error de la fórmula del trapecio si asumimos solo que $f \in C^1([a, b])$. Sugerencia: utilice la igualdad (4.3).

4.2.2. Fórmula de Simpson

Considere ahora el polinomio de interpolación de grado 2, con

$$x_0 = a, \quad x_1 = (a+b)/2, \quad x_2 = b.$$

En este caso podemos calcular los pesos que vienen dados por

$$\begin{aligned} w_0 &= \int_a^b \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}\, dx = \frac{(b-a)}{6}, \\ w_1 &= \int_a^b \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}\, dx = \frac{4(b-a)}{6}, \\ w_2 &= \int_a^b \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}\, dx = \frac{(b-a)}{6}. \end{aligned}$$

De esta forma definimos

$$\widetilde{I}_{\text{Simp}} := \int_a^b p_2(x)\, dx = \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right],$$

la cual es conocida como *fórmula de Simpson*. A continuación se presenta la fórmula del error correspondiente:

Teorema 4.6. *Suponga que $f \in C^4([a,b])$. Entonces existe $\xi \in (a,b)$ tal que*

$$\int_a^b f(x)\, dx = \frac{b-a}{6}\left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right] - \frac{f^{(4)}(\xi)}{2880}(b-a)^5.$$

Demostración. Considere el cambio de variable

$$x = \frac{a+b}{2} + \frac{b-a}{2}t, \ t \in [-1,1].$$

Defina $F(t) := f\left(\frac{a+b}{2} + \frac{b-a}{2}t\right)$. Podemos así escribir

$$I - \widetilde{I}_{\text{Simp}} = \frac{b-a}{2}\left[\int_{-1}^1 F(t)\, dt - \frac{1}{3}\left(F(-1) + 4F(0) + F(1)\right)\right] = \frac{b-a}{2}G(1),$$

donde definimos

$$G(t) := \int_{-t}^t F(t)\, dt - \frac{t}{3}\left(F(-t) + 4F(0) + F(t)\right).$$

Sea $H(t) := G(t) - t^5 G(1)$. Tenemos entonces que:

(a) $H(0) = H(1) = 0$, por lo que existe $\xi_1 \in (0,1)$ tal que $H'(\xi_1) = 0$.

(b) $H'(0) = H'(\xi_1) = 0$, por lo que existe $\xi_2 \in (0,\xi_1)$ tal que $H''(\xi_2) = 0$.

(b) $H''(0) = H''(\xi_2) = 0$, por lo que existe $\xi_3 \in (0,\xi_2)$ tal que $H'''(\xi_3) = 0$.

De un cálculo directo obtenemos que

$$0 = H'''(\xi_3) = -\frac{\xi_3}{3}\left[F'''(\xi_3) - F'''(-\xi_3)\right] - 60\xi_3^2 G(1).$$

Utilizando el Teorema del Valor Medio a la función F''', existe $\xi_4 \in (-\xi_3, \xi_3)$ tal que

$$0 = -\frac{\xi_3}{3}\left[2\xi_3 F^{(4)}(\xi_4)\right] - 60\xi_3^2 G(1).$$

Despejando $G(1)$ obtenemos el resultado. □

Comentario 4.7. Como el error depende de la cuarta derivada de la función, la fórmula de Simpson integra de manera exacta polinomios de grado a lo sumo tres. Note que a pesar de estar aproximando la función f con un polinomio cuadrático, ganamos un orden de exactitud en la integración. □

Comentario 4.8. Se pueden escribir fórmulas explícitas para $n \geqslant 3$. No obstante, al utilizar $n+1$ puntos equidistantes en el intervalo $[a,b]$ podríamos obtener una mala aproximación para f; ver Ejemplo 3.9. También, es posible que los pesos sean negativos, lo cual puede provocar errores de cancelación en la suma; ver Ejercicio 4.43. En lugar de aumentar el grado del polinomio, en la siguiente sección consideraremos particiones del intervalo $[a,b]$, y en cada partición usaremos la fórmula del Trapecio o Simpson. □

4.3. Fórmulas compuestas

Las expresiones para el error en las aproximaciones del trapecio y de Simpson presentan el inconveniente de depender del tamaño del intervalo de integración. Si consideramos una partición uniforme del intervalo dada por (4.2), podemos escribir

$$I_f = \int_a^b f(x)\ dx = \sum_{i=1}^{n} \int_{x_{i-1}}^{x_i} f(x)\ dx.$$

Defina $h := (b-a)/n$. Aplicando la regla del trapecio en cada subintervalo tenemos que

$$\int_{x_{i-1}}^{x_i} f(x)\ dx = \frac{h}{2}\left(f(x_{i-1}) + f(x_i)\right) - \frac{h^3}{12} f''(\xi_i)$$

para algún $\xi_i \in [x_{i-1}, x_i]$. De esta forma,

$$I_f = \frac{h}{2} \sum_{i=1}^{n} (f(x_{i-1}) + f(x_i)) - \frac{h^3}{12} \sum_{i=1}^{n} f''(\xi_i).$$

Por el teorema del valor intermedio aplicado a f'', tenemos que existe $\xi \in (a, b)$ tal que

$$\frac{1}{n} \sum_{i=1}^{n} f''(\xi_i) = f''(\xi),$$

por lo que tenemos que

$$I_f = \frac{b-a}{n} \sum_{i=0}^{n}{}' f(x_i) - \frac{(b-a)^3}{12n^2} f''(\xi),$$

donde el símbolo de prima en la sumatoria significa que el primer y último término se deben dividir por dos. De esta manera, hemos probado el siguiente resultado:

Teorema 4.9. *Considere una función $f \in C^2([a, b])$ y la partición uniforme de $n + 1$ puntos de $[a, b]$ dada en (4.2). Entonces existe $\xi \in (a, b)$ tal que*

$$I_f - \widetilde{I}_{\mathrm{TC}} = -\frac{(b-a)^3}{12n^2} f''(\xi),$$

donde

$$\widetilde{I}_{\mathrm{TC}} := \frac{b-a}{n} \sum_{i=0}^{n}{}' f(x_i)$$

se define como la fórmula compuesta del trapecio, donde el símbolo de prima en la sumatoria significa que el primer y último término se deben dividir por dos.

Si utilizamos la regla de Simpson, después de un cálculo análogo obtenemos el siguiente resultado:

Teorema 4.10. *Considere $f \in C^4([a, b])$ y la partición uniforme de $n + 1$ puntos de $[a, b]$ dada en (4.2). Entonces existe $\xi \in (a, b)$ tal que*

$$I_f - \widetilde{I}_{\mathrm{SC}} = -\frac{(b-a)^5}{2880n^4} f^{(4)}(\xi),$$

donde

$$\widetilde{I}_{\mathrm{SC}} := \frac{b-a}{6n} \sum_{i=1}^{n} \left(f(x_{i-1}) + 4f\left(\frac{x_{i-1} + x_i}{2}\right) + f(x_i) \right)$$

se define como la fórmula compuesta de Simpson.

Comentario 4.11. En los dos resultados anteriores, note que al aumentar el número de intervalos n podemos controlar el término del error, por lo que es posible obtener una aproximación precisa para la integral si tomamos n suficientemente grande. □

4.4. Método adaptivo de Simpson

Las fórmulas compuestas permiten mejorar la aproximación de la integral conforme aumentamos el número de subintervalos. No obstante, el hecho de utilizar una partición uniforme podría no ser eficiente. Por ejemplo, podríamos ahorrar tiempo en la evaluación de f si supiéramos que en cierta parte de su dominio se comporta de manera adecuada.

Asuma que inicialmente tenemos una partición (no necesariamente uniforme)

$$a = x_0 < x_1 < \ldots < x_{n-1} < x_n = b.$$

Defina $h_i := x_{i+1} - x_i,\ i = 0, \ldots, n-1$. En el intervalo $[x_i, x_{i+1}]$ podemos aproximar la contribución de la integral de dos maneras. Primero, mediante la fórmula de Simpson defina

$$S_i := \frac{h_i}{6}\left(f(x_i) + 4f(x_i + h_i/2) + f(x_{i+1})\right).$$

Segundo, si dividimos $[x_i, x_{i+1}]$ en dos subintervalos $[x_i, x_i + h_i/2]$ y $[x_i + h_i/2, x_{i+1}]$, podemos utilizar la regla de Simpson en cada uno de ellos para escribir

$$\widetilde{S}_i = \frac{h_i}{12}\left(f(x_i) + 4f\left(x_i + \frac{h_i}{4}\right) + 2f\left(x_i + \frac{h_i}{2}\right) + \right.$$
$$\left. + 4f\left(x_i + \frac{3h_i}{4}\right) + f\left(x_{i+1}\right)\right).$$

Denote por I_i la integral de f en el intervalo $[x_i, x_{i+1}]$. Del Teorema 4.6 podemos escribir

$$I_i - S_i = -\frac{f^{(4)}(\xi_1)}{2880}h_i^5,$$
$$I_i - \widetilde{S}_i = -\frac{f^{(4)}(\xi_2) + f^{(4)}(\xi_3)}{2880}\left(\frac{h_i}{2}\right)^5 = -\frac{2f^{(4)}(\xi_4)}{2880}\left(\frac{h_i}{2}\right)^5,$$

donde hemos utilizado el Teorema del Valor Intermedio en la última igualdad, para $\xi_1, \xi_4 \in (x_i, x_{i+1})$, $\xi_2 \in (x_i, x_i + h_i/2)$, $\xi_3 \in (x_i + h_i/2, x_{i+1})$. Restando las dos igualdades anteriores obtenemos que

$$\widetilde{S}_i - S_i = \frac{h_i^5}{2880}\left(\frac{f^{(4)}(\xi_4)}{16} - f^{(4)}(\xi_1)\right).$$

Si asumimos que $f^{(4)}$ varía lentamente en $[x_i, x_{i+1}]$, podemos aproximar

$$\widetilde{S}_i - S_i \approx -\frac{h_i^5}{2880} \cdot \frac{15 f^{(4)}(\xi_1)}{16}.$$

De esta forma, tenemos que

$$I_i - \widetilde{S}_i \approx \frac{\widetilde{S}_i - S_i}{15}.$$

Dada una tolerancia $\varepsilon > 0$, si se cumple que

$$\frac{1}{15}|\widetilde{S}_i - S_i| < \frac{h_i}{b-a} \cdot \varepsilon, \tag{4.4}$$

entonces la aproximación

$$\widetilde{I}_{\mathrm{SA}} := \sum_{i=0}^{n-1} \widetilde{S}_i$$

satisface que

$$|I_f - \widetilde{I}_{SA}| \leqslant \sum_{i=0}^{n-1} |I_i - \widetilde{S}_i| < \sum_{i=0}^{n-1} \frac{h_i}{b-a} \cdot \varepsilon = \varepsilon.$$

Note que en este análisis iniciamos con una partición (no uniforme) del intervalo $[a, b]$, la cual debe satisfacer (4.4) y en donde $f^{(4)}$ varía lentamente en cada subintervalo de la partición. Es claro que si iniciamos con una partición fija, dichas condiciones no serán satisfechas. Por lo tanto, deberá ser *adaptada* mediante la inclusión de nuevos nodos, refinando la partición inicial. Esto motiva el *método adaptivo de Simpson*:

Data: Funcion continua f, intervalo de integración $[a, b]$, tolerancia ε
Result: Aproximación $\widetilde{I}_{\mathrm{SA}}$ de la integral de f sobre $[a, b]$
Calcule los valores de S_i y $\widetilde{S}_i$ para el intervalo $[a, (a+b)/2]$;
if $|\widetilde{S}_i - S_i| < \dfrac{15h_i}{b-a} \cdot \varepsilon$ **then**
  Tome $\widetilde{S}_i$ como la aproximación de $\int_a^{(a+b)/2} f(x)\ dx$;
else
  Calcule de manera separada $\int_a^{(a+b)/4} f(x)\ dx$ y $\int_{(a+b)/4}^{(a+b)/2} f(x)\ dx$;
end
Repita para calcular la integral sobre $[(a+b)/2, b]$;

Algoritmo 4: Método adaptivo de Simpson

El algoritmo anterior se llama *adaptivo* porque apriori no sabemos el número de intervalos necesarios (ni su tamaño) para calcular I_f de manera precisa. El algoritmo se encarga de decidir si debe agregar o no un nuevo subintervalo utilizando la condición dada en (4.4). A continuación presentamos un ejemplo para esclarecer cómo funciona el método.

Ejemplo 4.12. Deseamos utilizar el Algoritmo 4 para aproximar

$$I = \int_0^1 \sqrt{x}\ dx$$

con una tolerancia de $\varepsilon = 0.0005$. Conociendo el gráfico de f, esperaríamos tener más puntos cerca de $x = 0$, pues para valores cercanos a $x = 1$ la función crece lentamente.

Empezamos aproximando $\int_{1/2}^1 \sqrt{x}\ dx$. En este intervalo tenemos que

$$S = 0.43093403,$$
$$\widetilde{S} = 0.43096219.$$

Como estos valores satisfacen (4.4), tomamos la aproximación

$$\int_{1/2}^1 \sqrt{x}\ dx \approx 0.43096219.$$

Falta aproximar $\int_0^{1/2} \sqrt{x}\ dx$. En este intervalo,

$$S \approx 0.22559223,$$
$$\widetilde{S} \approx 0.23211709.$$

Al no satisfacerse (4.4), debemos dividir el intervalo en dos. De manera similar, al considerar el subintervalo $[1/4, 1/2]$ se puede verificar (4.4), por lo que tomamos la aproximación

$$\int_{1/4}^{1/2} \sqrt{x}\, dx \approx 0.15236814.$$

Falta aproximar ahora $\int_0^{1/4} \sqrt{x}\, dx$. Esta nuevamente se debe dividir en dos. De esta manera se termina con la partición $\{0, 1/8, 1/4, 1/2, 1\}$ para obtener $I \approx 0.6662$. □

4.5. Polinomios ortogonales

En esta sección se introduce el concepto de polinomios ortogonales, de utilidad para poder establecer cuadraturas de mayor orden.

Definición 4.13. Sea $w : (a, b) \to \mathbb{R}^+$ una función integrable, la cual llamaremos *función peso*. Dadas dos funciones f, g definimos

$$(f, g)_w := \int_a^b f(x)g(x)w(x)\, dx.$$

Es fácil ver que $(f, g)_w$ es un producto interno definido en el espacio de funciones integrables con respecto al peso w, por lo que induce la norma natural

$$\|f\| = (f, f)_w^{1/2}.$$

Definición 4.14. Sea $\{\phi_j\}_{j=0}^{\infty}$ un conjunto de polinomios, donde cada ϕ_j tiene exactamente grado j. Dicho conjunto se llama un *sistema de polinomios ortogonales* en el intervalo $[a, b]$ si $(\phi_j, \phi_k)_w = 0$ si y solo si $j \neq k$.

Ejercicio 4.15. Demuestre que si $\{\phi_j\}_{j=0}^{\infty}$ es un conjunto de polinomios ortogonales, entonces son linealmente independientes. De esta forma, $\{\phi_j\}_{j=0}^{n}$ genera el espacio de polinomios de grado n.

Ejemplo 4.16. Del capítulo anterior, se tiene que los polinomios de Chebyshev son un sistema de polinomios ortogonales en el intervalo $[-1, 1]$ con respecto al peso $w(x) = (1 - x^2)^{-1/2}$; ver Ejercicio 3.21. □

Ejercicio 4.17. Considere un conjunto de polinomios ortogonales $\{\phi_j\}_{j=0}^{\infty}$ en el intervalo $[a, b]$, correspondientes a una función peso $w(x) > 0$. Demuestre que existen sucesiones $\{a_n\}_{n=1}^{\infty}$, $\{b_n\}_{n=1}^{\infty}$, $\{c_n\}_{n=1}^{\infty}$ tales que los polinomios ortogonales satisfacen la relación de recurrencia

$$\phi_{n+1}(x) = (a_n x + b_n)\phi_n(x) + c_n\phi_{n-1}(x) \ \forall \ n \geqslant 1.$$

Sugerencia: Por algoritmo de la división, para cada $n \geqslant 1$, existe a_n tal que

$$\phi_{n+1}(x) - a_n x\phi_n(x) \in \mathbb{P}_n.$$

Concluya utilizando condiciones de ortogonalidad.

Ejemplo 4.18. Los polinomios ortogonales en $[-1, 1]$ para $w(x) = 1$ se llaman *Polinomios de Legendre*. Dichos polinomios se pueden definir mediante la fórmula

$$P_n(x) = \frac{1}{2^n n!}\frac{d^n}{dx^n}(x^2 - 1)^n.$$

En esta definición, los polinomios se normalizan de forma tal que $P_n(1) = 1$. □

Ejercicio 4.19. Considere los polinomios de Legendre definidos anteriormente.

(a) Demuestre que es un sistema de polinomios ortogonales.

(b) Escriba explícitamente P_0, P_1, P_2 y P_3.

(c) Demuestre que satisfacen la relación de recurrencia

$$(n + 1)P_{n+1}(x) = (2n + 1)xP_n(x) - nP_{n-1}(x).$$

Comentario 4.20. En general, dado cualquier peso w, es posible obtener un sistema de polinomios ortogonales con respecto a w. La manera más sencilla es utilizar el proceso de ortogonalización de Gram-Schmidt a la base de polinomios $\{1, x, x^2, \ldots, x^n, \ldots\}$. Inductivamente, iniciamos con $\phi_0(x) = 1$.

Si ya tenemos un conjunto de polinomios ortogonales $\{\phi_0, \ldots, \phi_n\}$ $(n \geqslant 0)$, donde cada ϕ_j es un polinomio de grado j, podemos definir

$$\phi_{n+1} := x^{n+1} - \sum_{j=0}^{n} a_j \phi_j,$$

donde

$$a_j := \frac{(x^{n+1}, \phi_j)_w}{(\phi_j, \phi_j)_w}.$$

Es fácil ver que $(\phi_{n+1}, \phi_j) = 0$ para $j \leqslant n$, por lo que $\{\phi_0, \ldots, \phi_n, \phi_{n+1}\}$ es un conjunto de polinomios ortogonales. □

Ejemplo 4.21. Suponga que deseamos construir los polinomios ortogonales en $[0, 1]$ para $w = 1$. Tome primero $\phi_0(x) = 1$. Intuitivamente, deseamos encontrar un polinomio lineal ϕ_1 tal que $(\phi_1, \phi_0)_w = 0$. Si tomamos ϕ_1 mónico, podemos escribir $\phi_1(x) = x + a$. De la condición de ortogonalidad entonces necesitamos que

$$\int_0^1 (x + a)\, dx = 0,$$

de donde $a = -1/2$. Luego, $\phi_1(x) = x - 1/2$ (también podemos multiplicarlo por cualquier constante). Directamente, pudimos haber calculado

$$a_0 = \frac{(x, 1)_w}{(1, 1)_w} = -\frac{1}{2}.$$

De manera similar, $\phi_2(x) = x^2 - d_1\phi_1(x) - d_0\phi_0(x)$, donde

$$d_1 = \frac{(x^2, \phi_1)_w}{(\phi_1, \phi_1)_w} = 1,$$
$$d_0 = \frac{(x^2, \phi_0)_w}{(\phi_0, \phi_0)_w} = -\frac{1}{6}.$$

Así, $\phi_2(x) = x^2 - x + 1/6$. De esta manera podemos continuar para obtener polinomios ortogonales de grado mayor. Note que en este caso estamos normalizando los polinomios al considerar que son mónicos. Si los normalizamos de forma tal que la imagen de 1 es 1, entonces obtendríamos $\phi_1(x) = 2x - 1$, $\phi_2(x) = 6x^2 - 6x + 1$. □

Ejemplo 4.22. Los primero polinomios de Legendre vienen dados por

$$P_0(x) = 1,\ P_1(x) = x,\ P_2(x) = \frac{3}{2}x^2 - \frac{1}{2},$$

los cuales son ortogonales en $[-1,1]$ para $w(x)=1$. Para $j \neq k$ tenemos que

$$\int_{-1}^{1} P_j(x)P_k(x)\ dx = 0.$$

Haciendo el cambio de variable $t = \frac{1}{2}x + \frac{1}{2}$, obtenemos que

$$\int_{0}^{1} P_j(2t-1)P_k(2t-1)\ dt = 0.$$

Por lo tanto, podemos obtener directamente los polinomios ortogonales en el intervalo $[0,1]$ conociendo los polinomios de Legendre. En este caso,

$$\phi_1(x) = P_1(2x-1) = 2x-1, \text{ y}$$

$$\phi_2(x) = P_2(2x-1) = \frac{3}{2}(2x-1)^2 - \frac{1}{2} = 6x^2 - 6x + 1.$$

Compare estos resultados con el ejemplo anterior. □

Concluimos con un resultado de utilidad para la defición de la cuadratura de Gauss que se estudiará en la siguiente sección:

Teorema 4.23. *Considere un sistema de polinomios ortogonales $\{\phi_j\}_{j=0}^{\infty}$ en el intervalo $[a,b]$ con respecto a un peso dado $w(x)$. Entonces ϕ_j $(j \geqslant 1)$ tiene exactamente j ceros reales y distintos, los cuales se encuentran en el intervalo (a,b).*

Demostración. Por ortogonalidad tenemos que

$$\int_{a}^{b} \phi_j(x)w(x)\ dx = 0.$$

Al ser w positivo, debe ocurrir que ϕ_j cambia al menos una vez de signo. Defina $\xi_1, \ldots, \xi_k$ los $k \geqslant 1$ puntos en donde ϕ_j cambia de signo en (a,b). Demostraremos que $j = k$, lo que implica que ϕ_j tiene exactamente j cambios de signo en (a,b), los cuales corresponden a sus j ceros distintos. Defina $\pi_k(x) := (x-\xi_1)\cdots(x-\xi_k)$. Tenemos entonces que el polinomio $\phi_j\pi_k$ no cambia de signo en (a,b), por lo que

$$\int_{a}^{b} w(x)\phi_j(x)\pi_k(x)\ dx \neq 0.$$

Al ser ϕ_j un polinomio de un sistema de polinomios ortogonales, debe ocurrir que el grado de π_k sea mayor o igual que j, pues en caso contrario la integral anterior sería cero (por ortogonalidad). Finalmente, si $k > j$ se tendría que ϕ_j cambia de signo un número mayor de veces que su grado, lo cual es una contradicción. Por lo tanto, $k = j$. □

4.6. Cuadraturas de Gauss

En esta sección seguimos el desarrollo de [27, Capítulo 10]. Sean $f \in C^1([a,b])$ y w positiva e integrable en (a,b). Deseamos obtener una aproximación para

$$I = \int_a^b f(x)w(x)\ dx.$$

En lugar de aproximar f mediante su polinomio de Legendre tal como se estudió en el Apartado 4.2, podríamos utilizar su polinomio de interpolación de Hermite. Dicho polinomio viene dado por

$$h_{2n+1}(x) = \sum_{k=0}^{n} f(x_k)H_k(x) + f'(x_k)K_k(x) \in \mathbb{P}_{2n+1},$$

donde $\{x_k\}_{k=0}^n$ son nodos que definiremos posteriormente. Defina

$$\mathcal{G}_n(f) := \int_a^b h_{2n+1}(x)w(x)\ dx.$$

Si definimos

$$W_k := \int_a^b H_k(x)w(x)\ dx,$$
$$V_k := \int_a^b K_k(x)w(x)\ dx,$$

podemos escribir

$$\mathcal{G}_n(f) = \sum_{k=0}^{n} W_k f(x_k) + V_k f'(x_k).$$

Sería muy conveniente si en la expresión anterior se cumpliera que $V_k = 0$ para todo k, con el fin de evitar evaluar la derivada de la función. Note que

$$V_k = \int_a^b w(x)L_k(x)\left[L_k(x)\left(x - x_k\right)\right]\ dx.$$

El término entre paréntesis cuadrados es un polinomio de grado $n+1$ cuyos ceros son $x_0, \ldots, x_n$. Por lo tanto, si escogemos los nodos $\{x_i\}_{i=0}^n$ como los

ceros del polinomio de grado $n+1$ de un sistema de polinomios ortogonales con respecto a $w(x)$ en $[a,b]$ tendremos que

$$V_k = 0,\ k = 0, \ldots, n,$$

pues L_k y $L_k(x)\,(x - x_k)$ serían ortogonales. Luego, al calcular las constantes W_k obtenemos que

$$\begin{aligned} W_k &= \int_a^b w(x) L_k^2(x) \cdot \big(1 - 2L_k'(x_k)(x - x_k)\big)\ dx \\ &= \int_a^b w(x) L_k^2(x)\ dx - 2L_k'(x_k) V_k \\ &= \int_a^b w(x) L_k^2(x)\ dx, \end{aligned}$$

de donde se deduce que los pesos W_k son positivos. Además, como el polinomio de Hermite interpola de manera exacta cualquier polinomio de grado a lo sumo $2n+1$, tenemos que $\mathcal{G}_n(f)$ da el valor exacto de la integral I si $f \in \mathbb{P}_{2n+1}$. Por lo tanto, como $L_k(x)$ es un polinomio de grado n, tenemos la igualdad

$$\int_a^b L_k(x) w(x)\ dx = \sum_{k=0}^{n} W_k L_k(x_k) = W_k.$$

De esta manera podemos simplificar el cálculo de W_k, lo que nos lleva a la siguiente definición:

Definición 4.24. Considere f una función continua en (a,b) y w una función positiva, continua e integrable en (a,b). La *cuadratura de Gauss* de f de orden n en el intervalo $[a,b]$ se define como la suma

$$\mathcal{G}_n(f) := \sum_{k=0}^{n} W_k f(x_k),$$

donde los nodos $\{x_k\}_{k=0}^n$ son los $n+1$ ceros distintos de un polinomio de grado $n+1$ que pertenece a un sistema de polinomios ortogonal con respecto a $w(x)$ en $[a,b]$, y los pesos W_k vienen dados por

$$W_k = \int_a^b L_k(x) w(x)\ dx > 0.$$

Comentario 4.25. Al inicio del capítulo cuando dedujimos las fórmulas del trapecio y la de Simpson (casos $n = 1$ y $n = 2$ de las fórmulas de Newton-Cotes), fijamos los nodos como puntos equidistantes en el intervalo $[a,b]$.

En el caso de la cuadratura de Gauss, dichos nodos se seleccionan como los ceros de un polinomio ortogonal. De esta forma, al tener $n+1$ nodos y $n+1$ pesos que determinar, podemos integrar de manera exacta polinomios de grado a lo sumo $2n+1$. □

Con respecto al error en la aproximación, tenemos el siguiente resultado:

Teorema 4.26. *Sea $f \in C^{2n+2}([a,b])$ y w una función positiva e integrable en (a,b). Entonces*

$$\left| \int_a^b f(x)w(x)\ dx - \mathcal{G}_n(f) \right| \leqslant \frac{M}{(2n+2)!} \int_a^b \pi_{n+1}^2(x)w(x)\ dx$$

donde $\pi_{n+1}(x) := (x-x_0)\dots(x-x_n) \in \mathbb{P}_{n+1}$ y $M = \max\limits_{x\in[a,b]} |f^{(2n+2)}(x)|$.

Demostración. Este resultado es consecuencia directa del Teorema 3.8. □

Además, en este caso podemos probar convergencia:

Teorema 4.27. *Sea f una función continua en $[a,b]$ y w una función positiva, continua e integrable en (a,b). Entonces,*

$$\lim_{n\to\infty} \mathcal{G}_n(f) = \int_a^b f(x)w(x)\ dx.$$

Demostración. Sea $\epsilon > 0$. Defina $W := \int_a^b w(x)\ dx > 0$. Por el Teorema de Aproximación de Weierstrass, al ser f continua, puede ser aproximada por un polinomio p_N de grado exactamente N tal que

$$|f(x) - p_N(x)| < \epsilon/2W \quad \forall\ x \in [a,b]. \tag{4.5}$$

Por desigualdad triangular tenemos que

$$\begin{aligned}\left| \int_a^b f(x)w(x)\ dx - \mathcal{G}_n(f) \right| \leqslant & \int_a^b |f(x) - p_N(x)|\, w(x)\ dx \\ & + \left| \int_a^b p_N(x)w(x)\ dx - \mathcal{G}_n(p_N) \right| \\ & + |\mathcal{G}_n(p_N) - \mathcal{G}_n(f)|\,.\end{aligned}$$

El primer término es fácilmente acotado por

$$\int_a^b |f(x) - p_N(x)|\, w(x)\ dx \leqslant \epsilon/2$$

al utilizar (4.5). De manera similar para el tercer sumando,

$$\begin{aligned}|\mathcal{G}_n(p_N) - \mathcal{G}_n(f)| &\leqslant \sum_{k=0}^{n} W_k |p_N(x_k) - f(x_k)| \\ &< \frac{\epsilon}{2W} \sum_{k=0}^{n} W_k = \frac{\epsilon}{2W} \int_a^b w(x)\ dx \\ &= \epsilon/2,\end{aligned}$$

donde hemos usado el hecho que la cuadratura integra exactamente la función constante $f(x) = 1$. Finalmente, defina $N_0 := \lfloor N/2 \rfloor$ (parte entera). De esta forma, si $n > N_0$ tenemos que

$$\int_a^b p_N(x) w(x)\ dx - \mathcal{G}_n(p_N) = 0,$$

al ser la fórmula de cuadratura exacta para polinomios de grado $2n + 1$ y $N \leqslant 2N_0 + 1 < 2n + 1$. De esta forma, dado ϵ, existe N_0 tal que

$$\left| \int_a^b f(x) w(x)\ dx - \mathcal{G}_n(f) \right| < \epsilon \quad \text{si} \quad n > N_0,$$

de donde concluimos el resultado. □

Comentario 4.28. En este caso podemos hablar también de fórmulas compuestas. Asuma por simplicidad que $w \equiv 1$. Si dividimos el intervalo $[a, b]$ en m subintervalos de igual tamaño, podemos escribir

$$\int_a^b f(x)\ dx = \sum_{j=1}^{m} \int_{x_{j-1}}^{x_j} f(x)\ dx,$$

donde

$$x_j := a + jh\ (j = 0, \ldots, m), \quad \text{con} \quad h = \frac{b-a}{m}.$$

Cambiando el intervalo de integración de $x \in [x_{j-1}, x_j]$ a $t \in [-1, 1]$ mediante el cambio de variable

$$x = \frac{1}{2}(x_{j-1} + x_j) + \frac{1}{2} ht,$$

podemos escribir

$$\int_a^b f(x)\ dx = \frac{h}{2} \sum_{j=1}^{m} \int_{-1}^{1} f\left(\frac{1}{2}(x_{j-1} + x_j) + \frac{1}{2} ht \right)\ dt,$$

y al utilizar la cuadratura de Gauss para el intervalo $[-1,1]$ tenemos que

$$\int_a^b f(x)\ dx \approx \frac{h}{2}\sum_{j=1}^{m}\sum_{k=0}^{n} W_k f\left(\frac{1}{2}(x_{j-1}+x_j)+\frac{1}{2}h\widetilde{x}_k\right)\ dt,$$

donde $\{\widetilde{x}_k\}_{k=0}^n$ son los ceros de un polinomio ortogonal de grado $n+1$ en el intervalo $[-1,1]$. Note que el cambio de variable es importante pues permite utilizar el conjunto de polinomios ortogonales en $[-1,1]$ en todos los casos, sin necesidad de calcular los polinomios ortogonales para cada subintervalo $[x_{j-1},x_j]$. □

4.7. Otras cuadraturas

En algunos casos es posible y deseable fijar algunos de los nodos de la fórmula de cuadratura de Gauss. Por ejemplo, si fijamos ambos extremos $x_0 = a$ y $x_n = b$ la fórmula se conoce como *cuadratura de Gauss-Lobatto*. De esta forma, queremos calcular una cuadratura de la forma

$$\int_a^b f(x)w(x)\ dx \approx W_0 f(a) + \sum_{k=1}^{n-1} W_k f(x_k) + W_n f(b). \tag{4.6}$$

Debe ser claro que al tener que definir $n-1$ nodos y $n+1$ pesos, la fórmula será exacta para polinomios de grado a lo sumo $2n-1$. Por tal razón, si p_{2n-1} es un polinomio de grado $2n-1$, por el algoritmo de la división de polinomios podemos escribir

$$p_{2n-1}(x) = (x-a)(b-x)q_{2n-3}(x) + \frac{p_{2n-1}(b)}{b-a}(x-a) + \frac{p_{2n-1}(a)}{b-a}(b-x), \tag{4.7}$$

donde $q_{2n-3} \in \mathbb{P}_{2n-3}$. Si consideramos el peso

$$\widetilde{w}(x) := (x-a)(b-x)w(x),$$

podemos integrar

$$\int_a^b w(x)(x-a)(b-x)q_{2n-3}(x)\ dx = \sum_{k=1}^{n-1} \widetilde{W}_k\ q_{2n-3}(\widetilde{x}_k),$$

donde $\{\widetilde{x}_k\}_{k=1}^{n-1}$ son los $n-1$ nodos de la cuadratura de Gauss aplicada al peso $\widetilde{w}$, y $\widetilde{W}_k$ son sus respectivos pesos. Despejando q_{2n-3} de (4.7) y sustituyendo

en la ecuación anterior obtenemos

$$\begin{aligned}\int_a^b p_{2n-1}(x)w(x)\ dx = \frac{p_{2n-1}(a)}{b-a}\int_a^b (b-x)w(x)\ dx - \frac{p_{2n-1}(a)}{b-a}\sum_{k=1}^{n-1}\frac{\widetilde{W}_k}{b-\widetilde{x}_k} \\ + \sum_{k=1}^{n-1}\frac{\widetilde{W}_k}{(\widetilde{x}_k-a)(b-\widetilde{x}_k)}p_{2n-1}(\widetilde{x}_k) \\ + \frac{p_{2n-1}(b)}{b-a}\int_a^b (x-a)w(x)\ dx - \frac{p_{2n-1}(b)}{b-a}\sum_{k=1}^{n-1}\frac{\widetilde{W}_k}{\widetilde{x}_k-a}.\end{aligned}$$

De esta forma hemos construido la fórmula dada en (4.6), donde $x_k = \widetilde{x}_k$ $(1 \leqslant k \leqslant n-1)$ son los nodos de cuadratura de Gauss para $\widetilde{w}$,

$$\begin{aligned}W_k &:= \frac{\widetilde{W}_k}{(\widetilde{x}_k-a)(b-\widetilde{x}_k)}\ (1 \leqslant k \leqslant n-1), \\ W_0 &:= \frac{1}{b-a}\left(\int_a^b (b-x)w(x)\ dx - \sum_{k=1}^{n-1}\frac{\widetilde{W}_k}{b-\widetilde{x}_k}\right), \\ W_n &:= \frac{1}{b-a}\left(\int_a^b (x-a)w(x)\ dx - \sum_{k=1}^{n-1}\frac{\widetilde{W}_k}{\widetilde{x}_k-a}\right).\end{aligned}$$

Para $1 \leqslant k \leqslant n-1$ es claro que $W_k > 0$ pues $\widetilde{W}_k > 0$. Para $k = 0$, si tomamos $P(x) = (b-x)\prod_{k=1}^{n-1}(x-x_k)^2 \in \mathbb{P}_{2n-1}$, la cuadratura (4.6) implica que

$$\int_a^b P(x)w(x)\ dx = W_0 P(a),$$

por lo que $W_0 > 0$. De igual manera se puede comprobar que $W_n > 0$.

Existe también la posibilidad de fijar solamente uno de los extremos. En este caso la fórmula es conocida como la *cuadratura de Radau*; ver Ejercicio 4.36.

4.8. Métodos de Monte Carlo para integración

A continuación describiremos un método utilizado en la solución de ecuaciones diferenciales estocásticas y cuantificación de incertidumbre, que permite calcular integrales definidas en varias variables. Dada una función f definida en una variable aleatoria X con función de densidad de probabilidad p, la esperanza de f viene dada por

$$\mu(f) := \int_{\mathbb{R}^n} f(X)p(X)\ dX.$$

Si identificamos un punto $\boldsymbol{x} = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$ con la variable aleatoria $X = (X_1, \ldots, X_n)^T$ tenemos que

$$\int_\Omega f(\boldsymbol{x})\ d\boldsymbol{x} = |\Omega| \int_{\mathbb{R}^n} |\Omega|^{-1} \chi_\Omega(X) f(X)\ dX = |\Omega| \mu(f),$$

donde

- $|\Omega|$ es la medida $n-$dimensional de Ω.
- $\chi_\Omega(X)$ es la función característica del conjunto Ω; esto es, $\chi_\Omega(X) = 1$ si $X \in \Omega$ y cero en caso contrario.
- $p(X) = |\Omega|^{-1} \chi_\Omega(X)$ la función de densidad de probabilidad asociada a la variable aleatoria X.

Tome N muestras mutuamente independientes $X_1, \ldots, X_N$. Por la ley fuerte de los grandes números, sabemos que

$$I_N(f) := \frac{1}{N} \sum_{i=1}^{N} f(X_i) \to \mu(f) \quad (N \to \infty).$$

De esta manera tiene sentido aproximar

$$\int_\Omega f(\boldsymbol{x})\ d\boldsymbol{x} \approx |\Omega| I_N(f). \tag{4.8}$$

Es posible además probar que el error en la aproximación decrece en el orden $N^{-1/2}$; ver [21, Sección 4.6]. Note que esta razón de convergencia no depende de la dimensión del espacio, lo cual hace que el método sea atractivo en altas dimensiones; ver Ejercicio 4.42. En este caso también es posible considerar fórmulas compuestas, como las estudiadas anteriormente.

4.9. Ejercicios

Ejercicio 4.29. La fórmula de Newton-Cotes para $n = 3$ en el intervalo $[-1, 1]$ es

$$\int_{-1}^{1} f(x)\ dx \approx w_0 f(-1) + w_1 f(-1/3) + w_2 f(1/3) + w_3 f(1).$$

Determine los valores de los pesos w_i.

Ejercicio 4.30. Construya los polinomios ortogonales de grado $0, 1, 2$ en el intervalo $(0, 1)$ para $w(x) = -\ln(x)$. Determine la cuadratura de Gauss para $\int_0^1 f(x)w(x)\ dx$ en este caso (asumiendo condiciones sobre f para que la integral exista).

Ejercicio 4.31. Considere los polinomios de Legendre P_n. Demuestre que los polinomios

$$\left\{ P_n\left(\frac{2x-a-b}{b-a}\right)\right\}$$

son un sistema de polinomios ortogonales en el intervalo (a, b) con peso $w(x) = 1$.

Ejercicio 4.32. Suponga que $f \in C^2([0, 1])$. Demuestre que existe $\xi \in (0, 1)$ tal que

$$\int_0^1 xf(x)\ dx = \frac{1}{2}f\left(\frac{2}{3}\right) + \frac{1}{72}f''(\xi).$$

Ejercicio 4.33. Demuestre que los pesos W_k de la cuadratura de Gauss para el caso de $w(x) = (1-x^2)^{-1/2}$ en el intervalo $(-1, 1)$ vienen dados por

$$W_k = \frac{\pi}{n+1}.$$

Sugerencia: la fórmula

$$\sum_{j=0}^{n} \cos(2j+1)\theta = \frac{\sin(2n+2)\theta}{2\sin\theta}\ (\theta \neq k\pi, k \in \mathbb{Z})$$

puede ser de utilidad.

Ejercicio 4.34. Verifique que para $n = 1$ la cuadratura de Gauss en el intervalo $[-1, 1]$ para $w(x) = 1$ corresponde a la fórmula

$$\int_{-1}^{1} f(x)\ dx \approx \frac{1}{2}f\left(\frac{1}{2} - \sqrt{\frac{1}{12}}\right) + \frac{1}{2}f\left(\frac{1}{2} + \sqrt{\frac{1}{12}}\right).$$

Ejercicio 4.35. Construya la fórmula de Gauss-Lobatto para $n = 2$:

$$\int_{-1}^{1} f(x)\ dx \approx W_0 f(-1) + W_1 f(x_1) + W_2 f(1).$$

Sugerencia: Escriba un sistema de cuatro ecuaciones para obtener W_0, W_1, W_2 y x_1.

Ejercicio 4.36. (Cuadratura de Radau) El caso de fijar únicamente un extremo $x_0 = a$ o $x_n = b$ es llamada *Cuadratura de Radau*. Construya la fórmula de cuadratura

$$\int_a^b f(x)w(x)\ dx \approx W_0 f(a) + \sum_{k=1}^{n} W_k f(x_k),$$

donde hemos fijado $x_0 = a$. Sugerencia: la fórmula debe ser exacta para polinomios de grado $2n$. Escriba

$$p_{2n}(x) = (x-a)q_{2n-1}(x) + r,$$

con $r \in \mathbb{R}$.

Ejercicio 4.37. (a) Construya polinomios ortogonales ϕ_0, ϕ_1, ϕ_2 en el intervalo $[0, \infty)$ con respecto al peso $w(x) = e^{-x}$. Sugerencia: puede usar el hecho que

$$\int_0^\infty x^n e^{-x}\ dx = n!$$

(b) Construya la cuadratura de Gauss con dos nodos ($n = 1$) para aproximar

$$\int_0^\infty e^{-x} f(x)\ dx;$$

esto es, calcule x_0, x_1 y w_0, w_1 tales que

$$\int_0^\infty e^{-x} f(x)\ dx \approx w_0 f(x_0) + w_1 f(x_1).$$

Indique de forma explícita los valores de x_0, x_1; los valores de w_0 y w_1 puede expresarlos en términos de integrales (dando de forma explícita el integrando, en función de x_0 y x_1).

Ejercicio 4.38. (Polinomios de Laguerre) [27, Ejercicio 10.6]

(a) Demuestre que para k entero, con $0 \leqslant k \leqslant j$, se tiene que

$$\frac{d^k}{dx^k}\left(x^j e^{-x}\right) = x^{j-k} q_k(x) e^{-x},$$

donde q_k es un polinomio de grado k.

(b) Defina

$$L_j(x) := e^x \frac{d^j}{dx^j}\left(x^j e^{-x}\right).$$

Demuestre que L_j tiene exactamente grado j y que $\{L_j\}_{j\geqslant 0}$ es un sistema de polinomios ortogonales para $w(x) = e^{-x}$ en el intervalo $(0, \infty)$.

(c) Calcule L_0, L_1, L_2.

(d) Demuestre que

$$\int_0^\infty e^{-x} x (L_j(x) - L_j'(x)) p_r(x)\ dx = 0$$

si p_r es un polinomio de grado $r < j$.

(e) Asuma que fijamos el primer nodo de cuadratura $x_0 = 0$. Considere la fórmula de Radau

$$\int_0^\infty e^{-x} p_{2n}(x)\ dx = W_0 p_{2n}(0) + \sum_{k=1}^{n} W_k p_{2n}(x_k).$$

Demuestre que los otros puntos de cuadratura x_k $(k = 1, \ldots, n)$ son los ceros del polinomio $L_n - L_n'$. Sugerencia: escriba $p_{2n}(x) = x q_{2n-1}(x) + r$, con $r \in \mathbb{R}$.

(f) Del inciso anterior, deduzca que

$$\int_0^\infty e^{-x} p_2(x) = \frac{1}{2} p_2(0) + \frac{1}{2} p_2(2),$$

la cual corresponde a la cuadratura de Radau para $n = 1$.

Ejercicio 4.39. [MATLAB] (Simpson adaptivo)

(a) Escriba un programa para el método adaptivo de la cuadratura Simpson, donde las entradas de la función son la función por integrar f, los extremos del intervalo de integración a, b y cierta tolerancia ϵ. Dicho método es recursivo e idealmente debe evaluar $f(x)$ una sola vez para cada valor x. Durante la ejecución, hay un subintervalo I_k activo cuya contribución para el valor de la integral debe ser calculado de manera precisa, de manera tal que su error es menor que $\epsilon|I_k|/|b - a|$.

El primer intervalo activo es $[a, b]$. Si se decide que la fórmula de cuadratura no es lo suficientemente precisa, el intervalo $[a, (a+b)/2]$ se

convierte en activo. Una vez que la aproximación para dicho intervalo es precisa, se calcula la contribución del nuevo intervalo activo $[(a+b)/2, b]$.

(b) Utilizando la función del punto anterior, aproxime

$$I_1 = \int_0^1 x^{1/2}\ dx,$$
$$I_2 = \int_0^1 (1-x^2)^{3/2}\ dx,$$
$$I_3 = \int_0^1 \frac{\sin(x)}{x^{3/2}}\ dx,$$

utilizando como tolerancia el vector `10.^(-linspace(2,8))`. Grafique el error relativo en cada caso, en función de ϵ. (Nota: para I_3 debe garantizar que su resultado satisface el valor de la tolerancia, tratando de manera particular el problema que pueda existir en $x = 0$. Además, utilice como valor teórico $I_3 = 1.9351549819852953$.)

(c) Utilice las funciones de `Matlab` dadas por `trapz` (regla del trapecio) y `quad` (que corresponde precisamente a la cuadratura adaptiva de Simpson) para aproximar las tres integrales. Compare sus resultados.

Ejercicio 4.40. [MATLAB] (Tomado de [8]) Dado $n \in \mathbb{N}$ y f continua en $[0, \infty)$, deseamos encontrar una fórmula de cuadratura para la integral impropia

$$I := \int_0^\infty f(x)\ \frac{x^\alpha}{(1+x)^\beta}\ dx = \sum_{k=1}^{n} a_k f(x_k) + r_n(f), \tag{4.9}$$

donde $r_n(f)$ representa el error entre el valor exacto de la integral y la fórmula de la cuadratura. Asumimos que $\alpha > -1$ y $2n < \beta - \alpha$ para garantizar que la integral converge.

(a) Indique el mayor $M \in \mathbb{N}$ tal que $r_n(p_M) = 0$, donde p_M es un polinomio de grado M. Esto es, ¿cuál es el mayor grado (en función de n) de los polinomios que podemos integrar de manera exacta? (Sugerencia: cuente la cantidad de nodos y pesos que ocupamos determinar).

(b) Realice el cambio de variable

$$t = \frac{1-x}{1+x}$$

en la integral I. Escriba las ecuaciones correspondientes que se deben cumplir para integrar de manera exacta $f(x) = x^\lambda$, $\lambda = 0, 1, \ldots, M$ (dichas ecuaciones deben escribirse en función de t).

(c) Considere el peso $w(t) := (1-t)^\alpha(1+t)^{\beta-\alpha-2n-1}$. El conjunto de polinomios ortogonales con respecto a w en $[-1,1]$ son llamados *polinomios de Jacobi* con parámetros $(\alpha, \beta-\alpha-2n-1)$. Determine la fórmula de cuadratura; esto es, defina los pesos a_k y los nodos x_k en términos de los nodos y pesos de la cuadratura asociada con los polinomios de Jacobi. Sugerencia: los polinomios $\{(1-t)^\lambda(1+t)^{2n-1-\lambda}\}_{\lambda=0}^{M}$ son una base de $\mathbb{P}_M$.

(d) Implemente un código que calcule la cuadratura dada en la ecuación (4.9). Como entradas debe incluir la función f, el número de nodos n y los parámetros α y β. Verifique la exactitud de la fórmula para $f(x) = x^5$ en el caso $n = 4$, $\alpha = 1$ y $\beta = 10$. Para calcular los nodos y pesos relacionados con la cuadratura de los polinomios ortogonales de Jacobi en el intervalo $[a, b]$, utilice la función

```
function [nodes, weigths] = gw(n, a, b)
% nodos y pesos para polinomio Jacobi de grado n
% de parametros (a,b) en el intervalo [-1,1]
   ab = a + b;
  ii = (2:n-1)';
  abi = 2*ii + ab;
  aa = [(b - a)/(2 + ab)
        (b^2 - a^2)./((abi - 2).*abi)
        (b^2 - a^2)./((2*n - 2+ab).*(2*n+ab))];
  bb = [2*sqrt( (1 + a)*(1 + b)/(ab + 3))/(ab + 2) ;
        2*sqrt(ii.*(ii + a).*(ii + b).*(ii + ab)./(abi.^2 ...
            - 1))./abi];
  % Jacobi matrix.
  TT = diag(bb,1) + diag(aa) + diag(bb,-1);
  % Eigenvalue decomp
  [V, x] = eig( TT );
  % Jacobi points.
  nodes = diag(x);
  % Quadrature weights.
  weigths = V(1,:).^2*2^(ab+1)*beta(a+1, b+1);
end
```

Ejercicio 4.41. Para cada n entero no negativo, considere los polinomios de Hermite definidos por

$$H_n(x) := (-1)^n e^{x^2} \frac{d^n}{dx^n}\left(e^{-x^2}\right), \ x \in \mathbb{R}.$$

(a) Verifique que H_n es un polinomio de grado exactamente n.

(b) Demuestre que los polinomios $\{H_n\}$ satisfacen la relación de recurrencia

$$\begin{aligned} H_0(x) &= 1, \\ H_1(x) &= 2x, \\ H_{n+1}(x) &= 2xH_n(x) - 2nU_{n-1}(x) \ \forall n \geqslant 1. \end{aligned}$$

(c) Demuestre que la familia de polinomios $\{H_n\}_{n=0}^{\infty}$ es una familia de polinomios ortogonales con respecto al peso $w(x) = e^{-x^2}$ en el intervalo $(-\infty, \infty)$. Sugerencia: utilice integración por partes.

(d) Obtenga la fórmula de cuadratura de Gauss de tres puntos para aproximar

$$I_f := \int_{-\infty}^{\infty} f(x)e^{-x^2} \, dx;$$

esto es, obtenga w_0, w_1, w_2 y x_0, x_1, x_2 tales que

$$I_f = \sum_{n=0}^{2} w_k f(x_k) + \mathcal{E}_3, \tag{4.10}$$

donde $\mathcal{E}_3$ es el error asociado a la aproximación. Escriba explícitamente una fórmula para $\mathcal{E}_3$ (puede expresar $\mathcal{E}_3$, w_0, w_1, w_2 en términos de integrales).

(e) Indique el grado máximo para el cual la aproximación (4.11) es exacta. Justifique su respuesta.

Ejercicio 4.42. [MATLAB] En este ejercicio deseamos aproximar el valor de la integral

$$I := \int_0^1 \int_0^1 \int_0^1 \sin(\pi x)\sin(\pi y)\sin(\pi z) \, dz \, dy \, dx = \frac{8}{\pi^3}$$

mediante el Método de Monte Carlo descrito en la Sección 4.8.

(a) Utilice el comando de `Matlab integral3(f,0,1,0,1,0,1)` para obtener una primera aproximación para I.

(b) Defina el vector `N = 10.^(1:8)`. Para cada entrada de N, utilice la aproximación (4.8) para calcular I. Grafique el error relativo en función de N. Repita este proceso cinco veces, mostrando en una misma

gráfica (`loglog`) el error relativo en cada uno de los cinco experimentos. (Sugerencia: utilice la función `rand(n,1)` para generar un vector fila de tamaño n con n números aleatorios en el intervalo $[0, 1]$ para cada uno de los cinco experimentos).

(c) En el mismo gráfico del punto anterior dibuje la función $N^{-1/2}$ mediante el comando `loglog(N,1./(sqrt(N)),'--')`. Interprete el comportamiento de la gráfica del error relativo con la función $N^{-1/2}$.

(d) Considere ahora

$$J := \int_{[0,1]^{10}} \sin(\pi x_1) \dots \sin(\pi x_{10})\ dx_{10} \dots dx_1 = \left(\frac{2}{\pi}\right)^{10}.$$

Utilice (4.8) para obtener una aproximación para cada entrada de N. Grafique el error relativo en el mismo gráfico de la parte (b) y (c). ¿Hubo algún cambio significativo al aumentar la dimensión del espacio de 3 a 10? Sugerencia: comente sobre el error observado, el tiempo de ejecución y la memoria utilizada para guardar los vectores de valores aleatorios.

Ejercicio 4.43. Considere la fórmula de Newton-Cotes para aproximar:

$$\int_0^1 f(x)\ dx \approx \sum_{k=0}^{n} W_k f(x_k),$$

donde $x_k = k/n$ $(k = 0, 1, \dots, n)$, $W_k = \int_0^1 L_k(x)\ dx$.

(a) Demuestre que la suma de los pesos es igual a $b - a$.

(b) Escriba una función

```
function pesos = newtonCotes(n)
```

que calcule los pesos de la cuadratura. Sugerencia: puede utilizar las funciones `polyval`, `polyfit`, `polyint`.

(c) Verifique que su código brinda los pesos correctos para la fórmula del trapecio ($n = 1$) y la fórmula de Simpson ($n = 2$).

(d) Calcule los pesos para $n = 8$. ¿Son las entradas positivas? ¿Los pesos suman 1?

(e) Grafique la norma infinito los pesos para $n \in \{1, 2, \ldots, 20\}$ en función de n. ¿Qué observa en la gráfica?

(f) En la quadratura de Gauss, los pesos son positivos. ¿Observa alguna ventaja numérica poder garantizar que sean positivos?

Ejercicio 4.44. Para cada n entero no negativo, considere los polinomios de Chebyshev dados por

$$T_n(x) = \cos(n \arc\cos x), \ x \in [-1, 1].$$

Denotamos por $U_n(x)$ el polinomio de Chebyshev de segundo orden, el cual se define mediante la fórmula

$$U_n(x) := \frac{1}{n+1} T'_{n+1}(x).$$

(a) Verifique que $U_n(x) = \dfrac{\operatorname{sen}((n+1)\theta)}{\operatorname{sen}\theta}$, donde $x = \cos\theta$.

(b) Demuestre que los polinomios $\{U_n\}$ satisfacen la relación de recurrencia

$$\begin{aligned} U_0(x) &= 1, \\ U_1(x) &= 2x, \\ U_{n+1}(x) &= 2xU_n(x) - U_{n-1}(x) \ \forall n \geqslant 1. \end{aligned}$$

(c) Demuestre que la familia de polinomios $\{U_n\}_{n=0}^{\infty}$ es una familia de polinomios ortogonales con respecto al peso $w(x) = \sqrt{1-x^2}$ en el intervalo $[-1, 1]$.

(d) Obtenga la fórmula de cuadratura de Gauss de tres puntos para aproximar

$$I_f := \int_{-1}^{1} f(x)\sqrt{1-x^2}\, dx;$$

esto es, obtenga w_0, w_1, w_2 y x_0, x_1, x_2 tales que

$$I_f = \sum_{n=0}^{2} w_k f(x_k) + \mathcal{E}_3, \tag{4.11}$$

donde $\mathcal{E}_3$ es el error asociado a la aproximación. Escriba explícitamente una fórmula para $\mathcal{E}_3$.

(e) Indique el grado máximo para el cual la aproximación (4.11) es exacta. Justifique su respuesta.

Ejercicio 4.45. Al utilizar métodos de resolución de Ecuaciones Diferenciales Parciales, tales como Métodos de Elementos Finitos o Métodos de Elementos Virtuales, es necesario calcular integrales sobre regiones poligonales en general. Para este ejercicio, considere el triángulo de referencia T con vértices $A(0,0)$, $B(1,0)$, $C(0,1)$. En este ejercicio buscamos una fórmula de cuadratura para la integral

$$I = \iint_T f(x,y)\ dy\ dx \tag{4.12}$$

que sea exacta para polinomios en dos variables de grado a lo sumo n.

(a) Primero suponga que $n = 1$. Un polinomio en dos variables de grado uno tiene la forma

$$p_1(x,y) = ax + by + c,$$

donde $a, b, c \in \mathbb{R}$. Obtenga los valores de los pesos w_j y los nodos (x_j, y_j) tales que la fórmula

$$\iint_T p_1(x,y)\ dy\ dx = \sum_{j=0}^{k} w_j p_1(x_j, y_j) \tag{4.13}$$

sea válida. ¿Cuál debe ser el valor de k? Sugerencia: escriba un sistema de ecuaciones para la base canónica de los polinomios en dos variables de grado uno.

(b) Calcule el error en la aproximación del inciso anterior; esto es, calcule una fórmula para

$$\mathcal{E}_1 = \iint_T f(x,y)\ dy\ dx - \sum_{j=0}^{k} w_j f(x_j, y_j).$$

(c) Repita el inciso (a) para $n = 2$; esto es, calcule nuevos pesos y nodos tales que la ecuación (4.13) sea válida para todo polinomio de grado dos, $p_2(x,y) = a + bx + cy + dx^2 + exy + fy^2$.

(d) Escriba un código que implemente las fórmulas de cuadraturas obtenidas en los incisos (a) y (c). Verifique el comportamiento para las funciones $f_1(x,y) = y - x + 1$ y $f_2(x,y) = x^2 - xy + x - y$. Compare sus resultados con el valor exacto.

(e) Verifique el comportamiento de las cuadraturas obtenidas para

$$f(x,y) = \operatorname{sen}(\pi x)\cos(\pi y).$$

Calcule el error relativo en cada caso.

(f) Considere ahora el triángulo $\widehat{T}$ de vértices (x_0,y_0), (x_1,y_1), (x_2,y_2). Mediante un cambio de variable, realice un cambio en la región de integración

$$\iint_{\widehat{T}} \widehat{f}(\hat{x},\hat{y})\ d\hat{y}\ d\hat{x} = \iint_{T} f(x,y)\ dy\ dx.$$

De esta manera, no es necesario calcular los nodos y pesos para cada nuevo triángulo $\widehat{T}$. Aproxime de esta manera la integral

$$\iint_{\widehat{T}} x^2 e^{-(x+y)}\ dy\ dx,$$

donde $\widehat{T}$ es el triángulo con vértices $(-1,1)$, $(1,3)$, $(0,5)$, con las cuadraturas de orden uno y dos.

Ejercicio 4.46. Considere un dominio poligonal $\Omega \subset \mathbb{R}^2$ de n lados, cuyos vértices vienen dados por los puntos $(x_1,y_1),\ldots,(x_n,y_n)$ (ordenados en sentido antihorario). Dado un polinomio en dos variables $p:\Omega \to R$, en este ejercicio se desea calcular el valor de

$$I_p = \int_E p(x,y)\ dxdy.$$

(a) Fije α,β enteros no negativos. Considere $p(x,y) = x^\alpha y^\beta$. Utilice el teorema de la divergencia para determinar una función $\hat{p}$ tal que $I_p = \int_{\partial E} \hat{p}(s)\ ds$. De esta manera, el problema se reduce a calcular integrales de línea sobre cada arista e_i del polígono, donde e_i une el punto (x_i,y_i) con el punto (x_{i+1},y_{i+1}); esto es,

$$I_p = \sum_i \int_{e_i} \hat{p}(s)\ ds.$$

(b) Para calcular cada integral sobre e_i, consideraremos la cuadratura de Gauss-Lobatto con $k+1$ nodos sobre cada arista. Escriba una fórmula para $\int_{e_i} \hat{p}(s)\ ds$ en términos de los nodos y pesos de la fórmula de cuadratura. ¿Cuál debe ser el menor valor de k para garantizar que la cuadratura dé el valor exacto para I_p?

(c) [MATLAB] Implemente una función `integraMonomio` que reciba como entrada los exponentes $[\alpha, \beta]$ y los vértices de Ω, cuya salida sea I_p. Sugerencia: la función

```
function [nodes,weights] = quadInfoGL(k)
% GL quadrature rule in [0,1]
j = 1:k-2;
v = sqrt(j.*(j+2)./((2*j+1).*(2*j+3)));
A = diag(v,-1)+diag(v,1);
[V,D] = eig(A);
x  = diag(D);                          % quadrature nodes
w  = V(1,:).^2*4/3;                    % quadrature weights
w1 = 1-sum(w./(1-x'))/2;               % weight at ...
    initial node
w2 = 1-sum(w./(1+x'))/2;               % weight at final ...
    node
nodes   = ([-1; x; 1]+1)/2;            % translate to [0,1]
weights = [w1, w./((1-x').*(1+x')), w2]'/2;
end
```

genera de forma eficiente los nodos y pesos de Gauss-Lobatto en el intervalo $[0, 1]$ para cualquier valor k.

(d) [MATLAB] Verifique el comportamiento de su código para calcular $\int_0^1 \int_0^1 x^3 y^2 dydx$.

(e) [MATLAB] Considere ahora $p(x, y) = \sum_{\alpha+\beta \leqslant n} c_{\alpha,\beta} x^\alpha y^\beta$ un polinomio en dos variables de grado a lo sumo n. Escriba una función `integraPoli` (que utilice la función `integraMonomio`) que calcule I_p de forma exacta, cuya entrada es el vector de coeficientes $c_{\alpha,\beta}$. Verifique su comportamiento para calcular

$$\int_E (1 + xy + x^2 y^2)\ dydx,$$

donde E es el polígono con vértices $(0, 0), (2, 0), (2, 2), (1, 4), (0, 4)$.

Ejercicio 4.47. Dado un intervalo $[a, b]$ y una partición de nodos equidistantes $\{x_i\}_{i=0}^n$ (dada por $x_i = a+hi$, con $h = (b-a)/n$), deseamos aproximar el valor de $I_f = \int_a^b f(x)\ dx$. Considere la fórmula compuesta del trapecio

$$I_{TC} = \frac{b-a}{2n} \sum_{i=1}^{n} \left(f(x_{i-1}) + f(x_i)\right).$$

En clase vimos una fórmula para el error cuando $f \in C^2([a, b])$, no obstante, la idea de este ejercicio es considerar condiciones más generales.

(a) Suponga primero que f es derivable y que f' es integrable (no necesariamente continua). Determine una cota superior para el error $|I_f - I_{TC}|$ en este caso. Sugerencia: utilice un argumento similar al estudiado en clase, pues en cada subintervalo se cumple que

$$\int_{x_{i-1}}^{x_i} f(x)\ dx - \frac{b-a}{2n}\left(f(x_{i-1}) + f(x_i)\right) = -\int_{x_{i-1}}^{x_i} (x - c_i) f'(x)\ dx,$$

con $c_i = (x_{i-1} + x_i)/2$. La cota debe ser de la forma C/n^α, para C y α apropiados.

(b) Suponga ahora que f es derivable dos veces y que f'' es integrable (no necesariamente continua). Determine una cota superior para el error $|I_f - I_{TC}|$ en este caso. Dicha cota debe ser función de $1/n$.

(c) [MATLAB] Implemente una función en Matlab que reciba como entradas el intervalo $[a, b]$, el número de intervalos n y la función f, y que calcule la fórmula de aproximación I_{TC}.

(d) [MATLAB] Verifique el comportamiento de su función para las funciones $f_1(x) = x^{1/2}$, $f_2(x) = x^{3/2}$. Indique la cota teórica en cada caso, y compare numéricamente que dicho comportamiento es el esperado, según los incisos (a) y (b). Sugerencia: grafique el error absoluto y curvas de la forma $y = c/n^\alpha$, para valores de c y α apropiados.

Ejercicio 4.48. Considere las fórmulas compuestas con nodos equidistantes para aproximar $\int_a^b f(x)\ dx$:

$$I_{TC} = T(n) = \frac{b-a}{2n} \sum_{i=1}^{n} \left(f(x_{i-1}) + f(x_i)\right)$$

$$I_S = S(n) = \frac{b-a}{6n} \sum_{i=1}^{n} \left[f(x_{i-1}) + 4f\left(\frac{x_{i-1} + x_i}{2}\right) + f(x_i) \right],$$

donde hacemos clara la dependencia en el número de subintervalos n.

(a) Demuestre que $T(2^{n+1}) = \dfrac{1}{2} T(2^n) + \dfrac{b-a}{2^{n+1}} \displaystyle\sum_{j=1}^{2^n} f(x_{2j-1})$.

(b) Demuestre que $S(2n) = \frac{4}{3} T(2n) - \frac{1}{3} T(n)$.

(c) [MATLAB] Implemente una función que reciba f y calcule la aproximación mediante ambas fórmulas, de forma recursiva. Para ello, calcule

$$T(1), T(2), T(4), \ldots, T(2^n), \ldots \text{ y } \ S(2), S(4), \ldots$$

Grafique el error relativo para la función $f(x) = 1/(1+25x^2)$ en función de n, para ambas sucesiones. Tome como valor exacto el valor que brinda la función `integral`. Utilice como condición de parada $|S(m) - S(2m)| < \epsilon$, donde ϵ es la precisión de la máquina. ¿Qué ventaja tiene utilizar la recursión en lugar de las fórmulas explícitas?

Ejercicio 4.49. (a) Construya la cuadratura de Gauss con un nodo ($n = 0$) para aproximar

$$\int_0^\infty e^{-x^2} f(x)\ dx;$$

esto es, calcule x_0 y w_0 tales que

$$\int_0^\infty e^{-x^2} f(x)\ dx = w_0 f(x_0) + \mathcal{E}_0.$$

Indique de forma explícita los valores de x_0 y w_0. Sugerencia: puede usar el hecho que

$$\int_0^\infty e^{-x^2}\ dx = \sqrt{\pi}/2, \quad \int_0^\infty x e^{-x^2}\ dx = 1/2, \quad \int_0^\infty x^2 e^{-x^2}\ dx = \sqrt{\pi}/4.$$

(b) Brinde una cota superior para el término del error $\mathcal{E}_0$, asumiendo que $f \in C^2([0, \infty])$. En su respuesta puede aparecer f o sus derivadas, pero ninguna integral.

Ejercicio 4.50. [`MATLAB`] En este ejercicio se implementará de forma numérica la cuadratura de Gauss para $w(x) = 1$ (polinomios ortogonales de Legendre) en el intervalo $[-1, 1]$. Recuerde que dichos polinomios satisfacen la recurrencia

$$\begin{aligned} P_0(x) &= 1, \\ P_1(x) &= x, \\ (n+1)P_{n+1}(x) &= (2n+1)xP_n(x) - nP_{n-1}(x). \end{aligned} \tag{4.14}$$

(a) Programe una función que reciba como entrada n y devuelva los $n+1$ ceros del polinomio de Legendre de grado $n+1$. Sugerencia: puede usar la función `roots` o cualquier algoritmo para resolver ecuaciones visto en clase. Considere generar los coeficientes de P_{n+1} a partir de P_n y P_{n-1}.

(b) Programe una función que reciba los ceros del polinomio ortogonal y calcule los pesos correspondientes de la cuadratura de Gauss. Sugerencia: puede usar las funciones `polyfit`, `polyval`, `polyint`; la función que implementamos en clase para Newton-Cotes puede ser de utilidad.

(c) Verifique que para $n = 0$ su programa retorna $x_0 = 0, w_0 = 2$, y que para $n = 1$ retorna $x_0 = -1/\sqrt{3}, x_1 = 1/\sqrt{3}, w_0 = w_1 = 1$.

(d) Construya una cuadratura que permita integrar $f(x) = x^8 + 2x^2 + x$ de manera exacta. Utilice el mínimo valor de n posible. Escriba los nodos y pesos que utilizó, y verifique que obtiene el resultado correcto.

(e) Implemente una función que calcule la fórmula compuesta de la cuadratura de Gauss. Para ello, considere como entrada el intervalo $[a, b]$, el número de subintervalos m, el valor n para la cuadratura de Gauss y la función f. Dicha función debe calcular una partición uniforme $\{x_j\}_{j=0}^m$ dada por

$$x_j := a + jh \ (j = 0, \ldots, m), \quad \text{con} \ \ h = \frac{b-a}{m},$$

y calcular la fórmula

$$\int_a^b f(x)\ dx \approx \frac{h}{2} \sum_{j=1}^{m} \sum_{k=0}^{n} W_k f\left(\frac{1}{2}(x_{j-1} + x_j) + \frac{1}{2}h\widetilde{x}_k\right)\ dt,$$

donde $\{\widetilde{x}_k\}_{k=0}^n$ son los ceros de un polinomio ortogonal de grado $n+1$ en el intervalo $[-1, 1]$ y W_k los pesos asociados (que calculó anteriormente).

(f) Considere $g(x) = \text{sen}(100\pi x)(1-x)^{1/2}\log(1-x)$ en el intervalo $[0, 1]$. Grafique, para $n \in \{0, 2, 5, 8\}$, el error absoluto de esta cuadratura en función de m (una curva para cada valor de n). Comente sus resultados. Utilice $I_g = 0.003940021793519$ como valor exacto.

Capítulo 5

Factorización de matrices

5.1. Introducción

En este capítulo estudiaremos algunas herramientas teóricas de importancia para la resolución numérica de sistemas de ecuaciones lineales. Primero, definiremos normas en el espacio de matrices. Hablaremos luego de la *Descomposición en Valores Singulares* (DVS) y la *Factorización QR* de matrices, útiles en diversos problemas de aplicación. Luego presentaremos el método de *Eliminación Gaussiana* y la *Factorización LU*. Concluiremos con un caso particular llamado *Factorización de Cholesky* y algunas observaciones para la resolución de sistemas de ecuaciones. Algunas aplicaciones de interés serán detalladas en el siguiente capítulo. Nuestro estudio se basa en [29], y como referencias adicionales, se recomienda leer [11, 12].

Los vectores serán representados con letras minúsculas en negrita (por ejemplo, $\boldsymbol{x}$) y sus entradas serán denotadas por $x_1, \ldots, x_m$. Las matrices serán denotadas por letras mayúsculas. Dada una matriz A, denotaremos por $\boldsymbol{a}_j$ a la j-ésima columna de A, y por a_{ij} a la entrada en la fila i y columna j. Además, el espacio generado por un conjunto de vectores se denotará por $\langle \boldsymbol{x}_1, \ldots, \boldsymbol{x}_n \rangle$. Empezamos con algunas definiciones importantes:

Definición 5.1. Considere una matriz con entradas reales $A \in \mathbb{R}^{m \times n}$.

1. Su *matriz traspuesta* A^T es la matriz que se obtiene al intercambiar sus filas por sus columnas; esto es, $a_{ij}^T := a_{ji}$.

2. Una matriz cuadrada $A \in \mathbb{R}^{m \times m}$ se llama *simétrica* si $A^T = A$.

3. Una matriz cuadrada $A \in \mathbb{R}^{m \times m}$ se llama *ortogonal* si $AA^T = A^T A = I_{m \times m}$, donde I es la matriz identidad. Note que si $\boldsymbol{a}_i$ es una columna

de A, entonces $\boldsymbol{a}_i^T \boldsymbol{a}_j = \delta_{ij}$; esto es, sus columnas forman un conjunto ortonormal de vectores. Además, la inversa de A es A^T.

Definición 5.2. Considere una matriz con entradas complejas $A \in \mathbb{C}^{m\times n}$.

1. Su *matriz conjugada traspuesta* o *adjunta* $A^* \in \mathbb{C}^{n\times m}$ se define como la matriz con entradas $a_{ij}^* := (a_{ji})^*$. Recuerde que el conjugado de un número complejo $z = a + bi$ viene dado por $z^* = a - bi$.

2. Una matriz cuadrada $A \in \mathbb{C}^{m\times m}$ se llama *Hermitiana* o *Hermítica* si $A^* = A$.

3. Una matriz cuadrada $A \in \mathbb{C}^{m\times m}$ se llama *unitaria* si $AA^* = A^*A = I_{m\times m}$, donde I es la matriz identidad.

Tenemos además la siguiente definición:

Definición 5.3. *Considere un espacio vectorial V sobre un cuerpo $\mathbb{K}$. Una norma es una función*

$$\|\cdot\| : V \to [0, \infty)$$

tal que, para todo $\boldsymbol{x}, \boldsymbol{y} \in V$ y $\alpha \in \mathbb{K}$, se tiene que

(a) $\|\boldsymbol{x}\| = 0 \iff \boldsymbol{x} = 0$,

(b) $\|\alpha\boldsymbol{x}\| = |\alpha|\ \|\boldsymbol{x}\|$,

(c) $\|\boldsymbol{x} + \boldsymbol{y}\| \leqslant \|\boldsymbol{x}\| + \|\boldsymbol{y}\|$.

Ejemplo 5.4. Considere $V = \mathbb{C}^n$ el espacio de vectores (columna) con n entradas complejas y fije $p \in [1, \infty)$. La $p-$norma de un vector $\boldsymbol{x} \in \mathbb{C}^n$ se define como

$$\|\boldsymbol{x}\|_p := \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}.$$

Para $p = \infty$ se define

$$\|\boldsymbol{x}\|_\infty = \max_{1\leqslant i\leqslant n} |x_i|.$$

Asumimos que se es familiar con la prueba que $\|\cdot\|_p$ es efectivamente una norma. □

Ejemplo 5.5. (Normas inducidas en matrices) Considere $A \in \mathbb{C}^{m\times n}$ una matriz de tamaño $m \times n$. Dicha matriz es un operador $A : \mathbb{C}^n \to \mathbb{C}^m$, donde a cada vector $\boldsymbol{x} \in \mathbb{C}^n$ se le asigna su imagen $\boldsymbol{y} = A\boldsymbol{x} \in \mathbb{C}^m$. Considere

dos normas $\|\cdot\|_{(m)}$, $\|\cdot\|_{(n)}$ definidas en los espacios vectoriales $\mathbb{C}^m$ y $\mathbb{C}^n$, respectivamente. La norma de A inducida por estas normas es definida como

$$\|A\|_{(m,n)} := \sup_{\boldsymbol{x}\in\mathbb{C}^n,\boldsymbol{x}\neq 0} \frac{\|A\boldsymbol{x}\|_{(m)}}{\|\boldsymbol{x}\|_{(n)}}.$$

Note que si tomamos $\boldsymbol{y} = \boldsymbol{x}/\|\boldsymbol{x}\|_{(n)}$, podemos escribir

$$\|A\|_{(m,n)} = \sup_{\|\boldsymbol{y}\|_{(n)}=1} \|A\boldsymbol{y}\|_{(m)}. \tag{5.1}$$

Utilizando el hecho que $\|\cdot\|_{(m)}$, $\|\cdot\|_{(n)}$ son normas, podemos deducir que (5.1) satisface también las propiedades de ser norma.

Ejercicio 5.6. Verifique que (5.1) es una norma definida en el espacio de matrices. Sugerencia: utilice el hecho que $\|\cdot\|_{(m)}$, $\|\cdot\|_{(n)}$ son normas en $\mathbb{C}^m$ y $\mathbb{C}^n$, respectivamente.

En particular, para una matriz cuadrada $A \in \mathbb{C}^{m\times m}$ y $p \in [1,\infty]$ escribiremos

$$\|A\|_p := \sup_{\|\boldsymbol{y}\|_p=1} \|A\boldsymbol{y}\|_p, \tag{5.2}$$

donde $\|\cdot\|_p$ es la $p-$norma definida en el Ejemplo 5.4. □

Lema 5.7. *Considere dos matrices $A_{m\times p}$, $B_{p\times n}$. Entonces, se cumple que*

$$\|AB\|_{(m,n)} \leqslant \|A\|_{(m,p)}\|B\|_{(p,n)}.$$

Demostración. De la definición, tenemos que

$$\|AB\boldsymbol{x}\|_{(m)} \leqslant \|A\|_{(m,p)}\|B\boldsymbol{x}\|_{(p)} \leqslant \|A\|_{(m,p)}\|B\|_{(p,n)}\|\boldsymbol{x}\|_{(n)},$$

de donde se concluye el resultado. □

Ejemplo 5.8. (Norma de una matriz diagonal) Sea $p \in [1,\infty)$. Considere una matriz diagonal $D \in \mathbb{C}^{m\times m}$

$$D = \begin{bmatrix} d_1 & & & \\ & d_2 & & \\ & & \ddots & \\ & & & d_m \end{bmatrix},$$

donde las entradas omitidas son iguales a cero. Sean $\boldsymbol{x} \in \mathbb{C}^n$ y k el índice (no necesariamente único) para el cual

$$\max_{1 \leqslant i \leqslant m} |d_i| = |d_k|.$$

Como

$$D\boldsymbol{x} = \begin{bmatrix} d_1 x_1 \\ d_2 x_2 \\ \vdots \\ d_m x_m \end{bmatrix},$$

se tiene que

$$\begin{aligned} \|D\|_p &= \sup_{\|\boldsymbol{x}\|_p = 1} \|D\boldsymbol{x}\|_p = \sup_{\|\boldsymbol{x}\|_p = 1} \left(\sum_{i=1}^{m} |d_i x_i|^p \right)^{1/p} \\ &\leqslant |d_k| \sup_{\|\boldsymbol{x}\|_p = 1} \left(\sum_{i=1}^{m} |x_i|^p \right)^{1/p} = |d_k| \sup_{\|\boldsymbol{x}\|_p = 1} \|\boldsymbol{x}\|_p \\ &= |d_k|. \end{aligned}$$

Si tomamos el vector canónico $\boldsymbol{e}_k$ (vector donde todas las entradas son cero, a excepción de la entrada k que toma el valor 1), tenemos que

$$\|D\boldsymbol{e}_k\|_p = |d_k|,$$

por lo que en la última desigualdad, la igual se alcanza para $\boldsymbol{x} = \boldsymbol{e}_k$ (es claro que $\|\boldsymbol{e}_k\|_p = 1$). De esta manera concluimos que

$$\|D\|_p = \max_{1 \leqslant i \leqslant m} |d_i|.$$

Similarmente se puede hacer el análisis para $p = \infty$. □

Ejemplo 5.9. (1−norma de una matriz) Dada $A \in \mathbb{C}^{m \times n}$, escriba

$$A = \left[\begin{array}{c|c|c} & & \\ \boldsymbol{a}_1 & \cdots & \boldsymbol{a}_n \\ & & \\ \end{array}\right]_{m \times n,}$$

donde $\{\boldsymbol{a}_j\}_{j=1}^n \in \mathbb{C}^m$ son sus n vectores columna. Dado $\boldsymbol{x} \in \mathbb{C}^n$ con $\|\boldsymbol{x}\|_1 = 1$, por desigualdad triangular se tiene que

$$\|A\boldsymbol{x}\|_1 = \left\| \sum_{j=1}^{n} x_j \boldsymbol{a}_j \right\|_1 \leqslant \sum_{j=1}^{n} |x_j| \|\boldsymbol{a}_j\|_1 \leqslant \max_{1 \leqslant j \leqslant n} \|\boldsymbol{a}_j\|_1 \sum_{j=1}^{n} |x_j| = \max_{1 \leqslant j \leqslant n} \|\boldsymbol{a}_j\|_1.$$

Si tomamos $\boldsymbol{e}_k$, donde k es el índice que maximiza $\max_{1\leqslant j\leqslant n}\|\boldsymbol{a}_j\|_1$, tenemos que

$$A\boldsymbol{e}_k = \max_{1\leqslant j\leqslant n}\|\boldsymbol{a}_j\|_1.$$

De esta forma, la cota para $\|A\boldsymbol{x}\|_1$ en la última desigualdad se alcanza, por lo que se concluye que

$$\|A\|_1 = \max_{1\leqslant j\leqslant n}\|\boldsymbol{a}_j\|_1.$$

Esto es, la $1-$norma de una matriz corresponde a la máxima suma de los valores absolutos de las entradas de cada columna. □

Ejercicio 5.10. ($\infty-$norma de una matriz) Si denotamos por $\boldsymbol{a}_i^*$ a las m filas de $A\in\mathbb{C}^{m\times n}$, demostrar que

$$\|A\|_\infty = \max_{1\leqslant i\leqslant m}\|\boldsymbol{a}_i^*\|_1.$$

Esto es, la $\infty-$norma de una matriz corresponde al máximo de la suma de los valores absolutos de cada fila.

Ejemplo 5.11. Considere $\boldsymbol{x}\in\mathbb{C}^m$ y $\boldsymbol{y}\in\mathbb{C}^n$. El producto externo

$$A = \boldsymbol{x}\boldsymbol{y}^*\in\mathbb{C}^{m\times n}$$

es la matriz de rango uno

$$A = \begin{bmatrix} x_1y_1 & x_1y_2 & \dots & x_1y_n \\ \vdots & \vdots & \dots & \vdots \\ x_my_1 & x_my_2 & \dots & x_my_n \end{bmatrix}_{m\times n.}$$

Note que todas las filas (columnas) son múltiplos de la primer fila (columna). Además, como $\boldsymbol{y}^*\boldsymbol{z}$ es un escalar, mediante la desigualdad de Cauchy-Schwarz se tiene que

$$\|A\boldsymbol{z}\|_2 = \|\boldsymbol{x}\boldsymbol{y}^*\boldsymbol{z}\|_2 = |\boldsymbol{y}^*\boldsymbol{z}|\|\boldsymbol{x}\|_2 \leqslant \|\boldsymbol{x}\|_2\|\boldsymbol{y}\|_2\|\boldsymbol{z}\|_2.$$

Además, la desigualdad anterior se convierte en igualdad para $\boldsymbol{z}=\boldsymbol{y}$. Así, concluimos que

$$\|A\|_2 = \|\boldsymbol{x}\|_2\|\boldsymbol{y}\|_2.$$

□

Ejemplo 5.12. *(Norma de Frobenius) En la definición de norma, podemos tomar V como el espacio vectorial de matrices $A \in \mathbb{C}^{m\times n}$. En este caso, podemos definir normas que no son inducidas por normas de los espacios $\mathbb{C}^m$ y $\mathbb{C}^n$. Por ejemplo, la norma de Frobenius*

$$\|\cdot\|_F : \mathbb{C}^{m\times n} \to [0,\infty)$$

se define como

$$\|A\|_F := \left(\sum_{i=1}^{m}\sum_{j=1}^{n} |a_{ij}|^2\right)^{1/2}.$$

Es fácil ver que dicha función es efectivamente una norma. De manera equivalente podemos escribir

$$\|A\|_F = (tr(AA^*))^{1/2} = (tr(A^*A))^{1/2},$$

donde $tr(B)$ denota la suma de las entradas de la diagonal de la matriz cuadrada B. Ahora, considere $C = A_{m\times p}B_{p\times n} \in \mathbb{C}^{m\times n}$. La entrada en la fija i y columna j es $c_{ij} = \boldsymbol{a}_i^*\boldsymbol{b}_j$, y por Cauchy-Schwarz, $|c_{ij}| \leqslant \|\boldsymbol{a}_i\|_2\|\boldsymbol{b}_j\|_2$. De esta manera,

$$\begin{aligned}\|C\|_F^2 = \sum_{i=1}^{m}\sum_{j=1}^{n} |c_{ij}|^2 &\leqslant \sum_{i=1}^{m}\sum_{j=1}^{n} \|\boldsymbol{a}_i\|_2^2\|\boldsymbol{b}_j\|_2^2 \\ &= \sum_{i=1}^{m} \|\boldsymbol{a}_i\|_2^2 \sum_{j=1}^{n} \|\boldsymbol{b}_j\|_2^2 = \|A\|_F^2\|B\|_F^2,\end{aligned}$$

por lo que

$$\|AB\|_F \leqslant \|A\|_F\|B\|_F;$$

compare este resultado con el Lema 5.7. □

Concluimos esta sección con el siguiente resultado:

Lema 5.13. *Considere $A \in \mathbb{C}^{m\times n}$ y $Q \in \mathbb{C}^{m\times m}$ unitaria. Entonces,*

$$\|QA\|_2 = \|A\|_2 \quad y \quad \|QA\|_F = \|A\|_F.$$

Demostración. Para la primer igualdad utilizamos el hecho que

$$\|Q\boldsymbol{x}\|_2^2 = \boldsymbol{x}^*Q^*Q\boldsymbol{x} = \boldsymbol{x}^*\boldsymbol{x} = \|\boldsymbol{x}\|_2^2$$

y la definición de la norma inducida $\|QA\|_2$. Para la segunda igualdad, note que

$$\|QA\|_F^2 = \mathrm{tr}(A^*Q^*QA) = \mathrm{tr}(A^*A) = \|A\|_F^2.$$

□

5.2. Descomposición en Valores Singulares

La descomposición en valores singulares tiene un gran número de aplicaciones, de las cuales estudiaremos algunas en esta sección. Los algoritmos para calcular numéricamente dicha factorización los estudiaremos posteriormente. Iniciamos con la definición de dicha factorización:

Definición 5.14. Se dice que $A \in \mathbb{C}^{m\times n}$ admite una *factorización en valores singulares* si existen $U \in \mathbb{C}^{m\times m}$ unitaria, $V \in \mathbb{C}^{n\times n}$ unitaria y $\Sigma \in \mathbb{R}^{m\times n}$ diagonal tales que

$$A = U\Sigma V^*,$$

donde las entradas de la diagonal de Σ satisfacen $\sigma_1 \geqslant \sigma_2 \geqslant \ldots \geqslant \sigma_p \geqslant 0$, con $p := \text{mín}\{m, n\}$. Los números $\{\sigma_k\}_{k=1}^p$ son llamados *valores singulares*, las columnas de U son llamadas *vectores singulares izquierdos* y las columnas de V son llamadas *vectores singulares derechos.*

Dividimos la existencia y unicidad de dicha descomposición en dos teoremas:

Teorema 5.15 (Existencia de la descomposición en valores singulares)**.** *Toda matriz $A \in \mathbb{C}^{m\times n}$ admite una descomposición en valores singulares. Además, los valores singulares $\{\sigma_k\}_{k=1}^p$ son únicos.*

Demostración. La prueba se demuestra por inducción en la dimensión de A. Defina el primer valor singular $\sigma_1 := \|A\|_2$. Por definición, existen $\boldsymbol{u}_1 \in \mathbb{C}^m$, $\boldsymbol{v}_1 \in \mathbb{C}^n$ con $\|\boldsymbol{u}\|_2 = \|\boldsymbol{v}\|_2 = 1$, tales que

$$A\boldsymbol{v}_1 = \sigma_1\boldsymbol{u}_1.$$

Extienda $\{\boldsymbol{u}_1\}$ a una base unitaria $\{\boldsymbol{u}_1 \ldots, \boldsymbol{u}_m\}$ de $\mathbb{C}^m$ y defina U_1 como la matriz con columnas $\{\boldsymbol{u}_1 \ldots, \boldsymbol{u}_m\}$. Similarmente, extienda $\{\boldsymbol{v}_1\}$ a una base unitaria de $\mathbb{C}^n$ y defina V_1 como la matriz con dicha base como columnas. De esta forma, podemos calcular el producto $S := U_1^* A V_1$. Como $A\boldsymbol{v}_1 = \sigma_1\boldsymbol{u}_1$, la primer columna de $U_1^* A V_1$ es un vector donde su primer entrada es σ_1 y el resto de entradas son nulas. Podemos escribir entonces

$$U_1^* A V_1 = S = \begin{bmatrix} \sigma_1 & \boldsymbol{w}^* \\ \underline{0} & B \end{bmatrix},$$

donde $\underline{0}$ es un vector columna de dimension $m-1$, $\boldsymbol{w}^*$ es un vector fila de dimensión $n-1$, y $B \in \mathbb{C}^{(m-1)\times(n-1)}$. Note que en la primer columna de S se tienen entradas de la forma $\boldsymbol{u}_j^T A\boldsymbol{v}_1 = \sigma_1\boldsymbol{u}_j^T\boldsymbol{u}_1$.

Además, se cumple que

$$\left\| \begin{bmatrix} \sigma_1 & \boldsymbol{w}^* \\ \underline{0} & B \end{bmatrix} \begin{bmatrix} \sigma_1 \\ \boldsymbol{w} \end{bmatrix} \right\|_2 \geqslant \sigma_1^2 + \boldsymbol{w}^*\boldsymbol{w} = \sqrt{\sigma_1^2 + \boldsymbol{w}^*\boldsymbol{w}} \left\| \begin{bmatrix} \sigma_1 \\ \boldsymbol{w} \end{bmatrix} \right\|_2,$$

por lo que $\|S\|_2 \geqslant \sqrt{\sigma_1^2 + \boldsymbol{w}^*\boldsymbol{w}}$. Pero al ser U_1 y V_1 unitarias, se tiene que $\|S\|_2 = \|A\|_2 = \sigma_1$, por lo que $\boldsymbol{w} = \underline{0}$. De esta forma,

$$U_1^* A V_1 = \begin{bmatrix} \sigma_1 & \underline{0} \\ \underline{0} & B \end{bmatrix},$$

donde usamos indistintamente la notación $\underline{0}$ para vectores fila y columna nulos (del tamaño correspondiente).

Si $m = 1$ o $n = 1$, el resultado se cumple pues $A = U_1 S V_1$. En caso contrario, por inducción, la submatriz $B \in \mathbb{C}^{(m-1)\times(n-1)}$ tiene una descomposición

$$B = U_2 \Sigma_2 V_2.$$

De esta forma podemos escribir

$$A = U_1 \begin{bmatrix} 1 & \underline{0} \\ \underline{0} & U_2 \end{bmatrix} \begin{bmatrix} \sigma_1 & \underline{0} \\ \underline{0} & \Sigma_2 \end{bmatrix} \begin{bmatrix} 1 & \underline{0} \\ \underline{0} & V_2 \end{bmatrix} V_1^*,$$

que es una factorización en valores singulares de A. Finalmente, note que $\|A\|_2 = \|S\|_2 \geqslant \|B\|_2$, por lo que $\sigma_1 \geqslant \sigma_2$. Inductivamente concluimos que $\sigma_1 \geqslant \sigma_2 \geqslant \ldots \geqslant \sigma_p \geqslant 0$, y por construcción, los valores singulares son únicos. □

Teorema 5.16 (Unicidad)**.** *Considere $A \in \mathbb{C}^{m\times m}$ donde todos sus valores singulares son diferentes. Entonces, los valores singulares izquierdos y derechos están determinados de manera única, a excepción de multiplicación por constantes complejas de magnitud 1.*

Demostración. Sabemos que existe una factorización en valores singulares $A = U\Sigma V^*$. Como $A^*A \in \mathbb{C}^{m\times m}$ es Hermitiana, se sigue que A^*A es diagonalizable y sus valores propios son reales. Tenemos además que

$$A^*A = V\Sigma^* U^* U \Sigma V^* = V\Sigma^2 V^*,$$

por lo que A^*A y Σ^2 son similares y tienen los mismos valores propios. Los valores propios de Σ^2 son $\{\sigma_1^2, \ldots, \sigma_m^2\}$. De esta forma, los valores singulares positivos corresponden a la raíz cuadrada de los valores propios de A^*A (o de los de AA^* pues son los mismos; ver Ejercicio 5.17). Note que cada columna

v_j de V es entonces un vector propio que genera el subespacio asociado al valor propio σ_j^2. Estos subespacios son de dimensión uno (al ser todos los valores propios diferentes por hipótesis), por lo que está determinado de manera única, a excepción de multiplicación por constantes de módulo unitario. Similarmente, $AA^* = U\Sigma^2 U^*$ y se razona de manera similar para concluir que los vectores singulares izquierdos son únicos, a excepción de multiplicación por constantes de módulo unitario. □

Ejercicio 5.17. Considere una matriz $A \in \mathbb{C}^{m\times n}$. Verifique que si $\lambda \neq 0$ es un valor propio de A^*A, entonces también es un valor propio de AA^*. ¿Qué pasa si $\lambda = 0$?

De esta última prueba, podemos generar un algoritmo para calcular la factorización en valores singulares de una matriz A, el cual ejemplificamos a continuación. No obstante, debe ser claro que numéricamente el método que se utiliza es diferente, e incluimos el ejemplo solo por fines didácticos.

Ejemplo 5.18. Considere la matriz

$$A = \begin{bmatrix} 3 & 1 & 1 \\ -1 & 3 & 1 \end{bmatrix}.$$

Mediante un cálculo directo se obtiene

$$AA^T = \begin{bmatrix} 11 & 1 \\ 1 & 11 \end{bmatrix},$$

cuyos valores singulares son $\lambda_1 = 10$, $\lambda_2 = 12$, con vectores propios asociados $u_1 = [1, -1]^T$, $u_2 = [1, 1]^T$, respectivamente. De esta manera, los valores singulares son

$$\sigma_1 = \sqrt{12}, \quad \sigma_2 = \sqrt{10},$$

y así

$$\Sigma = \begin{bmatrix} \sqrt{12} & 0 & 0 \\ 0 & \sqrt{10} & 0 \end{bmatrix}.$$

Como $AA^T = U\Sigma^2 U^T$, basta entonces tomar

$$U = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}.$$

De manera análoga, al calcular A^TA obtenemos

$$A^TA = \begin{bmatrix} 10 & 0 & 2 \\ 0 & 10 & 4 \\ 2 & 4 & 2 \end{bmatrix}.$$

Al calcular los valores y vectores propios, obtenemos $\widetilde{\lambda_1} = 12$, $\widetilde{\lambda_2} = 10$, $\widetilde{\lambda_3} = 0$, con vectores propios $v_1 = [1, 2, 1]^T$, $v_2 = [2, -1, 0]^T$, $v_3 = [1, 2, -5]^T$. De esta manera,

$$V = \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{5}} & \frac{1}{\sqrt{30}} \\ \frac{2}{\sqrt{6}} & -\frac{1}{\sqrt{5}} & \frac{2}{\sqrt{30}} \\ \frac{1}{\sqrt{6}} & 0 & -\frac{5}{\sqrt{30}} \end{bmatrix}.$$

De esta manera, podemos verificar que $A = U\Sigma V^T$. □

Como consecuencia directa de la factorización en valores singulares tenemos los siguientes resultados:

Lema 5.19. *Sea $A = U\Sigma V^*$ una factorización en valores singulares de A y defina r como el número de valores singulares positivos de A. Entonces, podemos expresar la matriz A como suma de r matrices de rango 1. Explícitamente,*

$$A = \sum_{j=1}^{r} \sigma_j \boldsymbol{u}_j \boldsymbol{v}_j^*, \tag{5.3}$$

donde $\{\boldsymbol{u}_j\}_{j=1}^r$ y $\{\boldsymbol{v}_j\}_{j=1}^r$ son los r vectores singulares izquierdos y derechos, respectivamente, asociados a los valores singulares $\{\sigma_j\}_{j=1}^r$.

Demostración. La matriz diagonal Σ puede ser separada como suma de r matrices Σ_j donde todos sus elementos son cero, a excepción de la entrada $\Sigma_{jj} = \sigma_j$, $1 \leqslant j \leqslant r$. Multiplicar $U\Sigma_j V^*$ equivale a seleccionar la columna j de U y la fila j de V, escalada con el factor σ_j. Con esto se concluye el resultado. □

Comentario 5.20. Note que en la suma (5.3) solo incluimos los valores singulares no nulos. Por lo tanto, podemos definir la *descomposición en valores singulares reducida* como

$$A = \widehat{U}\widehat{\Sigma}\widehat{V}^*,$$

con $\widehat{U} \in \mathbb{C}^{m\times r}$, $\widehat{\Sigma} \in \mathbb{C}^{r\times r}$ y $\widehat{V} \in \mathbb{C}^{n\times r}$, donde $\widehat{U}$ y $\widehat{V}$ incluyen las primeras r columnas de U y V, respectivamente. Además, $\widehat{\Sigma}$ es la matriz cuadrada y diagonal cuyas entradas son los r valores singulares positivos. □

Ejemplo 5.21. Para la matriz A del Ejemplo 5.18, podemos escribir la factorización reducida como

$$A = \begin{bmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix} \begin{bmatrix} \sqrt{12} & 0 \\ 0 & \sqrt{10} \end{bmatrix} \begin{bmatrix} \frac{1}{\sqrt{6}} & \frac{2}{\sqrt{5}} \\ \frac{2}{\sqrt{6}} & -\frac{1}{\sqrt{5}} \\ \frac{1}{\sqrt{6}} & 0 \end{bmatrix}^T,$$

la cual se obtiene de eliminar la última columna de Σ. □

Lema 5.22. *Sea $A = U\Sigma V^* \in \mathbb{C}^{m\times n}$. La imagen de A es generada por $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$ y el núcleo de A es generado por $\{\boldsymbol{v}_{r+1}, \ldots, \boldsymbol{v}_n\}$. De esta forma, el rango r de una matriz $A \in \mathbb{C}^{m\times n}$ es igual al número de valores singulares positivos.*

Demostración. Del Lema 5.19 se deduce de inmediato que la imagen es generada por $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$. Al ser vectores linealmente independientes, se concluye que el rango es r. Además, $\boldsymbol{v}_j^* \boldsymbol{v}_k = 0$ para $1 \leqslant j \leqslant r$ y $r+1 \leqslant k \leqslant n$. Como el núcleo debe tener dimensión $n - r$, se concluye que el núcleo es generado por $\{\boldsymbol{v}_{r+1}, \ldots, \boldsymbol{v}_n\}$. □

Lema 5.23. *Considere una matriz $A \in \mathbb{C}^{m\times n}$ con valores singulares positivos $\sigma_1, \ldots, \sigma_r$. Entonces,*

$$\|A\|_2 = \sigma_1 \quad y \quad \|A\|_F = \sqrt{\sigma_1^2 + \ldots \sigma_r^2}.$$

Demostración. La primer igualdad se deduce de la prueba del Teorema 5.15. La segunda es consecuencia inmediata del Lema 5.13 y la definición de la norma $\|\cdot\|_F$. □

Lema 5.24. *El conjunto de valores singulares positivos de $A \in \mathbb{C}^{m\times n}$ es igual al conjunto de las raíces cuadradas de los valores propios no nulos de A^*A o AA^*.*

Demostración. Este resultado se concluye de la prueba del Lema 5.16. □

Lema 5.25. *Si $A = A^*$, el conjunto de valores singulares de A es igual al conjunto de valores absolutos de los valores propios de A.*

Demostración. Al ser A una matriz Hermitiana, podemos escribir

$$A = Q\Lambda Q^* = Q|\Lambda|\text{sign}(\Lambda)Q^*,$$

donde $|\Lambda|$ y $\text{sign}(\Lambda)$ son matrices diagonales con entradas $|\lambda_j|$ y $\text{sign}(\lambda_j)$, respectivamente. Si ordenamos los valores propios en orden creciente, la

igualdad anterior es una descomposición en valores singulares con lo que concluimos la prueba. □

Lema 5.26. *Para* $A \in \mathbb{C}^{m\times m}$, $|\det(A)| = \prod_{i=1}^{m} \sigma_i$.

Demostración. Sigue del hecho que $|\det(U)| = 1 = |\det(V)|$. □

Comentario 5.27. Sea $A \in \mathbb{C}^{m\times n}$ y considere la esfera unitaria

$$\mathcal{B} = \{\boldsymbol{x} \in \mathbb{C}^n : \|\boldsymbol{x}\|_2 = 1\}.$$

Por el Lema 5.13, al calcular la imagen de $\mathcal{B}$, tenemos que V^* preserva la esfera. Luego, la matriz diagonal Σ genera una *hiperelipse* al cambiar de tamaño los vectores de la base de las columnas de V (eliminando los vectores asociados a valores singulares nulos). Finalmente, U preserva la hiperelipse. De esta manera, la imagen de la esfera unitaria en $\mathbb{R}^n$ bajo el operador A es una hiperelipse en $\mathbb{R}^m$, cuyos ejes principales vienen dados por $\{\sigma_k \boldsymbol{u}_k\}_{k=1}^{r}$. □

Comentario 5.28. Considere $\boldsymbol{x} \in \mathbb{C}^n$, $\boldsymbol{b} \in \mathbb{C}^m$. Sean $\boldsymbol{x}'$ y $\boldsymbol{b}'$ las coordenadas de $\boldsymbol{x}$ y $\boldsymbol{b}$ en las bases de las columnas de V y U, respectivamente. Esto es, $\boldsymbol{x} = V\boldsymbol{x}'$ y $\boldsymbol{b} = U\boldsymbol{b}'$. Al ser U y V unitarias, tenemos que

$$\boldsymbol{x}' = V^*\boldsymbol{x} \quad \text{y} \quad \boldsymbol{b}' = U^*\boldsymbol{b}.$$

Luego, $A\boldsymbol{x} = \boldsymbol{b} \iff \boldsymbol{b}' = U^*\boldsymbol{b} = U^*A\boldsymbol{x} = U^*(U\Sigma V^*)\boldsymbol{x} = \Sigma\boldsymbol{x}'$. De esta forma, $A\boldsymbol{x} = \boldsymbol{b} \iff \boldsymbol{b}' = \Sigma\boldsymbol{x}'$. En otras palabras, A actúa en las bases unitarias dadas por las columnas de U y V de manera diagonal. □

Concluimos con uno de los teoremas más importantes de esta sección:

Teorema 5.29 (Aproximación de bajo rango)**.** *Para cualquier* ν *con* $1 \leqslant \nu \leqslant r$*, defina la aproximación*

$$A_\nu := \sum_{j=1}^{\nu} \sigma_j \boldsymbol{u}_j \boldsymbol{v}_j^*.$$

Entonces,

$$\sigma_{\nu+1} = \|A - A_\nu\|_2 = \inf_{B\in\mathbb{C}^{m\times n}, rank(B)\leqslant\nu} \|A - B\|_2,$$

donde $\sigma_{v+1} = 0$ *si* $\nu = r$*. De esta manera,* A_ν *brinda la mejor aproximación en la* $2-$*norma para* A *en el espacio de matrices de rango* ν*.*

Demostración. Es claro que $\sigma_{\nu+1} = \|A - A_\nu\|_2$, pues

$$A - A_\nu = \sum_{j=\nu+1}^{r} \sigma_j \boldsymbol{u}_j \boldsymbol{v}_j^*$$

brinda una descomposición en valores singulares para $A - A_\nu$, cuyo mayor valor singular es $\sigma_{\nu+1}$.

Asuma que existe $B \in \mathbb{C}^{m\times n}$ con $\operatorname{rank}(B) \leqslant \nu$ tal que $\|A - B\|_2 < \sigma_{\nu+1}$. El núcleo de B tiene dimensión al menos $n - \nu$, y si $\boldsymbol{w} \in \ker(B)$, entonces

$$\|A\boldsymbol{w}\|_2 = \|(A - B)\boldsymbol{w}\|_2 \leqslant \|A - B\|_2 \|\boldsymbol{w}\|_2 < \sigma_{\nu+1}\|\boldsymbol{w}\|_2.$$

Por otro lado, el espacio $\mathcal{B}$ generado por $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_{\nu+1}\}$ tiene dimensión $\nu+1$. Si se toma

$$\boldsymbol{w} = \sum_{k=1}^{\nu+1} \alpha_k \boldsymbol{v}_k \in \mathcal{B},$$

por ortogonalidad se tiene que

$$A\boldsymbol{w} = \sum_{k=1}^{\nu+1} \alpha_k A\boldsymbol{v}_k = \sum_{k=1}^{\nu+1} \alpha_k \sigma_k \boldsymbol{u}_k$$

y así podemos verificar que

$$\|A\boldsymbol{w}\|_2^2 = \sum_{k=1}^{\nu+1} \alpha_k^2 \sigma_k^2 \geqslant \sigma_{\nu+1} \sum_{k=1}^{\nu+1} \alpha_k^2 = \sigma_{\nu+1}\|\boldsymbol{w}\|_2^2.$$

La suma de la dimensión de estos dos subespacios es mayor que n, por lo que su intersección es no vacía. Pero entonces existe $\boldsymbol{w}$ tal que satisface dos desigualdades contradictorias. De esta forma concluimos el resultado. □

5.3. Factorización QR

En esta sección asumimos que $A \in \mathbb{C}^{m\times n}$ con $m \geqslant n$. Si consideramos las n columnas de A, denotadas por $\boldsymbol{a}_j \in \mathbb{C}^m$ $(1 \leqslant j \leqslant n)$, podemos obtener una base ortogonal de vectores unitarios $\boldsymbol{q}_j \in \mathbb{R}^m$ $(1 \leqslant j \leqslant n)$, tales que

$$\langle \boldsymbol{a}_1, \ldots, \boldsymbol{a}_k \rangle = \langle \boldsymbol{q}_1, \ldots, \boldsymbol{q}_k \rangle, \quad k = 1, \ldots, n.$$

Si A tiene rango completo (es decir, sus columnas son linealmente independientes), deseamos relacionar los vectores $\boldsymbol{a}_j$ y $\boldsymbol{q}_j$ mediante las ecuaciones

$$\begin{aligned}
\boldsymbol{a}_1 &= r_{11}\boldsymbol{q}_1 \\
\boldsymbol{a}_2 &= r_{12}\boldsymbol{q}_1 + r_{22}\boldsymbol{q}_2 \\
\boldsymbol{a}_3 &= r_{13}\boldsymbol{q}_1 + r_{23}\boldsymbol{q}_2 + r_{33}\boldsymbol{q}_3 \\
&\vdots \\
\boldsymbol{a}_n &= r_{1n}\boldsymbol{q}_1 + r_{2n}\boldsymbol{q}_2 + r_{3n}\boldsymbol{q}_3 + \ldots + r_{nn}\boldsymbol{q}_n.
\end{aligned} \tag{5.4}$$

Como los vectores $\boldsymbol{q}_j$ son ortogonales, obtenemos que los coeficientes vienen dados por

$$r_{ij} = \boldsymbol{q}_i^* \boldsymbol{a}_j \ (i \neq j), \quad |r_{jj}| = \left\| \boldsymbol{a}_j - \sum_{i=1}^{j-1} r_{ij}\boldsymbol{q}_i \right\|_2 . \tag{5.5}$$

Esto nos permite escribir

$$\left[\begin{array}{c|c|c|c} \boldsymbol{a}_1 & \boldsymbol{a}_2 & \cdots & \boldsymbol{a}_n \end{array}\right] = \left[\begin{array}{c|c|c|c} \boldsymbol{q}_1 & \boldsymbol{q}_2 & \cdots & \boldsymbol{q}_n \end{array}\right] \begin{bmatrix} r_{11} & r_{12} & \cdots & r_{1n} \\ & r_{22} & r_{23} & r_{2n} \\ & & \ddots & \vdots \\ & & & r_{nn} \end{bmatrix};$$

esto es,

$$A = \widehat{Q}\widehat{R},$$

donde $\widehat{Q} \in \mathbb{C}^{m\times n}$ satisface que $\widehat{Q}^*\widehat{Q} = I_{n\times n}$, y $\widehat{R} \in \mathbb{C}^{n\times n}$ es triangular superior. Esta descomposición es llamada *factorización QR reducida*.

Si A no tiene rango completo, entonces algún $\boldsymbol{a}_k \in \langle \boldsymbol{a}_1, \ldots, \boldsymbol{a}_{k-1} \rangle$, con lo que $r_{kk} = 0$. En tal caso $\boldsymbol{q}_k$ no está bien definido, pero basta con seleccionarlo de manera tal que sea ortogonal a $\langle \boldsymbol{q}_1, \ldots, \boldsymbol{q}_{k-1} \rangle$. De esta manera también podemos escribir $A = \widehat{Q}\widehat{R}$.

Si extendemos $\{\boldsymbol{q}_1, \ldots, \boldsymbol{q}_n\}$ a una base de $\mathbb{C}^m$, y denotamos por Q a la matriz unitaria cuyas columnas son $\{\boldsymbol{q}_1, \ldots, \boldsymbol{q}_m\}$, podemos escribir la *factorización QR completa de A* dada por

$$A = QR,$$

donde $Q \in \mathbb{C}^{m\times m}$ es unitaria y $R \in \mathbb{C}^{m\times n}$ es triangular superior. La matriz R se obtiene agregando $m - n$ filas nulas al final de $\widehat{R}$.

De esta manera, hemos comprobado el siguiente resultado:

Teorema 5.30. *Toda matriz $A \in \mathbb{C}^{m\times n}$ $(m \geqslant n)$ admite una descomposición reducida $\widehat{Q}\widehat{R}$, donde $\widehat{Q} \in \mathbb{C}^{m\times m}$, $\widehat{Q}^*\widehat{Q} = I$ y $\widehat{R}$ es triangular superior. Además admite una factorización completa $A = QR$, con $Q \in \mathbb{C}^{m\times m}$ unitaria y $R \in \mathbb{C}^{m\times n}$ triangular superior.*

Note que r_{jj} están determinados solamente por su módulo. Tenemos entonces el siguiente resultado relacionado con la unicidad de la factorización:

Teorema 5.31. *Toda matriz $A \in \mathbb{C}^{m\times n}$ $(m \geqslant n)$ de rango completo tiene una única factorización reducida $\widehat{Q}\widehat{R}$ donde las entradas de la diagonal de $\widehat{R}$ son positivas.*

Demostración. Al ser A de rango completo, todas las entradas r_{jj} son diferentes de cero. Luego, de (5.5) las entradas de $\widehat{R}$ están determinadas de manera única al elegir $r_{jj} > 0$. □

A continuación discutiremos los dos algoritmos fundamentales para calcular numéricamente la factorización QR de una matriz A:

5.3.1. Ortogonalización de Gram-Schmidt

Las ecuaciones dadas en (5.4) se pueden reescribir de la forma

$$\begin{aligned}
\boldsymbol{q}_1 &= \frac{\boldsymbol{a}_1}{r_{11}} \\
\boldsymbol{q}_2 &= \frac{\boldsymbol{a}_2 - r_{12}\boldsymbol{q}_1}{r_{22}} \\
\boldsymbol{q}_3 &= \frac{\boldsymbol{a}_3 - r_{13}\boldsymbol{q}_1 - r_{23}\boldsymbol{q}_2}{r_{33}} \\
&\vdots \\
\boldsymbol{q}_n &= \frac{\boldsymbol{a}_n - \sum_{i=1}^{n-1} r_{in}\boldsymbol{q}_i}{r_{nn}}.
\end{aligned} \tag{5.6}$$

El lector debe reconocer en (5.6) el proceso de *ortogonalización de Gram-Schmidt*. Esto permite definir el siguiente algoritmo para calcular la factorización QR de una matriz A:

Data: Matriz $A \in \mathbb{C}^{m\times n}$
Result: Matriz unitaria Q y triangular superior R tales que $A = QR$
for $j = 1 : n$ **do**
  $\boldsymbol{v}_j = \boldsymbol{a}_j$;
  for $i = 1 : j-1$ **do**
    $r_{ij} = \boldsymbol{q}_i^* \boldsymbol{a}_j$;
    $\boldsymbol{v}_j = \boldsymbol{v}_j - r_{ij}\boldsymbol{q}_i$;
  end
  $r_{jj} = \|\boldsymbol{v}_j\|_2$;
  $\boldsymbol{q}_j = \boldsymbol{v}_j / r_{jj}$;
end

Algoritmo 5: Ortogonalización de Gram-Schmidt (inestable)

No obstante, el algoritmo anterior puede ser inestable debido a errores de redondeo. Si A es de rango completo, podemos definir las matrices triangulares superiores

$$R_1 = \begin{bmatrix} \frac{1}{r_{11}} & -\frac{r_{12}}{r_{11}} & -\frac{r_{13}}{r_{11}} & \cdots & -\frac{r_{1n}}{r_{11}} \\ & 1 & & & \\ & & 1 & & \\ & & & \ddots & \\ & & & & 1 \end{bmatrix},$$

$$R_2 = \begin{bmatrix} 1 & & & & \\ & \frac{1}{r_{22}} & -\frac{r_{23}}{r_{22}} & \cdots & -\frac{r_{2n}}{r_{22}} \\ & & 1 & & \\ & & & \ddots & \\ & & & & 1 \end{bmatrix}, \ \ldots, \ R_n = \begin{bmatrix} 1 & & & & \\ & 1 & & & \\ & & 1 & & \\ & & & \ddots & \\ & & & & \frac{1}{r_{nn}} \end{bmatrix}.$$

Al calcular el producto $AR_1R_2 \ldots R_n$ se obtiene la matriz $\widehat{Q} \in \mathbb{C}^{m\times n}$ cuyas columnas son los vectores $\boldsymbol{q}_i$ dados en (5.6). De esta forma, si definimos $\widehat{R} = (R_1 \ldots R_n)^{-1}$, tenemos que $A = \widehat{Q}\widehat{R}$ es la factorización reducida QR de A, por lo que en la práctica se utiliza el siguiente algoritmo el cual es equivalente al Algoritmo 5:

Data: Matriz $A \in \mathbb{C}^{m\times n}$
Result: Matriz unitaria Q y triangular superior R tales que
$A = QR$
$V = A$;
for $i = 1 : n$ **do**
  $r_{ii} = \|\boldsymbol{v}_i\|_2$;
  $\boldsymbol{q}_i = \boldsymbol{v}_i / r_{ii}$;
  for $j = i + 1 : n$ **do**
    $r_{ij} = \boldsymbol{q}_i^* \boldsymbol{v}_j$;
    $\boldsymbol{v}_j = \boldsymbol{v}_j - r_{ij}\boldsymbol{q}_i$;
  end
end

Algoritmo 6: Ortogonalización de Gram-Schmidt (estable)

Los Algoritmos 5 y 6 satisfacen el siguiente lema:

Lema 5.32. *El número de operaciones en aritmética de punto flotante (flops) para calcular la factorización QR de una matriz $A \in \mathbb{C}^{m\times n}$ dada en los Algoritmos 5 y 6 es de orden $\mathcal{O}(2mn^2)$.*

Demostración. En ambos algoritmos, la mayor cantidad de operaciones ocurren al calcular r_{ij} (m multiplicaciones y $m-1$ sumas) y al actualizar $\boldsymbol{v}_j$ (m productos y m restas). Para el algoritmo 5 el número total de operaciones es de orden

$$\sum_{j=1}^{n}\sum_{i=1}^{j-1} 4m = 4m\sum_{j=1}^{n}(j-1) = 4m\left(\frac{n(n+1)}{2} - n\right) = \mathcal{O}(2mn^2).$$

De igual manera se puede contabilizar el número de operaciones en el Algoritmo 6. □

5.3.2. Triangularización de Householder

En la última sección indicamos que podemos multiplicar a la derecha de A por matrices triangulares superiores para obtener $\widehat{Q}$; este proceso es llamado *ortogonalización triangular*. Una idea análoga es definir matrices ortogonales $Q_1, \ldots, Q_n$ tales que

$$Q_n \ldots Q_1 A = \widehat{R}.$$

De esta manera, si definimos $\widehat{Q} := (Q_n \ldots Q_1)^* = Q_1^* \ldots Q_n^*$, obtenemos la factorización deseada $A = \widehat{Q}\widehat{R}$. Este proceso se conoce como *triangularización ortogonal*.

Por ejemplo, para $A \in \mathbb{C}^{5\times 3}$, deseamos construir Q_1, Q_2 y Q_3 tales que

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \\ a_{51} & a_{52} & a_{53} \end{bmatrix} \underset{\rightarrow}{Q_1 A} \begin{bmatrix} a'_{11} & a'_{12} & a'_{13} \\ 0 & a'_{22} & a'_{23} \\ 0 & a'_{32} & a'_{33} \\ 0 & a'_{42} & a'_{43} \\ 0 & a'_{52} & a'_{53} \end{bmatrix}$$

$$\underset{\rightarrow}{Q_2 Q_1 A} \begin{bmatrix} a'_{11} & a'_{12} & a''_{13} \\ 0 & a''_{22} & a''_{23} \\ 0 & 0 & a''_{33} \\ 0 & 0 & a''_{43} \\ 0 & 0 & a''_{53} \end{bmatrix} \underset{\rightarrow}{Q_3 Q_2 Q_1 A} \begin{bmatrix} a'_{11} & a'_{12} & a''_{13} \\ 0 & a''_{22} & a''_{23} \\ 0 & 0 & a'''_{33} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \widehat{R}.$$

Note que en cada paso Q_j actúa sobre la columna j generando ceros en las entradas bajo la diagonal, sin cambiar los valores de las columnas anteriores ni los valores por encima de la diagonal. Así, cada Q_j debe ser escogida en forma de bloque

$$Q_j = \begin{bmatrix} I_j & \underline{0} \\ \underline{0} & F_j \end{bmatrix}_{m\times m}.$$

donde $I_j \in \mathbb{C}^{j-1\times j-1}$ es la matriz identidad y $F_j \in \mathbb{C}^{m-j+1\times m-j+1}$ debe ser unitaria. A continuación detallamos cómo definir el operador F_j, el cual es llamado *Reflector de Householder*.

Considere $\boldsymbol{x} \in \mathbb{C}^{m-j+1}$. Como F_j debe ser unitaria, por el Lema 5.13 sabemos que

$$\|F_j \boldsymbol{x}\|_2 = \|\boldsymbol{x}\|_2.$$

Deseamos entonces que

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_{m-j+1} \end{bmatrix} \underset{\rightarrow}{F_j} \begin{bmatrix} \|\boldsymbol{x}\|_2 \\ 0 \\ \vdots \\ 0 \end{bmatrix} = |F_j \boldsymbol{x}|.$$

Note que $F_j \boldsymbol{x} = z\|\boldsymbol{x}\|_2 \boldsymbol{e}_1$, donde $z \in \mathbb{C}$ con $|z| = 1$, y $\boldsymbol{e}_1$ es el vector canónico. Geométricamente, F_j corresponde a la reflexión sobre el hiperplano ortogonal a

$$\boldsymbol{v} := z\|\boldsymbol{x}\|_2 \boldsymbol{e}_1 - \boldsymbol{x};$$

ver Figura 5.1. Dicha reflexión viene dada por

$$F_j \boldsymbol{x} = \boldsymbol{x} - 2\frac{\boldsymbol{v}^* \boldsymbol{x}}{\boldsymbol{v}^* \boldsymbol{v}} \boldsymbol{v} = \left(I - 2\frac{\boldsymbol{v}\boldsymbol{v}^*}{\boldsymbol{v}^* \boldsymbol{v}} \right) \boldsymbol{x},$$

por lo que definimos el reflector de Householder como el operador dado por

$$F_j := I - 2\frac{\boldsymbol{v}\boldsymbol{v}^*}{\boldsymbol{v}^*\boldsymbol{v}}.$$

Note que debemos seleccionar apropiadamente z. En aritmética exacta, no importa el valor que se escoja para z. No obstante, si tomamos por ejemplo $z = 1$, numéricamente es posible obtener errores de cancelación. Por lo tanto, es deseable que la imagen $F_j\boldsymbol{x}$ se encuentre tan lejos como sea posible de $\boldsymbol{x}$. Por tal razón, seleccionamos $z = -\text{sign}(x_1)$ (donde definimos $\text{sign}(0) = 1$). De esta forma, el operador F_j está definido por las ecuaciones

$$\boldsymbol{v} := \text{sign}(x_1)\|\boldsymbol{x}\|_2\boldsymbol{e}_1 + \boldsymbol{x}, \quad F_j := I - 2\frac{\boldsymbol{v}\boldsymbol{v}^*}{\boldsymbol{v}^*\boldsymbol{v}}. \tag{5.7}$$

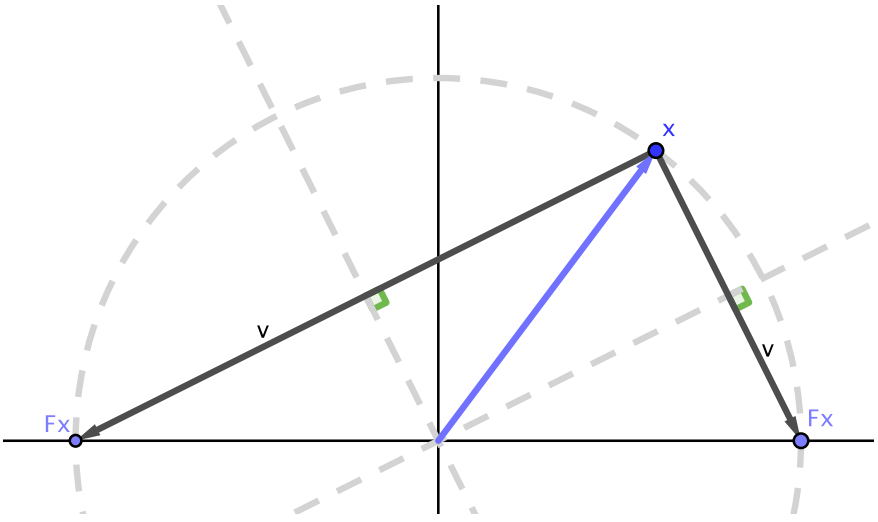


Figura 5.1: Reflexión de Householder.

Así, obtenemos el algoritmo de Householder para obtener la factorización QR de A.

Data: Matriz $A \in \mathbb{C}^{m\times n}$
Result: Matriz unitaria Q y triangular superior R tales que
$A = QR$
for $j = 1 : n$ **do**
  $\boldsymbol{x} = A_{j:m,j}$ % vector de valores entre filas j y m de la columna j;
  $\boldsymbol{v}_j = \text{sign}(x_1)\|\boldsymbol{x}\|_2\boldsymbol{e}_1 + \boldsymbol{x}$;
  $\boldsymbol{v}_j = \boldsymbol{v}_j/\|\boldsymbol{v}_j\|_2$;
  $A_{j:m,j:n} = A_{j:m,j:n} - 2\boldsymbol{v}_j\left(\boldsymbol{v}_j^* A_{j:m,j:n}\right)$;
end

Algoritmo 7: Triangularización de Householder

Ejercicio 5.33. Demuestre que el algoritmo 7 requiere un orden de $2mn^2 - \frac{2}{3}n^3$ operaciones aritméticas en punto flotante.

Además, cuando resolvamos sistemas de ecuaciones, necesitaremos calcular productos de la forma $Q\boldsymbol{x}$ y $Q^*\boldsymbol{b}$. Esto se puede realizar en $\mathcal{O}(mn)$ operaciones con los siguientes algoritmos:

Data: Matriz $A = QR \in \mathbb{C}^{m\times n}$, vector $\boldsymbol{b} \in \mathbb{C}^m$
Result: Producto $Q^*\boldsymbol{b}$
for $j = 1 : n$ **do**
  $\boldsymbol{b}_{j:m} = \boldsymbol{b}_{j:m} - 2\boldsymbol{v}_j\left(\boldsymbol{v}_j^*\boldsymbol{b}_{j:m}\right)$;
end

Algoritmo 8: Cálculo implícito de $Q^*\boldsymbol{b}$

Data: Matriz $A = QR \in \mathbb{C}^{m\times n}$, vector $\boldsymbol{x} \in \mathbb{C}^n$
Result: Producto $Q\boldsymbol{x}$
for $j = n : 1$ **do**
  $\boldsymbol{x}_{j:m} = \boldsymbol{x}_{j:m} - 2\boldsymbol{v}_j\left(\boldsymbol{v}_j^*\boldsymbol{x}_{j:m}\right)$;
end

Algoritmo 9: Cálculo implícito de $Q\boldsymbol{x}$

5.4. Factorización LU

5.4.1. Eliminación Gaussiana sin pivoteo

En esta sección consideraremos $A \in \mathbb{C}^{m\times m}$. Iniciamos con un ejemplo para recordar el proceso de Eliminación Gaussiana. Primero consideraremos

el caso en donde no intercambiamos filas en dicho proceso.

Ejemplo 5.34. Considere la matriz

$$A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}.$$

Para colocar ceros de bajo de la diagonal en la primer columna, debemos empezar con las operaciones $-2f_1 + f_2$, $-4f_1 + f_3$ y $-3f_1 + f_4$. Dichas operaciones pueden expresarse de manera matricial. Defina

$$L_1 := \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -4 & 0 & 1 & 0 \\ -3 & 0 & 0 & 1 \end{bmatrix}.$$

Luego calculamos

$$L_1 A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ -2 & 1 & 0 & 0 \\ -4 & 0 & 1 & 0 \\ -3 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 3 & 5 & 5 \\ 0 & 4 & 6 & 8 \end{bmatrix}.$$

En el siguiente paso necesitamos realizar las operaciones $-3f_2 + f_3$ y $-4f_2 + f_4$. Similarmente definimos

$$L_2 := \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -3 & 1 & 0 \\ 0 & -4 & 0 & 1 \end{bmatrix}$$

Obtenemos así

$$L_2 L_1 A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & -3 & 1 & 0 \\ 0 & -4 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 3 & 5 & 5 \\ 0 & 4 & 6 & 8 \end{bmatrix} = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 2 & 2 \\ 0 & 0 & 2 & 4 \end{bmatrix}.$$

Finalmente, debemos restar las dos últimas filas. De esta manera definimos

$$L_3 := \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 1 \end{bmatrix} \quad \text{y} \quad L_3 L_2 L_1 A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 2 & 2 \\ 0 & 0 & 0 & 2 \end{bmatrix} =: U.$$

De esta manera, hemos obtenido una matriz triangular superior U como resultado de multiplicar por tres matrices triangulares inferiores. Así, si definimos

$$L := L_1^{-1}L_2^{-1}L_3^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 4 & 3 & 1 & 0 \\ 3 & 4 & 1 & 1 \end{bmatrix}$$

hemos escrito la matriz A como producto de una matriz triangular inferior con unos en la diagonal y una matriz triangular superior, $A = LU$. Note que L se obtiene de colocar el opuesto de cada entrada de las matrices L_1, L_2 y L_3. Esto lo demostraremos más adelante. □

Definición 5.35. Considere $A \in \mathbb{C}^{m\times m}$. Diremos que A admite una factorización LU si existen $U \in \mathbb{C}^{m\times m}$ triangular superior, $L \in \mathbb{C}^{m\times m}$ triangular inferior unitaria (todos sus elementos de la diagonal son unos) tales que $A = LU$.

Es claro que en $m-1$ pasos podemos obtener una secuencia de matrices triangulares inferiores unitarias $L_1, \ldots, L_{m-1}$ tales que

$$L_{m-1} \ldots L_1 A = U,$$

donde U es una matriz triangular superior (siempre y cuando, al aplicar el proceso de eliminación Gaussiana, ninguna entrada en la diagonal sea cero). Si definimos

$$L := L_1^{-1} \ldots L_{m-1}^{-1},$$

entonces tenemos una factorización

$$A = LU$$

Podemos construir explícitamente L. En el paso k buscamos colocar ceros debajo del elemento de la diagonal en la columna k. En otras palabras, la acción de L_k sobre la columna k viene dada por

$$\begin{bmatrix} a_{1,k} \\ \vdots \\ a_{k,k} \\ a_{k+1,k} \\ \vdots \\ a_{m,k} \end{bmatrix} \begin{matrix} \\ \\ \\ L_k \\ \rightarrow \\ \\ \\ \end{matrix} \begin{bmatrix} a_{1,k} \\ \vdots \\ a_{k,k} \\ 0 \\ \vdots \\ 0 \end{bmatrix}.$$

Defina

$$\ell_{j,k} := \frac{a_{j,k}}{a_{k,k}} \ (k < j \leqslant m).$$

De esta forma,

$$L_k := \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & -\ell_{k+1,k} & 1 & & \\ & & \vdots & & \ddots & \\ & & -\ell_{m,k} & & & 1 \end{bmatrix}_{m\times m} = I - \boldsymbol{\ell}_k \boldsymbol{e}_k^*,$$

donde las entradas omitidas son ceros, $I \in \mathbb{C}^{m\times m}$ es la matriz identidad, $\boldsymbol{\ell}_k = [0, \ldots 0, \ell_{k+1,k}, \ldots, \ell_{m,k}]^T \in \mathbb{C}^m$ es el vector de multiplicadores y $\boldsymbol{e}_k \in \mathbb{C}^m$ es el vector canónico (todas sus entradas son cero, a excepción de un uno en la posición k).

Note que

$$L_k(I + \boldsymbol{\ell}_k \boldsymbol{e}_k^*) = (I - \boldsymbol{\ell}_k \boldsymbol{e}_k^*)(I + \boldsymbol{\ell}_k \boldsymbol{e}_k^*) = I - \boldsymbol{\ell}_k \boldsymbol{e}_k^* \boldsymbol{\ell}_k \boldsymbol{e}_k^* = I,$$

pues $\boldsymbol{e}_k^* \boldsymbol{\ell}_k = 0$. De esta manera, invertir L_k equivale a cambiar los signos de sus entradas debajo de la diagonal; esto es

$$L_k^{-1} = I + \boldsymbol{\ell}_k \boldsymbol{e}_k^* = \begin{bmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & \ell_{k+1,k} & 1 & & \\ & & \vdots & & \ddots & \\ & & \ell_{m,k} & & & 1 \end{bmatrix}_{m\times m}.$$

Luego, si multiplicamos

$$\begin{aligned} L_k^{-1} L_{k+1}^{-1} &= (I + \boldsymbol{\ell}_k \boldsymbol{e}_k^*)(I + \boldsymbol{\ell}_{k+1} \boldsymbol{e}_{k+1}^*) \\ &= I + \boldsymbol{\ell}_k \boldsymbol{e}_k^* + \boldsymbol{\ell}_{k+1} \boldsymbol{e}_{k+1}^* + \boldsymbol{\ell}_k \boldsymbol{e}_k^* \boldsymbol{\ell}_{k+1} \boldsymbol{e}_{k+1}^* \\ &= I + \boldsymbol{\ell}_k \boldsymbol{e}_k^* + \boldsymbol{\ell}_{k+1} \boldsymbol{e}_{k+1}^*, \end{aligned}$$

pues $\boldsymbol{e}_k^* + \boldsymbol{\ell}_{k+1} = 0$. En palabras sencillas, multiplicar dos inversas equivale a tomar la identidad y colocar los valores de los multiplicadores $\ell_{j,k}$ debajo

de la diagonal. Por lo tanto,

$$L = L_1^{-1} \dots L_{m-1}^{-1} = \begin{bmatrix} 1 & & & & \\ \ell_{2,1} & 1 & & & \\ \ell_{3,1} & \ell_{3,2} & 1 & & \\ \vdots & \vdots & \ddots & \ddots & \\ \ell_{m,1} & \ell_{m,2} & \dots & \ell_{m,m-1} & 1 \end{bmatrix}.$$

De esta forma, tenemos el siguiente algoritmo:

Data: Matriz $A \in \mathbb{C}^{m\times m}$
Result: Factorización $A = LU$
$U = A$, $L = I$;
for $k = 1 : m-1$ **do**
  for $j = k+1 : m$ **do**
    $L_{j,k} = U_{j,k}/U_{k,k}$;
    $U_{j,k:m} = U_{j,k:m} - L_{j,k}U_{k,k:m}$;
  end
end

Algoritmo 10: Eliminación Gaussiana sin pivoteo

El algoritmo anterior presenta dos posibles inconvenientes:

(a) Note que en aritmética exacta,

$$A = \begin{bmatrix} 10^{-20} & 1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 10^{20} & 1 \end{bmatrix} \begin{bmatrix} 10^{-20} & 1 \\ 0 & 1-10^{20} \end{bmatrix} = LU.$$

Trabajando con precisión doble, tenemos que $1 \ominus 10^{20} = 10^{20}$. Así, el resultado numérico sería

$$\widetilde{L}\widetilde{U} = \begin{bmatrix} 1 & 0 \\ 10^{20} & 1 \end{bmatrix} \begin{bmatrix} 10^{-20} & 1 \\ 0 & -10^{20} \end{bmatrix} = \begin{bmatrix} 10^{-20} & 1 \\ 1 & 0 \end{bmatrix} \neq A.$$

Si intercambiamos las filas de A al inicio, obtenemos

$$A_1 = \begin{bmatrix} 1 & 1 \\ 10^{-20} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 10^{-20} & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1-10^{-20} \end{bmatrix} = L_1U_1,$$

y numéricamente

$$\widetilde{L}_1\widetilde{U}_1 = \begin{bmatrix} 1 & 0 \\ 10^{-20} & 1 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 10^{-20} & 1 \end{bmatrix} = A_1.$$

Note que en el segundo caso, los elementos de la matriz L son menores o iguales que 1. Para ello, debemos intercambiar filas de manera adecuada.

(b) Si algún elemento de la diagonal es cero, no podemos efectuar el proceso de eliminación Gaussiana tal como lo hicimos en el Ejemplo 5.34. Así, para

$$A = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$

es claro que debemos intercambiar filas primero.

Ambos problemas pueden ser evitados si utilizamos *pivoteo parcial*, el cual estudiamos a continuación.

5.4.2. Eliminación Gaussiana con pivoteo parcial

Antes de multiplicar a la izquierda por cada matriz L_k, consideraremos intercambiar dos filas, de manera tal que en el elemento de la diagonal de la columna k se ubique el mayor elemento. Tal valor será llamado *pivote*. Veamos el siguiente ejemplo, donde utilizamos la misma matriz del Ejemplo 5.34.

Ejemplo 5.36. Para

$$A = \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix},$$

iniciamos intercambiando las filas 1 y 3, pues el pivote se encuentra en la posición $(3, 1)$. Esto lo logramos mediante la acción de la matriz

$$P_1 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

la cual se obtiene de permutar las filas 1 y 3 de la matriz identidad. De esta forma,

$$P_1 A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ 4 & 3 & 3 & 1 \\ 2 & 1 & 1 & 0 \\ 6 & 7 & 9 & 8 \end{bmatrix}.$$

Luego, siguiendo el mismo proceso del ejemplo 5.34, calculamos

$$L_1 P_1 A = \begin{bmatrix} 1 & & & \\ -\frac{1}{2} & 1 & & \\ -\frac{1}{4} & & 1 & \\ -\frac{3}{4} & & & 1 \end{bmatrix} \begin{bmatrix} 8 & 7 & 9 & 5 \\ 4 & 3 & 3 & 1 \\ 2 & 1 & 1 & 0 \\ 6 & 7 & 9 & 8 \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ 0 & -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} \\ 0 & -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} \\ 0 & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \end{bmatrix}.$$

Ahora, en la segunda columna debemos colocar la mayor entrada en la diagonal (sin considerar la primer fila). Por ello, intercambiamos las filas 2 y 4 al multiplicar por la matriz

$$P_2L_1P_1A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 8 & 7 & 9 & 5 \\ 0 & -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} \\ 0 & -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} \\ 0 & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ 0 & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ 0 & -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} \\ 0 & -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} \end{bmatrix}.$$

De esta forma,

$$L_2 = \begin{bmatrix} 1 & & & \\ & 1 & & \\ & \frac{3}{7} & 1 & \\ & \frac{2}{7} & & 1 \end{bmatrix}$$

y

$$L_2P_2L_1P_1A = \begin{bmatrix} 1 & & & \\ & 1 & & \\ & \frac{3}{7} & 1 & \\ & \frac{2}{7} & & 1 \end{bmatrix} \begin{bmatrix} 8 & 7 & 9 & 5 \\ 0 & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ 0 & -\frac{3}{4} & -\frac{5}{4} & -\frac{5}{4} \\ 0 & -\frac{1}{2} & -\frac{3}{2} & -\frac{3}{2} \end{bmatrix} = \begin{bmatrix} 8 & 7 & 9 & 5 \\ 0 & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ 0 & 0 & -\frac{2}{7} & -\frac{4}{7} \\ 0 & 0 & -\frac{6}{7} & -\frac{2}{7} \end{bmatrix}.$$

Note ahora que debemos intercambiar las filas 3 y 4. De esta forma,

$$P_3 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}, \quad L_3 = \begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & -\frac{1}{3} & 1 \end{bmatrix},$$

y

$$L_3P_3L_2P_2L_1P_1A = \begin{bmatrix} 8 & 7 & 9 & 5 \\ 0 & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ 0 & 0 & -\frac{6}{7} & -\frac{2}{7} \\ 0 & 0 & 0 & \frac{2}{3} \end{bmatrix} =: U.$$

Si definimos la matriz de permutaciones

$$P := P_3P_2P_1 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

y construimos L de manera similar al Ejemplo 5.34, podemos verificar que

$$PA = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 2 & 1 & 1 & 0 \\ 4 & 3 & 3 & 1 \\ 8 & 7 & 9 & 5 \\ 6 & 7 & 9 & 8 \end{bmatrix}$$
$$= \begin{bmatrix} 1 & & & \\ \frac{3}{4} & 1 & & \\ \frac{1}{2} & -\frac{2}{7} & 1 & \\ \frac{1}{4} & -\frac{3}{7} & \frac{1}{3} & 1 \end{bmatrix} \begin{bmatrix} 8 & 7 & 9 & 5 \\ 0 & \frac{7}{4} & \frac{9}{4} & \frac{17}{4} \\ 0 & 0 & -\frac{6}{7} & -\frac{2}{7} \\ 0 & 0 & 0 & \frac{2}{3} \end{bmatrix} = LU.$$

De esta forma, hemos calculado la factorización LU de la matriz PA, la cual corresponde a la matriz original A con sus filas permutadas. □

Tal como se demostró en el ejemplo anterior, dada una matriz $A \in \mathbb{C}^{m\times m}$, siempre es posible escribir

$$L_{m-1}P_{m-1}\dots L_2P_2L_1P_1A = U,$$

donde U es triangular superior y las entradas de las matrices L_k están acotadas por 1. Note que en el caso que el pivote sea 0 (es decir, si en la columna k donde estamos aplicando el paso de eliminación Gaussiana todas las entradas por considerar son cero), no se realiza ninguna permutación, se toma $L_k = I$ y se continua en la columna $k+1$.

Definimos las matrices

$$\begin{aligned} L'_1 &= P_{m-1}\dots P_2L_1P_2^{-1}\dots P_{m-1}^{-1} \\ &\vdots \\ L'_k &= P_{m-1}\dots P_{k+1}L_kP_{k+1}^{-1}\dots P_{m-1}^{-1} \\ &\vdots \\ L'_{m-1} &= L_{m-1} \end{aligned}$$

Directamente se puede verificar que

$$(L'_{m-1}\dots L'_1)(P_{m-1}\dots P_1)A = U,$$

por lo que si definimos $P := P_{m-1}\dots P_1$ y $L := (L'_1)^{-1}\dots\left(L'_{m-1}\right)^{-1}$ obtenemos la factorización

$$PA = LU.$$

De esta manera, hemos demostrado el siguiente resultado:

Teorema 5.37. *Toda matriz $A \in \mathbb{C}^{m\times m}$ tiene una factorización $PA = LU$, donde P es una matriz de permutaciones de fila, L es una matriz unitaria triangular inferior donde todas sus entradas están acotadas por 1, y U es una matriz triangular superior.*

El algoritmo para producir la factorización $PA = LU$ es el siguiente:
Data: Matriz $A \in \mathbb{C}^{m\times m}$
Result: Factorización $A = LU$
$U = A$, $L = I$, $P = I$;
for $k = 1 : m - 1$ **do**
  Seleccione $i \geqslant k$ tal que $|U_{i,k}|$ es máximo.;
  $U_{k,k:m} \longleftrightarrow U_{i,k:m}$;
  $L_{k,1:k-1} \longleftrightarrow L_{i,1:k-1}$;
  $P_{k,:} \longleftrightarrow P_{i,:}$;
  for $j = k + 1 : m$ **do**
    $L_{j,k} = U_{j,k}/U_{k,k}$;
    $U_{j,k:m} = U_{j,k:m} - L_{j,k}U_{k,k:m}$;
  end
end

Algoritmo 11: Eliminación Gaussiana con pivoteo parcial

Ejercicio 5.38. Demuestre que el algoritmo 11 requiere un orden de $\frac{2}{3}m^3$ operaciones aritméticas en punto flotante.

5.4.3. Pivoteo completo

En el pivoteo parcial, en el paso k buscamos el mayor elemento del conjunto $\{A_{k,k}, A_{k+1,k}, \ldots, A_{m,k}\}$. No obstante, podríamos buscar el mayor elemento

$$\max_{k\leqslant i,j\leqslant m,} |A_{i,j}|.$$

Esto es, buscamos el máximo en todo el bloque restante de la matriz A. Para colocar el pivote en la posición deseada, necesitamos intercambiar dos filas y dos columnas. De esta forma se puede demostrar que se obtiene una factorización

$$PAQ = LU,$$

donde Q representa los cambios de columna que se realizan en cada paso. En práctica, buscar el pivote puede ser un proceso lento (por ejemplo, en el

primer paso se debe realizar una búsqueda en un espacio de n^2 elementos), por lo que usualmente es satisfactorio utilizar solo pivoteo parcial.

5.5. Factorización de Cholesky

Considere el caso particular de una matriz A Hermitiana. En tal caso, para todo $\boldsymbol{x} \in \mathbb{C}^m$ se tiene que $(\boldsymbol{x}^* Ax)^* = \boldsymbol{x}^* A\boldsymbol{x}$, por lo que

$$\boldsymbol{x}^* A\boldsymbol{x} \in \mathbb{R} \ \forall \ \boldsymbol{x} \in \mathbb{C}^m.$$

En esta sección consideraremos matrices definidas positivas, las cuales definimos a continuación:

Definición 5.39. *Una matriz Hermitiana $A \in \mathbb{C}^{m\times m}$ es definida positiva si $\boldsymbol{x}^* A\boldsymbol{x} > 0$ para todo $\boldsymbol{x} \neq 0$.*

Tenemos los siguientes dos lemas importantes:

Lema 5.40. *Una matriz Hermitiana se llama definida positiva si y solo si todos sus valores propios son positivos.*

Demostración. Si A es definida positiva, considere λ un valor propio asociado al vector propio $\boldsymbol{x} \neq 0$. Se tiene así que $\lambda \boldsymbol{x}^* \boldsymbol{x} = \boldsymbol{x}^* A\boldsymbol{x} > 0$, por lo que $\lambda > 0$. Suponga ahora que todos los valores propios son positivos. Sabemos que existe una base $\{\boldsymbol{v}_i\}$ de vectores propios ortogonales (pues A es Hermitiana). Entonces, dado $\boldsymbol{x} \neq 0$, podemos escribir $\boldsymbol{x} = \sum_i \alpha_i \boldsymbol{v}_i$. Luego, $A\boldsymbol{x} = \sum_i \alpha_i \lambda_i \boldsymbol{v}_i$ y por ortogonalidad, $\boldsymbol{x}^* A\boldsymbol{x} = \sum_i \alpha_i^2 \lambda_i \boldsymbol{v}^* \boldsymbol{v} > 0$. □

Lema 5.41. *Sean $A \in \mathbb{C}^{m\times m}$ Hermitiana y $X \in \mathbb{C}^{m\times n}$ con $m \geqslant n$. Suponga que A es definida definida positiva y que las columnas de X son linealmente independientes. Entonces X^*AX es Hermitiana y definida positiva.*

Demostración. Es claro que $(X^*AX)^* = X^*AX$. Dado $\boldsymbol{x} \neq 0$, $X\boldsymbol{x} \neq 0$ y así $\boldsymbol{x}^*(X^*AX)\boldsymbol{x} = (X\boldsymbol{x})^* A(X\boldsymbol{x}) > 0$. □

En particular, el lema anterior implica que las entradas de la diagonal de una matriz A Hermitiana definida positiva son positivas (al tomar $X = \boldsymbol{e}_k$ se tiene que $X^*AX = a_{kk}$). De manera similar se puede concluir que cualquier submatriz principal de A también es definida positiva.

Si consideramos la factorización LU de una matriz Hermitiana A, uno esperaría obtener una factorización donde U sea la transpuesta conjugada de L. De esta forma, consideramos la posibilidad de escribir

$$A = LL^* = R^*R, \tag{5.8}$$

donde L es triangular inferior y $R = L^*$ es triangular superior. En este caso, no esperamos que las entradas de la diagonal de L sean 1. Tal descomposición recibe el nombre de *factorización de Cholesky*, para la cual se tiene el siguiente resultado:

Teorema 5.42. *Toda matriz $A \in \mathbb{C}^{m\times m}$ Hermitiana definida positiva tiene una única factorización de Cholesky* (5.8) *donde las entradas de la diagonal son positivas.*

Demostración. Tome $\alpha = (a_{1,1})^{1/2}$ (recuerde que $a_{1,1} > 0$ por el Lema 5.41). Si escribimos la matriz A en bloques

$$A = \begin{bmatrix} a_{1,1} & \boldsymbol{w}^* \\ \boldsymbol{w} & K \end{bmatrix},$$

donde $K \in \mathbb{C}^{m-1\times m-1}$, $\boldsymbol{w} \in \mathbb{C}^{m-1}$, es fácil ver que

$$\begin{aligned} A &= \begin{bmatrix} a_{1,1} & \boldsymbol{w}^* \\ \boldsymbol{w} & K \end{bmatrix} \\ &= \begin{bmatrix} \alpha & 0 \\ \boldsymbol{w}/\alpha & I \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & K - \boldsymbol{w}\boldsymbol{w}^*/\alpha^2 \end{bmatrix} \begin{bmatrix} \alpha & \boldsymbol{w}^*/\alpha \\ 0 & I \end{bmatrix} \\ &= L_1 A_1 L_1^*. \end{aligned}$$

Note que L_1 es invertible, y así $L_1^{-1}A(L_1^*)^{-1} = A_1$ es definida positiva. Luego, $K - \boldsymbol{w}\boldsymbol{w}^*/\alpha^2$ es definida positiva por ser una submatriz principal de A_1. Si $m = 1$ es claro que tal factorización existe. Si $m > 1$, por inducción podemos escribir $A_1 = L_2 L_2^*$, de donde $A = LL^*$, donde $L = L_1 L_2$, $L_{k,k} > 0$. □

De esta manera tenemos el algoritmo para obtener la factorización de Cholesky de una matriz definida positiva:

Data: Matriz $A \in \mathbb{C}^{m\times m}$
Result: Factorización $A = R^*R$
$R = A$;
for $k = 1 : m$ **do**
  for $j = k + 1 : m$ **do**
    $R_{j,j:m} = R_{j,j:m} - R_{k,j:m}\overline{R}_{kj}/R_{k,k}$;
  end
  $R_{k,k:m} = R_{k,k:m}/\sqrt{R_{k,k}}$;
end

Algoritmo 12: Factorización de Cholesky

Ejercicio 5.43. Demuestre que el algoritmo 12 requiere un orden de $\frac{1}{3}m^3$ operaciones aritméticas en punto flotante.

5.6. Resolución de sistemas de ecuaciones

Si deseamos resolver el sistema de ecuaciones

$$A\boldsymbol{x} = \boldsymbol{b},\ A \in \mathbb{C}^{m\times m},$$

con A invertible, y conocemos alguna factorización de A,

$$\begin{array}{cc} A = U\Sigma V^*, & A = QR, \\ A = LU, & A = LL^*, \end{array}$$

podemos resolver el sistema como detallamos a continuación:

(a) $U\Sigma V^*\boldsymbol{x} = \boldsymbol{b} \Rightarrow \boldsymbol{x} = V\Sigma^{-1}U^*\boldsymbol{b}$, en donde Σ^{-1} se obtiene al invertir cada elemento de la diagonal de Σ.

(b) $QR\boldsymbol{x} = \boldsymbol{b} \Rightarrow R\boldsymbol{x} = Q^*\boldsymbol{b}$, y este último sistema es triangular superior.

(c) $LU\boldsymbol{x} = \boldsymbol{b}$, en donde primero encontramos $\boldsymbol{y}$ tal que $L\boldsymbol{y} = \boldsymbol{b}$ y luego $\boldsymbol{x}$ tal que $U\boldsymbol{x} = \boldsymbol{y}$. En este caso ocupamos resolver un sistema con una matriz triangular inferior y otro con una matriz triangular superior.

(d) $LL^*\boldsymbol{x} = \boldsymbol{b}$, en donde primero encontramos $\boldsymbol{y}$ tal que $L\boldsymbol{y} = \boldsymbol{b}$ y luego $\boldsymbol{x}$ tal que $L^*\boldsymbol{x} = \boldsymbol{y}$.

Para resolver un sistema con una matriz triangular superior, se utiliza el método de *sustitución hacia atrás*:

Data: Matriz $R \in \mathbb{C}^{m\times m}$ triangular superior, $\boldsymbol{b} \in \mathbb{C}^m$
Result: Solución $\boldsymbol{x} = R^{-1}\boldsymbol{b}$
for $k = m : 1$ **do**
  $x_k = (b_k - \sum_{j=k+1}^{m} R_{k,j}x_j)/R_{k,k}$;
end

Algoritmo 13: Sustitución hacia atrás

Si por el contrario, tenemos que resolver un sistema con una matriz triangular inferior, utilizamos el método de *sustitución hacia adelante*:

Data: Matriz $L \in \mathbb{C}^{m\times m}$ triangular inferior, $\boldsymbol{b} \in \mathbb{C}^m$
Result: Solución $\boldsymbol{x} = L^{-1}\boldsymbol{b}$
for $k = 1 : m$ **do**
  $x_k = (b_k - \sum_{j=1}^{k-1} L_{k,j}x_j)/L_{k,k}$;
end

Algoritmo 14: Sustitución hacia adelante

Ejercicio 5.44. Determine el número de operaciones requeridas al ejecutar los Algoritmos 13 y 14.

Debe ser importante notar que luego de calcular alguna factorización de A, no es necesario invertir directamente ninguna matriz para resolver la ecuación $A\boldsymbol{x} = \boldsymbol{b}$ según los algoritmos descritos en (a), (b), (c), (d). En la práctica, calcular directamente A^{-1} debe ser evitado, por problemas de precisión e inestabilidad. En el siguiente capítulo hablaremos de estos temas.

5.7. Ejercicios Adicionales[1]

Ejercicio 5.45. Demuestre que la inversa de una matriz triangular superior invertible es también triangular superior.

Ejercicio 5.46. Demuestre que si una matriz A es triangular y unitaria, entonces es diagonal.

Ejercicio 5.47. Si $\boldsymbol{u}, \boldsymbol{v} \in \mathbb{C}^m$, la matriz $A = I + \boldsymbol{u}\boldsymbol{v}^*$ es llamada *perturbación de rango uno de la identidad*.

(a) Muestre que si A es invertible, entonces su inversa tiene la forma $A^{-1} = I + \alpha\boldsymbol{u}\boldsymbol{v}^*$ con $\alpha \in \mathbb{C}$. Encuentre el valor de α explícitamente.

(b) ¿Para qué valores de $\boldsymbol{u}$ y $\boldsymbol{v}$ es A singular? En este caso, determine el núcleo de A.

Ejercicio 5.48. Suponga que $\boldsymbol{x} \in \mathbb{C}^m$, $A \in \mathbb{C}^{m\times n}$. Demuestre las siguientes desigualdades. En cada caso, indique un vector o una matriz (no nulos) para los cuales la igualdad se cumple.

[1] Algunos ejercicios teóricos fueron tomados de [29].

(a) $\|\boldsymbol{x}\|_\infty \leqslant \|\boldsymbol{x}\|_2$.

(b) $\|\boldsymbol{x}\|_2 \leqslant \sqrt{m}\|\boldsymbol{x}\|_\infty$.

(c) $\|A\|_\infty \leqslant \sqrt{n}\|A\|_2$.

(d) $\|A\|_2 \leqslant \sqrt{m}\|A\|_\infty$.

Ejercicio 5.49. Sea $A = \boldsymbol{x}\boldsymbol{y}^*$, con $\boldsymbol{x} \in \mathbb{C}^m$, $\boldsymbol{y} \in \mathbb{C}^n$. ¿Es cierto que $\|A\|_F = \|\boldsymbol{x}\|_F\|\boldsymbol{y}\|_F$? Demuestre la igualdad o dé un contraejemplo.

Ejercicio 5.50. Considere la matriz

$$A = \begin{bmatrix} -2 & 11 \\ -10 & 5 \end{bmatrix}.$$

(a) Determine una descomposición en valores singulares de A.

(b) Calcule $\|A\|_1$, $\|A\|_2$, $\|A\|_\infty$ y $\|A\|_F$.

(c) Utilice la descomposición de la parte (a) para calcular A^{-1}.

(d) Encuentre los valores propios λ_1 y λ_2 de A.

(e) Verifique que $\det A = \lambda_1\lambda_2$ y $|\det A| = \sigma_1\sigma_2$.

Ejercicio 5.51. Dada $A \in \mathbb{C}^{m\times n}$, con $m \geqslant n$, demuestre que A^*A es invertible si y solo si A tiene rango completo.

Ejercicio 5.52. Considere la matriz

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

Determine una factorización reducida QR y una factorización completa QR de A, donde las entradas de la diagonal de R sean no negativas.

Ejercicio 5.53. Sea $A \in \mathbb{C}^{m\times m}$. Demuestre la *desigualdad de Hadamard*,

$$|\det A| \leqslant \prod_{j=1}^{m} \|\boldsymbol{a}_j\|_2,$$

donde $\boldsymbol{a}_j$ denota la columna j de A.

Ejercicio 5.54. Sea $A \in \mathbb{C}^{m\times n}$ $(m \geqslant n)$. Considere $A = \widehat{Q}\widehat{R}$ una factorización QR reducida de A.

(a) Demuestre que A tiene rango n si y solo si las entradas de la diagonal de $\widehat{R}$ son diferentes de cero.

(b) Suponga que $\widehat{R}$ tiene k entradas en la diagonal diferente de cero, con $0 \leqslant k < n$. Determine si el rango de A es k, mayor que k, o menor que k. Pruebe formalmente su respuesta.

Ejercicio 5.55. Considere el reflector de Householder F_j definido en la Ecuación (5.7). Determine sus valores propios, su determinante y sus valores singulares.

Ejercicio 5.56. Sea $A \in \mathbb{C}^{m\times m}$ invertible.

(a) Muestre que A tiene una factorización $A = LU$ si y solo si, para cada k con $1 \leqslant k \leqslant m$, el bloque $A_{1:k,1:k}$ es invertible.

(b) Suponga que $A = LU$ y que existe un natural p tal que $A_{i,j} = 0$ si $|i-j| > p$ (en este caso se dice que A tiene banda $2p+1$). ¿Qué puede concluir de la forma de L y U? ¿Cuáles elementos puede garantizar que son ceros, por ejemplo?

Ejercicio 5.57. Describa cómo utilizar la factorización $A = LU$ para calcular $\det A$.

Ejercicio 5.58. Suponga que $A \in \mathbb{C}^{m\times m}$ satisface que para todo k,

$$|A_{k,k}| > \sum_{j\neq k} |A_{j,k}|.$$

Dicha condición es llamada *dominancia estricta de la diagonal en las filas*. Muestre que si utiliza el algoritmo de eliminación Gaussiana con pivoteo parcial, entonces no es necesario intercambiar ninguna fila.

Ejercicio 5.59. Sea $A \in \mathbb{C}^{m\times m}$ invertible y asuma que $A = QR$ y $A = U^*U$ son sus factorizaciones QR y de Cholesky, con $r_{j,j} > 0$, $u_{j,j} > 0$. ¿Es cierto que $R = U$?

Ejercicio 5.60. `[MATLAB]` (Matriz de Hilbert) Sea $H \in \mathbb{R}^{n\times n}$ la matriz cuyas entradas vienen dadas por

$$h_{ij} = \frac{1}{i+j-1}, \quad i,j \in \{1,\ldots,n\};$$

ver Sección 1.6.2. Se puede demostrar que esta matriz es invertible pero mal condicionada.

(a) Considere las funciones dadas por

```
function H = hilb_def(n)
H=zeros(n);
  for i = 1:n
    for j = 1:n
      H(i,j) = 1/(i+j-1);
  end
end

function H = hilb(n)
  j = 1:n;
  H = 1./(j'+j-1);
end
```

Ambas calculan la matriz de Hilbert de tamaño n. La función `hilb_def` hace los cálculos de manera directa mediante dos lazos *for*, recorriendo la matriz entrada por entrada. En cambio, la función `hilb` la calcula mediante el uso de vectores (para lo cual `Matlab` está optimizado). Compare los tiempos de ejecución al correr los comandos

```
n = 1e4; % matriz de tamaño 10 mil
tic, H = hilb_def(n); toc
tic, H = hilb(n); toc
```

y comente los resultados.

(b) Fije `n = 15`. Defina `A = hilb(n)` y `x_exact = ones(n,1)`, el cual corresponde a un vector columna de tamaño n cuyas entradas son iguales a 1. Calcule `b = A*x_exact`.

(c) Deseamos resolver el sistema de ecuaciones $Ax = b$. En `Matlab`, esto se ejecuta de manera directa con el comando `x_exp = A\b`. Calcule la $2-$norma del error $\|x_{\text{exact}} - x_{\text{exp}}\|_2$.

(d) Calcule una descomposición en valores singulares `[U,S,V] = svd(A)`. Grafique los valores singulares mediante el comando

`semilogy(diag(S))`,

donde `diag(S)` extrae las entradas de la diagonal de la matriz `S`. Numéricamente, ¿cuál diría es el rango de la matriz? (recuerde que el rango es el número de valores singulares no nulos).

(e) Considere una matriz con descomposición en valores singulares $A = USV^T$. La *pseudoinversa* se define como

$$A^+ = VS^+U^T,$$

donde

$$S^+ = \begin{bmatrix} 1/\sigma_1 & & & \\ & 1/\sigma_2 & & \\ & & \ddots & \\ & & & 1/\sigma_p \end{bmatrix}$$

es la matriz diagonal obtenida de invertir los valores singulares positivos de S. Defina un vector de tolerancias

```
tol = 10.^(-linspace(2,17))
```

Para cada entrada `j` de `tol`, defina S_j como la matriz obtenida a partir de S, reemplazando los valores singulares menores que `tol(j)`$*\sigma_1$ por cero. Aproxime `x_svd =` $(VS_j^+U^T)b$ y calcule el error relativo $\|x_{\text{exact}} - x_{\text{svd}}\|_2 / \|x_{\text{exact}}\|_2$. Note que estamos aproximando la pseudoinversa con algunos valores singulares de A. Grafique el error relativo en función del vector de tolerancias. Comente sus resultados y explique el comportamiento del gráfico.

(f) `Matlab` incluye la función `invhilb(n)` para calcular la inversa de la matriz de Hilbert de tamaño $n \times n$. Para `n = 1:20`, grafique el error relativo

$$\|\texttt{hilb(n)*invhilb(n) - eye(n)}\|$$

en función de n, donde `eye(n)` representa la matriz identidad de tamaño $n \times n$. Concluya que la función `invhilb` no es confiable para valores $n \geqslant 10$. De esta forma, es mejor utilizar la pseudoinversa.

Ejercicio 5.61 (`MATLAB`)**.** Considere los Algoritmos 5, 6, 7 para calcular la factorización QR de una matriz A.

(a) Escriba tres funciones que implemente cada uno de los algoritmos.

(b) Defina la matriz A mediante las siguiente líneas:

```
m = 80;
[U,~] = qr(randn(m));
[V,~] = qr(randn(m));
S = diag(2.^(-1:-1:-80));
A = U*S*V;
```

(c) Calcule la factorización QR de A utilizando los tres algoritmos mencionados. Para cada uno calcule $\|A - QR\|_2$ y $\|Q'Q - I\|_2$. Comente sus resultados.¿Cuál algoritmo es mejor? Compare sus resultados con el comando de `Matlab` $[q, r] = qr(A)$.

Ejercicio 5.62 (`MATLAB`)**.** (Compresión de imágenes) En este ejercicio utilizamos la descomposición en valores singulares para comprimir imágenes de una manera muy básica. Una imagen de dimensión $m \times n$ consiste de mn píxeles (la resolución se refiere al número de píxeles utilizados). En una imagen a color, cada píxel contiene tres valores enteros $(r_p, g_p, b_p) \in [0, 255]^3$, que corresponden a los componentes rojo, verde y azul. De esta forma, la imagen es guardada como una matriz de tamaño $m \times n \times 3$.

(a) (Leyendo la imagen) Considere una imagen `'photo.jpg'`. Para leer la imagen ejecute el comando `RGB = imread('photo.jpg')` (*image read*). ¿Cuántos píxeles tiene la imagen guardada en `RGB`? ¿Qué tipo de entradas tiene dicha matriz? Sugerencia: imprima el comando `whos RGB` y comente el resultado.

(b) (Convirtiendo la imagen a formato doble) Para poder utilizar la DVS, convierta la matriz `RGB` a formato doble mediante el comando `RGB = im2double(RGB)`.

(c) (Comprimiendo la imagen) Calcule la descomposición en valores singulares de `RGB` $= USV^T$ (para cada matriz de color rojo, verge y azul). Calcule la aproximación de rango 5 dada por

$$A_5 = \sum_{i=1}^{5} \sigma_i u_i v_i^T.$$

En una misma ventana (utilizando el comando subplot) muestre la imagen original, la imagen guardada en A_5 y el error $|\texttt{RGB} - A_5|$. Comente sus resultados. Sugerencia: el comando `imshow(RGB)` muestra la imagen dada den `RGB`.

(d) Para cada matriz `RGB(:,:,i)`, determine el número r_i de valores singulares σ_k tales que $\sigma_k > 0.01\sigma_1$ y tome $R_1 = \text{máx}\{r_i\}$. Para este valor de R_1 calcule la mejor aproximación A_{R_1} de rango R_1 de la matriz `RGB`. Repita para $\sigma_k > 0.005\sigma_1$; denote este nuevo rango R_2 y la respectiva aproximación A_{R_2}. Dibuje en una misma ventana ambas aproximaciones. Comente las diferencias entre ambas imágenes. Sugerencia: el comando `diag(S)` extrae los elementos de la diagonal de la matriz `S`.

(e) En práctica, uno debe guardar solamente los valores y vectores singulares para ahorrar espacio. Determine el tamaño en memoria que utiliza la matriz A_{R_2} mediante la función `whos`. Compare con el tamaño requerido para guardar los valores y vectores singulares necesarios para calcular A_{R_2}.

(f) Finalmente guarde la imagen A_{R_2} como archivos `bmp` y `jpg` mediante los comandos

`imwrite(A_R2,'photo_comp.bmp')` y `imwrite(A_R2,'photo_comp.jpg')`.

Compare el tamaño de estos dos archivos con el tamaño original de la foto inicial. (Note que el formato `jpg` por defecto comprime la imagen y está optimizado para tal fin).

Ejercicio 5.63. [`MATLAB`] (Matriz de Vandermonde) Considere el problema de interpolación de Lagrange dado por: Encontrar un polinomio p de grado a lo sumo n cuya gráfica incluya los puntos $\{(x_i, y_i)\}_{i=0}^n$ (el cual sabemos tiene solución única). En este ejercicio plantearemos y resolveremos este problema mediante un sistemas de ecuaciones. Escriba

$$p(x) = \sum_{k=0}^{n} a_k x^k.$$

El problema de interpolar es equivalente a encontrar los coeficientes $\{a_k\}$ que satisfcen las $n+1$ ecuaciones

$$p(x_i) = \sum_{k=0}^{n} a_k x_i^k = y_i \quad \forall\ i = 0, \ldots, n,$$

el cual se puede escribir matricialmente de la forma $A\boldsymbol{x} = \boldsymbol{b}$, donde

$$\boldsymbol{x} = \begin{bmatrix} a_0 \\ \vdots \\ a_n \end{bmatrix}, \quad \boldsymbol{b} = \begin{bmatrix} y_0 \\ \vdots \\ y_n \end{bmatrix},$$

y A es la Matriz de Vandermonde

$$A = \begin{bmatrix} 1 & x_0 & \ldots & x_0^n \\ 1 & x_1 & \ldots & x_1^n \\ \vdots & & \ddots & \vdots \\ 1 & x_n & \ldots & x_n^n \end{bmatrix} \in \mathbb{R}^{n+1 \times n+1}$$

En este ejercicio utilizaremos nodos equidistantes en el intervalo $[-1, 1]$; esto es,

$$x_i = -1 + \frac{2i}{n} \quad (i = 0, \ldots, n).$$

(a) Demuestre que A es invertible.

(b) Para $n = 40$, construya la matriz A. Calcule el número de condición de A, su rango y su determinante. ¿Es la matriz invertible? Sugerencia: puede utilizar los comandos `vander, rank, det, cond` de `MATLAB`.

(c) Para resolver el sistema de ecuaciones $A\boldsymbol{x} = \boldsymbol{b}$ en `MATLAB`, basta ejecutar el comando $\widetilde{\boldsymbol{x}} =$ `A\b`. Considere $f(x) = (1 + 25x^2)^{-1}$. Estime $\|f(\texttt{xx}) - p(\texttt{xx})\|_\infty$, donde `xx = linspace(-1,1,1e3)`.

(d) Calcule la Descomposición en Valores Singulares de A mediante el comando

`[U,S,V] = svd(A)`.

De esta manera,

$$\widetilde{\boldsymbol{x}} = VS^{-1}U^T b.$$

Estime nuevamente $\|f(\texttt{xx}) - p(\texttt{xx})\|_\infty$. ¿Existe alguna mejora en la aproximación de la solución? Plotee los valores singulares de A en una escala semilogarítmica. Comente el comportamiento. ¿Cuál es el rango numérico de la matriz? Sugerencia: la función `diag` puede ser de utilidad.

(e) Considere una matriz con descomposición en valores singulares $A = USV^T$. La *pseudoinversa* se define como

$$A^+ = VS^+U^T,$$

donde

$$S^+ = \begin{bmatrix} 1/\sigma_1 & & & \\ & 1/\sigma_2 & & \\ & & \ddots & \\ & & & 1/\sigma_p \\ & & & \end{bmatrix}$$

es la matriz diagonal obtenida de invertir los valores singulares positivos de S. Defina un vector de tolerancias `tol = 10.^(-linspace(2,17))`. Para cada entrada `j` de `tol`, defina S_j como la matriz obtenida a partir

de S, reemplazando los valores singulares menores que `tol(j)*`σ_1 por cero. Calcule

$$\texttt{x_svd} = (VS_j^+U^T)b$$

y calcule la norma $\|f(\texttt{xx})-p(\texttt{xx})\|_\infty$. Note que estamos aproximando la pseudoinversa con solamente algunos valores singulares de A. Grafique el error en función del vector de tolerancias. Comente sus resultados y explique el comportamiento del gráfico.

(f) Para un valor de tolerancia que minimice el error del inciso anterior, plotee en un mismo gráfico `f(xx)`, `p(xx)` y los nodos de interpolación. ¿Qué tan buena es la aproximación?

(g) Ahora utilice `tol = 1e-5`. Plotee en un mismo gráfico `f(xx)`, `p(xx)` y los nodos de interpolación. ¿Se observa el fenónemo de Runge? Explique el porqué.

Ejercicio 5.64. Dada $A = U\Sigma V^T \in \mathbb{R}^{m\times n}$ una descomposición en valores singulares de A, defina su pseudoinversa como $A^+ = V\Sigma^+U^T$, donde Σ^+ se obtiene de Σ al invertir sus entradas no nulas; esto es$(\Sigma^+)_{ii} = 1/\sigma_{ii}$ si $\sigma_{ii} > 0$. Demuestre que si $\boldsymbol{x}$ está en el espacio fila de A y $\boldsymbol{y} = A\boldsymbol{x}$ entonces $\boldsymbol{x} = A^+\boldsymbol{y}$.

Ejercicio 5.65. Para graficar una función en dos variables $z = f(x, y)$, se puede generar una malla (matriz) de puntos `xx,yy` en las cuales se evalúa f. De esta forma, `zz = f(xx,yy)` es una matriz del mismo tamaño de `xx` y `yy`.

(a) Defina los vectores

`x = linspace(0,1,100)`, `y = linspace(0,2,200)`.

La malla de puntos equidistantes en el dominio $D = [0, 1]\times[0, 2]$ se genera mediante el comando `[xx,yy]=meshgrid(x,y)` (note que `xx` y `yy` son matrices). Grafique la función $f(x, y) = \sin(2\pi(x+y))\sin(\pi(x-y))$ en esta malla del dominio D. Sugerencia: utilice el comando `surf(xx,yy,f(xx,yy))`.

(b) Defina `zz = f(xx,yy)`, donde f es la función del inciso anterior. Calcule la descomposición en valores singulares de `zz`. Mediante el comando `subplot`, grafique en una misma ventana las mejores aproximaciones de rango 1,2,3 y 4. En una nueva ventana, grafique el error absoluto para cada una de las cuatro aproximaciones. ¿Qué observa?

(c) ¿Cuáles son los valores singulares y el rango de `zz`?

(d) Demuestre (a mano) que la matriz tiene el rango y los valores singulares correspondientes. ¿Tiene sentido construir la matriz `zz`?

Capítulo 6

Condición y estabilidad

6.1. Introducción

Considere un problema como una función $f : X \to Y$ entre dos espacios vectoriales normados. El espacio X representa los datos de entrada y el espacio Y es el espacio de soluciones. En este capítulo contestaremos dos preguntas fundamentales:

1. Es natural tener errores de redondeo en los datos de entrada x de un algoritmo, al menos. ¿Podemos medir cuánto se amplifican estos errores en la ejecución del algoritmo al calcular los valores de salida $f(x)$? Esto lo relacionaremos con el *número de condición* del problema.

2. Por la naturaleza discreta del sistema en punto flotante, el problema que resolvemos numéricamente es diferente al problema exacto. ¿Es posible que ambos resultados estén lo suficientemente *cerca*? Esto lo relacionaremos con la *estabilidad* del algoritmo, y particularmente consideraremos la *estabilidad hacia atrás*.

En el caso particular del problema de resolver sistemas de ecuaciones lineales, podremos obtener una cota para el error relativo del valor exacto y el valor numérico obtenido según los algoritmos de factorización de matrices discutidos en el Capítulo 5. Referimos al lector interesado a profundizar en estos temas en [29, Sección 3].

6.2. Número de condición

Empezamos definiendo formalmente el número de condición de f, el cual mencionamos brevemente en el fenómeno del ejemplo de la Sección 1.6.2:

Definición 6.1. Denote por δx una pequeña perturbación en x, y escriba $\delta f = f(x+\delta x) - f(x)$. El *número de condición absoluto de* f se define como

$$\hat{\kappa}(x) = \hat{\kappa} := \lim_{\delta \to 0} \sup_{\|\delta x\| < \delta} \frac{\|\delta f\|}{\|\delta x\|}.$$

Si f es diferenciable, entonces podemos reducir la definición anterior a

$$\hat{\kappa} = \|J(x)\|,$$

donde $J(x)$ es el Jacobiano de f.

Note que $\hat{\kappa}(x)$ es una medida de cuánto cambia la salida al tener pequeños cambios en la entrada x. No obstante, es usual medir dichos cambios de forma relativa:

Definición 6.2. El número de condición relativo se define como el cociente

$$\kappa(x) := \lim_{\delta \to 0} \sup_{\|\delta x\| < \delta} \frac{\|\delta f\|}{\|f(x)\|} \frac{\|x\|}{\|\delta x\|}.$$

Si f es diferenciable, entonces

$$\kappa = \frac{\|J(x)\|}{\|f(x)\|/\|x\|}.$$

Por simplicidad escribiremos $\kappa = \kappa(x)$, pero debe ser claro que el número de condición depende de la entrada x. En general, diremos que un problema f ese *bien condicionado* si cambios pequeños en $x \in X$ implican cambios pequeños en la solución (κ pequeño). En caso contrario, diremos que el problema es *mal condicionado*. Es claro que al decir "cambios pequeños en la solución", el orden del cambio es relativo con respecto a diversos factores y dependerá de la aplicación en particular.

Ejemplo 6.3. Para $f(x) = \sqrt{x}$, es fácil ver que $\kappa = 1/2$. En este caso, buscar la raíz cuadrada de un número real es un problema bien condicionado. □

Ejemplo 6.4. Para $f(\boldsymbol{x}) = x_1 - x_2$, con $\boldsymbol{x} = [x_1, x_2]^T \in \mathbb{R}^2$ y $\|\cdot\|_\infty$, es fácil ver que $J(x) = [\frac{\partial f}{\partial x_1} \frac{\partial f}{\partial x_2}] = [1 \; -1]$.. Luego, $\|J(x)\|_\infty = 2$. Así,

$$\kappa = \frac{2\,\text{máx}\{x_1, x_2\}}{|x_1 - x_2|}.$$

De esta forma, el problema de restar dos números es mal condicionado si x_1 y x_2 son de un orden similar (tal como lo mencionamos en el Capítulo 1 al hablar de errores de cancelación). □

Ejemplo 6.5. Fije una matriz $A \in \mathbb{C}^{m\times m}$ invertible. Considere el problema de calcular el producto $A\boldsymbol{x}$ donde la entrada es $\boldsymbol{x}$ (consideramos perturbaciones $\boldsymbol{\delta x} \in \mathbb{C}^m$ para $\boldsymbol{x}$). De esta forma,

$$\kappa = \sup_{\delta x} \frac{\|A(\boldsymbol{x}+\boldsymbol{\delta x}) - A\boldsymbol{x}\|}{\|A\boldsymbol{x}\|} \frac{\|\boldsymbol{x}\|}{\|\boldsymbol{\delta x}\|} = \|A\| \frac{\|\boldsymbol{x}\|}{\|A\boldsymbol{x}\|}.$$

Por definición,

$$\|A^{-1}\| = \sup_{\boldsymbol{y}\neq \underline{0}} \frac{\|A^{-1}\boldsymbol{y}\|}{\|\boldsymbol{y}\|} \geqslant \frac{\|\boldsymbol{x}\|}{\|A\boldsymbol{x}\|}$$

tomando $\boldsymbol{y} = A\boldsymbol{x}$. De esta forma,

$$\kappa \leqslant \|A\|\|A^{-1}\|.$$

En particular, si tomamos $\|\cdot\|_2$, la igualdad anterior se cumple si $\boldsymbol{x}$ es múltiplo del vector singular derecho asociado al menor valor singular. □

Ejemplo 6.6. Fije una matriz $A \in \mathbb{C}^{m\times m}$ invertible. Considere ahora el problema de encontrar $\boldsymbol{x}$ tal que $A\boldsymbol{x} = \boldsymbol{b}$, donde $\boldsymbol{b}$ es fijo. Esto es, queremos calcular $\boldsymbol{x} = A^{-1}\boldsymbol{b}$. Siguiendo el razonamiento del ejemplo anterior, tenemos que

$$\kappa \leqslant \|A\|\|A^{-1}\|.$$

En particular, si tomamos $\|\cdot\|_2$, la igualdad anterior se cumple si $\boldsymbol{b}$ es múltiplo del vector singular izquierdo asociado al menor valor propio. □

Definición 6.7. Dada una matriz $A \in \mathbb{C}^{m\times m}$, se define el número de condición de la matriz como

$$\kappa(A) := \|A\|\|A^{-1}\|.$$

Si A no es invertible, es usual escribir $\kappa(A) = \infty$.

Ejercicio 6.8. *Si utilizamos la 2-norma, demuestre que* $\kappa(A) = \frac{\sigma_1}{\sigma_m}$, *donde* σ_1 *y* σ_m *son el mayor y menor valor singular de* A.

Comentario 6.9. En general, si el número de condición de A es alto, al resolver el problema $A\boldsymbol{x} = \boldsymbol{b}$ se espera obtener grandes perturbaciones en la salida (la solución $\boldsymbol{x}$) sin importar las perturbaciones en la entrada (A y $\boldsymbol{b}$). De esta forma, el resultado no será preciso, tal como ocurre en la Sección 1.6.2. Esto lo discutiremos con más detalle al hablar de estabilidad. □

6.3. Estabilidad

Considere un algoritmo $\widetilde{f} : X \to Y$. El error relativo de la solución obtenida $\tilde{y} = \widetilde{f}(x)$ es

$$\frac{\|\widetilde{f}(x) - f(x)\|}{\|f(x)\|}.$$

De esta forma, diremos que un algoritmo es *preciso* si

$$\frac{\|\widetilde{f}(x) - f(x)\|}{\|f(x)\|} = \mathcal{O}(\epsilon_m).$$

No obstante, si f es mal condicionado, en general no es posible hablar de precisión (al menos tendremos perturbaciones por redondeo en la entrada). En su lugar, hablamos de estabilidad:

Definición 6.10. Se dice que el algoritmo $\widetilde{f}$ para resolver f es *estable* si para todo $x \in X$,

$$\frac{\|\widetilde{f}(x) - f(\widetilde{x})\|}{\|f(\widetilde{x})\|} = \mathcal{O}(\epsilon_m)$$

para algún $\widetilde{x}$ con

$$\frac{\|\widetilde{x} - x\|}{\|x\|} = \mathcal{O}(\epsilon_m).$$

En palabras sencillas, un algoritmo estable da casi la respuesta correcta a casi el mismo problema.

Definición 6.11. Se dice que el algoritmo $\widetilde{f}$ para resolver f es *estable hacia atrás* si para todo $x \in X$,

$$\widetilde{f}(x) = f(\widetilde{x})$$

para algún $\widetilde{x}$ con

$$\frac{\|\widetilde{x} - x\|}{\|x\|} = \mathcal{O}(\epsilon_m).$$

En palabras sencillas, un algoritmo estable hacia atrás da la respuesta correcta a casi el mismo problema.

Antes de considerar algunos ejemplos, demostramos el siguiente teorema:

Teorema 6.12 (Precisión de un algoritmo estable hacia atrás)**.** *Suponga que $\widetilde{f}$ es un algoritmo estable hacia atrás para resolver $f : X \to Y$. Sea κ el número de condición de f. Entonces, el error relativo satisface*

$$\frac{\|\widetilde{f}(x) - f(x)\|}{\|f(x)\|} = \mathcal{O}(\kappa(x)\epsilon_m).$$

Demostración. De la definición de estabilidad hacia atrás sabemos que existe $\widetilde{x}$ con

$$\frac{\|\widetilde{x} - x\|}{\|x\|} = \mathcal{O}(\epsilon_m),$$

tal que $\widetilde{f}(x) = f(\widetilde{x})$. Tome $\delta x = \widetilde{x} - x$ en la definición del número de condición. Así,

$$\kappa > \frac{\|f(\widetilde{x}) - f(x)\|}{\|f(x)\|} \frac{\|x\|}{\|\widetilde{x} - x\|}.$$

Luego, como $\widetilde{f}(x) = f(\widetilde{x})$, concluimos el resultado. □

Comentario 6.13. Como consecuencia, si el problema es bien condicionado, esperamos tener una solución precisa. Por el contrario, si $\kappa \approx 10^m$, esperamos perder en general m dígitos de precisión en la respuesta. □

A continuación mostramos dos casos del Algoritmo 13 que permiten demostrar que es estable hacia atrás. Dichas pruebas escapan a los objetivos del curso, por lo que posteriormente solo enunciaremos resultados similares sobre las factorizaciones discutidas en el Capítulo 5.

Teorema 6.14. *Sea $R \in \mathbb{C}^{m\times m}$ una matriz triangular superior invertible. Considere el método de sustitución hacia atrás dado en el Algoritmo 13 para resolver el sistema $R\boldsymbol{x} = \boldsymbol{b}$. Dicho algoritmo es estable hacia atrás; específicamente, existe una matriz triangular superior $\delta R \in \mathbb{C}^{m\times m}$ tal que*

$$(R + \delta R)\widetilde{\boldsymbol{x}} = \boldsymbol{b},$$

con

$$\frac{\|\delta R\|}{\|R\|} = \mathcal{O}(\epsilon_m).$$

En particular,

$$\frac{|\delta r_{ij}|}{|r_{ij}|} \leqslant m\epsilon_m + \mathcal{O}(\epsilon_m^2).$$

Demostración. Por simplicidad y brevedad presentamos solo el caso $m = 1$ y $m = 2$. Note que la idea en general es expresar el error en la aproximación sobre la matriz de entrada R.

Considere primero que $m = 1$. En este caso solo debemos resolver

$$\widetilde{x}_1 = b_1 \oslash r_{11}.$$

Del Teorema 1.3, podemos deducir que

$$\widetilde{x}_1 = \frac{b_1}{r_{11}}(1+\varepsilon_1),\ \text{con}\ |\varepsilon_1| \leqslant \epsilon_m.$$

Defina $\varepsilon_1' := -\varepsilon_1/(1+\varepsilon_1)$. Entonces, $1+\varepsilon_1' = 1/(1+\varepsilon_1)$, y así

$$\widetilde{x}_1 = \frac{b_1}{r_{11}(1+\varepsilon_1')},\ \text{con}\ |\varepsilon_1'| \leqslant \epsilon_m + \mathcal{O}(\epsilon_m^2).$$

De esta forma,

$$(r_{11}+\delta r_{11})\widetilde{x}_1 = b_1,\ \text{con}\ \ \delta r_{11} = \varepsilon_1' r_{11},$$

de donde deducimos el resultado para $m=1$.

Para $m=2$, primero debemos obtener

$$\widetilde{x}_2 = b_2 \oslash r_{22} = \frac{b_2}{r_{22}(1+\varepsilon_1)},\ \text{con}\ |\varepsilon_1| \leqslant \epsilon_m + \mathcal{O}(\epsilon_m^2)$$

tal como obtuvimos para el caso $m=1$. Luego, debemos calcular

$$\begin{aligned}
\widetilde{x}_1 &= (b_1 \ominus (\widetilde{x}_2 \otimes r_{12})) \oslash r_{11} \\
&= (b_1 \ominus (\widetilde{x}_2 r_{12}(1+\varepsilon_2)) \oslash r_{11} \\
&= (b_1 - \widetilde{x}_2 r_{12}(1+\varepsilon_2))(1+\varepsilon_3) \oslash r_{11} \\
&= \frac{(b_1 - \widetilde{x}_2 r_{12}(1+\varepsilon_2))(1+\varepsilon_3)}{r_{11}}(1+\varepsilon_4).
\end{aligned}$$

Defina $\varepsilon_3' := -\varepsilon_3/(1+\varepsilon_3)$, $\varepsilon_4' := -\varepsilon_4/(1+\varepsilon_4)$. De esta forma,

$$\begin{aligned}
\widetilde{x}_1 &= \frac{b_1 - \widetilde{x}_2 r_{12}(1+\varepsilon_2)}{r_{11}(1+\varepsilon_3')(1+\varepsilon_4')} \\
&= \frac{b_1 - \widetilde{x}_2 r_{12}(1+\varepsilon_2)}{r_{11}(1+2\varepsilon_5)},
\end{aligned}$$

con $|\varepsilon_2| \leqslant \epsilon_m$, $|\varepsilon_3'|, |\varepsilon_4'|, |\varepsilon_5| \leqslant \epsilon_m + \mathcal{O}(\epsilon_m^2)$. De esta forma,

$$(R+\delta R)\widetilde{\boldsymbol{x}} = \boldsymbol{b},$$

donde

$$\delta R = \begin{bmatrix} \delta r_{11} & \delta r_{12} \\ 0 & \delta r_{22} \end{bmatrix} = \begin{bmatrix} 2\varepsilon_5 r_{11} & \varepsilon_2 r_{12} \\ 0 & \varepsilon_1 r_{22} \end{bmatrix}.$$

De esta forma, podemos verificar el resultado. □

A continuación enunciamos la estabilidad hacia atrás de otros algoritmos que hemos estudiado hasta el momento. Por simplicidad, omitimos sus pruebas.

Teorema 6.15. *Sea $A = QR \in \mathbb{C}^{m\times n}$. Si $\widetilde{Q}$ y $\widetilde{R}$ son las matrices obtenidas del Algoritmo 7 (con Householder), entonces existe $\delta A \in \mathbb{C}^{m\times n}$ tal que*

$$\widetilde{Q}\widetilde{R} = A + \delta A, \ \ con \ \ \frac{\|\delta A\|}{\|A\|} = \mathcal{O}(\epsilon_m).$$

Teorema 6.16. *Sea $\boldsymbol{y} = Q^*\boldsymbol{b}$. Si $\widetilde{\boldsymbol{y}}$ es el vector producido por el Algoritmo 8, entonces existe $\delta Q \in \mathbb{C}^{m\times n}$ tal que*

$$(Q + \delta Q)\widetilde{\boldsymbol{y}} = \boldsymbol{b}, \ \ con \ \ \|\delta Q\| = \mathcal{O}(\epsilon_m).$$

Teorema 6.17. *Sea $A = R^*R \in \mathbb{C}^{m\times m}$ la factorización de Cholesky de A. Si $\widetilde{R}$ es la matriz obtenida del Algoritmo 12, entonces existe $\delta A \in \mathbb{C}^{m\times m}$ tal que*

$$\widetilde{R}^*\widetilde{R} = A + \delta A, \ \ con \ \ \frac{\|\delta A\|}{\|A\|} = \mathcal{O}(\epsilon_m).$$

Teorema 6.18. *Sea $A \in \mathbb{C}^{m\times m}$, donde $PA = LU$ es la factorización LU de A. Si $\widetilde{L}$, $\widetilde{U}$ y $\widetilde{P}$ son las matrices obtenidas del Algoritmo 11 (con pivoteo parcial), entonces existe $\delta A \in \mathbb{C}^{m\times m}$ tal que*

$$\widetilde{L}\widetilde{U} = \widetilde{P}A + \delta A, \ \ con \ \ \frac{\|\delta A\|}{\|A\|} = \mathcal{O}(\rho\epsilon_m),$$

donde ρ es el factor de crecimiento de A, definido como

$$\rho(A) = \frac{\max_{i,j} |u_{ij}|}{\max_{i,j} |a_{ij}|} \tag{6.1}$$

Comentario 6.19. Considere la matriz $A \in \mathbb{R}^{m\times m}$ dada por

$$A = \begin{bmatrix} 1 & & & & & 1 \\ -1 & 1 & & & & 1 \\ -1 & -1 & 1 & & & 1 \\ \vdots & \vdots & \ddots & \ddots & & \vdots \\ -1 & -1 & \dots & -1 & 1 & 1 \\ -1 & -1 & \dots & -1 & -1 & 1 \end{bmatrix},$$

donde las entradas de la diagonal y la última columna son 1, y las entradas debajo de la diagonal son -1. Al aplicar eliminación Gaussiana, se obtiene que

$$U = \begin{bmatrix} 1 & & & & & 1 \\ & 1 & & & & 2 \\ & & 1 & & & 4 \\ & & & \ddots & & \vdots \\ & & & & 1 & 2^{m-2} \\ & & & & & 2^{m-1} \end{bmatrix}.$$

De esta forma $\rho = 2^{m-1}$, y el estimado del teorema anterior es del orden $2^{m-1}\epsilon_m$. Este crecimiento implica una pérdida de $m-1$ bits de precisión. No obstante, en la práctica al resolver problemas físicos de interés, dichos factores de crecimiento no son un problema en general.

6.4. Error al resolver sistemas de ecuaciones

Suponga que queremos resolver el sistema $A\boldsymbol{x} = \boldsymbol{b}$ (con $A \in \mathbb{C}^{m\times m}$ invertible) con el siguiente algoritmo:

(a) Obtenga la factorización $A = QR$ mediante el Algoritmo 7 (Householder).
(b) Calcule $\boldsymbol{y} = Q^*\boldsymbol{b}$ mediante el Algoritmo 8.
(c) Resuelva $\boldsymbol{x} = R^{-1}\boldsymbol{y}$ mediante el Algoritmo 13 (sustitución hacia atrás).

Dicho algoritmo es estable hacia atrás, tal como lo demostramos a continuación:

Teorema 6.20. *Dados $A \in \mathbb{C}^{m\times m}$ invertible y $\boldsymbol{b} \in \mathbb{C}^m$, considere el problema de resolver el sistema $A\boldsymbol{x} = \boldsymbol{b}$ mediante el algoritmo descrito anteriormente. Existe $\Delta A \in \mathbb{C}^{m\times m}$ tal que*

$$(A + \Delta A)\widetilde{\boldsymbol{x}} = \boldsymbol{b}, \text{ con } \frac{\|\Delta A\|}{\|A\|} = \mathcal{O}(\epsilon_m),$$

donde $\widetilde{\boldsymbol{x}}$ es la solución numérica del algoritmo.

Demostración. De la descripción del algoritmo, tenemos que

(a) Existe $\delta A \in \mathbb{C}^{m\times n}$ tal que

$$\widetilde{Q}\widetilde{R} = A + \delta A, \text{ con } \frac{\|\delta A\|}{\|A\|} = \mathcal{O}(\epsilon_m).$$

(b) Existe $\delta Q \in \mathbb{C}^{m\times n}$ tal que

$$(\widetilde{Q} + \delta Q)\widetilde{\boldsymbol{y}} = \boldsymbol{b}, \text{ con } \|\delta Q\| = \mathcal{O}(\epsilon_m).$$

(c) Existe una matriz triangular superior $\delta R \in \mathbb{C}^{m\times m}$ tal que

$$(\widetilde{R} + \delta R)\widetilde{\boldsymbol{x}} = \widetilde{\boldsymbol{y}},$$

con

$$\frac{\|\delta R\|}{\|\widetilde{R}\|} = \mathcal{O}(\epsilon_m).$$

Así,

$$\begin{aligned}\boldsymbol{b} &= (\widetilde{Q} + \delta Q)\widetilde{\boldsymbol{y}} = (\widetilde{Q} + \delta Q)(\widetilde{R} + \delta R)\widetilde{\boldsymbol{x}}\\ &= (\widetilde{Q}\widetilde{R} + \delta Q\widetilde{R} + \widetilde{Q}\delta R + \delta Q\delta R)\widetilde{\boldsymbol{x}}\\ &= (A + \Delta A)\widetilde{\boldsymbol{x}},\end{aligned}$$

donde $\Delta A := \delta A + +\delta Q\widetilde{R} + \widetilde{Q}\delta R + \delta Q\delta R$. Como $\widetilde{Q}\widetilde{R} = A + \delta A$ y $\widetilde{Q}$ es unitaria, tenemos que

$$\frac{\|\widetilde{R}\|}{\|A\|} \leqslant \|\widetilde{Q}\|\frac{\|A + \delta A\|}{\|A\|} = \mathcal{O}(1).$$

Así,

$$\begin{aligned}\frac{\|\delta A\|}{\|A\|} &= \mathcal{O}(\epsilon_m)\\ \frac{\|(\delta Q)\widetilde{R}\|}{\|A\|} &\leqslant \|\delta Q\|\frac{\|\widetilde{R}\|}{\|A\|} \leqslant \mathcal{O}(\epsilon_m)\\ \frac{\|\widetilde{Q}(\delta R)\|}{\|A\|} &\leqslant \|\widetilde{Q}\|\frac{\|\delta R\|}{\|\widetilde{R}\|}\,\frac{\|\widetilde{R}\|}{\|A\|} \leqslant \mathcal{O}(\epsilon_m)\\ \frac{\|(\delta Q)(\delta R)\|}{\|A\|} &\leqslant \|\delta Q\|\frac{\|\delta R\|}{\|A\|} \leqslant \mathcal{O}(\epsilon_m^2).\end{aligned}$$

Por lo tanto ΔA satisface el teorema. □

De esta forma, podemos concluir el siguiente resultado:

Teorema 6.21. *La solución $\widetilde{\boldsymbol{x}}$ del algoritmo anterior satisface que*

$$\frac{\|\boldsymbol{x} - \widetilde{\boldsymbol{x}}\|}{\|\boldsymbol{x}\|} = \mathcal{O}(\kappa(A)\epsilon_m).$$

Demostración. La prueba es consecuencia del Teorema 6.12 y del Teorema 6.20. □

Comentario 6.22. El teorema anterior es válido si utilizamos la factorización de Cholesky o la factorización en valores singulares (a pesar que no hemos mencionado el algoritmo con el que se produce dicha factorización). En el caso del algoritmo de eliminación Gaussiana, el factor de crecimiento ρ se hace presente en la cota del error, el cual puede tener efectos significativos. No obstante, como se mencionó en el Comentario 6.19, en la mayoría de los casos ρ es pequeño y no trae consecuencias importantes al utilizar eliminación Gaussiana con pivoteo. □

Concluimos entonces enunciando el Teorema 6.21 en su forma más general:

Teorema 6.23. *Al resolver el sistema de ecuaciones $A\boldsymbol{x} = \boldsymbol{b}$ (ya sea mediante la factorización en valores singulares, QR o Cholesky), la solución $\widetilde{\boldsymbol{x}}$ satisface que*

$$\frac{\|\boldsymbol{x} - \widetilde{\boldsymbol{x}}\|}{\|\boldsymbol{x}\|} = \mathcal{O}(\kappa(A)\epsilon_m).$$

En el caso de la factorización LU con pivoteo, se tiene que

$$\frac{\|\boldsymbol{x} - \widetilde{\boldsymbol{x}}\|}{\|\boldsymbol{x}\|} = \mathcal{O}(\rho\kappa(A)\epsilon_m).$$

6.5. Ejercicios

Ejercicio 6.24 (`MATLAB`)**.** Sea $A \in \mathbb{R}^{m\times m}$ la matriz de Hilbert. Utilice el comando `cond(A)` en `MATLAB` para calcular el número de condición (en la norma 2) de A. Grafique `cond(A)` en función de m. ¿Qué indica entonces el Teorema 6.21?

Ejercicio 6.25. Demuestre que el factor de crecimiento definido en (6.1) (al utilizar eliminación Gaussiana con pivoteo) satisface $\rho \leqslant 2^{m-1}$ para cualquier matriz $A \in \mathbb{C}^{m\times m}$.

Ejercicio 6.26. Sean $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{C}^m$. Demuestre que calcular el producto interno $\boldsymbol{x}^T\boldsymbol{y}$ es estable hacia atrás.

Ejercicio 6.27. Sean $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{C}^m$. Demuestre que calcular el producto externo $\boldsymbol{x}\boldsymbol{y}^T$ no es estable hacia atrás. (Sugerencia: el producto tiene rango 1).

Ejercicio 6.28. Considere el Teorema 6.15. ¿Se puede concluir algo sobre la norma $\|Q - \widetilde{Q}\|$? ¿Y sobre la norma $\|R - \widetilde{R}\|$?

Ejercicio 6.29 (`MATLAB`)**.** (Efectos del factor de crecimiento)

(a) Escriba una función que reciba como entrada $m \in \mathbb{N}$ y devuelva la matriz $A \in \mathbb{R}^{m\times m}$ dada por

$$A = \begin{bmatrix} 1 & & & & & 1 \\ -1 & 1 & & & & 1 \\ -1 & -1 & 1 & & & 1 \\ \vdots & \vdots & \ddots & \ddots & & \vdots \\ -1 & -1 & \dots & -1 & 1 & 1 \\ -1 & -1 & \dots & -1 & -1 & 1 \end{bmatrix},$$

donde las entradas de la diagonal y la última columna son 1, y las entradas debajo de la diagonal son -1.

(b) Para $m = 50$, calcule el factor de crecimiento de A,

$$\rho(A) = \frac{\text{máx}\,|u_{ij}|}{\text{máx}\,|a_{ij}|},$$

y la norma $\|A - LU\|_2$. Comente sobre los resultados.

(c) Para cada `m = 10:60`, defina `xExact = rand(m,1)` y calcule

`b = A * xExact`.

Resuelva el sistema `A * xApr = b` mediante la factorización LU y la factorización QR (utilice los comandos `lu` y `qr` de `Matlab`). Obtenga una gráfica de $\|$`xExact - xApr`$\|_2$ en función de m. Justifique el comportamiento del gráfico.

Ejercicio 6.30 (`MATLAB`)**.** Defina $A \in \mathbb{C}^{m\times m}$ como una matriz aleatoria con entradas de una distribución normal con media cero y desviación estándar $m^{-1/2}$. Para ello, basta definir `A = randn(m,m)/sqrt(m)` en `Matlab`. Obtenga el factor de crecimiento para A. Repita este experimento para `m = 2.^(2:8)`. Repita 5 veces para dichos valores de m y grafique ρ en función de m. Grafique además la función $\sqrt{m}$. ¿Qué observa en el gráfico? (Recuerde que en general $\rho \leqslant 2^{m-1}$).

Capítulo 7

Aplicaciones de sistemas de ecuaciones

En este capítulo discutiremos algunas aplicaciones en donde se requiere resolver sistemas de ecuaciones lineales. En el siguiente capítulo también veremos algunos casos en donde al resolver ecuaciones diferenciales ordinarias es necesario resolver sistemas lineales de ecuaciones.

7.1. Problemas de mínimos cuadrados

En esta sección seguimos la exposición de [29]. Considere una matriz $A \in \mathbb{C}^{m\times n}$, con $m > n$. Dado $\boldsymbol{b} \in \mathbb{C}^m$, encontrar $\boldsymbol{x} \in \mathbb{C}^n$ tal que $A\boldsymbol{x} = \boldsymbol{b}$ es posible si y solo si $\boldsymbol{b} \in \text{Rang}(A)$, debido a que tenemos un sistema sobredeterminado con más ecuaciones que incógnitas. De esta manera, definimos el problema de mínimos cuadrados de la siguiente manera: encontrar $\boldsymbol{x} \in \mathbb{C}^m$ tal que

$$\|\boldsymbol{r}\|_2 = \|A\boldsymbol{x} - \boldsymbol{b}\|_2 \to \text{mín}$$

donde $\boldsymbol{r}$ es llamado *residuo*. Para un estudio completo en este tema, ver [1]. Tenemos el siguiente teorema que caracteriza la existencia y unicidad:

Teorema 7.1. *Sean $A \in \mathbb{C}^{m\times n}$ $(m > n)$ y $\boldsymbol{b} \in \mathbb{C}^m$ dados. Un vector $\boldsymbol{x} \in \mathbb{C}^n$ minimiza el residuo*

$$\|\boldsymbol{r}\|_2 = \|A\boldsymbol{x} - \boldsymbol{b}\|_2$$

si y solo si $P\boldsymbol{b} = A\boldsymbol{x}$ donde $P \in \mathbb{C}^{m\times m}$ es el proyector ortogonal sobre $\text{Rang}(A)$.

Demostración. Veamos que $\boldsymbol{y} := P\boldsymbol{b} \in \text{Rang}(A)$ es el único punto que minimiza la norma del residuo. Sea $\boldsymbol{z} \in \text{Rang}(A)$ con $\boldsymbol{z} \neq \boldsymbol{y}$. Como $\boldsymbol{z} - \boldsymbol{y}$ es ortogonal a $\boldsymbol{b} - \boldsymbol{y}$, tenemos que para cualquier $\boldsymbol{z} \neq \boldsymbol{y}$,

$$\|\boldsymbol{b} - \boldsymbol{z}\|_2^2 = \|\boldsymbol{b} - \boldsymbol{y}\|_2^2 + \|\boldsymbol{y} - \boldsymbol{z}\|_2^2 > \|\boldsymbol{b} - \boldsymbol{y}\|_2^2.$$

De esta forma $\boldsymbol{y}$ minimiza la norma $\|\boldsymbol{b} - \boldsymbol{y}\|_2$ y $\boldsymbol{x}$ debe satisfacer la igualdad $A\boldsymbol{x} = P\boldsymbol{b}$. □

Comentario 7.2. Geométricamente, la condición dada en el teorema anterior es equivalente a tener que $\boldsymbol{r} \perp \text{Rang}(A)$, lo cual ocurre sii

$$A^*\boldsymbol{r} = 0.$$

De esta forma, resolver el problema de mínimos cuadrados es equivalente a resolver el sistema

$$A^*A\boldsymbol{x} = A^*\boldsymbol{b},$$

el cual es comúnmente llamado *sistema de ecuaciones normales.*

Teorema 7.3. *El problema de mínimos cuadrados tiene solución única si y solo si A tiene rango completo (i.e., si sus columnas son linealmente independientes).*

Demostración. Es consecuencia inmediata del Ejercicio 5.51. □

Comentario 7.4. Si A tiene rango completo, la solución del problema de mínimos cuadrados viene dada por

$$\boldsymbol{x} = (A^*A)^{-1}A^*\boldsymbol{b}.$$

A la matriz

$$A^+ := (A^*A)^{-1}A^* \in \mathbb{C}^{n\times m}$$

se le llama *pseudoinversa de* A, y así podemos escribir

$$\boldsymbol{x} = A^+\boldsymbol{b}.$$

Para obtener numéricamente $\boldsymbol{x}$, presentamos tres diferentes algoritmos donde asumimos que A es de rango completo:

1. Como A^*A es Hermitiana y definida positiva, podemos encontrar su factorización de Cholesky $A^*A = R^*R$, y resolver dicho sistema de ecuaciones; ver el Algoritmo 15. El número total de operaciones es de orden $mn^2 + n^3/3$.

   Data: Matriz $A \in \mathbb{C}^{m\times n}$, $\boldsymbol{b} \in \mathbb{C}^m$
   Result: $\boldsymbol{x} \in \mathbb{C}^n$ tal que $\|\boldsymbol{b} - A\boldsymbol{x}\|_2 \to$ mín
   Forme A^*A, $A^*\boldsymbol{b}$;
   Factorice $A^*A = R^*R$;
   Resuelva el sistema triangular inferior $R^*\boldsymbol{w} = A^*\boldsymbol{b}$ para $\boldsymbol{w}$;
   Resuelva el sistema triangular superior $R\boldsymbol{x} = \boldsymbol{w}$ para $\boldsymbol{x}$;
   Algoritmo 15: Mínimos cuadrados con ecuaciones normales

2. Utilizando la factorización reducida $A = \widehat{Q}\widehat{R}$, el proyector ortogonal P sobre el rango de A se puede escribir como $P = \widehat{Q}\widehat{Q}^*$. Como $A\boldsymbol{x} = P\boldsymbol{b}$, entonces $\widehat{Q}\widehat{R}\boldsymbol{x} = \widehat{Q}\widehat{Q}^*\boldsymbol{b}$ y basta resolver $\widehat{R}\boldsymbol{x} = \widehat{Q}^*\boldsymbol{b}$; ver el Algoritmo 16. En este caso, el número de operaciones es de orden $2mn^2 - 2n^3/3$.

   Data: Matriz $A \in \mathbb{C}^{m\times n}$, $\boldsymbol{b} \in \mathbb{C}^m$
   Result: $\boldsymbol{x} \in \mathbb{C}^n$ tal que $\|\boldsymbol{b} - A\boldsymbol{x}\|_2 \to$ mín
   Factorice $A = \widehat{Q}\widehat{R}$;
   Calcule $\widehat{Q}^*\boldsymbol{b}$;
   Resuelva el sistema triangular superior $\widehat{R}\boldsymbol{x} = \widehat{Q}^*\boldsymbol{b}$ para $\boldsymbol{x}$;
   Algoritmo 16: Mínimos cuadrados con factorización QR

3. Si consideramos la factorización reducida $A = \widehat{U}\widehat{\Sigma}V^*$, al sustituir en las ecuaciones normales concluimos que $\widehat{\Sigma}V^*\boldsymbol{x} = \widehat{U}^*\boldsymbol{b}$, con lo que $\boldsymbol{x} = V(\widehat{\Sigma})^{-1}\widehat{U}^*\boldsymbol{b}$; ver Algoritmo 17. En este caso, el número de operaciones es de orden $2mn^2 + 11n^3$.

   Data: Matriz $A \in \mathbb{C}^{m\times n}$, $\boldsymbol{b} \in \mathbb{C}^m$
   Result: $\boldsymbol{x} \in \mathbb{C}^n$ tal que $\|\boldsymbol{b} - A\boldsymbol{x}\|_2 \to$ mín
   Factorice $A = \widehat{U}\widehat{\Sigma}V^*$;
   Calcule $\widehat{U}^*\boldsymbol{b}$;
   Resuelva el sistema diagonal $\widehat{\Sigma}\boldsymbol{w} = \widehat{U}^*\boldsymbol{b}$ para $\boldsymbol{w}$;
   Calcule $\boldsymbol{x} = V\boldsymbol{w}$;
   Algoritmo 17: Mínimos cuadrados con valores singulares

Ejemplo 7.5. Considere un conjunto de puntos $\{(x_i, y_i)\}_{i=1}^m$, donde los x_i son valores diferentes del intervalo $[a, b]$. Considere el polinomio de interpolación de Lagrange

$$p_{m-1}(x) = c_0 + c_1 x + \ldots + c_{m-1}x^{m-1},$$

que satisface $p_{m-1}(x_i) = y_i \ \forall i = 1, 2, \ldots, m$, el cual sabemos existe y es único; ver Capítulo 3. No obstante, podemos encontrar dichos coeficientes si resolvemos el sistema de ecuaciones dado por

$$\begin{bmatrix} 1 & x_1 & x_1^2 & \ldots & x_1^{m-1} \\ 1 & x_2 & x_2^2 & \ldots & x_2^{m-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \ldots & x_m^{m-1} \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{m-1} \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

Sin embargo, sabemos que el fenómeno de Runge puede presentarse al interpolar si el grado del polinomio es alto. Una estrategia diferente es considerar un polinomio de grado menor. Defina

$$p_{n-1}(x) = c_0 + c_1 x + \ldots + c_{n-1} x^{n-1},$$

donde $p_{n-1} \in \mathbb{P}_{n-1}$ con $n < m$. Al tener más ecuaciones que incógnitas, el problema de interpolación en general no tiene solución. En su lugar, buscamos resolver el problema de mínimos cuadrados:

$$\sum_{i=1}^{m} |y_i - p_{n-1}(x_i)|^2 \to \text{mín}\,.$$

En este caso, podemos escribir el problema de la forma

$$\|A\boldsymbol{c} - \boldsymbol{y}\|_2 \to \text{mín},$$

donde $A \in \mathbb{C}^{m \times n}$ es la llamada *matriz de Vandermonde*,

$$A := \begin{bmatrix} 1 & x_1 & x_1^2 & \ldots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \ldots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & \ldots & x_m^{n-1} \end{bmatrix}, \quad \boldsymbol{c} := \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{n-1} \end{bmatrix}, \quad \boldsymbol{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}.$$

Al resolver el problema, obtenemos la función mostrada en la Figura 7.1.

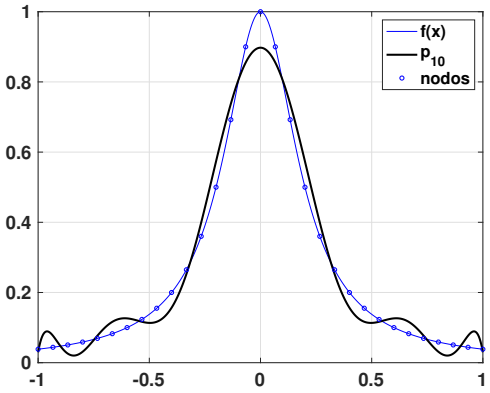


Figura 7.1: Polinomio de mínimos cuadrados para $f(x) = 1/(1 + 25x^2)$, $m = 31$, $n = 10$.

7.2. Mejor aproximación en la 2-norma

En el Ejemplo 7.5 consideramos minimizar la 2-norma del vector de desviaciones cuyas entradas son los valores $|y_i - p(x_i)|$. En esta sección consideramos el problema: Dada $f \in L^2(a, b)$, encuentre $p_n \in \mathbb{P}_n$ tal que

$$E := \|f - p_n\|_2^2 = \int_a^b |f(x) - p_n(x)|^2 \, dx \to \text{mín}\,.$$

Ejercicio 7.6. Si escribimos

$$p_n(x) = c_0 + c_1 x + \ldots c_n x^n,$$

tenemos que $E = E(c_0, c_1, \ldots, c_n)$ es una función que depende de los coeficientes de p_n. Obtenga las derivadas parciales

$$\partial E(c_0, \ldots, c_{n-1})/\partial c_k \ (k = 0, \ldots, n-1).$$

Para obtener el punto crítico, exprese el resultado como un sistema de n ecuaciones con n incógnitas, $M\boldsymbol{c} = \boldsymbol{b}$. ¿Qué problema tiene este sistema?

Del ejercicio anterior se concluye que utilizar un sistema de ecuaciones para obtener la mejor aproximación en la 2-norma no es la mejor idea. Para

resolver estos inconvenientes, recurrimos al uso de polinomios ortogonales. A continuación demostramos dos teoremas que garantizan la existencia y unicidad del problema de mejor aproximación en la 2-norma.

Teorema 7.7. *Dada $f \in L^2(a,b)$, existe un único polinomio $p_n \in \mathbb{P}_n$ tal que*

$$\|f - p_n\|_{L^2(a,b)} = \min_{q \in \mathbb{P}_n} \|f - q\|_{L^2(a,b)}.$$

Demostración. Considere el conjunto de polinomios ortonormales $\{\psi_j\}_{j=0}^n$ en el intervalo $[a,b]$ con respecto al peso $w(x) = 1$. Al ser ortonormales, satisfacen que $\langle \psi_i, \psi_j \rangle = \delta_{ij}$. Si expandimos $q \in \mathbb{P}_n$ en la base dada por los polinomios ortonormales, podemos escribir

$$q(x) = \beta_0 \psi_0(x) + \ldots + \beta_n \psi_n(x).$$

Luego,

$$\begin{aligned} E(\beta_0, \ldots, \beta_n) &= \langle f - q, f - q \rangle = \langle f, f \rangle - 2\langle f, q \rangle + \langle q, q \rangle \\ &= \|f\|_{L^2}^2 - 2 \sum_{j=0}^n \beta_j \langle f, \psi_j \rangle + \sum_{j,k=0}^n \beta_j \beta_k \langle \psi_j, \psi_k \rangle \\ &= \|f\|_{L^2}^2 - 2 \sum_{j=0}^n \beta_j \langle f, \psi_j \rangle + \sum_{j=0}^n \beta_j^2 \\ &= \sum_{j=0}^n (\beta_j - \langle f, \psi_j \rangle)^2 + \|f\|_{L^2}^2 - \sum_{j=0}^n (\langle f, \psi_j \rangle)^2 . \end{aligned}$$

De esta forma, E alcanza su mínimo sii los coeficientes β_j se escogen de manera tal que

$$\beta_j = \langle f, \psi_j \rangle \ (j = 0, 1, \ldots, n),$$

y así,

$$p_n(x) = \langle f, \psi_0 \rangle \psi_0(x) + \ldots + \langle f, \psi_n \rangle \psi_n(x)$$

es el polinomio de mejor aproximación. □

Teorema 7.8. *Un polinomio $p_n \in \mathbb{P}_n$ minimiza $\|f - p_n\|_{L^2(a,b)}$ si y solo si*

$$\langle f - p_n, q \rangle = 0 \ \forall \ q \in \mathbb{P}_n.$$

Geométricamente, p_n es la L^2-proyección de f sobre el espacio de polinomios de grado a lo sumo n.

Demostración. Se deja como ejercicio. □

Ejercicio 7.9. Con la misma notación del Ejercicio 7.6, calcule p para $n = 10$, $f(x) = \cos(4\pi x)$ utilizando la construcción del Teorema 7.7. Compare sus resultados al resolver el sistema de Vandermonde asociado. Dibuje en la misma gráfica la nueva aproximación y calcule la 2-norma del error.

7.3. Sistemas de ecuaciones no lineales

Considere una función continua en varias variables $\boldsymbol{g} : D \subset \mathbb{R}^n \to \mathbb{R}^n$, donde D es un cerrado de $\mathbb{R}^n$. Escribimos

$$\boldsymbol{g}(\boldsymbol{x}) = \begin{bmatrix} g_1(\boldsymbol{x}) \\ g_2(\boldsymbol{x}) \\ \vdots \\ g_n(\boldsymbol{x}) \end{bmatrix}, \ \boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}.$$

Estamos interesados en encontrar puntos fijos de $\boldsymbol{g}$. De manera similar al caso de una dimensión, consideramos la *iteración simultánea*:

Dado $\boldsymbol{x}^{(0)} \in D$, defina la secuencia

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{g}(\boldsymbol{x}^{(k)}), \ k = 0, 1, \ldots \tag{7.1}$$

(note que los superíndices denotan el número de término en la sucesión). Del Teorema 2.6, si utilizamos $d(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|_\infty$, garantizamos la existencia de un único punto fijo en D si $\boldsymbol{g}$ es una contracción; esto es, si existe $L \in (0, 1)$ tal que

$$\|\boldsymbol{g}(\boldsymbol{x}) - \boldsymbol{g}(\boldsymbol{y})\|_\infty \leqslant L\|\boldsymbol{x} - \boldsymbol{y}\|_\infty \ \forall \boldsymbol{x}, \boldsymbol{y} \in D.$$

No obstante, se pueden obtener estimados locales si conocemos la norma del Jacobiano $J_{\boldsymbol{g}}(\boldsymbol{x}) \in \mathbb{R}^{n \times n}$,

$$J_{\boldsymbol{g}}(\boldsymbol{x}) := \begin{bmatrix} \dfrac{\partial g_1(\boldsymbol{x})}{\partial x_1} & \dfrac{\partial g_1(\boldsymbol{x})}{\partial x_2} & \cdots & \dfrac{\partial g_1(\boldsymbol{x})}{\partial x_n} \\ \dfrac{\partial g_2(\boldsymbol{x})}{\partial x_1} & \dfrac{\partial g_2(\boldsymbol{x})}{\partial x_2} & \cdots & \dfrac{\partial g_2(\boldsymbol{x})}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \dfrac{\partial g_n(\boldsymbol{x})}{\partial x_1} & \dfrac{\partial g_n(\boldsymbol{x})}{\partial x_2} & \cdots & \dfrac{\partial g_n(\boldsymbol{x})}{\partial x_n} \end{bmatrix}.$$

Teorema 7.10. *Suponga $\boldsymbol{g}$ es continua en un cerrado $D \in \mathbb{R}^n$ y sea $\boldsymbol{c} \in D$ un punto fijo de $\boldsymbol{g}$. Suponga que las primeras derivadas parciales son continuas en un vecindario abierto $N(\boldsymbol{c}) \subset D$ de $\boldsymbol{c}$, con*

$$\|J_{\boldsymbol{g}}(\boldsymbol{c})\|_\infty < 1.$$

Entonces, existe $\epsilon > 0$ tal que $\boldsymbol{g}(\overline{B}_\epsilon(\boldsymbol{c})) \subset \overline{B}_\epsilon(\boldsymbol{c})$, donde $\overline{B}_\epsilon(\boldsymbol{c})$ es la bola cerrada de centro $\boldsymbol{c}$ y radio ϵ. Además, la secuencia definida en (7.1) converge a $\boldsymbol{c}$ para todo $\boldsymbol{x}^{(0)} \in \overline{B}_\epsilon(\boldsymbol{c})$.

Demostración. Primero, como las derivadas parciales son continuas en $N(\boldsymbol{c})$, podemos encontrar una bola cerrada $\overline{B}_\epsilon(\boldsymbol{c})$ de radio ϵ y centro $\boldsymbol{c}$, tal que

$$\|J_{\boldsymbol{g}}(\boldsymbol{z})\|_\infty \leqslant L < 1 \ \forall \ \boldsymbol{z} \in \overline{B}_\epsilon(\boldsymbol{c}).$$

Para $\boldsymbol{x}, \boldsymbol{y} \in \overline{B}_\epsilon(\boldsymbol{c})$ e $i \in \{1, \ldots, n\}$, defina la función $\phi_i : [0,1] \to \mathbb{R}$ por $\phi_i(t) := g_i\left(t\boldsymbol{x} + (1-t)\boldsymbol{y}\right)$. Al ser $\phi_i \in C^1([0,1])$, por el Teorema del Valor Medio, existe $\eta \in (0,1)$ tal que

$$g_i(\boldsymbol{x}) - g_i(\boldsymbol{y}) = \phi_i(1) - \phi_i(0) = \phi_i'(\eta).$$

Derivando, tenemos entonces que

$$g_i(\boldsymbol{x}) - g_i(\boldsymbol{y}) = \sum_{j=1}^{n} (x_j - y_j) \frac{\partial g_i}{\partial x_j}\left(\eta\boldsymbol{x} + (1-\eta)\boldsymbol{y}\right) \text{ para } \ i \in \{1, \ldots, n\}.$$

Como $|x_j - y_j| \leqslant \|\boldsymbol{x} - \boldsymbol{y}\|_\infty$, deducimos que

$$|g_i(\boldsymbol{x}) - g_i(\boldsymbol{y})| \leqslant \|\boldsymbol{x} - \boldsymbol{y}\|_\infty \|J_{\boldsymbol{g}}\left(\eta\boldsymbol{x} + (1-\eta)\boldsymbol{y}\right)\|_\infty \text{ para } \ i \in \{1, \ldots, n\}.$$

De esta manera,

$$\|\boldsymbol{g}(\boldsymbol{x}) - \boldsymbol{g}(\boldsymbol{y})\|_\infty \leqslant L\|\boldsymbol{x} - \boldsymbol{y}\|_\infty.$$

En la última desigualdad, si tomamos $\boldsymbol{y} = \boldsymbol{c}$, entonces tenemos que

$$\|\boldsymbol{g}(\boldsymbol{x}) - \boldsymbol{c}\|_\infty \leqslant L\|\boldsymbol{x} - \boldsymbol{c}\|_\infty < \|\boldsymbol{x} - \boldsymbol{c}\|_\infty \leqslant \epsilon \ \forall \boldsymbol{x} \in \overline{B}_\epsilon(\boldsymbol{c}).$$

De esta forma, $\boldsymbol{g}(\overline{B}_\epsilon(\boldsymbol{c})) \subset \overline{B}_\epsilon(\boldsymbol{c})$. La convergencia es resultado inmediato del Teorema 2.6. □

Continuando de manera análoga al caso unidimensional, para resolver $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{0}$, consideramos la generalización del Método de Newton al buscar puntos fijos de la función $\boldsymbol{g}(x) := \boldsymbol{x} - \lambda(\boldsymbol{x})\boldsymbol{f}(\boldsymbol{x})$ $(\lambda(\boldsymbol{x}) \neq 0)$. De esta forma, tenemos el *Método de Newton para funciones en varias variables*:

Data: Función continua y derivable $\boldsymbol{f}$, valor inicial $\boldsymbol{x}^{(0)}$
Result: Aproximación $\boldsymbol{x}^{(k)}$ de la raíz $\boldsymbol{c}$ de $\boldsymbol{f}$
for $k = 1, 2, \ldots$ **do**
  $\boldsymbol{x}^{(k)} := \boldsymbol{x}^{(k-1)} - \left(J_{\boldsymbol{f}}(\boldsymbol{x}^{(k-1)})\right)^{-1} \boldsymbol{f}(\boldsymbol{x}^{(k-1)})$;
end

Algoritmo 18: Método de Newton en varias variables

Note que en el caso que $n = 1$, el Jacobiano corresponde a la derivada de f y así el método coincide con el que estudiamos en el Capítulo 2. A continuación demostramos el teorema sobre la convergencia del Método de Newton:

Teorema 7.11. *Suponga que $\boldsymbol{f}(\boldsymbol{c}) = \boldsymbol{0}$ con $f \in C^2(N(\boldsymbol{c}))$, donde $N(\boldsymbol{c})$ es un vecindario abierto que contiene a $\boldsymbol{c}$. Asuma que $J_{\boldsymbol{f}}(\boldsymbol{c})$ es invertible. Entonces, la secuencia definida en el Algoritmo 18 converge a $\boldsymbol{c}$ si $\boldsymbol{x}^{(0)}$ se encuentra suficientemente cerca de $\boldsymbol{c}$. Además, la convergencia es cuadrática.*

Demostración. Para demostrar la convergencia, considere la función

$$\boldsymbol{g}(\boldsymbol{x}) = \boldsymbol{x} - (J_{\boldsymbol{f}}(\boldsymbol{x}))^{-1} \boldsymbol{f}(\boldsymbol{x}).$$

Es claro que $\boldsymbol{g}(\boldsymbol{c}) = \boldsymbol{c}$. Por continuidad, al ser $J_{\boldsymbol{f}}(\boldsymbol{c})$ invertible y depender continuamente de sus entradas, tenemos que existe $\epsilon > 0$ tal que $\boldsymbol{g}$ es continua en la bola cerrada $\overline{B}_\epsilon(\boldsymbol{c}) \subset N(\boldsymbol{c})$ de centro $\boldsymbol{c}$ y radio ϵ. Es fácil ver que $\|J_{\boldsymbol{g}}(\boldsymbol{c})\|_\infty = 0$. De esta forma, el Teorema 7.10 implica que la iteración simultánea (7.1) converge a $\boldsymbol{c}$; esto es, la secuencia de vectores obtenida de la iteración de Newton converge a $\boldsymbol{c}$.

Para obtener la convergencia cuadrática, procedemos de manera similar al caso unidimensional. Por Taylor, tenemos que

$$\boldsymbol{0} = \boldsymbol{f}(\boldsymbol{c}) = \boldsymbol{f}\left(\boldsymbol{x}^{(k)}\right) + J_{\boldsymbol{f}}\left(\boldsymbol{x}^{(k)}\right)\left(\boldsymbol{c} - \boldsymbol{x}^{(k)}\right) + E_{\boldsymbol{f}},$$

donde el término $E_{\boldsymbol{f}}$ incluye todos los términos de las segundas derivadas y satisface

$$\|E_{\boldsymbol{f}}\|_\infty \leqslant \frac{1}{2} n^2 A_{\boldsymbol{f}} \left\|\boldsymbol{c} - \boldsymbol{x}^{(k)}\right\|_\infty^2,$$

donde

$$A_{\boldsymbol{f}} := \max_{1 \leqslant i,j,l \leqslant n} \max_{\boldsymbol{x} \in \overline{B}_\epsilon(\boldsymbol{c})} \left| \frac{\partial^2 f_i}{\partial x_j \partial x_l}(\boldsymbol{x}) \right|$$

y el término n^2 se obtiene al acotar cada una de sus n^2 derivadas de orden dos. Luego, del desarrollo de Taylor se deduce que

$$\boldsymbol{x}^{(k+1)} - \boldsymbol{c} = \boldsymbol{x}^{(k)} - \boldsymbol{c} - \left(J_{\boldsymbol{f}}(\boldsymbol{x}^{(k)})\right)^{-1} \boldsymbol{f}(\boldsymbol{x}^{(k)}) = \left(J_{\boldsymbol{f}}(\boldsymbol{x}^{(k)})\right)^{-1} E_{\boldsymbol{f}},$$

y así

$$\|\boldsymbol{x}^{(k+1)} - \boldsymbol{c}\|_\infty \leqslant \frac{1}{2} C n^2 A_{\boldsymbol{f}} \|\boldsymbol{x}^{(k)} - \boldsymbol{c}\|_\infty,$$

donde

$$C := \max_{\boldsymbol{x} \in \overline{B}_\epsilon(\boldsymbol{c})} \left\| (J_{\boldsymbol{f}}(\boldsymbol{x}))^{-1} \right\|_\infty^2.$$

De esta desigualdad podemos deducir la convergencia cuadrática. □

7.4. Trazadores cúbicos naturales

En la Sección 3.6 describimos los trazadores cúbicos de Hermite y mencionamos brevemente los llamados *trazadores cúbicos naturales*. Considere una partición del intervalo $[a, b]$ dada por

$$a = x_0 < x_1 < \ldots < x_{n-1} < x_n = b,$$

y defina $h_i := x_i - x_{i-1}$ $(i = 1, \ldots, n)$ (note que los nodos no necesariamente son equidistantes).

De esta forma, definimos el spline cúbico como la función s que satisface las siguientes condiciones:

1. s interpola los datos $\{(x_i, y_i)\}$; esto es, $s(x_i) = y_i$ $\forall i = 0, 1, \ldots, n$.
2. $s \in C^2([a, b])$; esto es, sus derivadas de orden menor o igual que dos son continuas.
3. La restricción de $s(x)$ a cada subintervalo $[x_{i-1}, x_i]$ $(i = 1, \ldots, n)$ es un polinomio de grado tres.

Para definir completamente $s(x)$, debemos especificar un polinomio de grado tres en cada uno de los n subintervalos de la partición. De esta manera, ocupamos determinar $4n$ coeficientes en total. Como deseamos que $s \in C^2([a, b])$, tenemos las siguientes restricciones:

- $(n + 1)$ condiciones de interpolación $s(x_i) = f(x_i),\ i = 0, 1, \ldots, n$.
- $(n - 1)$ condiciones de continuidad para s (en los nodos internos de la partición).

- $(n-1)$ condiciones de continuidad para la primer derivada de s.
- $(n-1)$ condiciones de continuidad para la segunda derivada de s.

Así, tenemos $4n-2$ condiciones para encontrar $4n$ coeficientes. Por lo tanto, deben agregarse dos condiciones extra. Para los *splines naturales* se considera que

$$s''(a) = s''(b) = 0.$$

De esta manera se obtiene un sistema de $4n$ ecuaciones con $4n$ incógnitas que debe ser resuelto. A continuación presentamos dicha construcción.

Defina $\sigma_i := s''(x_i)$ $(i = 0, 1, \ldots, n)$. La restricción de la segunda derivada de s en cada subintervalo $I_i := [x_{i-1}, x_i]$ es una función lineal. Como sabemos que en los extremos de I_i tenemos continuidad en la segunda derivada, la restricción de s'' en I_i es el interpolante lineal de σ_{i-1} y σ_i. De esta forma,

$$s''(x) = \frac{x_i - x}{h_i}\sigma_{i-1} + \frac{x - x_{i-1}}{h_i}\sigma_i, \ x \in I_i.$$

Al integrar dos veces, podemos escribir

$$s(x) = \frac{(x_i - x)^3}{6h_i}\sigma_{i-1} + \frac{(x - x_{i-1})^3}{6h_i}\sigma_i + \alpha_i(x - x_{i-1}) + \beta_i(x_i - x), \ x \in I_i. \tag{7.2}$$

De las condiciones de continuidad, obtenemos que

$$f(x_{i-1}) = s(x_{i-1}^+) = \frac{1}{6}h_i^2\sigma_{i-1} + h_i\beta_i \Rightarrow \beta_i = \frac{1}{h_i}\left(f_{i-1} - \frac{1}{6}h_i^2\sigma_{i-1}\right),$$

$$f(x_i) = s(x_i^-) = \frac{1}{6}h_i^2\sigma_i + h_i\alpha_i \Rightarrow \alpha_i = \frac{1}{h_i}\left(f_i - \frac{1}{6}h_i^2\sigma_i\right), \tag{7.3}$$

donde hemos escrito por simplicidad $f_{i-1} = f(x_{i-1})$ y $f_i = f(x_i)$. Falta todavía garantizar la continuidad de $s'(x)$; esto es, necesitamos que

$$s'(x_i^-) = s'(x_i^+) \ (i = 1, \ldots, n-1),$$

de donde se sigue que

$$h_i\sigma_{i-1} + 2(h_i + h_{i+1})\sigma_i + h_{i+1}\sigma_{i+1} = \frac{6}{h_{i+1}}(f_{i+1} - f_i) - \frac{6}{h_i}(f_i - f_{i-1})$$

para todo $i = 1, 2, \ldots, n-1$. Este es un sistema de $n-1$ ecuaciones con incógnitas σ_i $(i = 0, 1, \ldots, n)$. Al agregar las condiciones $\sigma_0 = s''(a) = 0$ y $\sigma_n = s''(b) = 0$, obtenemos el sistema de ecuaciones

$$A\boldsymbol{\sigma} = \boldsymbol{b}, \tag{7.4}$$

donde

$$A := \begin{bmatrix} 1 & & & & \\ h_1 & 2(h_1+h_2) & h_2 & & \\ & \ddots & \ddots & \ddots & \\ & & h_{n-1} & 2(h_{n-1}+h_n) & h_n \\ & & & & 1 \end{bmatrix}, \ \boldsymbol{\sigma} := \begin{bmatrix} \sigma_0 \\ \sigma_1 \\ \vdots \\ \sigma_{n-1} \\ \sigma_n \end{bmatrix}$$

y

$$\boldsymbol{b} = \begin{bmatrix} 0 \\ \frac{6}{h_2}(f_2 - f_1) - \frac{6}{h_1}(f_1 - f_0) \\ \vdots \\ \frac{6}{h_n}(f_n - f_{n-1}) - \frac{6}{h_{n-1}}(f_{n-1} - f_{n-2}) \\ 0 \end{bmatrix}$$

De esta forma, para construir s, resolvemos primero el sistema (7.4) de donde obtenemos los valores σ_i. Luego, de (7.3) calculamos los valores α_i y β_i. Finalmente, la restricción de $s(x)$ en el intervalo $[x_{i-1}, x_i]$ se obtiene al sustituir los valores encontrados en (7.2).

A continuación demostramos un lema que garantizará que la matriz A es invertible, por lo que s existe y es único. Primero, definiremos lo que es una matriz tridiagonal:

Definición 7.12. Una matriz T se llama *tridiagonal* si $t_{ij} = 0$ para $|i-j| > 1$; esto es, sus elementos no nulos se encuentran en la diagonal, en la super-diagonal (elementos encima de la diagonal) y en la sub-diagonal (elementos debajo de la diagonal). En general, la escribiremos de la forma

$$T = \begin{bmatrix} b_1 & c_1 & & & & \\ a_2 & b_2 & c_2 & & & \\ & a_3 & b_3 & c_3 & & \\ & & \ddots & \ddots & \ddots & \\ & & & a_{n-1} & b_{n-1} & c_{n-1} \\ & & & & a_n & b_n \end{bmatrix}.$$

Si aplicamos eliminación Gaussiana sin pivoteo a una matriz tridiagonal T, obtenemos que

$$T = \begin{bmatrix} 1 & & & & \\ l_2 & 1 & & & \\ & l_3 & 1 & & \\ & & \ddots & \ddots & \\ & & & l_n & 1 \end{bmatrix} \begin{bmatrix} u_1 & v_1 & & & \\ & u_2 & v_2 & & \\ & & \ddots & \ddots & \\ & & & u_{n-1} & v_{n-1} \\ & & & & u_n \end{bmatrix},$$

donde podemos calcular explícitamente que

$$\begin{aligned} l_j &= \frac{a_j}{u_{j-1}} \ (j = 2, 3, \ldots, n), \\ u_1 &= b_1, \\ u_j &= b_j - l_j c_{j-1} \ (j = 2, 3, \ldots, n). \end{aligned}$$

Tenemos así el siguiente lema:

Lema 7.13. *Suponga que $T \in \mathbb{R}^{n\times n} (n \geqslant 3)$ es tridiagonal, con*

$$|b_j| > |a_j| + |c_j|, \ j = 1, \ldots, n,$$

donde por conveniencia definimos $a_1 \equiv 0$, $c_n \equiv 0$. Entonces T es invertible.

Demostración. Veamos por inducción que $|u_j| > |c_j|$. Para $j = 1$ es claro que $|u_1| = |b_1| > |c_1|$ por hipótesis. Suponga ahora que $|u_{k-1}| > |c_{k-1}|$. Así,

$$|u_j| \geqslant \left| |b_j| - |a_j| \frac{|c_{j-1}|}{|u_{j-1}|} \right| \geqslant ||b_j| - |a_j|| > |c_j|.$$

Por inducción se sigue que $|u_j| > |c_j| \ \forall j$. En particular, $u_j \neq 0$ para $j = 1, \ldots, n$ y así $\det A = \det L \det U = u_1 \ldots u_n \neq 0$. □

Comentario 7.14. Es necesario hacer dos observaciones:

1. El número de operaciones necesarias para calcular la factorización LU de una matriz tridiagonal T y resolver el sistema correspondiente es de orden $\mathcal{O}(n)$ (a diferencia del caso general con una matriz densa que es de orden $\mathcal{O}(n^3)$). De esta forma, construir el trazador cúbico natural es lineal con respecto al número de nodos de interpolación.

2. Se tiene que $|u_j| \leqslant |b_j| + |l_j||c_{j-1}| \leqslant |b_j| + |a_j|$, por lo que el factor de crecimiento no es un problema en la precisión de la respuesta que se obtiene numéricamente. □

7.5. Ejercicios

Ejercicio 7.15. Sean $A \in \mathbb{C}^{m\times m}$ y $\boldsymbol{x} \in \mathbb{C}^m$ dados. Considere el problema de mínimos de cuadrados: Encontrar $\alpha \in \mathbb{C}$ tal que $\|\alpha \boldsymbol{x} - A\boldsymbol{x}\|_2 \to$ mín. Encuentre el valor de α (en términos de A y $\boldsymbol{x}$).

Ejercicio 7.16. Dada $A \in \mathbb{C}^{m\times m}$ de rango n y $\boldsymbol{b} \in \mathbb{C}^m$, considere el sistema de ecuaciones escrito en bloques dado por

$$\begin{bmatrix} I & A \\ A^* & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{r} \\ \boldsymbol{x} \end{bmatrix} = \begin{bmatrix} \boldsymbol{b} \\ \boldsymbol{0} \end{bmatrix},$$

donde $I \in \mathbb{C}^{m\times m}$ es la identidad. Demuestre que este sistema tiene solución única, y que los vectores $\boldsymbol{r}$ y $\boldsymbol{x}$ son el residuo y la solución del problema de mínimos cuadrados.

Ejercicio 7.17 (`MATLAB`)**.** Considere m puntos $\{(x_i, y_i)\}_{i=1}^m$, donde $x_i \neq x_j \ \forall i \neq j$. Queremos resolver el siguiente problema de mínimos cuadrados: encontrar

$$p(x) = c_0 + c_1 x + \ldots + c_{n-1}x^{n-1}, \ x \in [a, b],$$

tal que

$$\sum_{i=1}^{m} |y_i - p(x_i)|^2 \to \text{mín},$$

donde $m > n$.

Tome $m = 101$ y $n = 21$. Defina el vector `x = linspace(-1,1,m)` y tome los y_i como la imagen de x bajo la función $f(x) = 1/(1 + 25x^2)$, con $x \in [-1, 1]$. Escriba el sistema de ecuaciones correspondiente con la matriz de Vandermonde. Resuelva utilizando los tres algoritmos estudiados en clase (ecuaciones normales, QR y valores singulares) y con el comando `polyfit`. Compare los resultados obtenidos. Grafique en una misma gráfica f y los polinomios obtenidos. Además, grafique los errores en cada caso. ¿Hay algún algoritmo mejor o peor?

Ejercicio 7.18 (`MATLAB`)**.** (Mejor aproximación en la norma $\|\cdot\|_2$) Sea

$$p(x) = c_0 + c_1 x + \ldots + c_{n-1}x^{n-1}, \ x \in [0, 1].$$

Considere el problema de minimizar

$$E(c_0, \ldots, c_{n-1}) := \|f - p\|_2^2 = \int_0^1 |f(x) - p(x)|^2 \, dx \to \text{mín}\,.$$

(a) Obtenga las derivadas parciales $\partial E(c_0, \ldots, c_{n-1})/\partial c_k$ $(k = 0, \ldots, n-1)$. Para obtener el punto crítico, exprese el resultado como un sistema de n ecuaciones con n incógnitas, $M\boldsymbol{c} = \boldsymbol{b}$. ¿Qué problema tiene este sistema?

(b) Resuelva el sistema correspondiente para $n = 10$, $f(x) = \cos(4\pi x)$. Dibuje en un mismo gráfico f y p. ¿Cuál es el valor de $\|f(x) - p(x)\|_2^2$? (Sugerencia: puede utilizar el comando `integral(f,0,1)` para integrar funciones).

(c) Para obtener una mejor aproximación, considere la base de polinomios ortogonales de Legendre $\{\phi_j\}_{j\geqslant 0}$. La proyección de f sobre el espacio de polinomios de grado $n-1$ viene dada por

$$p(x) = \beta_0 + \beta_1\phi_1(x) + \ldots \beta_{n-1}\phi_{n-1}(x),$$

donde

$$\beta_k = \frac{\langle f, \phi_k\rangle}{\langle \phi_k, \phi_k\rangle}.$$

Calcule p para $n = 10$, $f(x) = \cos(4\pi x)$. Compare con los resultados del apartado anterior. Dibuje en la misma gráfica la nueva aproximación y calcule la 2-norma del error.

Ejercicio 7.19. Considere la función

$$H(x) = \begin{cases} 1 & \text{si } \ x > 0 \\ 0 & \text{en caso contrario} \end{cases}$$

Calcule las mejores aproximaciones en la 2-norma de grado 0, 1 y 2.

Ejercicio 7.20. Suponga que $\boldsymbol{g}$ es una contracción en la $\infty-$norma. Demuestre que $\boldsymbol{g}$ es una contracción en la $p-$norma $(1 \leqslant p < \infty)$ si $L < n^{-1/p}$.

Ejercicio 7.21 (`MATLAB`)**.** Grafique las tres superficies dadas de forma implícita por las ecuaciones

$$\begin{cases} x^2 + y^2 + z^2 & = 1 \\ 2x^2 + y^2 - 4z & = 0 \\ 3x^2 - 4y + z^2 & = 0 \end{cases}$$

Utilice el método de Newton para resolver el sistema anterior. Sugerencia: utilice el comando `fimplicit3` para graficar.

Ejercicio 7.22. Suponga que tenemos los $m+1$ nodos (no necesariamente equidistantes)

$$a = x_0 < x_1 < \ldots < x_{m-1} < x_m = b.$$

Queremos construir un trazador s_n de grado n (esto es, la restricción a cada subintervalo $[x_{i-1}, x_i]$ $(i = 1, \ldots, m)$ es un polinomio de grado n). Si deseamos que $s_n \in C^{n-1}([a, b])$, determine el número de condiciones extra

que debemos especificar para tener un único spline s_n. Sugerencia: para el caso de trazadores cúbicos naturales, se debe considerar dos condiciones extra: $s_3''(a) = s_3''(b) = 0$.

Ejercicio 7.23. Suponga que $A \in \mathbb{C}^{m\times m}$ es una matriz penta-diagonal; esto es, $a_{ij} = 0$ si $|i-j| > 2$. Suponga que existe la factorización LU de A sin pivoteo. ¿Qué forma tendrán L y U? ¿Qué sucede si utilizamos pivoteo?

Ejercicio 7.24. (Trazadores cuadráticos) Considere un conjunto de $m+1$ puntos $\{(x_i, y_i)\}_{i=0}^m$, donde

$$0 = x_0 < x_1 < \ldots < x_{m-1} < x_m = 1$$

es una partición del intervalo $[0,1]$ (no necesariamente con puntos equidistantes). En particular asumiremos que $y_0 = y_m$ pues consideraremos funciones periódicas. Deseamos construir una función $s_2 : \mathbb{R} \to \mathbb{R}$ tal que:

- s_2 es periódica con periodo 1; esto es, $s_2(x+1) = s_2(x)\ \forall\ x \in \mathbb{R}$.
- $s_2 \in C^1(\mathbb{R})$; esto es, s_2 tiene derivada continua.
- s_2 interpola los valores del conjunto de puntos; esto es, $s_2(x_i) = y_i\ \forall\ i = 0, 1, \ldots, m$.
- la restricción de s_2 a cada subintervalo $[x_i, x_{i+1}]$ $(i = 0, 1, \ldots, m-1)$ es un polinomio cuadrático.

Es claro que basta construir s_2 en el intervalo $[0,1]$ al ser periódica.

(a) Escriba un sistema de ecuaciones que permita determinar $s_2 : [0,1] \to \mathbb{R}$ (Sugerencia: puede seguir la construcción de los splines cúbicos naturales del Capítulo 7; en este caso defina $\sigma_i = s_2'(x_i),\ i = 0, 1, \ldots, m$ y genere un sistema para encontrar los σ_i. Otra idea es escribir la restricción de $s_2(x)$ en cada subintervalo $[x_i, x_{i+1}]$ de la forma $s_2(x) = \alpha_i + \sigma_i(x - x_i) + \beta_i(x - x_i)^2$. Recuerde además que s_2 debe ser periódica).

(b) Fije $m = 15$ y tome $f(x) := cos(2\pi x)$. Defina $\boldsymbol{x} = [x_0, \ldots, x_m]^T$ utilizando valores aleatorios con la función `rand` y calcule $y_i = f(x_i)\ \forall\ i = 0, 1, \ldots, m$. Construya s_2 para este conjunto de datos. Grafique f, s_2 y los puntos $\{(x_i, y_i)\}_{i=0}^m$ en un mismo gráfico. Grafique además el error absoluto $|f(x) - s_2(x)|$ y estime $\|f - s_2\|_{L^\infty([0,1])}$.

(c) Repita el ejercicio anterior para $\boldsymbol{x}$ = `linspace(0,1,m+1)'`. Compare con los resultados del inciso anterior y comente sus resultados.

(d) Uno puede aproximar $f'(x)$ mediante $s_2'(x)$. Utilizando s_2 tal como se calculó en el inciso anterior, escriba un programa que calcule $s_2'(x)$. Grafique f' y s_2' en el mismo gráfico. Grafique además el error absoluto $|f'(x) - s_2'(x)|$ y estime $\|f' - s_2'\|_{L^\infty([0,1])}$. Note que s_2' es diferenciable a trozos. ¿Cómo podría aproximar f' por una función s de manera tal que s'' sea continua?

Ejercicio 7.25. Considere un conjunto de mediciones $\{(x_i, y_i)\}_{i=1}^m$. En este ejercicio deseamos resolver el problema de mínimos cuadrados, donde la función buscada $f(x, \boldsymbol{c})$ no depende linealmente de los coeficientes $\boldsymbol{c} = [c_1, \ldots, c_n]^T \in \mathbb{R}^n$ (en clase analizamos el caso donde f es un polinomio). De esta forma, deseamos hallar $\boldsymbol{c}$ tal que

$$\sum_{i=1}^{m} |f(x_i, \boldsymbol{c}) - y_i|^2 \to \text{mín}\,.$$

Para x_i fijo y f con derivadas continuas (con respecto a c), considere la aproximación de primer orden con centro $\boldsymbol{c}^{(k)}$ dada por

$$f(x_i, \boldsymbol{c}) \approx f(x_i, \boldsymbol{c}^{(k)}) + J_f(x_i, \boldsymbol{c}^{(k)})(\boldsymbol{c} - \boldsymbol{c}^{(k)}),$$

donde

$$J_f(x_i, \boldsymbol{c}) = \nabla_{\boldsymbol{c}} f(x_i, \boldsymbol{c}) \in \mathbb{R}^{1\times n}$$

es el Jacobiano de f con derivadas parciales $\dfrac{\partial f}{\partial c_j}$ evaluado en x_i.

De esta forma, deseamos que

$$J_f(x_i, \boldsymbol{c}^{(k)})(\boldsymbol{c} - \boldsymbol{c}^{(k)}) \approx y_i - f(x_i, \boldsymbol{c}^{(k)}) \;\forall\; i = 1, \ldots, m. \tag{7.5}$$

Defina la matriz $J(\boldsymbol{c}^{(k)}) \in \mathbb{R}^{m\times n}$ donde su fila i viene dada por $J_f(x_i, \boldsymbol{c}^{(k)})$. Defina además

$$\boldsymbol{y} := [y_1, \ldots, y_m]^T \in \mathbb{R}^m,$$
$$\boldsymbol{g}(\boldsymbol{c}) := [f(x_1, \boldsymbol{c}), \ldots, f(x_m, \boldsymbol{c})]^T \in \mathbb{R}^m.$$

Podemos escribir así las ecuaciones dadas en (7.5) de forma matricial. Si sabemos que $\boldsymbol{c}^{(k)}$ es cercano a $\boldsymbol{c}$, buscamos encontrar $\boldsymbol{c}$ tal que

$$J(\boldsymbol{c}^{(k)})(\boldsymbol{c} - \boldsymbol{c}^{(k)}) \approx \boldsymbol{y} - \boldsymbol{g}(\boldsymbol{c}^{(k)}). \tag{7.6}$$

Para tal efecto, consideramos el siguiente algoritmo:

- Considere $\boldsymbol{c}^{(0)}$ dado.

- Para $k = 0, 1, 2, \ldots$
  - Obtenga $\Delta \boldsymbol{c}^{(k)}$ tal que $\|J(\boldsymbol{c}^{(k)})\Delta \boldsymbol{c}^{(k)} - (\boldsymbol{y} - \boldsymbol{g}(\boldsymbol{c}^{(k)}))\|_2 \to$ mín (por ejemplo, mediante el uso de las ecuaciones normales).
  - Defina $\boldsymbol{c}^{(k+1)} = \boldsymbol{c}^{(k)} + \Delta \boldsymbol{c}^{(k)}$.

En este ejercicio asumimos que f tiene la forma

$$f(x, \boldsymbol{c}) = c_1 e^{-c_2 x} \sin(c_3 x + c_4), x \in [0, 10],$$

donde $\boldsymbol{c} = [c_1, c_2, c_3, c_4]^T \in \mathbb{R}^4$ es el vector de coeficientes que debemos de encontrar (los cuales corresponden a la amplitud, el decaimiento, el periodo y la fase de f, respectivamente). Para obtener cotas del error, asumiremos que la solución exacta viene dada por el vector

$$\boldsymbol{c}_{\text{exact}} = [1, 1/2, 2, 0]^T.$$

(a) Utilice el comando `load data` para cargar los datos guardados en el archivo `data.dat` suministrado. Este archivo incluye los valores $\{(x_i, y_i)\}_{i=1}^m$ para $m = 100$, guardados en dos vectores $\boldsymbol{x}$ y $\boldsymbol{y}$.

(b) Implemente un programa que ejecute el algoritmo descrito. Para detener la iteración, utilice la condición $\|\Delta \boldsymbol{c}^{(k)}\|_\infty < 10^{-8}$.

(c) Corra su programa para $\boldsymbol{c}_0 = [1.1; 0.4; 2.1; 0.2]$. Reporte el número de iteraciones requeridas. Grafique los nodos $\{x_i, y_i\}_{i=1}^m$, la función exacta $f(x, \boldsymbol{c}_{\text{exact}})$ y la función aproximada $f(x, \boldsymbol{c}^{(k)})$. Grafique además $\|\Delta \boldsymbol{c}^{(k)}\|_\infty$ en función de k. ¿Qué puede deducir sobre la velocidad de convergencia?

(d) Para el valor inicial del inciso anterior, grafique el número de condición $\kappa(J(\boldsymbol{c}^{(k)}))$ en función de k. ¿Qué nos indica este gráfico? Sugerencia: utilice la función `cond(J)`.

(e) (1 puntos) Ahora considere $\boldsymbol{c}_0 = [1, 1, 1, 1]^T$. ¿Qué ocurre en este caso?

Para lograr mejores resultados, considere el sistema

(7.7)
$$\left[J(\boldsymbol{c}^{(k)})^T J(\boldsymbol{c}^{(k)}) + \lambda_k \text{Diag}(J(\boldsymbol{c}^{(k)})^T J(\boldsymbol{c}^{(k)}))\right] \Delta \boldsymbol{c}^{(k)} = J(\boldsymbol{c}^{(k)})^T (\boldsymbol{y} - \boldsymbol{g}(\boldsymbol{c}^{(k)})),$$

donde λ_k se conoce como parámetro de *damping* (que garantiza que la norma del residuo disminuye en cada iteración) y $\text{Diag}(J(\boldsymbol{c}^{(k)})^T J(\boldsymbol{c}^{(k)}))$ es una

matriz diagonal cuyas entradas son la diagonal de $J(\boldsymbol{c}^{(k)})^T J(\boldsymbol{c}^{(k)})$. Se puede demostrar que si λ_k toma valores altos, el método converge (pero lentamente), mientras que si λ_k es pequeño, el método - en caso de convergencia - converge rápidamente. Note que si $\lambda_k = 0$, entonces el sistema corresponde a las ecuaciones normales del problema (7.6). Considere así el siguiente algoritmo:

- Considere $\boldsymbol{c}^{(0)}$ dado y defina $\lambda = 1$.
- Para $k = 0, 1, \ldots$ hasta convergencia:
  - Evalúe el residuo $r_k = (\boldsymbol{g}(\boldsymbol{c}^{(k)}) - \boldsymbol{y})^T(\boldsymbol{g}(\boldsymbol{c}^{(k)}) - \boldsymbol{y})$.
  - Usando $\lambda_k = \lambda$, resuelva el sistema (7.7) y calcule un nuevo valor de prueba $\boldsymbol{c}^{(k+1)}$. Para este valor $\boldsymbol{c}^{(k+1)}$, calcule el nuevo residuo $r_{k+1} = (\boldsymbol{g}(\boldsymbol{c}^{(k+1)}) - \boldsymbol{y})^T(\boldsymbol{g}(\boldsymbol{c}^{(k+1)}) - \boldsymbol{y})$.
  - Usando $\lambda_k = \lambda/2$, resuelva el sistema (7.7) y calcule un nuevo valor de prueba $\tilde{\boldsymbol{c}}^{(k+1)}$. Para este valor $\tilde{\boldsymbol{c}}^{(k+1)}$, calcule el nuevo residuo $\tilde{r}_{k+1} = (\boldsymbol{g}(\tilde{\boldsymbol{c}}^{(k+1)}) - \boldsymbol{y})^T(\boldsymbol{g}(\tilde{\boldsymbol{c}}^{(k+1)}) - \boldsymbol{y})$.
  - Si r_{k+1} es menor que r_k y $\tilde{r}_{k+1}$, acepte $\boldsymbol{c}^{(k+1)}$.
  - Si $\tilde{r}_{k+1}$ es menor que r_k y r_{k+1}, tome $\boldsymbol{c}^{(k+1)} := \tilde{\boldsymbol{c}}^{(k+1)}$ y haga $\lambda \leftarrow \lambda/2$.
  - Si r_k es menor que $\tilde{r}_{k+1}$ y r_{k+1}, haga $\lambda \leftarrow 2\lambda$ y calcule el nuevo residuo. Continúe duplicando λ hasta que el nuevo residuo sea menor que r_k. Acepte $\boldsymbol{c}^{(k+1)}$.

(f) Implemente un programa que ejecute el algoritmo descrito anteriormente. Para detener la iteración, utilice la condición $\|\Delta \boldsymbol{c}_k\|_\infty < 10^{-8}$. Además, contabilice el número de sistemas de ecuaciones que son resueltos en su código.

(g) Corra su nuevo programa para $\boldsymbol{c}_0 = [1.1; 0.4; 2.1; 0.2]$. Reporte el número de iteraciones requeridas y compare con los resultados del inciso (c). ¿Cuántos sistemas de ecuaciones se resolvieron en ambos casos?

(g) Ahora considere $\boldsymbol{c}_0 = [1, 1, 1, 1]^T$. Reporte el número de iteraciones requeridas. Grafique los nodos $\{x_i, y_i\}_{i=1}^m$, la función exacta $f(x, \boldsymbol{c}_{\text{exact}})$ y la función aproximada $f(x, \boldsymbol{c}^{(k)})$. Grafique además $\|\Delta \boldsymbol{c}^{(k)}\|_\infty$ en función de k. ¿Qué puede deducir sobre la velocidad de convergencia?

Capítulo 8

Ecuaciones Diferenciales Ordinarias

8.1. Introducción

En este capítulo exploraremos métodos básicos para la resolución numérica de Ecuaciones Diferenciales Ordinarias (EDOs). En general, dada una función $\boldsymbol{f} : [t_0, \infty) \times \mathbb{R}^d \to \mathbb{R}^d$ y una condición inicial $\boldsymbol{y}_0$, estamos interesados en encontrar $\boldsymbol{y} : [t_0, \infty) \to \mathbb{R}^d$ de manera tal que satisfaga el sistema de primer orden

$$\begin{cases} \boldsymbol{y}' = \boldsymbol{f}(t, \boldsymbol{y}), & t > t_0, \\ \boldsymbol{y}(t_0) = \boldsymbol{y}_0 \end{cases} . \tag{8.1}$$

Recuerde que para garantizar existencia y unicidad basta asumir que existe L tal que

$$\|\boldsymbol{f}(t, \boldsymbol{x}) - \boldsymbol{f}(t, \boldsymbol{y})\| \leqslant L\|\boldsymbol{x} - \boldsymbol{y}\| \quad \forall\ \boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d,\ t \geqslant t_0. \tag{8.2}$$

En particular estudiaremos sistemas de ecuaciones de primer orden de la forma (8.1). Recuerde que ecuaciones de mayor orden pueden ser escritas como un sistema de ecuaciones de primer orden. En la sección 8.7 hablaremos sobre algunas consideraciones prácticas en el caso de tener una ODE implícita de la forma

$$\boldsymbol{g}(t, \boldsymbol{y}, \boldsymbol{y}') = \underline{0}.$$

Buscaremos una aproximación para $\boldsymbol{y}$ en un intervalo cerrado $[t_0, t_n]$. Consideraremos una partición uniforme dada por los nodos

$$t_0 < t_1 < \ldots < t_n, \quad t_k := t_0 + kh,$$

donde $h := (t_n - t_0)/n$ es el tamaño del incremento en el tiempo, y obtendremos aproximaciones $\boldsymbol{y}_k \approx \boldsymbol{y}(t_k)$.

Si integramos (8.1) tenemos que la ecuación diferencial es equivalente a encontrar $\boldsymbol{y}$ tal que

$$\boldsymbol{y}(t) = \boldsymbol{y}_0 + \int_{t_0}^{t} \boldsymbol{f}(\tau, \boldsymbol{y}(\tau))\ d\tau.$$

De esta manera, basta aproximar la integral de manera precisa para obtener diferentes algoritmos. Discutiremos los más básicos a continuación, en donde asumimos que $\boldsymbol{f}$ es lo suficientemente suave para obtener sus desarrollos de Taylor, según corresponda. En nuestra exposición seguimos las ideas de [15], y referimos también a [4, 3] para profundizar en estos temas.

8.2. Método de Euler

Si integramos (8.1) sobre el intervalo $[t_k, t_{k+1}]$, tenemos que

$$\boldsymbol{y}(t_{k+1}) = \boldsymbol{y}(t_k) + \int_{t_k}^{t_{k+1}} \boldsymbol{f}(\tau, \boldsymbol{y}(\tau))\ d\tau.$$

Si aproximamos la integral mediante su valor en el extremo izquierdo del intervalo,

$$\int_{t_k}^{t_{k+1}} \boldsymbol{f}(\tau, \boldsymbol{y}(\tau))\ d\tau \approx (t_{k+1} - t_k)\boldsymbol{f}(t_k, \boldsymbol{y}(t_k)),$$

podemos considerar la iteración explícita

$$\boldsymbol{y}_{k+1} = \boldsymbol{y}_k + h\boldsymbol{f}(t_k, \boldsymbol{y}_k)\ (k \geqslant 0), \quad \boldsymbol{y}_0 = \boldsymbol{y}(t_0), \tag{8.3}$$

la cual es llamada *Método de Euler*. Dicho método es el más simple de todos, y como veremos a continuación, converge en el sentido que

$$\lim_{h \to 0^+} \max_{k=1,\ldots,n} \|\boldsymbol{y}_k - \boldsymbol{y}(t_k)\| = 0;$$

esto es, la norma del error local $\|\boldsymbol{y}_k - \boldsymbol{y}(t_k)\|$ tiende a cero en todos los nodos de la discretización en t, conforme disminuimos el paso del tiempo h.

Comentario 8.1. La iteración dada en (8.3) puede deducirse de manera sencilla, al menos en el caso unidimensional, si consideramos el desarrollo

$$y'(t_k) = \frac{y(t_k + h) - y(t_k)}{h} + \mathcal{O}(h).$$

Al truncar el término de orden h y aproximar $y(t_k)$ por y_k, obtenemos (8.3). □

Teorema 8.2. *La iteración* (8.3) *converge a la solución de la ecuación diferencial* (8.1).

Demostración. Sea $h > 0$. Defina el error en el nodo t_k por

$$\boldsymbol{e}_k := \boldsymbol{y}_k - \boldsymbol{y}(t_k).$$

Vamos a demostrar que

$$\lim_{h\to 0^+} \max_{k=1,\ldots,n} \|\boldsymbol{e}_k\| = 0.$$

Primero, al utilizar un desarrollo de Taylor, tenemos que

$$\begin{aligned}\boldsymbol{y}(t_{k+1}) &= \boldsymbol{y}(t_k) + h\boldsymbol{y}'(t_k) + \mathcal{O}(h^2)\\ &= \boldsymbol{y}(t_k) + h\boldsymbol{f}(t_k, \boldsymbol{y}(t_k)) + \mathcal{O}(h^2).\end{aligned}$$

Utilizando (8.3), obtenemos que

$$\boldsymbol{e}_{k+1} = \boldsymbol{e}_k + h\left[\boldsymbol{f}(t_k, \boldsymbol{y}(t_k) + \boldsymbol{e}_k) - \boldsymbol{f}(t_k, \boldsymbol{y}(t_k))\right] + \mathcal{O}(h^2),$$

y utilizando (8.2) deducimos que

$$\|\boldsymbol{e}_{k+1}\| \leqslant (1 + hL)\|\boldsymbol{e}_k\| + ch^2.$$

Por inducción es fácil ver que

$$\|\boldsymbol{e}_k\| \leqslant \frac{ch}{L}\left[(1 + hL)^k - 1\right],\ k = 0, 1, \ldots, n,$$

y como $1 + x < e^x$ para $x > 0$, entonces

$$\|\boldsymbol{e}_k\| < \frac{ch}{L}\left(e^{Lhk} - 1\right) \leqslant \frac{ch}{L}\left(e^{L(t_n - t_0)} - 1\right),\ k = 0, 1, \ldots, n.$$

De esta manera podemos concluir que

$$\max_{k=1,\ldots,n} \|\boldsymbol{e}_k\| \to 0 \quad \text{cuando} \quad h \to 0,$$

por lo que el método de Euler converge. □

Definición 8.3. Se dice que un método para resolver la ecuación diferencial (8.1) tiene orden p si dicho método obtiene de manera exacta soluciones polinomiales de grado p.

Teorema 8.4. *El Método de Euler es de orden 1; esto es, si la solución de la ecuación diferencial* (8.1) *es un polinomio de grado 1, entonces el método recupera dicha solución de manera exacta.*

Demostración. Como

$$\boldsymbol{y}_{k+1} - (\boldsymbol{y}_k + h\boldsymbol{f}(t_k, \boldsymbol{y}_k)) = 0,$$

al sustituir la solución exacta en el lado de la izquierda obtenemos que

$$\begin{aligned}\boldsymbol{y}(t_{k+1}) &- (\boldsymbol{y}(t_k) + h\boldsymbol{f}(t_k, \boldsymbol{y}(t_k))) \\ &= \boldsymbol{y}(t_k) + h\boldsymbol{y}'(t_k) + \mathcal{O}(h^2) - \big(\boldsymbol{y}(t_k) + h\boldsymbol{y}'(t_k)\big) \\ &= \mathcal{O}(h^2).\end{aligned}$$

De esta forma, obtenemos la solución exacta si $\boldsymbol{y}$ es un polinomio de grado uno. □

8.3. Método del trapecio

En este caso consideramos la aproximación dada por la regla del trapecio,

$$\int_{t_k}^{t_{k+1}} \boldsymbol{f}(\tau, \boldsymbol{y}(\tau))\, d\tau \approx \frac{1}{2}(t_{k+1} - t_k)\left(\boldsymbol{f}(t_k, \boldsymbol{y}(t_k)) + \boldsymbol{f}(t_{k+1}, \boldsymbol{y}(t_{k+1}))\right).$$

Definimos así la iteración

$$\boldsymbol{y}_{k+1} = \boldsymbol{y}_k + \frac{1}{2}h\left[\boldsymbol{f}(t_k, \boldsymbol{y}_k) + \boldsymbol{f}(t_{k+1}, \boldsymbol{y}_{k+1})\right] \ (k \geqslant 0), \quad \boldsymbol{y}_0 = \boldsymbol{y}(t_0), \tag{8.4}$$

la cual es llamada *Método del Trapecio*. Note que el término $\boldsymbol{y}_{k+1}$ está definido de manera implícita y que en cada iteración debe resolverse una ecuación (en general no lineal) para encontrar $\boldsymbol{y}_{k+1}$.

Para este método, tenemos el siguiente resultado:

Teorema 8.5. *La iteración* (8.4) *es convergente y tiene orden dos.*

Demostración. Sabemos que

$$\boldsymbol{y}'(t_k) = \boldsymbol{f}(t_k, \boldsymbol{y}(t_k)) \quad \text{y} \quad \boldsymbol{y}'(t_{k+1}) = \boldsymbol{f}(t_{k+1}, \boldsymbol{y}(t_{k+1})),$$

por lo que $\boldsymbol{f}(t_{k+1}, \boldsymbol{y}(t_{k+1})) = \boldsymbol{y}'(t_k) + h\boldsymbol{y}''(t_k) + \mathcal{O}(h^2)$. De esta forma,

$$\begin{aligned} \boldsymbol{y}(t_{k+1}) &= \boldsymbol{y}(t_k) + h\boldsymbol{y}'(t_k) + \frac{1}{2}h^2\boldsymbol{y}''(t_k) + \mathcal{O}(h^3) \\ &= \boldsymbol{y}(t_k) + \frac{1}{2}h\left[\boldsymbol{f}(t_k, \boldsymbol{y}(t_k)) + \boldsymbol{f}(t_{k+1}, \boldsymbol{y}(t_{k+1}))\right] + \mathcal{O}(h^3). \end{aligned} \tag{8.5}$$

Restando (8.4) tenemos que

$$\|\boldsymbol{e}_{k+1}\| \leqslant \|\boldsymbol{e}_k\| + \frac{1}{2}hL\left(\|\boldsymbol{e}_k\| + \|\boldsymbol{e}_{k+1}\|\right) + ch^3.$$

Asumiendo que $hL < 2$ (lo cual tiene sentido pues h es pequeño), tenemos que

$$\|\boldsymbol{e}_{k+1}\| \leqslant \left(\frac{1 + \frac{1}{2}hL}{1 - \frac{1}{2}hL}\right)\|\boldsymbol{e}_k\| + \frac{c}{1 - \frac{1}{2}hL}h^3,$$

y por inducción se puede probar que

$$\|\boldsymbol{e}_k\| \leqslant \frac{c}{L}\left[\left(\frac{1 + \frac{1}{2}hL}{1 - \frac{1}{2}hL}\right)^k - 1\right]h^2 \quad (k = 0, \ldots, n).$$

De esta forma,

$$\begin{aligned} \|\boldsymbol{e}_k\| &\leqslant \frac{ch^2}{L}e^{\dfrac{nhL}{1 - \frac{1}{2}hL}} \\ &\leqslant \frac{ch^2}{L}e^{\dfrac{L(t_n - t_0)}{1 - \frac{1}{2}hL}} \to 0 \quad \text{cuando} \quad h \to 0. \end{aligned}$$

De esta forma comprobamos la convergencia. Para ver que el método es de orden dos, al sustituir la solución exacta en (8.4) y utilizando (8.5) se deduce que

$$\boldsymbol{y}(t_{k+1}) - \left(\boldsymbol{y}(t_k) + \frac{1}{2}h\left[\boldsymbol{f}(t_k, \boldsymbol{y}(t_k)) + \boldsymbol{f}(t_{k+1}, \boldsymbol{y}(t_{k+1}))\right]\right) = \mathcal{O}(h^3),$$

lo que implica que el orden del método del trapecio es dos. □

Comentario 8.6. Los métodos implícitos tienen la desventaja que en cada iteración se debe resolver un sistema de ecuaciones, usualmente no lineal. Una idea alternativa consiste en combinar métodos explícitos (para predecir el valor de la siguiente iteración) con métodos implícitos (para corregir el valor que se predijo); para una mayor discusión referimos a [3].

El Método predictor-corrector combina el Método de Euler (8.3) (explícito) con el Método del Trapecio (8.4) (implícito). En la iteración k, predecimos el valor $\boldsymbol{y}_{k+1}$ con un paso de Euler:

$$\widetilde{\boldsymbol{y}}_{k+1} = \boldsymbol{y}_k + h\boldsymbol{f}(t_k, \boldsymbol{y}_k).$$

Luego, corregimos dicha aproximación mediante el Método del Trapecio:

$$\boldsymbol{y}_{k+1} = \boldsymbol{y}_k + \frac{1}{2}h\left[\boldsymbol{f}(t_k, \boldsymbol{y}_k) + \boldsymbol{f}(t_{k+1}, \widetilde{\boldsymbol{y}}_{k+1})\right].$$

De esta manera, evitamos resolver el sistema asociado a la iteración del Método del Trapecio, y solamente necesitamos evaluar la función en dos puntos por cada iteración.

Comentario 8.7. Es posible evaluar la función f solamente una vez por iteración mediante el esquema

$$\begin{aligned}\widetilde{\boldsymbol{y}}_{k+1} &= \boldsymbol{y}_k + h\boldsymbol{f}(t_k, \widetilde{\boldsymbol{y}}_k),\\ \boldsymbol{y}_{k+1} &= \boldsymbol{y}_k + \frac{1}{2}h\left[\boldsymbol{f}(t_k, \widetilde{\boldsymbol{y}}_k) + \boldsymbol{f}(t_{k+1}, \widetilde{\boldsymbol{y}}_{k+1})\right],\end{aligned}$$

donde tomamos $\widetilde{\boldsymbol{y}}_0 = \boldsymbol{y}_0$. Este método es conocido como *Predecir – Evaluar – Corregir* (PEC).

Comentario 8.8. En general, podemos considerar la iteración

(8.6)
$$\boldsymbol{y}_{k+1} = \boldsymbol{y}_k + h\left[\theta\boldsymbol{f}(t_k, \boldsymbol{y}_k) + (1-\theta)\boldsymbol{f}(t_{k+1}, \boldsymbol{y}_{k+1})\right] \ (k \geqslant 0), \quad \boldsymbol{y}_0 = \boldsymbol{y}(t_0),$$

con $\theta \in [0, 1]$. Es claro que el método de Euler corresponde a $\theta = 1$ y el método del trapecio corresponde a $\theta = 1/2$. En general, dicho método tiene orden 1 si $\theta \neq 1/2$ y orden dos solo si $\theta = 1/2$. Además, el método es explícito si y solo si $\theta = 1$. □

Ejercicio 8.9. Verifique que la iteración (8.6) tiene orden dos si y solo si $\theta = 1/2$.

Comentario 8.10. Se pueden definir además los llamados *métodos multipasos*, en donde utilizamos el conocimiento de valores anteriores ya calculados. Por ejemplo, los famoso métodos explícitos de Adams con dos y tres pasos vienen dados por las iteraciones

$$\boldsymbol{y}_{k+2} = \boldsymbol{y}_{k+1} + h\left[\frac{3}{2}(t_{k+1}, \boldsymbol{y}_{k+1}) - \frac{1}{2}\boldsymbol{f}(t_k, \boldsymbol{y}_k)\right], \quad \text{y}$$

$$\boldsymbol{y}_{k+3} = \boldsymbol{y}_{k+2} + h\left[\frac{23}{12}(t_{k+2}, \boldsymbol{y}_{k+2}) - \frac{4}{3}\boldsymbol{f}(t_{k+1}, \boldsymbol{y}_{k+1}) + \frac{5}{12}\boldsymbol{f}(t_k, \boldsymbol{y}_k)\right],$$

los cuales tienen orden 2 y 3, respectivamente. De esta manera, incluir valores pasados conocidos permite aumentar el orden del método. □

Ejercicio 8.11. Verifique que el orden de los algoritmos multipaso descritos en el Comentario 8.10 tienen orden dos y tres, respectivamente.

8.4. Métodos de Runge-Kuta Explícitos

De la ecuación diferencial, al hacer la sustitución $\tau \leftarrow t_k + h\tau$, sabemos que

(8.7)

$$\boldsymbol{y}(t_{k+1}) = \boldsymbol{y}(t_k) + \int_{t_k}^{t_{k+1}} \boldsymbol{f}(\tau, \boldsymbol{y}(\tau))\, d\tau = \boldsymbol{y}(t_k) + h\int_0^1 \boldsymbol{f}(t_k + h\tau, \boldsymbol{y}(t_k + h\tau))\, d\tau.$$

Una manera más precisa de aproximar la integral corresponde a utilizar fórmulas de cuadratura. De esta manera, dados ν nodos $\{c_1 \ldots, c_\nu\}$ y ν pesos $\{b_1, \ldots, b_\nu\}$, aproximamos

$$\int_0^1 \boldsymbol{f}(t_k + h\tau, \boldsymbol{y}(t_k + h\tau))\, d\tau \approx \sum_{j=1}^{\nu} b_j \boldsymbol{f}(t_k + c_j h, \boldsymbol{y}(t_k + c_j h)). \tag{8.8}$$

No obstante, los valores $\boldsymbol{y}(t_k + c_j h)$ son desconocidos por lo que debemos aproximarlos. De esta manera, denote por ξ_j a una aproximación de $\boldsymbol{y}(t_k + c_j h)$. Utilizando ideas similares al método de Euler y del trapecio, expresamos dichas aproximaciones como un sistema de ecuaciones (explícito)

dado por

$$\begin{aligned}
\xi_1 &= \boldsymbol{y}_k, \quad \text{(donde tomamos } \ c_1 = 0) \\
\xi_2 &= \boldsymbol{y}_k + h a_{2,1} \boldsymbol{f}(t_k, \xi_1), \\
\xi_3 &= \boldsymbol{y}_k + h a_{3,1} \boldsymbol{f}(t_k, \xi_1) + h a_{3,2} \boldsymbol{f}(t_k + c_2 h, \xi_2), \\
&\vdots \\
\xi_\nu &= \boldsymbol{y}_k + h \sum_{j=1}^{\nu-1} a_{\nu,j} \boldsymbol{f}(t_k + c_j h, \xi_j), \qquad (8.9)
\end{aligned}$$

y luego, de (8.7) y (8.8) tomamos

$$\boldsymbol{y}_{k+1} = \boldsymbol{y}_k + h \sum_{j=1}^{\nu} b_j \boldsymbol{f}(t_k + c_j h, \xi_j). \qquad (8.10)$$

La iteración anterior se conoce como *Método de Runge-Kuta explícito*, donde los valores ξ_j se calculan secuencialmente. Note que debemos determinar el vector de pesos $\boldsymbol{b} = [b_1, \ldots, b_\nu]^T$, el vector de nodos $\boldsymbol{c} = [c_1, \ldots, c_\nu]^T$ y los coeficientes de la matriz triangular inferior A, cuyas entradas son

$$(A)_{i,j} = a_{i,j},$$

de manera tal que el orden del algoritmo sea el mayor posible.

Por ejemplo, considere $\nu = 2$. En este caso, podemos calcular

$$\begin{aligned}
\boldsymbol{f}(t_k + c_2 h, \xi_2) &= \boldsymbol{f}(t_k + c_2 h, \boldsymbol{y}_k + a_{2,1} h \boldsymbol{f}(t_k, \boldsymbol{y}_k)) \\
&= \boldsymbol{f}(t_k, \boldsymbol{y}_k) + h \left[c_2 \frac{\partial \boldsymbol{f}}{\partial t}(t_k, \boldsymbol{y}_k) + a_{2,1} \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{y}}(t_k, \boldsymbol{y}_k) \boldsymbol{f}(t_k, \boldsymbol{y}_k) \right] + \mathcal{O}(h^2).
\end{aligned}$$

De esta manera,

$$\begin{aligned}
\boldsymbol{y}_{k+1} &= \boldsymbol{y}_k + h \left[b_1 \boldsymbol{f}(t_k, \boldsymbol{y}_k) + b_2 \boldsymbol{f}(t_k + c_2 h, \xi_2) \right] \\
&= \boldsymbol{y}_k + h(b_1 + b_2) \boldsymbol{f}(t_k, \boldsymbol{y}_k) + \\
&\quad + h^2 b_2 \left[c_2 \frac{\partial \boldsymbol{f}}{\partial t}(t_k, \boldsymbol{y}_k) + a_{2,1} \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{y}}(t_k, \boldsymbol{y}_k) \boldsymbol{f}(t_k, \boldsymbol{y}_k) \right] + \mathcal{O}(h^3). \qquad (8.11)
\end{aligned}$$

Como $\boldsymbol{y}'(t) = \boldsymbol{f}(t, \boldsymbol{y})$, entonces $\boldsymbol{y}''(t) = \dfrac{\partial \boldsymbol{f}}{\partial t}(t, \boldsymbol{y}) + \dfrac{\partial \boldsymbol{f}}{\partial \boldsymbol{y}}(t, \boldsymbol{y}) \boldsymbol{f}(t, \boldsymbol{y})$. Luego, si $\widehat{\boldsymbol{y}}$ es la solución exacta con condición inicial $\widehat{\boldsymbol{y}}(t_k) = \boldsymbol{y}_k$ en el intervalo $[t_k, t_{k+1}]$, podemos escribir

$$\begin{aligned}
\widehat{\boldsymbol{y}}(t_{k+1}) &= \boldsymbol{y}_k + h \boldsymbol{f}(t_k, \boldsymbol{y}_k) + \\
&\quad + \frac{1}{2} h^2 \left[\frac{\partial \boldsymbol{f}}{\partial t}(t_k, \boldsymbol{y}_k) + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{y}}(t_k, \boldsymbol{y}_k) \boldsymbol{f}(t_k, \boldsymbol{y}_k) \right] + \mathcal{O}(h^3). \qquad (8.12)
\end{aligned}$$

Por lo tanto, al comparar (8.11) y (8.12), el método tendrá orden dos si escogemos los nodos, pesos y coeficientes tales que

$$\begin{aligned} b_1 + b_2 &= 1, \\ b_2 c_2 &= 1/2, \\ a_{2,1} &= c_2. \end{aligned}$$

Algunas escogencias usuales son las siguientes:

$$\begin{array}{c|cc} 0 & & \\ \frac{1}{2} & \frac{1}{2} & \\ \hline & 0 & 1 \end{array}, \qquad \begin{array}{c|cc} 0 & & \\ \frac{2}{3} & \frac{2}{3} & \\ \hline & \frac{1}{4} & \frac{3}{4} \end{array}, \qquad \begin{array}{c|cc} 0 & & \\ 1 & 1 & \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array},$$

donde omitimos las entradas nulas y utilizamos la notación usual

$$\begin{array}{c|c} \boldsymbol{c} & A \\ \hline & \boldsymbol{b}^T \end{array}.$$

Para el caso $\nu = 3$ se puede hacer un análisis similar; ver Ejercicio 8.12. Los dos métodos más comunes de orden tres corresponden a las escogencias

$$\begin{array}{c|ccc} 0 & & & \\ 1/2 & 1/2 & & \\ 1 & -1 & 2 & \\ \hline & 1 & 2/3 & 1/6 \end{array}, \qquad \begin{array}{c|ccc} 0 & & & \\ 2/3 & 2/3 & & \\ 2/3 & 0 & 2/3 & \\ \hline & 1/4 & 3/8 & 3/8 \end{array}, \tag{8.13}$$

los cuales se conocen con el nombre de *Runge-Kuta clásico* y *Nystrom*, respectivamente. Finalmente, para $\nu = 4$, un método de orden cuatro viene dado por los coeficientes

$$\begin{array}{c|cccc} 0 & & & & \\ 1/2 & 1/2 & & & \\ 1/2 & 0 & 1/2 & & \\ 1 & 0 & 0 & 1 & \\ \hline & 1/6 & 1/3 & 1/3 & 1/6 \end{array}. \tag{8.14}$$

En general, considerar mayores valores para ν no es usual. Primero, el análisis se vuelve un poco tedioso. Segundo, si deseamos un método de Runge-Kuta de orden cinco se necesita $\nu = 6$, y para mayores órdenes la situación empeora. En tales casos, es mejor utilizar métodos implícitos.

Ejercicio 8.12. Verifique que los métodos explícitos dados en (8.13) tienen orden tres.

8.5. Métodos de Runge-Kuta Implícitos

En las aproximaciones (8.9), cada valor ξ_i se calcula a partir de los valores anteriores $\xi_1, \ldots, \xi_{i-1}$. No obstante, podemos considerar el método implícito

$$\xi_i = \boldsymbol{y}_k + h \sum_{j=1}^{\nu} a_{i,j} \boldsymbol{f}(t_k + c_j h, \xi_j), \quad i = 1, \ldots, \nu, \tag{8.15}$$

para definir, de manera similar a (8.10),

$$\boldsymbol{y}_{k+1} = \boldsymbol{y}_k + h \sum_{j=1}^{\nu} b_j \boldsymbol{f}(t_k + c_j h, \xi_j). \tag{8.16}$$

En esta sección consideramos una subclase de dichos métodos implícitos, llamados *métodos de colocación*. Para ello, considere el problema de encontrar $\boldsymbol{u} \in (\mathcal{P}_\nu)^d$ tal que

$$\begin{cases} \boldsymbol{u}(t_k) = \boldsymbol{y}_k, \\ \boldsymbol{u}'(t_k + c_j h) = \boldsymbol{f}(t_k + c_j h, \boldsymbol{u}(t_k + c_j h)) \quad (j = 1, \ldots, \nu). \end{cases} \tag{8.17}$$

Esto es, $\boldsymbol{u}$ satisface la ecuación diferencial (8.1) en ν puntos diferentes. Una vez que conocemos $\boldsymbol{u}$, procedemos a definir

$$\boldsymbol{y}_{k+1} = \boldsymbol{u}(t_{k+1}). \tag{8.18}$$

En palabras sencillas, en cada intervalo $[t_k, t_{k+1}]$ aproximamos la solución por un polinomio de grado ν. El nuevo valor $\boldsymbol{y}_{k+1}$ se obtiene entonces al evaluar dicho polinomio en t_{k+1}. Dicho método de colocación corresponde a un Método de Runge-Kuta Implícito, tal como lo demostramos a continuación:

Lema 8.13. *Considere $c_1, \ldots, c_\nu$ dados. Defina*

$$\begin{aligned} q(t) &:= \prod_{j=1}^{\nu} (t - c_j), \quad q_l(t) = \frac{q(t)}{t - c_l} \quad (l = 1, \ldots, \nu), \\ a_{i,j} &= \int_0^{c_i} \frac{q_j(\tau)}{q_j(c_j)} \, d\tau \quad (i, j = 1, \ldots, \nu), \\ b_i &= \int_0^1 \frac{q_i(\tau)}{q_i(c_i)} \, d\tau \quad (j = 1, \ldots, nu). \end{aligned} \tag{8.19}$$

Entonces, el método de colocación (8.18) *corresponde al método de Runge-Kuta implícito* (8.15), (8.16).

Demostración. Podemos escribir el polinomio de interpolación de Lagrange para $\boldsymbol{u}'$ como

$$\boldsymbol{u}'(t) = \sum_{l=1}^{\nu} \frac{q_l\left(\frac{t-t_k}{h}\right)}{q_l(c_l)}\ \boldsymbol{u}'(t_k + c_l h) = \sum_{l=1}^{\nu} \frac{q_l\left(\frac{t-t_k}{h}\right)}{q_l(c_l)}\ \boldsymbol{f}(t_k + c_l h, \boldsymbol{u}(t_k + c_l h)).$$

Al integrar y utilizando la condición inicial para $\boldsymbol{u}$, tenemos entonces que

$$\begin{aligned}\boldsymbol{u}(t) &= \boldsymbol{y}_n + \int_{t_k}^{t} \boldsymbol{u}'(\tau)\ d\tau \\ &= \boldsymbol{y}_n + h\sum_{l=1}^{\nu} \boldsymbol{f}(t_k + c_l h, \boldsymbol{u}(t_k + c_l h)) \int_0^{\frac{t-t_k}{h}} \frac{q_l(\tau)}{q_l(c_l)}\ d\tau.\end{aligned}$$

Defina $\xi_l = \boldsymbol{u}(t_k + c_l h)$ para $l = 1, \ldots, \nu$. Al evaluar $\boldsymbol{u}$ en $t_k + c_i h$ obtenemos que

$$\xi_i = \boldsymbol{y}_k + h\sum_{l=1}^{\nu} a_{i,l}\boldsymbol{f}(t_k + c_l h, \xi_l),$$

que corresponde a (8.15). Similarmente, al evaluar $\boldsymbol{u}$ en t_{k+1} obtenemos (8.16), de donde concluimos la prueba. □

La ventaja de hablar de métodos de colocación radica en el siguiente teorema:

Teorema 8.14. *Sean $c_1, \ldots, c_\nu$ los nodos del polinomios de Legendre de grado ν en el intervalo $[0, 1]$. Entonces, el método de Runge-Kuta implícito* (8.15), (8.16) *con constantes definidas en el Lema 8.13 tiene orden 2ν.*

Demostración. Referimos al interesado a leer [15, Capítulo 3.3]. □

Ejemplo 8.15. Considere primero $\nu = 1$. En este caso, el polinomio de Legendre es $p_1(t) = t - 1/2$, por lo que $c_1 = 1/2$. Al calcular las constantes dadas en (8.19) obtenemos el método de orden dos dado por

$$\begin{array}{c|c} 1/2 & 1/2 \\ \hline & 1 \end{array}.$$

Para $\nu = 2$, $p_2(t) = t^2 - t + 1/6$, cuyas raíces son $\frac{1}{2} \pm \frac{\sqrt{3}}{6}$. Luego, el método de orden cuatro viene dado por

$$\begin{array}{c|cc} \frac{1}{2} - \frac{\sqrt{3}}{6} & \frac{1}{4} & \frac{1}{4} - \frac{\sqrt{3}}{6} \\ \frac{1}{2} + \frac{\sqrt{3}}{6} & \frac{1}{4} + \frac{\sqrt{3}}{6} & \frac{1}{4} \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}.$$

8.6. Ecuaciones Diferenciales de tipo "stiff"

Se dice que una ecuación es *stiff* si se debe restringir el valor de h para evitar inestabilidad en la solución. Explicamos esto con un ejemplo. Considere la ecuación dada por

$$\begin{cases} \boldsymbol{y}' = \Lambda \boldsymbol{y}, & t \geqslant 0, \\ \boldsymbol{y}(0) = \boldsymbol{y}_0 \end{cases}, \tag{8.20}$$

donde

$$\Lambda = \begin{bmatrix} -100 & 1 \\ 0 & -1/10 \end{bmatrix}.$$

Es fácil ver que $\Lambda = VDV^{-1}$, donde

$$D := \begin{bmatrix} -100 & 0 \\ 0 & -1/10 \end{bmatrix}, \quad V := \begin{bmatrix} 1 & 1 \\ 0 & 999/10 \end{bmatrix}.$$

Así, la solución exacta viene dada por

$$\boldsymbol{y}(t) = Ve^{tD}V^{-1}\boldsymbol{y}_0 = e^{-100t}\boldsymbol{x}_1 + e^{-t/10}\boldsymbol{x}_2, \tag{8.21}$$

donde $\boldsymbol{x}_1, \boldsymbol{x}_2$ son dos vectores que dependen de la condición inicial y los vectores columna de V. De esta forma, se tiene que $\boldsymbol{y}(t) \to 0$ cuando $t \to \infty$.

Si utilizamos el método de Euler (8.3) tenemos que

$$\boldsymbol{y}_n = (I + h\Lambda)^n \boldsymbol{y}_0,$$

y como

$$(I + hD)^n = \begin{bmatrix} (1 - 100h)^n & 0 \\ 0 & (1 - h/10)^n \end{bmatrix},$$

tenemos que

$$\boldsymbol{y}_n = (1 - 100h)^n \boldsymbol{x}_1 + \left(1 - \frac{1}{10}h\right)^n \boldsymbol{x}_2,$$

donde $\boldsymbol{x}_1$ y $\boldsymbol{x}_2$ coinciden con los vectores definidos en (8.21). En este caso tenemos que $\boldsymbol{y}_n \to \infty$ cuando $n \to \infty$ si $h > 1/50$. Por lo tanto, debemos considerar valores menores para h si queremos obtener estabilidad en la solución. No obstante, si escogemos $\boldsymbol{y}_0 = [1, 999/10]^T$ se tiene que $\boldsymbol{x}_1 = \underline{0}$, $\boldsymbol{x}_2 = \boldsymbol{y}_0$, por lo que

$$\boldsymbol{y}_n = \left(1 - \frac{1}{10}h\right)^n \boldsymbol{y}_0,$$

y es claro ahora que $\boldsymbol{y}_n \to 0$ si $h < 20$ (el cual es un paso de tiempo grande). No obstante, numéricamente el término $(1 - 100h)^n$ eventualmente se hace presente en las aproximaciones (por errores de redondeo, por ejemplo, $\boldsymbol{x}_1$ no es exactamente cero). De esta manera, aún si $h < 20$ uno observa que la solución tiende a infinito para valores grandes de n. De esta manera, ocupamos siempre que $h < 1/50$ para obtener estabilidad en la solución numérica.

Por el contrario, si utilizamos la regla del trapecio, obtenemos que

$$\boldsymbol{y}_n = \left(\frac{1 - 50h}{1 + 50h}\right)^n \boldsymbol{z}_1 + \left(\frac{1 - h/20}{1 + h/20}\right)^n \boldsymbol{z}_2,$$

y en este caso el valor absoluto de ambas fracciones es menor que 1. De esta manera, no tenemos ninguna restricción para h para evitar que $\boldsymbol{y}_n$ tienda a infinito (solo consideramos h pequeño para obtener un error pequeño).

8.7. Resolviendo ecuaciones diferenciales ordinarias en MATLAB

Para resolver ODEs en MATLAB, se dispone de diferentes funciones definidas. En el Cuadro 8.1 se presenta un resumen de las diferentes opciones predeterminadas, basado en [17]. Ilustramos el uso de estos métodos en los siguientes dos ejemplos:

Ejemplo 8.16. Dado $\mu > 0$, considere la EDO de van der Pol dada por

$$u'' - \mu(1 - u^2)u' + u = 0,$$

el cual se puede reescribir como el sistema lineal (8.1) con

$$f(t, \boldsymbol{y}) = [y_2, \mu(1 - y_1^2)y_2 - y_1]^T, \tag{8.22}$$

donde $y_1 = u$, $y_2 = u'$. En MATLAB, podemos definir las opciones para resolver la ecuación diferencial de la siguiente manera:

Cuadro 8.1: Códigos para resolver EDOs en `MATLAB` (tomado de [17])

Comando	Tipo	Precisión
`ode15i`	Implícita	Baja
`ode15s`	Stiff	Baja - Media
`ode23s`	Stiff	Baja
`ode23t`	Stiff	Baja
`ode23tb`	Stiff	Baja
`ode23`	No stiff	Baja
`ode45`	No stiff	Media
`ode113`	No stiff	Baja - Alta

```
options = odeset('RelTol',1e-6,'AbsTol',1e-6,'Stats','on');
```

Una lista completa de las opciones se puede consultar en la ayuda de dicha función. Considere $\mu = 1000$ (para el cual la ecuación es tipo stiff). Para resolver (8.22), ejecutamos el siguiente código:

```
f   = @(t,y) [y(2); 1000*(1-y(1).^2).*y(2)-y(1)];
y0  = [2;0];
tic
[t,y] = ode23(f,[0,3000],y0,options);
toc
```

obteniendo la siguiente salida:

```
2.22766e+06 successful steps
40 failed attempts
6.68309e+06 function evaluations
Elapsed time is 21.319815 seconds.
```

Note que en este caso, se realizan casi 7 millones de evaluaciones de la función, y el resultado parece *extraño*; ver la solución en la Figura 8.1a. Si reemplazamos el *solver* por `ode15s` se obtiene ahora

```
1400 successful steps
215 failed attempts
3143 function evaluations
19 partial derivatives
315 LU decompositions
3085 solutions of linear systems
Elapsed time is 0.298213 seconds.
```

De esta forma, solo se necesita un segundo para obtener la solución. Note que en este caso el método incluye factorización de matrices, cálculo de derivadas parciales y resolución de sistemas lineales; ver la solución en la Figura 8.1b. Esto permite reducir el número de evaluaciones y obtener la solución en un tiempo mucho menor. Note que el comando `[t,y] = odexxx(...)` brinda un vector de valores de tiempo t y sus imágenes correspondientes. Uno también puede ejecutar el comando

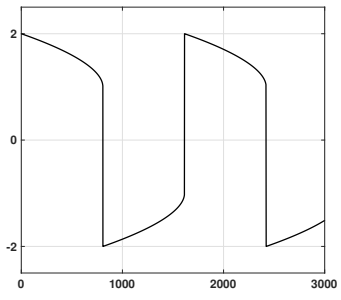
```
sol = ode23(f,[0,3000],y0,options);
```

para obtener una estructura

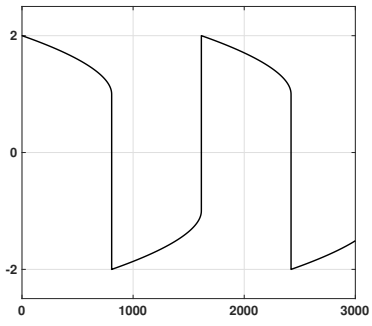
```
sol =
  struct with fields:
     solver: 'ode15s'
    extdata: [1x1 struct]
          x: [1x1401 double]
          y: [2x1401 double]
      stats: [1x1 struct]
      idata: [1x1 struct]
```

Dicha estructura puede ser evaluada mediante el comando `deval`:

```
tt = linspace(0,3000,1e5);
tic, uu = deval(sol,tt); toc
plot(tt,uu)
```



(a) `ode23`



(b) `ode15s`

Figura 8.1: Solución en `MATLAB` para la ED de van der Pol (8.22) con dos diferentes comandos.

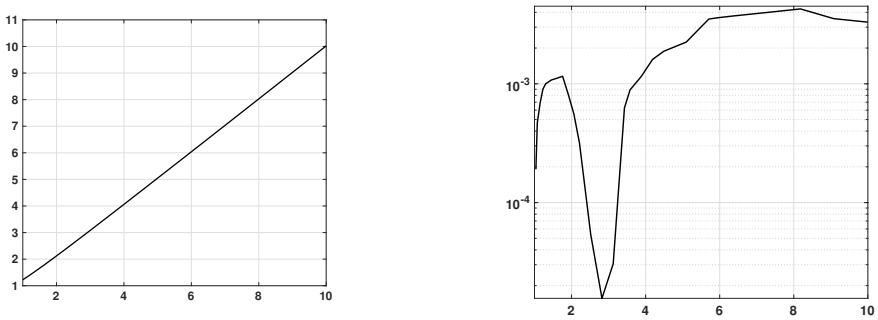
Ejemplo 8.17. Considere la ecuación diferencial implícita

$$ty^2(y')^3 - y^3(y')^2 + t(t^2+1)y' - t^2y = 0, \tag{8.23}$$

con solución exacta $y(t) = \sqrt{t^2 + 0.5}$. Para ello, `MATLAB` incluye la función `weissinger(t,y,yp)`, la cual calcula directamente el lado izquierdo de (8.23). Mediante el código

```
y0 = sqrt(3/2);
yp0 = sqrt(2/3);
[t,y] = ode15i(@weissinger,[1 10],y0,yp0);
```

se obtiene la solución mostrada en la Figura 8.2a; el error de la solución se observa en la Figura 8.2b.



(a) Solución

(b) Error

Figura 8.2: Solución en `MATLAB` para la ED implícita (8.23).

8.8. Método de Diferencias Finitas

Asumiendo que f es lo suficientemente suave, sabemos que

$$f(x+h) = f(x) + hf'(x) + \frac{1}{2}h^2f''(x) + \frac{1}{6}h^3f'''(x) + \mathcal{O}(h^4),$$
$$f(x-h) = f(x) - hf'(x) + \frac{1}{2}h^2f''(x) - \frac{1}{6}h^3f'''(x) + \mathcal{O}(h^4),$$

de donde tenemos que la primer derivada se puede expresar como

$$f'(x) = \frac{f(x+h) - f(x)}{h} + \mathcal{O}(h) \quad \text{(Diferencia hacia adelante)},$$
$$f'(x) = \frac{f(x) - f(x-h)}{h} + \mathcal{O}(h) \quad \text{(Diferencia hacia atrás)},$$
$$f'(x) = \frac{f(x+h) - f(x-h)}{2h} + \mathcal{O}(h^2) \quad \text{(Diferencia centrada)}. \tag{8.24}$$

Similarmente, podemos expresar la segunda derivada como

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + \mathcal{O}(h^2). \tag{8.25}$$

La idea básica del Método de Diferencias Finitas para resolver ecuaciones diferenciales consiste en aproximar las derivadas por expresiones similares a las anteriores. Exploraremos dos ejemplos que ilustran su uso para el caso de ecuaciones diferenciales ordinarias, pero este método se puede generalizar para resolver ecuaciones en derivadas parciales en geometrías sencillas y es tema de estudio en un curso posterior. Nuestro objetivo es ejemplificar cómo se puede utilizar para resolver ODEs de orden dos, evitando así reescribir el sistema como un sistema de dos EDOs.

Ejemplo 8.18. Considere la ecuación de segundo orden

$$-y''(t) + p(t)y'(t) + q(t)y(t) = f(t), \quad 0 < t < 1, \tag{8.26}$$

con condiciones de frontera $y(0) = \alpha$, $y(1) = \beta$. Asumimos por simplicidad que $p, q, f \in C([0,1])$. Considere una discretización uniforme del intervalo $[0,1]$ dada por $t_k = kh$ $(k = 0, \ldots, N)$, con $h = 1/N$ la distancia entre dos nodos consecutivos. Reemplazando las expresiones de orden dos dadas en (8.24) y (8.25) y evaluando en $t = t_k$ para $k = 1, \ldots, N-1$, se obtiene que

$$-\frac{y(t_{k+1}) - 2y(t_k) + y(t_{k-1})}{h^2} + p_k \frac{y(t_{k+1}) - y(t_{k-1})}{2h} + q_k y(t_k) + \mathcal{O}(h^2) = f_k, \tag{8.27}$$

donde $p_k := p(t_k)$, $q_k := q(t_k)$, $f_k := f(t_k)$ (note que para $k = 0$ y $k = N$ los valores $y(t_0)$ y $y(t_N)$ son conocidos por las condiciones de frontera). Truncando los términos de orden dos, aproximamos la solución mediante la expresión

$$-\frac{y_{k+1} - 2y_k + y_{k-1}}{h^2} + p_k \frac{y_{k+1} - y_{k-1}}{2h} + q_k y_k = f_k \quad (k = 1, \ldots, N-1), \tag{8.28}$$

de donde

$$\left(-1-\frac{h}{2}p_k\right)y_{k-1}+\left(2+h^2q_k\right)y_k+\left(-1+\frac{h}{2}p_k\right)y_{k+1}=h^2f_k. \tag{8.29}$$

Denotando $b_k := -1-\frac{h}{2}p_k$, $a_k := 2+h^2q_k$, $c_k := -1+\frac{h}{2}p_k$, las $N-1$ ecuaciones dadas en (8.29) se pueden escribir matricialmente como

(8.30)

$$\begin{bmatrix} a_1 & c_1 & & & & \\ b_2 & a_2 & c_2 & & & \\ & b_3 & a_3 & \ddots & & \\ & & \ddots & \ddots & c_{N-3} & \\ & & & b_{N-2} & a_{N-2} & c_{N-2} \\ & & & & b_{N-1} & a_{N-1} \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{N-2} \\ y_{N-1} \end{bmatrix} = \begin{bmatrix} h^2f_1-b_1\alpha \\ h^2f_2 \\ h^2f_3 \\ \vdots \\ h^2f_{N-2} \\ h^2f_{N-1}-c_{N-1}\beta \end{bmatrix}.$$

Note que $\boldsymbol{y}=[y_1,\ldots,y_{N-1}]^T\in\mathbb{R}^{N-1}$ solo incluye los valores que no conocemos en el interior del intervalo $[0,1]$, y los valores de frontera $y(0)=\alpha$, $y(1)=\beta$ se incluyen en el lado derecho. Esto pues al sustituir $k=1$ en (8.29) se obtiene el valor $y_0=\alpha$; de manera análoga, al sustituir $k=N-1$ se obtiene el valor $y_n=\beta$.

Note que por el Lema 7.13 podemos garantizar que la matriz es invertible en el caso particular que $q(t)>0$ (asumiendo que h es lo suficientemente pequeño). En un caso general, debe verificarse que dicha matriz es invertible.

Para acotar el error, defina $e_k := y(t_k)-y_k$ (note que $e_0=0=e_N$). Restando (8.27) y (8.28) obtenemos

$$-\frac{e_{k+1}-2e_k+e_{k-1}}{h^2}+p_k\frac{e_{k+1}-e_{k-1}}{2h}+q_ke_k=\mathcal{O}(h^2)\quad(k=1,\ldots,N-1),$$

de donde el vector $\boldsymbol{e}=[e_1,\ldots,e_{N-1}]^T\in\mathbb{R}^{N-1}$ satisface el sistema

$$A\boldsymbol{e}=\boldsymbol{b},$$

donde $\|\boldsymbol{b}\|_2=\mathcal{O}(h^4)$. De esta manera,

$$\|\boldsymbol{e}\|_2\leqslant Ch^4\|A^{-1}\|_2=C\frac{h^4}{\sigma_m},$$

donde σ_m es el menor valor singular positivo de A.

En el caso particular $p(t)\equiv 0$, $q(t)=q$ se tiene que la matriz A es simétrica, por lo que basta acotar el mínimo valor propio de A. En este

caso, A se reduce a la matriz tridiagonal

$$A = \begin{bmatrix} a & -1 & & & & \\ -1 & a & -1 & & & \\ & -1 & a & \ddots & & \\ & & \ddots & \ddots & -1 & \\ & & & -1 & a & -1 \\ & & & & -1 & a \end{bmatrix}_{N-1\times N-1},$$

con $a := 2 + h^2 q$. Se puede verificar que $A\boldsymbol{q}_j = \lambda_j \boldsymbol{q}_j$ para $j = 1, \ldots, N-1$, con $\lambda_j = a - 2\cos\frac{\pi j}{N}$ y $(\boldsymbol{q}_j)_l = \sqrt{\frac{2}{N}}\sin\frac{\pi jl}{N}$. De esta manera, el menor valor propio de A es

$$\lambda_m = a - 2\cos\frac{\pi}{N} = a - 2\cos\pi h = 2 + h^2 q - 2(1 + \mathcal{O}(h^2)) = \mathcal{O}(h^2),$$

con lo que

$$\|\boldsymbol{e}\|_2 = \mathcal{O}(h^2),$$

y así el método es convergente. □

Ejemplo 8.19. Considere la misma ecuación del ejemplo anterior,

$$-y''(t) + p(t)y'(t) + q(t)y(t) = f(t), \quad 0 < t < 1,$$

ahora con condiciones de frontera $y'(0) = \alpha$, $y(1) = \beta$. Note ahora que $y_0 = y(0)$ no es conocida, por lo que debemos incluirla en el sistema de ecuaciones. Así, obtenemos de manera similar que

$$\left(-1 - \frac{h}{2}p_k\right) y_{k-1} + \left(2 + h^2 q_k\right) y_k + \left(-1 + \frac{h}{2}p_k\right) y_{k+1} = h^2 f_k,$$

ahora para $k = 0, 1, \ldots, N-1$; compare con (8.29). Note que al sustituir $k = 0$ en la ecuación anterior ocupamos definir el llamado *punto fantasma* y_{-1}. Note que podemos aproximar

$$\alpha = y'(0) = \frac{y(h) - y(-h)}{2h} + \mathcal{O}(h^2) \approx \frac{y_1 - y_{-1}}{2h} \Rightarrow y_{-1} = y_1 - 2\alpha h.$$

De esta manera, cuando $k = 0$ se obtiene la ecuación

$$a_0 y_0 + c_0 y_1 = h^2 f_0 - b_0(y_1 - 2\alpha h),$$

la cual reemplaza a la primer ecuación del sistema dado en (8.30); como se desprecian términos de orden dos, el análisis del error se puede realizar de manera similar. □

8.9. A-estabilidad

Considere el problema de prueba

$$y' = \lambda y, \quad y(0) = 1 \quad \text{con} \quad \lambda \in \mathbb{C}.$$

La solución exacta es

$$y(t) = e^{\lambda t}$$

y es claro que

$$\lim_{t\to\infty} y(t) = 0 \Leftrightarrow \text{Re}\lambda < 0.$$

Considere cualquier método para resolver numéricamente el problema de prueba. El dominio de estabilidad de dicho método se define como

$$D = \{h\lambda \in \mathbb{C} : y_n \to 0 \text{ si } \ n \to \infty\},$$

y el método se llama A-estable si $\{z \in \mathbb{C} : \text{Re}z < 0\} \subset D$.

Por ejemplo, si utilizamos el Método de Euler tenemos que

$$y_n = (1 + h\lambda)^n,$$

por lo que

$$D_{\text{Euler}} = \{z \in \mathbb{C} : |1 + z| < 1\},$$

el cual corresponde al disco centrado en $z = -1$ de radio 1. Por lo tanto, el método de Euler no es A-estable. Tal como vimos en la sección anterior, se debe tener consideraciones sobre h para evitar que la solución sea estable.

Para el caso del trapecio, tenemos que

$$y_n = \left(\frac{1 + h\lambda/2}{1 - h\lambda/2}\right)^n,$$

por lo que

$$D_{\text{trap}} = \{z \in \mathbb{C} : \left|\frac{1 + z/2}{1 - z/2}\right| < 1\} = \{z \in \mathbb{C} : \text{Re}z < 0\},$$

por lo que el método del trapecio es A-estable, y no necesitamos consideraciones sobre h para tener una solución estable.

Para el caso de Runge-Kuta, tenemos que $\xi_j = y_n + \lambda h \sum_{i=1}^{\nu} a_{j,i}\xi_i$. Defina los vectores $\boldsymbol{\xi} = [\xi_1, \ldots, \xi_\nu]^T$, $\mathbf{1} = [1, \ldots, 1] \in \mathbb{R}^\nu$. De esta manera, podemos expresar de manera matricial

$$\boldsymbol{\xi} = \mathbf{1}y_n + \lambda h A\boldsymbol{\xi}, \quad \text{por lo que}$$

$$\boldsymbol{\xi} = (I - h\lambda A)^{-1}\mathbf{1}y_n,$$

y así

$$y_{n+1} = y_n + h\lambda \sum_{j=1}^{\nu} b_j \xi_j = \left[1 + h\lambda \boldsymbol{b}^T (I - h\lambda A)^{-1}\mathbf{1}\right] y_n,$$

por lo que

$$y_n = \left[r(h\lambda)\right]^n, \quad \text{donde}$$
$$r(z) = 1 + z\boldsymbol{b}^T (I - zA)^{-1}\mathbf{1}.$$

Concluimos así que

$$D_{\mathrm{RK}} = \{z \in \mathbb{C} : |r(z)| < 1\}.$$

De álgebra lineal sabemos que

$$(I - zA)^{-1} = \frac{\mathrm{adj}(I - zA)}{\det(I - zA)}.$$

De esta manera, si el método es explícito, A es triangular inferior con ceros en la diagonal. Por esta razón se puede deducir que $r(z)$ es un polinomio de grado ν, por lo que no es posible que los métodos de Runge-Kuta explícitos sean $A-$estables. Si el método es implícito, entonces $r(z)$ es el cociente de dos polinomios de grado ν. En este caso, se puede verificar que los métodos de Runge-Kuta implícitos son A-estables; ver, e.g., [15, Capítulo 4.3].

8.10. Ejercicios Adicionales

Ejercicio 8.20. Considere la ecuación diferencial $y' = y^{1/5}, y(0) = 0$, cuya solución no trivial es $y(x) = (4x/5)^{5/4}$. Verifique que el método de Euler no aproxima dicha solución. ¿Qué pasa si se utiliza el Método del Trapecio?

Ejercicio 8.21. Escriba una expresión para el polinomio de interpolación de Lagrange con nodos $\{(x_n, y_n), (x_{n+1}, y(x_{n+1})), (x_{n+2}), y(x_{n+2}))\}$, donde $x_{n+1} = x_n + h$, $x_{n+2} = x_n + 2h$. Derive este polinomio para demostrar que

$$y'(x_{n+2}) = \frac{1}{2h}\left(3y(x_{n+2} - 4y(x_{n+1}) + y(x_n)\right) + \mathcal{O}(h^2),$$

asumiendo que $y \in C^3([x_n, x_{n+2}])$. Deduzca que el método implícito

$$3y_{n+2} - 4y_{n+1} + y_n = 2hf(x_{n+2})$$

tiene orden uno.

Ejercicio 8.22. Escriba la fórmula del Método de Euler que se obtiene al resolver la ecuación

$$y' = xe^{-5x} - 5y, \quad y(0) = 0$$

en el intervalo $[0, 1]$ con nodos $t_k = kh$ $(k = 0, \ldots, n)$ $(nh = 1)$. Muestre directamente que $y_1 \to y(1)$ cuando $h \to 0$.

Ejercicio 8.23. Considere el Método de Euler para resolver la ecuación $y' = -y, \quad y(0) = 1$.

(a) Encuentre la solución exacta $y(t)$.

(b) Escriba una fórmula general para y_n.

(c) ¿Para qué valores de h la sucesión $y_0, y_1, \ldots$ es acotada?

Ejercicio 8.24. [MATLAB] Considere la ecuación diferencial

$$y' = \frac{y(1-y)}{2y-1}, \quad y(0) = \frac{5}{6}.$$

Sabemos que la solución exacta es

$$y(t) = \frac{1}{2} + \left(\frac{1}{4} - \frac{5}{36}e^{-t}\right)^{1/2}.$$

Aplique el Método del Trapecio y el Método Explícito de Runge-Kutta de orden 4 para medir el error en $t = 2$, para diferentes valores de h y verifique el orden de cada método.

Ejercicio 8.25. [MATLAB] Considere la ecuación diferencial

$$y'(t) = -100(y(t) - \text{sen}(t)), \quad y(0) = 1.$$

Sabemos que la solución decae rápidamente, y eventualmente es cercana a $\text{sen}(t)$.

(a) Presente la solución numérica para el Método del Trapecio y el Método Explícito de Runge-Kutta de orden 4. Discuta sobre la estabilidad de ambos métodos.

(b) Presente la solución numérica para el Método Predictor-Corrector del Ejemplo 8.7. Compare el tiempo de ejecución con el del Método del Trapecio.

Ejercicio 8.26. [MATLAB] (Runge-Kuta adaptivo) Considere el método de Runge-Kuta de cuarto orden explícito dado por el esquema (8.14). Para ese caso asumimos que h es constante en cada iteración. A continuación describimos un método adaptivo, en donde en cada paso se determina si el valor h debe ser aumentado o disminuido, según consideraciones locales de error. Asuma deseamos resolver el problema

$$\begin{cases} y' = f(t,y), \quad t \in [0, T_{\text{máx}}], \\ y(t_0) = y_0 \end{cases}$$

utilizando el método de Runge-Kuta explícito de orden cuatro, con *step size* variable.

El programa incluye tres parámetros: $h_{\text{máx}}$ (el máximo posible valor de h, por consideraciones de estabilidad), tol (la tolerancia prescrita) y h_0 (valor inicial de h).

En cada iteración k, el error local Δ se estima de la siguiente manera. Primero, se calcula la aproximación y_1 (en el tiempo $t_k + h$) mediante una iteración de Runge Kuta con incremento h. Luego, se calcula una aproximación y_2 (en el tiempo $t_k + h$) utilizando dos iteraciones de Runge-Kuta con incremento $h/2$ cada una. Se toma así $\Delta = y_2 - y_1$.

El valor de h se modifica de la siguiente manera. Calcule

$$\widetilde{h} = \begin{cases} Sh\left|\frac{tol}{\Delta}\right|^{0.20}, \text{si } |\Delta| \leqslant tol \\ Sh\left|\frac{tol}{\Delta}\right|^{0.25}, \text{si } |\Delta| > tol \end{cases}$$

donde S es un factor de *seguridad* cercano a 1; en nuestro caso tome $S = 0.9$. Luego, se toma $h_{\text{new}} = \text{mín}\{h_{\text{máx}}, \widetilde{h}\}$. Finalmente, si $|\Delta| < tol$, se toma el valor $y_{k+1} = y_2 + \Delta/15$; en caso contrario se repite el ciclo. Note que en cualquier caso se continua con el valor h_{new}.

(a) Escriba una función que implemente el método adaptivo descrito anteriormente. Las entradas deben ser f, y_0, t_0, $T_{\text{máx}}$, h_0, $h_{\text{máx}}$, tol.

(b) Utilice la función anterior para el caso

$$f(t) = \log(t + 0.01) + 1/(t - 4),$$

$y_0 = 0.8$, $t_0 = 0$, $T_{\text{máx}} = 3.99$. Tome $h_0 = 10^{-5}$, $h_{\text{máx}} = 10^{-2}$, $tol = 10^{-12}$. Dibuje h en función del tiempo (en una escala adecuada). Comente las variaciones observadas en las variaciones de h. Dibuje además el error $|y(t) - \tilde{y}(t)|$ en función de t.

Ejercicio 8.27. [MATLAB] Considere la ecuación diferencial dada por el sistema

$$\begin{cases} \dfrac{\partial^2 x}{\partial t^2} = x + 2\frac{\partial y}{\partial t} - u_c \cdot \dfrac{x+u}{((x+u)^2+y^2)^{3/2}} - u \cdot \dfrac{x-u_c}{((x-u_c)^2+y^2)^{3/2}} \\ \dfrac{\partial^2 y}{\partial t^2} = y - 2\frac{\partial x}{\partial t} - u_c \cdot \dfrac{y}{((x+u)^2+y^2)^{3/2}} - u \cdot \dfrac{y}{((x-u_c)^2+y^2)^{3/2}} \end{cases},$$

donde $x(t), y(t)$ son funciones $x, y : [0, \infty) \to \mathbb{R}$. Dicho sistema modela el movimiento bidimensional de un satélite, donde $(x(t), y(t))$ representa su ubicación en el tiempo t. En este caso $u = 0.012277471$ y $u_c = 1 - u$ son constantes dadas. Considere además las condiciones iniciales de posición y velocidad en $t = 0$ dadas por $x(0) = 0.994$, $y(0) = 0$, $x'(0) = 0$, $y'(0) = -2.001585106$. Sabemos a priori que el movimiento tiene periodo $T = 17.0652166$.

(a) Haga $z = x'$, $w = y'$ y obtenga un sistema de grado uno de ecuaciones diferenciales con cuatro ecuaciones de la forma $\boldsymbol{u}' = \boldsymbol{f}(t, \boldsymbol{u}(t))$, $\boldsymbol{u}(0) = \boldsymbol{u}_0$.

(b) Resuelva el sistema obtenido mediante el algoritmo adaptivo del Ejercicio 8.26. Utilice $T_{\text{máx}} = 40$, $h_0 = 10^{-5}$, $h_{\text{máx}} = 10^{-2}$, $tol = 10^{-6}$. Grafique la curva paramétrica $\{(x(t), y(t)), t \in [0, T_{\text{máx}}]\}$. Compare los valores $(x(0), y(0))$, $(x(T), y(T))$ y $(x(2T), y(2T))$. Comente sus resultados.

(c) Resuelva ahora el sistema mediante el comando

`[t,u] = ode113(f,[0 40],u0).`

Grafique la curva paramétrica $\{(x(t), y(t)), t \in [0, T_{\text{máx}}]\}$. ¿Qué observa en este caso?

(d) Defina el vector de opciones

```
options = odeset('RelTol',1e-10,'Stats','on','AbsTol',1e-10).
```

Ejecute ahora el comando `[t,u] = ode113(f,[0 40],u0,options).` Grafique la curva paramétrica $\{(x(t), y(t)), t \in [0, T_{\text{máx}}]\}$. ¿Qué observa en este caso?

Ejercicio 8.28. [MATLAB] (Movimiento de un péndulo) Una ecuación diferencial que modela el movimiento de un péndulo simple con amortiguamiento corresponde a

$$\frac{d^2\theta}{dt^2} + \gamma\frac{d\theta}{dt} + \omega^2 \operatorname{sen}(\theta) = 0,$$

donde asumimos que γ y ω son constantes (relacionadas con el amortiguamiento y la velocidad angular). Además, $\theta = \theta(t)$ representa la medida del ángulo del péndulo con respecto a la vertical en el tiempo $t > 0$; ver Figura 8.3. En este ejercicio tomaremos $\gamma = 0.1$, $\omega^2 = 5/4$.

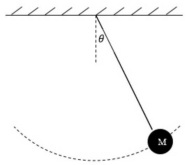


Figura 8.3: Péndulo simple.

(a) Considere las condiciones iniciales $\theta(0) = \pi/4$, $\theta'(0) = 0$. Mediante el comando `ode45` de `MATLAB`, grafique la solución θ en función de t, para $t \in [0, 50]$. Grafique además la velocidad angular θ' en función de θ. ¿Qué puede concluir del gráfico?

(b) Repita el inciso (a) con condiciones iniciales $\theta(0) = \pi$, $\theta'(0) = -0.5$.

(c) Repita el inciso (a) con condiciones iniciales $\theta(0) = \pi$, $\theta'(0) = 0$. ¿Qué ocurre en este caso? ¿Se puede decir que $(\theta, \theta') = (\pi, 0)$ es un punto estable o inestable a partir del gráfico?

(d) Construya una función que grafique el movimiento físico del péndulo (tal como uno lo observaría), con entradas t, θ y la longitud L de la cuerda. Grafique el movimiento del péndulo para los resultados obtenidos en la parte (a) y en la parte (c), tomando $L = 1$. ¿Tiene sentido físico el comportamiento del péndulo en cada caso?

(e) Utilice el comando `quiver` para graficar el diagrama de fase de la ecuación diferencial. Grafique en el mismo gráfico (θ, θ') para condiciones iniciales $(\theta, \theta') = (\pi/4, 2)$, $(\theta, \theta') = (\pi/4, 5/2)$. Comente sus resultados. Sugerencia: el comando `meshgrid` puede ser de utilidad.

Ejercicio 8.29. [MATLAB] (Sistema de Lorenz) Considere el sistema de ecuaciones diferenciales

$$\begin{aligned} x' &= \sigma(y - x) \\ y' &= x(\rho - z) - y \\ z' &= xy - \beta z \end{aligned}$$

donde σ, ρ, β son constantes, $x = x(t), y = y(t), z = z(t)$. En este ejercicio tomaremos $\sigma = 10, \rho = 28, \beta = 8/3$.

(a) Implemente una función en MATLAB que efectúe el Método de Newton para resolver la ecuación

$$\begin{aligned} 0 &= \sigma(y - x) \\ 0 &= x(\rho - z) - y \\ 0 &= xy - \beta z. \end{aligned}$$

Aproxime con dicha función todos los puntos de equilibrio del sistema de ecuaciones diferenciales.

(b) Resuelva la ecuación con condiciones iniciales $(x(0), y(0), z(0)) = (1; 1; 1)$ para $t \in [0, 100]$, y denote esta solución por (x_1, y_1, z_1).

(c) Resuelva la ecuación con condiciones iniciales $(x(0), y(0), z(0)) = (1; 1; 1.00001)$ para $t \in [0, 100]$, y denote esta solución por (x_2, y_2, z_2).

(d) En un mismo gráfico (en tres dimensiones) plotee las trayectorias $(x_i(t), y_i(t), z_i(t))$, $i \in \{1, 2\}$, así como los puntos de equilibrio. Comente sus resultados. Sugerencia: utilice el comando `plot3`

(e) Plotee la distancia

$$d(t) = \sqrt{(x_1(t) - x_2(t))^2 + (y_1(t) - y_2(t))^2 + (z_1(t) - z_2(t))^2}$$

en función de t. Comente el comportamiento de la gráfica. Sugerencia: el comando `deval` puede ser de utilidad.

Ejercicio 8.30. [MATLAB] Considere una población donde deseamos estudiar la propagación de una enfermedad. En el modelo más sencillo, considerados la población dividida en tres grupos: susceptibles, infectados y recuperados.

Considere el sistema del modelo dado por

$$\begin{cases} \dfrac{dS}{dt} = \Lambda - \mu S - \beta IS, \\ \dfrac{dI}{dt} = \beta IS - \gamma I - \mu I, \\ \dfrac{dR}{dt} = \gamma I - \mu R, \end{cases}$$

donde $S(t), I(t), R(t)$ representa el número de personas susceptibles, infectadas y recuperadas, respectivamente, en el tiempo t. En este ejercicio considere $\Lambda = 15$ (nacimientos por unidad de tiempo), $\gamma = 1/35$ (tasa de recuperación), $\mu = 1/75$ (tasa de mortalidad).

(a) Plotee (en un mismo gráfico) la solución $(S(t), I(t), R(t))$ para $\beta = 1/5500$, $t \in [0, 365]$, para las condiciones iniciales $(999, 1, 0)$.

(b) Para cada valor de $\beta =$ `1./linspace(25000,29000)`, obtenga $I(3650)$. Grafique dichos valores en función de $R_0(\beta) := \dfrac{\Lambda\beta}{\mu(\mu+\gamma)}$ (el cual es llamado *número básico de reproducción*. ¿Qué se puede deducir del gráfico?

(c) Obtenga los puntos de equilibrio del sistema. ¿Qué relación puede deducir que es necesaria para que el número de infectados tienda a cero?

Ejercicio 8.31. Considere el problema de valor inicial

$$\begin{cases} y'(t) - 3\displaystyle\int_0^t e^{-2(t-u)} y(u)du &= 4e^{-2t}, \qquad \text{para} \quad 0 < t < 1\,, \\ y(0) &= 4. \end{cases} \tag{8.31}$$

(a) Utilizando la Regla del Trapecio, aproxime la integral en la ecuación anterior.

(b) Usando la aproximación encontrada en a), reescriba el problema de valor inicial.

(c) Empleando el *método de RK4* con $h = 1/100$, encuentre una aproximación para $y(t)$ con $t \in [0, 1]$.

(d) Exprese el problema (8.31) como un sistema de ecuaciones diferenciales de primer orden (Sugerencia: derive (8.31) y haga la sustitución $z' = y$).

(e) Utilice el comando `ode23` para resolver el sistema del punto (d).

(f) Calcule la solución exacta de la ecuación diferencial. Determine el error para las aproximaciones numéricas efectuadas en los puntos anteriores.

Ejercicio 8.32. Considere el problema de valor inicial:

$$\left\{ \begin{array}{rcll} y'(t) & = & \dfrac{\cos^2(t)}{y} & \text{para} \quad 0 < t < 2\,, \\ y(0) & = & 0\,, & \end{array} \right. \tag{8.32}$$

el cual tiene además la restricción:

$$y(t) \ < \ 0 \qquad \text{para} \quad 0 < t \leqslant 2\,.$$

(a) Justifique el porqué no puede utilizarse un esquema numérico que posea el término $f(t_k, y_k)$ para aproximar la solución de (8.32).

(b) Empleando el método de Euler implícito dado por:

$$\left\{ \begin{array}{rcl} y_0 & = & y(t_0)\,, \\ y_{k+1} & = & y_k \ + \ hf(t_{k+1}, y_{k+1})\,. \end{array} \right.$$

Muestre que una iteración para aproximar (8.32) viene dada por:

$$(y_{k+1})^2 \ - \ y_k \cdot y_{k+1} \ - \ h\cos^2(t_{k+1}) \ = \ 0\,. \tag{8.33}$$

(c) Utilice la iteración (8.33) con $h = 1/100$ para hallar una aproximación para $y(t)$.

Ejercicio 8.33. Considere el método de Runge-Kutta implícito de orden cuatro dado por la tabla

$$\begin{array}{c|cc} \frac{1}{2} - \frac{\sqrt{3}}{6} & \frac{1}{4} & \frac{1}{4} - \frac{\sqrt{3}}{6} \\ \frac{1}{2} + \frac{\sqrt{3}}{6} & \frac{1}{4} + \frac{\sqrt{3}}{6} & \frac{1}{4} \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}\,.$$

(a) Escriba una función en `MATLAB` que implemente dicho algoritmo.

(b) Compruebe el desempeño de su programa para resolver la ecuación diferencial $y'(t) = \log(t + 0.01) + 1/(t - 4)$, $y(0) = 0.8$, para $t \in [0, 3.99]$. Determine un valor apropiado para h y grafique el error correspondiente.

Ejercicio 8.34. Considere la ecuación diferencial ordinaria con parámetro ε dado por

$$\varepsilon u'' + u = e^x,\ x \in (-1,1),\ u = u(x). \tag{8.34}$$

(a) Calcule la solución exacta (a mano) de (8.34). Para ello:

   a) Resuelva el problema homogéneo $\varepsilon u'' + u = 0$ para obtener una solución complementaria u_c (considere la ecuación auxiliar $\varepsilon m^2 + 1 = 0$).

   b) Utilice el método de variación de parámetros para obtener una solución particular de la forma $u_p = ce^x$.

   c) Concluya que $u = u_c + u_p$ resuelve (8.34).

(b) Fije $\varepsilon = 0.0001$. Considere ahora el problema de valor inicial (PVI) dado por (8.34) con condiciones $u(-1) = u'(-1) = 0$. Aproxime numéricamente las constantes de la solución u del inciso anterior. Sugerencia: puede utilizar la función `fsolve`.

(c) Utilice la función `ode45` para resolver el PVI. Grafique el error absoluto (tomando como solución exacta la del inciso anterior).

(d) Utilice el Método de Diferencias Divididas (DD) para resolver el PVI. Para N dado, utilice DD para obtener un sistema de ecuaciones $A\hat{u} = b$ que permita aproximar la solución de la ED en los nodos

```
xx = linspace(-1,1,N+1).
```

Para $N \in \{2^7, 2^8, \ldots, 2^{20}\}$ determine $\|u - \hat{u}\|_\infty$ y grafique dicha norma en función de h. ¿Qué velocidad de convergencia observa? ¿Es la convergencia esperada? ¿Qué forma particular tiene la matriz A?

(e) Repita el inciso anterior para el problema de frontera $u(-1) = 0, u(1) = 1$. Recuerde calcular la nueva solución exacta para estas nuevas condiciones. ¿Qué forma particular tiene la matriz A? Utilice el operador \ para resolver los sistemas de ecuaciones requeridos. Sugerencia: para N alto, no es posible construir la matriz completa con N^2 entradas por limitaciones de memoria. En su lugar, defina tres vectores $indI$, $indJ$, S tales que $A(indI(k), indJ(k)) = S(k)$; esto es, $indI$ e $indJ$ contienen el número de fila y columna de los valores no nulos S de A. Luego, defina $A = sparse(indI, indJ, S)$.

(f) Implemente el método de Seidel para resolver el sistema de ecuaciones del inciso anterior. Grafique la norma infinito del error absoluto en función de N. ¿Se observa converegencia en la iteración? Justifique su respuesta. Sugerencia: puede utilizar las funciones `diag`, `triu`, `tril`.

Capítulo 9

Valores y vectores propios

9.1. Introducción

En este capítulo deseamos estudiar cómo obtener aproximaciones numéricas para los valores y vectores propios de una matriz dada $A \in \mathbb{C}^{m\times m}$; esto es, encontrar $\boldsymbol{x} \in \mathbb{C}^m$ y $\lambda \in \mathbb{C}$ tales que

$$A\boldsymbol{x} = \lambda \boldsymbol{x}.$$

Para ello, iniciamos recordando algunas definiciones y resultados importantes:

Definición 9.1. *Dos matrices $A \in \mathbb{C}^{m\times m}$, $B \in \mathbb{C}^{m\times m}$ se llaman similares si existe una matriz $P \in \mathbb{C}^{m\times m}$ invertible tal que $P^{-1}AP = B$.*

La demostración del siguiente lema sigue de manera inmediata de la definición y se deja como ejercicio.

Lema 9.2. *Suponga que $A \in \mathbb{C}^{m\times m}$ y $B \in \mathbb{C}^{m\times m}$ son matrices similares. Entonces,*

(a) $\det A = \det B$.

(b) A es invertible sii B es invertible.

(c) A y B tienen el mismo rango.

(d) A y B tienen el mismo polinomio característico.

(e) A y B tienen los mismos valores propios.

Comentario 9.3. La idea general para encontrar valores propios será obtener una sucesión de matrices similares, de las cuales podemos obtener con facilidad los valores propios. En particular, no hay métodos directos para obtener en un número finito de operaciones los valores propios de una matriz dada. Esto se relaciona con el hecho que el problema de encontrar valores propios es equivalente a encontrar raíces de polinomios, para lo cual no hay una fórmula cerrada si el polinomio tiene grado mayor que cuatro. Así, cualquier algoritmo para encontrar valores propios debe ser iterativo. □

En general, podemos ubicar a priori la ubicación de los valores propios mediante los discos de Gerschgorin:

Definición 9.4 (Discos de Gerschgorin)**.** *Considere $A \in \mathbb{C}^{m\times m}$. Para $i = 1, \ldots, m$ defina el disco*

$$D_i := \{z \in \mathbb{C} : |z - a_{ii}| < R_i\}, \quad \textit{donde} \quad R_i := \sum_{j=1, j\neq i}^{m} |a_{ij}|.$$

Teorema 9.5. *Todos los valores propios de una matriz $A \in \mathbb{C}^{m\times m} (m \geqslant 2)$ se encuentran en la unión de los discos de Gerschgorin de la matriz A.*

Demostración. Suponga que $A\boldsymbol{x} = \lambda\boldsymbol{x}$ ($\boldsymbol{x} \neq \underline{0}$). De esta forma, al igualar los componentes en la posición i-ésima obtenemos que

$$\sum_{j=1}^{m} a_{ij}x_j = \lambda x_i \ \forall \ i = 1, \ldots, m.$$

Tome x_k como el máximo elemento de $\boldsymbol{x}$ (sin importar si dicho máximo se repite). Luego,

$$|\lambda - a_{kk}||x_k| = \left|\sum_{j=1}^{m} a_{kj}x_j - a_{kk}x_k\right| = \left|\sum_{j=1, j\neq k}^{m} a_{kj}x_j\right| \leqslant |x_k|R_k.$$

De esta forma, $|\lambda - a_{kk}| \leqslant R_k$ y λ pertenece a uno de los discos de Gerschgorin de la matriz A. □

Note que si pudiéramos escribir $A = QTQ^* \in \mathbb{C}^{m\times m}$, con $Q \in \mathbb{C}^{m\times m}$ unitaria y $T \in \mathbb{C}^{m\times m}$ triangular superior, entonces los valores propios de A coinciden con las entradas en la diagonal de T, pues

$$\det(A - \lambda I) = \det(T - \lambda I) = \prod_{i=1}^{n}(t_{ii} - \lambda).$$

De esta forma, tenemos la siguiente definición:

Definición 9.6 (Factorización de Schur)**.** *Dada $A \in \mathbb{C}^{m\times m}$, se dice que A admite una factorización de Schur si existen $Q \in \mathbb{C}^{m\times m}$ unitaria y $T \in \mathbb{C}^{m\times m}$ triangular superior tales que*

$$A = QTQ^*.$$

Teorema 9.7. *Toda $A \in \mathbb{C}^{m\times m}$ admite una factorización de Schur.*

Demostración. La prueba es por inducción sobre m. Si $m = 1$ es trivial (pues A es un escalar). Sea $m \geqslant 2$ y suponga ahora que toda matriz de tamaño $(m-1)\times(m-1)$ admite una factorización de Schur. Tome λ un valor propio de A con vector propio asociado $\boldsymbol{x}$; esto es, $A\boldsymbol{x} = \lambda\boldsymbol{x}$. Defina $U \in \mathbb{C}^{m\times m}$ unitaria, donde la primer columna de U es $\boldsymbol{x}$. Entonces, tenemos que

$$U^*AU = \begin{bmatrix} \lambda & \boldsymbol{b}^* \\ \underline{0} & C \end{bmatrix},$$

donde $\boldsymbol{b} \in \mathbb{C}^{m-1}$, $C \in \mathbb{C}^{(m-1)\times(m-1)}$ y $\underline{0} \in \mathbb{C}^{m-1}$. Por hipótesis inductiva, existen $V \in \mathbb{C}^{(m-1)\times(m-1)}$ unitaria y $T \in \mathbb{C}^{(m-1)\times(m-1)}$ triangular superior tales que $C = VTV^*$. Defina

$$Q := U \begin{bmatrix} 1 & \\ & V \end{bmatrix},$$

la cual es unitaria. Luego, después de calcular el producto Q^*AQ, se deduce que

$$Q^*AQ = \begin{bmatrix} \lambda & \boldsymbol{b}^*V \\ & T \end{bmatrix},$$

la cual es una matriz triangular superior. De esta manera, A admite una factorización de Schur para todo $m \in \mathbb{N}$. □

Por el Comentario 9.3, no es posible en general obtener en una cantidad finita de pasos la factorización de Schur de A. Por ello, dividiremos el estudio en dos etapas, en la cual primero obtendremos una matriz de *Hessenberg*:

Definición 9.8 (Matriz en forma de Hessenberg)**.** *Se dice que $H \in \mathbb{C}^{m\times m}$ es una matriz en forma de Hessenberg si todos sus elementos por debajo de la diagonal son nulos; esto es, si*

$$h_{ij} = 0 \quad si \quad i - j > 1.$$

Luego de obtener H similar a A, aproximaremos la factorización de Schur de H para obtener los valores propios de A.

9.2. Reducción a una matriz de Hessenberg

Considere $A \in \mathbb{C}^{m\times m}$. En esta sección describimos el algoritmo para encontrar una matriz de Hessenberg H similar a A. La manera más sencilla es utilizar reflexiones de Householder (ver Apartado 5.3.2).

La idea es similar a la factorización QR mediante Householder. En este caso, en el primer paso definimos una matriz Q_1 de manera tal que las entradas por debajo de la subdiagonal de Q_1A se anulen. Con el fin de obtener matrices similares, debemos calcular el producto $Q_1AQ_1^*$ en cada paso, tal como se muestra a continuación (note que los ceros no desaparecen al multiplicar por Q_1^* a la derecha):

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \\ a_{41} & a_{42} & a_{43} \\ a_{51} & a_{52} & a_{53} \end{bmatrix} \xrightarrow[]{Q_1A} \begin{bmatrix} a'_{11} & a'_{12} & a'_{13} \\ a'_{21} & a'_{22} & a'_{23} \\ 0 & a'_{32} & a'_{33} \\ 0 & a'_{42} & a'_{43} \\ 0 & a'_{52} & a'_{53} \end{bmatrix} \xrightarrow[]{Q_1AQ_1^*} \begin{bmatrix} a'_{11} & a'_{12} & a'_{13} \\ a'_{21} & a'_{22} & a'_{23} \\ 0 & a'_{32} & a'_{33} \\ 0 & a'_{42} & a'_{43} \\ 0 & a'_{52} & a'_{53} \end{bmatrix}$$

Continuando de esta manera, obtenemos $m-2$ operadores Q_k tales que

$$Q_{m-2}\dots Q_1AQ_1^*\dots Q_{m-2}^* = H,$$

donde H es de Hessenberg. Dicho método se presenta en el Algoritmo 19, el cual requiere $\mathcal{O}(10m^3/3)$ operaciones. Note en particular que si la matriz A es simétrica, entonces la matriz H es tridiagonal (pues es simétrica y de Hessenberg a la vez).

Data: $A \in \mathbb{C}^{m\times m}$
Result: Matriz de Hessenberg H similar a A
$H = A$; **for** $k = 1 : m-2$ **do**
  $\boldsymbol{x} = H_{k+1:m,k}$;
  $\boldsymbol{v}_k = \text{sign}(x_1)\|\boldsymbol{x}\|_2\boldsymbol{e}_1 + \boldsymbol{x}$;
  $\boldsymbol{v}_k = \boldsymbol{v}_k/\|\boldsymbol{v}_k\|_2$;
  $H_{k+1:m,k:m} = H_{k+1:m,k:m} - 2\boldsymbol{v}_k(\boldsymbol{v}_k^* H_{k+1:m,k:m})$;
  $H_{1:m,k+1:m} = H_{1:m,k+1:m} - 2(H_{1:m,k+1:m}\boldsymbol{v}_k)\boldsymbol{v}_k^*$;
end

Algoritmo 19: Forma de Hessenberg

Concluimos esta sección enunciando el teorema sobre la estabilidad hacia atrás del Algoritmo 19:

Teorema 9.9. *Sean $\widetilde{H}$ la aproximación de H obtenida al utilizar el Algoritmo 19 y $\widetilde{Q}$ la matriz unitaria de transformación. Entonces, existe $\delta A \in$*

$\mathbb{C}^{m\times m}$ *tal que*

$$\widetilde{Q}\widetilde{H}\widetilde{Q}^* = A + \delta A, \quad con \quad \frac{\|\delta A\|}{\|A\|} = \mathcal{O}(\varepsilon_m).$$

9.3. Secuencias de Sturm

Esta sección se basa en [27, Sección 5.6]. Asumimos que $A \in \mathbb{R}^{m\times m}$ es simétrica. De esta manera, A es similar a una matriz tridiagonal T que se obtiene al aplicar el Algoritmo 19. Escriba

$$T = \begin{bmatrix} a_1 & b_2 & & & & & \\ b_2 & a_2 & b_3 & & & & \\ & b_3 & a_3 & b_4 & & & \\ & & \ddots & \ddots & \ddots & & \\ & & & b_{m-1} & a_{m-1} & b_m \\ & & & & b_m & a_m \end{bmatrix}. \tag{9.1}$$

Comentario 9.10. Note que si $b_k = 0$ para algún k, entonces

$$T = \left[\begin{array}{ccccc|ccccc} a_1 & b_2 & & & & & & & & \\ b_2 & a_2 & \ddots & & & & & & & \\ & \ddots & \ddots & b_{k-1} & & & & & & \\ & & b_{k-1} & a_{k-1} & 0 & & & & & \\ \hline & & & 0 & a_k & b_{k+1} & & & & \\ & & & & b_{k+1} & a_{k+1} & \ddots & & & \\ & & & & & \ddots & \ddots & b_m & & \\ & & & & & & b_m & a_m & & \end{array}\right] = \begin{bmatrix} T_1 & & \\ & & \\ & & T_2 \end{bmatrix}$$

es una matriz con dos bloques diagonales. Es claro que el conjunto de valores propios de T es igual a la unión de los valores propios de T_1 y T_2. De esta manera, podemos calcular por separado los valores propios de T_1 y T_2 (que tienen dimensión menor). Dicho método se llama *método de deflación.*

Debido al Comentario 9.10, asumiremos entonces que $b_k \neq 0 \ \forall \ k = 2, \ldots, m$, pues en caso contrario podemos resolver dos problemas de menor dimensión. Los determinantes de los menores principales de $T - \lambda I$ tienen gran importancia:

Definición 9.11. *[Secuencia de Sturm] Considere la matriz T dada en (9.1). Los determinantes de los menores principales de $T - \lambda I$, los cuales vienen dados recursivamente por*

$$\begin{aligned} &p_0(\lambda) \equiv 1, \\ &p_1(\lambda) = a_1 - \theta, \\ &p_r(\lambda) = (a_r - \lambda)p_{r-1}(\lambda) - b_r^2 p_{r-2}(\lambda) \quad \textit{para} \quad r = 2, \ldots, m, \end{aligned}$$

generan una sucesión $\{p_r(\lambda)\}_{r=0}^m$ llamada secuencia de Sturm.

Tenemos el siguiente resultado importante:

Teorema 9.12. *Sea $n \geqslant 3$. Las raíces de $p_r(\lambda)$ separan las raíces de $p_{r+1}(\lambda)$ $(r = 1, 2, \ldots, m-1)$; esto es, entre dos raíces consecutivas de p_{r+1} se ubica una única raíz de p_r. En particular, las raíces de p_r son simples para todo $r = 1, \ldots, m$.*

Demostración. La prueba procede por inducción. En general, denotaremos por $\lambda_1^{(r)}, \ldots, \lambda_r^{(r)}$ a las r raíces del polinomio $p_r(\lambda)$ ordenadas de manera creciente. Para $r = 1$ la raíz de p_1 es $\lambda_1^{(1)} := a_1$ y las raíces de p_2 vienen dadas por

$$\begin{aligned} \lambda_1^{(2)} &:= \frac{1}{2}\left(a_1 + a_2 - \sqrt{(a_1 - a_2)^2 + 4b_2^2}\right) \\ \lambda_2^{(2)} &:= \frac{1}{2}\left(a_1 + a_2 + \sqrt{(a_1 - a_2)^2 + 4b_2^2}\right). \end{aligned}$$

Un cálculo directo muestra que $\lambda_1^{(2)} < \lambda_1^{(1)} < \lambda_2^{(2)}$ por lo que el teorema se cumple para $r = 1$. Asuma que p_{i-1} separa las raíces de p_i. Tome dos raíces consecutivas $\lambda_k^{(i)}$ y $\lambda_{k+1}^{(i)}$ de $p_i(\lambda)$ $(k = 1, \ldots, i-1)$. Como p_{i-1} tiene una única raíz en el intervalo $(\lambda_k^{(i)}, \lambda_{k+1}^{(i)})$, por continuidad se tiene que

$$p_{i-1}(\lambda_k^{(i)})p_{i-1}(\lambda_{k+1}^{(i)}) < 0.$$

Luego, como

$$p_{i+1}(\lambda) = (a_{i+1} - \lambda)p_i(\lambda) - b_{i+1}^2 p_{i-1}(\lambda),$$

directamente al sustituir obtenemos que

$$p_{i+1}(\lambda_k^{(i)})p_{i+1}(\lambda_{k+1}^{(i)}) < 0,$$

y de esta manera existe un cero $\lambda_{k+1}^{(i+1)}$ de p_{i+1} en el intervalo $(\lambda_k^{(i)}, \lambda_{k+1}^{(i)})$ para $k = 1, \ldots, i-1$. De esta forma hemos localizado $i-1$ raíces raíces las cuales se ubican entre raíces consecutivas de $p_i(\lambda)$. Para concluir la prueba, demostraremos que existe una raíz $\lambda_1^{(i+1)} < \lambda_1^{(i)}$ y una raíz $\lambda_{i+1}^{(i+1)} > \lambda_i^{(i)}$. De esta forma todas las raíces estaría separadas. Como $p_r(\lambda) \to +\infty$ cuando $\lambda \to -\infty$ para todo r, se tiene que $p_{i-1}(\lambda_1^{(i)}) > 0$ y así $p_{i+1}(\lambda_1^{(i)}) < 0$, por lo que existe una raíz $\lambda_1^{(i+1)} \in (-\infty, \lambda_1^{(i)})$. De manera similar se puede ver que existe una raíz $\lambda_{i+1}^{(i+1)} \in (\lambda_i^{(i)}, +\infty)$ (considerando los casos si $p_{i-1}(\lambda_i^{(i)})$ es positivo o negativo). □

El siguiente teorema permitirá ubicar valores propios de una matriz tridiagonal:

Teorema 9.13. *Sea $\theta \in \mathbb{R}$. Considere la secuencia de Sturm $\{p_r(\theta)\}_{r=0}^m$ definida en la Definición teorema 9.11. Entonces, el número de concordancias de signo entre términos consecutivos de la secuencia de Sturm es igual al número de valores propios de T mayores que θ.*

Demostración. Ver [27, Teorema 5.9]. □

Comentario 9.14. Si algún término de la secuencia de Sturm es cero, a dicho elemento se le asigna el signo igual al elemento anterior. Debe ser claro además que si $p_m(\theta) = 0$, entonces θ es un valor propio de T. □

Ejemplo 9.15. Veamos una aplicación del teorema anterior. Considere la matriz

$$T = \begin{bmatrix} 3 & 1 & & \\ 1 & -1 & 2 & \\ & 2 & 1 & 1 \\ & & 1 & 1 \end{bmatrix}$$

con valores propios $\lambda_1 \geqslant \lambda_2 \geqslant \lambda_3 \geqslant \lambda_4$. Por el Teorema 9.5, los valores propios de T se ubican en el intervalo $[-4, 4]$. Imagine deseamos aproximar λ_2. Iniciamos calculando la secuencia de Sturm para $\theta = 0$. En este caso, se obtiene

$$\{p_r(0)\}_{r=0}^4 = \{1, 3, -4, -16, -12\},$$

la cual tiene tres concordancias en signo. Por lo tanto, existen tres valores propios mayores que $\theta = 0$. Luego,

$$\{p_r(2)\}_{r=0}^4 = \{1, 1, -4, -0, 4\},$$

la cual tiene dos concordancias en signo. Por lo tanto, existen dos valores propios mayores que $\theta = 2$. Luego,

$$\{p_r(3)\}_{r=0}^{4} = \{1, +0, -1, 2, -3\},$$

la cual tiene una concordancia en signo. Por lo tanto, existe un valor propio mayor que $\theta = 3$. De esta manera $\lambda_2 \in [2, 3]$. Note que podemos utilizar el método de bisección para obtener una aproximación de cualquier valor propio. □

9.4. Método de Jacobi

Esta sección se basa en [27, Sección 5.3]. Estudiaremos un método iterativo que permite aproximar los valores propios de una matriz $A \in \mathbb{R}^{m\times m}$ simétrica. Para ello, sabemos que la matriz

$$R(\theta) = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}$$

corresponde a una matriz de rotación con ángulo θ en dos dimensiones (en sentido antihorario si $\theta > 0$ o en sentido horario si $\theta < 0$). Es claro que

$$I = R(-\theta)R(\theta) = R(\theta)^T R(\theta)$$

y por lo tanto R es ortogonal.

Definición 9.16. Dados $m \in \mathbb{N}$ y $1 \leqslant p < q \leqslant m$, defina la matriz $R^{(pq)}(\theta) \in \mathbb{R}^{m\times m}$ como la matriz cuyas entradas no nulas vienen dadas por

$$\begin{aligned} r_{pp} = \cos(\theta) \quad & r_{pq} = \sin(\theta) \\ r_{qp} = -\sin(\theta) \quad & r_{qq} = \cos(\theta) \\ r_{ii} = 1 \quad & \text{para } \ i \neq p, q. \end{aligned}$$

Es fácil ver que $(R^{(pq)}(\theta))^T = R^{(pq)}(-\theta)$ y que es ortogonal.

Los siguientes lemas permitirán desarrollar un algoritmo para obtener una sucesión de matrices similares a A que convergerá a una matriz diagonal:

Lema 9.17. *Sea $A \in \mathbb{R}^{m\times m}$ simétrica. Dados $1 \leqslant p < q \leqslant m$, existe $\varphi \in [-\pi/4, \pi/4]$ tal que la entrada (p, q) de $(R^{(pq)}(\varphi))^T A R^{(pq)}(\varphi)$ es cero.*

Demostración. Por simplicidad denote $R = R^{(pq)}(\varphi)$ y defina $B := R^T AR$. De un cálculo directo, se puede verificar que las entradas de B vienen dadas por

$$\begin{aligned}
b_{pp} &= a_{pp}\cos^2(\varphi) - 2a_{pq}\sin(\varphi)\cos(\varphi) + a_{qq}\sin^2(\varphi),\\
b_{qq} &= a_{pp}\sin^2(\varphi) + 2a_{pq}\sin(\varphi)\cos(\varphi) + a_{qq}\cos^2(\varphi),\\
b_{pq} = b_{qp} &= (a_{pp} - a_{qq})\sin(\varphi)\cos(\varphi) + a_{pq}(\cos^2(\varphi) - \sin^2(\varphi)),\\
b_{pi} = b_{ip} &= a_{ip}\cos(\varphi) - a_{iq}\sin(\varphi) \quad (i = 1,\dots,m, i \neq p, q),\\
b_{qi} = b_{iq} &= a_{ip}\sin(\varphi) + a_{iq}\cos(\varphi) \quad (i = 1,\dots,m, i \neq p, q),\\
b_{ij} &= a_{ij} \quad (i \neq p, q,\ j \neq p, q).
\end{aligned}$$

De esta forma, si deseamos que $b_{pq} = 0$, basta encontrar φ tal que

$$(a_{pp} - a_{qq})\sin(\varphi)\cos(\varphi) + a_{pq}(\cos^2(\varphi) - \sin^2(\varphi)) = 0,$$

lo cual es equivalente a resolver la ecuación

$$\tan(2\varphi) = \frac{2a_{pq}}{a_{qq} - a_{pp}}. \tag{9.2}$$

□

Lema 9.18. *Considere $A \in \mathbb{R}^{m\times m}$. Defina $R^{(pq)}(\varphi) \in \mathbb{R}^{m\times m}$ como en el lema anterior y sea*

$$B = (R^{(pq)}(\varphi))^T AR^{(pq)}(\varphi).$$

Se tiene que

$$\sum_{\substack{i,j=1\\ i\neq j}}^{m} b_{ij}^2 = \sum_{\substack{i,j=1\\ i\neq j}}^{m} a_{ij}^2 - 2a_{pq}^2;$$

esto es, la suma de los cuadrados de elementos fuera de la diagonal de B disminuye en $2a_{pq}^2$ con respecto a la suma de los cuadrados de elementos fuera de la diagonal de A.

Demostración. Sabemos que $\|A\|_F = \|B\|_F$ pues B se obtiene de A mediante transformaciones ortogonales. En particular, se tiene que

$$\begin{bmatrix} b_{pp} & b_{pq} \\ b_{qp} & b_{qq} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}^T \begin{bmatrix} a_{pp} & a_{pq} \\ a_{qp} & a_{qq} \end{bmatrix} \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix}$$

y así

$$a_{pp}^2 + 2a_{pq}^2 + a_{qq}^2 = b_{pp}^2 + 2b_{pq}^2 + b_{qq}^2 = b_{pp}^2 + b_{qq}^2,$$

pues por el lema anterior sabemos que $b_{pq} = 0$. De esta forma,

$$\sum_{i=1}^{m} b_{ii}^2 = b_{pp}^2 + b_{qq}^2 + \sum_{i \neq p,q}^{m} b_{ii}^2 = a_{pp}^2 + 2a_{pq}^2 + a_{qq}^2 + \sum_{i \neq p,q}^{m} a_{ii}^2,$$

por lo que

$$\sum_{i=1}^{m} b_{ii}^2 = 2a_{pq}^2 + \sum_{i=1}^{m} a_{ii}^2.$$

Ahora,

$$\sum_{\substack{i,\,j\,=\,1 \\ i\,\neq\,j}}^{m} a_{ij}^2 + \sum_{i=1}^{m} a_{ii}^2 = \|A\|_F^2 = \|B\|_F^2 = \sum_{\substack{i,\,j\,=\,1 \\ i\,\neq\,j}}^{m} b_{ij}^2 + \sum_{i=1}^{m} b_{ii}^2.$$

Juntando las dos ecuaciones anteriores obtenemos el resultado. □

De esta manera, tenemos el siguiente algoritmo:

Data: $A \in \mathbb{R}^{m \times m}$ simétrica
Result: Aproximación de una matriz diagonal similar a A
$A^{(0)} = A$;
for $k = 1, 2, 3, \ldots$ **do**
  Encuentre $p < q$ tales que $|a_{pq}| = \max_{i,j} |a_{i,j}|$;
  Calcule $\varphi_k = \frac{1}{2} \arctan\left(\frac{2a_{pq}^{(k)}}{a_{qq}^{(k)} - a_{pp}^{(k)}}\right)$;
  Defina $A^{(k+1)} := (R^{(pq)}(\phi_k))^T A^{(k)} R^{(pq)}(\phi_k)$;
end

Algoritmo 20: Método de Jacobi

Del Lema 9.18 concluimos que el Algoritmo 20 produce una sucesión de matrices $A^{(k)}$ de manera tal que la suma de los cuadrados de los elementos fuera de la diagonal disminuye paulatinamente (según el máximo de cada matriz). Por lo tanto, esperamos que la sucesión converja a una matriz diagonal D, la cual contiene los valores propios de la matriz A. Esto lo demostramos formalmente en el siguiente teorema:

Teorema 9.19. *Considere la sucesión $\{A^{(k)}\}_{k=0}^{\infty}$ obtenida en el Algoritmo 20. Se cumple que*

$$\lim_{k \to \infty} \sum_{\substack{i,\,j\,=\,1 \\ i\,\neq\,j}}^{m} (a_{ij}^{(k)})^2 = 0.$$

Demostración. Denote por $L(B)$ la suma de los cuadrados de los elementos fuera de la diagonal de una matriz B. En el paso k tenemos que

$$L\left(A^{(k)}\right) = \sum_{\substack{i,\,j\,=\,1 \\ i\,\neq\,j}}^{m} (a_{ij}^{(k)})^2 \leqslant m(m-1)(a_{pq}^{(k)})^2.$$

Combinando la desigualdad anterior con el Lema 9.18 tenemos que

$$L\left(A^{(k)}\right) = L\left(A^{(k-1)}\right) - 2(a_{pq}^{(k-1)})^2 \leqslant L\left(A^{(k-1)}\right)\left(1 - \frac{2}{m(m-1)}\right).$$

De esta forma

$$L\left(A^{(k)}\right) \leqslant L\left(A^{(0)}\right)\left(1 - \frac{2}{m(m-1)}\right)^k \to 0.$$

□

Comentario 9.20. En general, encontrar el máximo de las entradas fuera de la diagonal de una matriz puede ser costoso computacionalmente. Una variante del algoritmo consiste en hacer un barrido por todas las entradas fuera de la diagonal, y así evitar encontrar el máximo. De esta forma, en la práctica es normal realizar el siguiente algoritmo:

Data: $A \in \mathbb{R}^{m\times m}$ simétrica
Result: Aproximación de una matriz diagonal similar a A
for $k = 1, 2, 3, \ldots$ **do**
  for $i = 1 : m$ **do**
    for $j = i + 1 : m$ **do**
      Construya la matriz $R^{(ij)}$ donde la entrada ij de $R^{(ij)^T} A R^{(ij)}$ es cero;
      Haga $A \leftarrow R^{(ij)^T} A R^{(ij)}$;
    end
  end
end

Algoritmo 21: Método de Jacobi modificado

9.5. Iteración de potencias

En esta sección consideramos $A \in \mathbb{R}^{m\times m}$ simétrica. De esta manera, A tiene m valores propios $|\lambda_1| \geqslant |\lambda_2| \geqslant \ldots \geqslant |\lambda_m|$ con vectores propios asociados $\boldsymbol{q}_1, \ldots, \boldsymbol{q}_m$. En los métodos estudiados hasta el momento solo hemos

determinado cómo aproximar valores propios. ¿Qué debemos hacer si necesitamos además los vectores propios? El método iterativo más básico consiste en calcular potencias de la forma $\boldsymbol{v}, A\boldsymbol{v}, A^2\boldsymbol{v}, \ldots$. Bajo ciertas hipótesis, dicha secuencia converge a un vector propio asociado al mayor valor propio. Iniciamos antes definiendo el cociente de Rayleigh, que será de gran importancia para definir las iteraciones correspondientes.

9.5.1. Cociente de Rayleigh

Dada $A \in \mathbb{R}^{m\times m}$, se llama *cociente de Rayleigh* a la función $r : \mathbb{R}^m \to \mathbb{R}$ definida por

$$r(\boldsymbol{x}) := \frac{\boldsymbol{x}^T A\boldsymbol{x}}{\boldsymbol{x}^T\boldsymbol{x}}. \tag{9.3}$$

Se tiene que si $\boldsymbol{v}$ es un vector propio asociado al valor propio λ, entonces $r(\boldsymbol{v}) = \lambda$. Además, si calculamos el gradiente de r obtenemos que

$$\nabla r(\boldsymbol{x}) = \frac{2}{\boldsymbol{x}^T\boldsymbol{x}}(A\boldsymbol{x} - r(\boldsymbol{x})\boldsymbol{x}).$$

De esta manera, si $\boldsymbol{v}$ es un vector propio entonces $\nabla r(\boldsymbol{v}) = \underline{0}$. Recíprocamente, si $\nabla r(\boldsymbol{v}) = \underline{0}$ entonces $\boldsymbol{v}$ es un vector propio con valor propio asociado $r(\boldsymbol{v})$.

Note que el desarrollo de Taylor de $r(\boldsymbol{x})$ centrado en un vector propio $\boldsymbol{v}$ viene dado por

$$r(\boldsymbol{x}) = r(\boldsymbol{v}) + \mathcal{O}(\|\boldsymbol{x} - \boldsymbol{v}\|^2).$$

De esta manera, el cociente de Rayleigh es un estimado para los valores propios de orden cuadrático.

Finalmente, note que si expandimos $\boldsymbol{x} = \sum_j \alpha_j \boldsymbol{q}_j$ como combinación lineal de vectores propios de A asociados con valores propios λ_j, tenemos que

$$r(\boldsymbol{x}) = \frac{\sum \lambda_j \alpha_j^2}{\sum \alpha_j^2}.$$

De esta forma,

$$\lambda_{\text{mín}} \leqslant r(\boldsymbol{x}) \leqslant \lambda_{\text{máx}} \ \forall\ \boldsymbol{x} \in \mathbb{R}^m.$$

9.5.2. Iteración de Potencias

Considere $\boldsymbol{v}^{(0)}$ dado con $\|\boldsymbol{v}^{(0)}\|_2 = 1$. Si expresamos $\boldsymbol{v}^{(0)}$ como combinación lineal de vectores propios

$$\boldsymbol{v}^{(0)} = \alpha_1\boldsymbol{q}_1 + \alpha_2\boldsymbol{q}_2 + \ldots \alpha_m\boldsymbol{q}_m$$

(donde $\alpha_i = \boldsymbol{q}_i^T \boldsymbol{v}^{(0)}$), defina iterativamente

$$\begin{aligned}
\boldsymbol{v}^{(1)} &:= c_1 A\boldsymbol{v}^{(0)} = c_1(\alpha_1\lambda_1\boldsymbol{q}_1 + \alpha_2\lambda_2\boldsymbol{q}_2 + \ldots + \alpha_m\lambda_m\boldsymbol{q}_m),\\
\boldsymbol{v}^{(2)} &:= c_2 A\boldsymbol{v}^{(1)} = c_2(\alpha_1\lambda_1^2\boldsymbol{q}_1 + \alpha_2\lambda_2^2\boldsymbol{q}_2 + \ldots + \alpha_m\lambda_m^2\boldsymbol{q}_m),\\
&\vdots\\
\boldsymbol{v}^{(k)} &:= c_k A\boldsymbol{v}^{(k-1)} = c_k(\alpha_1\lambda_1^k\boldsymbol{q}_1 + \alpha_2\lambda_2^k\boldsymbol{q}_2 + \ldots + \alpha_m\lambda_m^k\boldsymbol{q}_m),
\end{aligned}$$

donde las constantes c_k se escogen de manera tal que $\|\boldsymbol{v}^{(k)}\|_2 = 1$. De esta manera, si $\lambda_1 > \lambda_2$, se tiene que

$$\boldsymbol{v}^{(k)} = c_k\lambda_1^k\left(\alpha_1\boldsymbol{q}_1 + \alpha_2\left(\frac{\lambda_2}{\lambda_1}\right)^k\boldsymbol{q}_2 + \ldots + \alpha_m\left(\frac{\lambda_m}{\lambda_1}\right)^k\boldsymbol{q}_m\right). \tag{9.4}$$

Por lo tanto, tenemos el siguiente resultado:

Teorema 9.21. *Suponga que* $|\lambda_1| > |\lambda_2| \geqslant \ldots \geqslant |\lambda_m| \geqslant 0$ *y que* $\boldsymbol{q}_1^T\boldsymbol{v}^{(0)} \neq 0$. *Entonces, la sucesión* $\{\boldsymbol{v}^{(k)}\}_{k\geqslant 0}$ *converge a* $\boldsymbol{q}_1$. *Además,*

$$\|\boldsymbol{v}^{(k)} - \boldsymbol{q}_1\| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right) \quad y \quad |\lambda^{(k)} - \lambda_1| = \mathcal{O}\left(\left|\frac{\lambda_2}{\lambda_1}\right|^{2k}\right),$$

donde $\lambda^{(k)} = \left(\boldsymbol{v}^{(k)}\right)^T A\boldsymbol{v}^{(k)}$ *es el cociente de Rayleigh de* $\boldsymbol{v}^{(k)}$.

Comentario 9.22. Al escribir que una sucesión de vectores converge a cierto vector $\boldsymbol{q}$, realmente el espacio generado por el límite de la sucesión es el mismo al espacio generado por $\boldsymbol{q}$. De esta forma, en el teorema anterior realmente $\boldsymbol{v}^{(k)} \to +\boldsymbol{q}_1$ o $\boldsymbol{v}^{(k)} \to -\boldsymbol{q}_1$, pero por simplicidad simplemente diremos que la sucesión converge al vector. □

De esta forma, tenemos el siguiente algoritmo:

Data: $A \in \mathbb{R}^{m\times m}$ simétrica, $\boldsymbol{v}^{(0)} \in \mathbb{R}^m$ unitario
Result: Aproximación del mayor valor propio (en valor absoluto) y su correspondiente vector propio
for $k = 1, 2, 3, \ldots$ **do**
  $\boldsymbol{w} = A\boldsymbol{v}^{(k-1)}$;
  $\boldsymbol{v}^{(k)} = \boldsymbol{w}/\|\boldsymbol{w}\|$;
  $\lambda^{(k)} = \left(\boldsymbol{v}^{(k)}\right)^T A\boldsymbol{v}^{(k)}$;
end

Algoritmo 22: Método de Iteración de Potencias

9.5.3. Iteración Inversa de Potencias

Suponga que μ no es un valor propio de A. Si $A\boldsymbol{x} = \lambda\boldsymbol{x}$, entonces es fácil ver que $(A - \mu I)^{-1}\boldsymbol{x} = (\lambda - \mu)^{-1}\boldsymbol{x}$. De esta forma, si μ es muy cercano a un valor propio λ_j, entonces se espera que

$$|\lambda_j - \mu|^{-1} \gg |\lambda_k - \mu|^{-1} \quad \text{para} \quad j \neq k.$$

De esta manera, si aplicamos el Método de Iteración de Potencias a la matriz $(A-\mu I)^{-1}$ esperamos que la sucesión converja a $\boldsymbol{q}_j$. De esta manera, tenemos el siguiente algoritmo:

Data: $A \in \mathbb{R}^{m\times m}$ simétrica, $\boldsymbol{v}^{(0)} \in \mathbb{R}^m$ unitario, $\mu \in \mathbb{R}$
Result: Aproximación del valor propio más cercano a μ y su correspondiente vector propio
for $k = 1, 2, 3, \ldots$ **do**
  Resuelva $(A - \mu I)\boldsymbol{w} = \boldsymbol{v}^{(k-1)}$ para $\boldsymbol{w}$;
  $\boldsymbol{v}^{(k)} = \boldsymbol{w}/\|\boldsymbol{w}\|$;
  $\lambda^{(k)} = \left(\boldsymbol{v}^{(k)}\right)^T A\boldsymbol{v}^{(k)}$;
end

Algoritmo 23: Método de Iteración Inversa de Potencias

Para el algoritmo descrito anteriormente, tenemos el siguiente resultado:

Teorema 9.23. *Suponga que* $|\mu - \lambda_J| < |\mu - \lambda_K| \leqslant |\mu - \lambda_j| \ \forall\ j \neq J$. *Suponga además que* $\boldsymbol{q}_J^T\boldsymbol{v}^{(0)} \neq 0$. *Entonces, la iteración del Algoritmo 23 satisface que*

$$\|\boldsymbol{v}^{(k)} - \boldsymbol{q}_J\| = \mathcal{O}\left(\left|\frac{\mu - \lambda_J}{\mu - \lambda_K}\right|^k\right) \quad y \quad |\lambda^{(k)} - \lambda_J| = \mathcal{O}\left(\left|\frac{\mu - \lambda_J}{\mu - \lambda_K}\right|^{2k}\right).$$

9.5.4. Iteración del cociente de Rayleigh

En la Iteración Inversa de Potencias, podríamos usar el hecho que $\lambda^{(k)}$ converge a un valor propio, y así podemos pensar en tomar $\mu = \lambda^{(k)}$. De esta forma, tenemos el algoritmo del cociente de Rayleigh:

Data: $A \in \mathbb{R}^{m\times m}$ simétrica, $\boldsymbol{v}^{(0)} \in \mathbb{R}^m$ unitario, $\lambda^{(0)} := (\boldsymbol{v}^{(0)})^T A\boldsymbol{v}^{(0)}$
Result: Aproximación del valor propio más cercano a $\lambda^{(0)}$ y su correspondiente vector propio
for $k = 1, 2, 3, \ldots$ **do**
  Resuelva $(A - \lambda^{(k-1)}I)\boldsymbol{w} = \boldsymbol{v}^{(k-1)}$ para $\boldsymbol{w}$;
  $\boldsymbol{v}^{(k)} = \boldsymbol{w}/\|\boldsymbol{w}\|$;
  $\lambda^{(k)} = \left(\boldsymbol{v}^{(k)}\right)^T A\boldsymbol{v}^{(k)}$;
end

Algoritmo 24: Método del cociente de Rayleigh

Note que este algoritmo es más costoso pues en cada iteración se debe resolver un sistema de ecuaciones diferente; por el contrario, en el método de iteración inversa de potencias basta obtener alguna factorización de $A - \mu I$ una vez. No obstante, la convergencia del método del cociente de Rayleigh es muy rápida:

Teorema 9.24. *El Método del cociente de Rayleigh converge a un valor propio λ_J y su correspondiente vector propio $\boldsymbol{q}_J$. Además, la convergencia es cúbica:*

$$\|\boldsymbol{v}^{(k+1)} - \boldsymbol{q}_J\| = \mathcal{O}(\|\boldsymbol{v}^{(k)} - \boldsymbol{q}_J)\|^3), \quad |\lambda^{(k+1)} - \lambda_J| = \mathcal{O}(|\lambda^{(k)} - \lambda_J|^3).$$

9.6. Algoritmo QR

Uno de los métodos más usados y sencillos corresponde al algoritmo QR, el cual presentamos a continuación en su forma básica:

Data: $A \in \mathbb{C}^{m\times m}$
Result: Aproximación $\widetilde{T}$ de la matriz triangular superior de la factorización de Schur de $A = QTQ^*$
for $k = 1, 2, 3, \ldots$ **do**
  Factorice $Q^{(k)}R^{(k)} = A^{(k-1)}$ para $\boldsymbol{w}$;
  Defina $A^{(k)} = R^{(k)}Q^{(k)}$;
end

Algoritmo 25: Método QR

Note que en el paso k, se tiene que $A^{(k)} = (Q^{(k)})^T A^{(k-1)}Q^{(k)}$. De esta forma, esperamos que $A^{(k)}$ converja a una matriz triangular superior (o tridiagonal si A es Hermitiana). No obstante, el algoritmo se puede mejorar tal como se muestra en el Algoritmo 27 (donde asumimos que A es hermitiana por simplicidad). Referimos al lector interesado a [29, Capítulos 28, 29, 30] para más detalles.

Data: $A \in \mathbb{C}^{m\times m}$ Hermitiana
Result: Aproximación $\widetilde{T}$ de la matriz diagonal de la factorización de Schur de $A = QTQ^*$
Calcule forma de Hessenberg $(Q^{(0)})^T A^{(0)} Q^{(0)} = A$ (T tridiagonal);
for $k = 1, 2, 3, \ldots$ **do**
  Escoja $\mu^{(k)}$ (por ejemplo, $\mu^{(k)} = A^{(k-1)}_{mm}$;
  Factorice $Q^{(k)}R^{(k)} = A^{k-1} - \mu^{(k)}I$;
  Calcule $A^{(k)} = R^{(k)}Q^{(k)} + \mu^{(k)}I$;
  Si algún elemento bajo la diagonal $A^{(k)}_{j,j+1}$ es cercano a cero, haga $A^{(k)}_{j,j+1} = A^{(k)}_{j+1,j} = 0$ y utilice deflación;
end

Algoritmo 26: Método QR práctico

9.7. Ejercicios Adicionales

Ejercicio 9.25. (Algoritmo de Jacobi para valores propios) Escriba un programa para encontrar los valores propios de una matriz $A \in \mathbb{R}^{m\times m}$ simétrica mediante el algoritmo de Jacobi. En cada iteración, utilice un barrido por todas las entradas de la matriz que se encuentran fuera de la diagonal. Dibuje la suma de los cuadrados de las entradas fuera de la diagonal en una escala logarítmica en función del número de barridos. Utilice matrices aleatorias de dimension 20, 40 y 80, y realice 10 barridos en cada caso. Comente sus resultados.

Ejercicio 9.26. (Secuencias de Sturm)

(a) Considere una matriz tridiagonal simétrica $T \in \mathbb{R}^{m\times m}$ dada por

$$T = \begin{bmatrix} a_1 & b_2 & & & & \\ b_2 & a_2 & b_3 & & & \\ & b_3 & a_3 & b_4 & & \\ & & \ddots & \ddots & \ddots & \\ & & & b_{m-1} & a_{m-1} & b_m \\ & & & & b_m & a_m \end{bmatrix}.$$

Escriba un programa cuyas entradas sean los vectores $\boldsymbol{a} \in \mathbb{R}^n$, $\boldsymbol{b} \in \mathbb{R}^{n-1}$ (que definen a T) y un valor θ, que calcule la secuencia de Sturm de T evaluada en θ y contabilice el número de concordancias en signo.

(b) Utilice la función anterior para encontrar el mayor y menor valor propio de la matriz T

$$A = \begin{bmatrix} 4 & -1 & & & & \\ -1 & 4 & -1 & & & \\ & \ddots & \ddots & \ddots & \ddots & \\ & & -1 & 4 & -1 & \\ & & & -1 & 4 & \end{bmatrix}$$

para $m = 10^6$ (las entradas en la diagonal son 4 y en la subdiagonal y superdiagonal son -1). Dicha matriz aparece de manera frecuente en la discretización de ecuaciones diferenciales. Utilice un método de bisección con una tolerancia `1e-6` y defina un intervalo inicial mediante el Teorema de Gershgorin.

(c) Por limitaciones de memoria, no es posible definir la matriz de manera *usual* mediante el comando

`T = diag(a)+diag(b,-1)+diag(b,1).`

Defina tres vectores `i`, `j`, `s` $\in \mathbb{R}^{3m-2}$ tales que $(T)_{\mathtt{i}(k),\mathtt{j}(k)} = \mathtt{s}(k)$ para $k = 1, \ldots, 3m-2$. Genere T mediante el comando `sparse(i,j,k,m,m)` (las últimas dos entradas indican el tamaño de la matriz).

(d) Los comandos `eigs(T,1,'LM')` y `eigs(T,1,'SM')` calculan un valor propio de mayor magnitud y un valor propio de menor magnitud, respectivamente (las banderas indican *largest magnitude* y *smallest magnitud*). Note además que como la matriz es *sparse* debe usarse el comando `eigs` en lugar de `eig`. Intente utilizar dicho comando para encontrar el mayor y menor valor propio de la matriz T. ¿Qué ocurre?

Ejercicio 9.27. Considere la matriz tridiagonal simétrica $T \in \mathbb{R}^{m\times m}$ dada por

$$T = \begin{bmatrix} \alpha & \beta & & & & \\ \beta & \alpha & \beta & & & \\ & \ddots & \ddots & \ddots & & \\ & & \beta & \alpha & \beta & \\ & & & \beta & \alpha & \end{bmatrix}.$$

(a) Demuestre que los valores propios de T vienen dados por

$$\lambda_j = \alpha + 2\beta \cos\left(\frac{\pi j}{m+1}\right) \quad (j = 1, \ldots, m)$$

donde su correspondiente vector propio $\boldsymbol{q}_j$ tiene entradas

$$\boldsymbol{q}_{j,l} = \sqrt{\frac{2}{m+1}} \sin\left(\frac{\pi j l}{m+1}\right) \quad (j,l = 1,\ldots,m).$$

(b) Tome $\alpha = 3$, $\beta = -1$. Estime teóricamente el número de condición de la matriz $\kappa_2(A)$ para m suficientemente grande. ¿La matriz es bien o mal condicionada?

(c) Para $m = 10^6$ defina T (como matriz *sparse*). Utilice la función `condest(T)` para obtener un estimado del número de condición $\kappa_\infty(A)$.

(d) Asuma que queremos resolver el sistema $T\boldsymbol{x} = \boldsymbol{b}$, donde $\boldsymbol{b} =$ `rand(m,1)`. ¿Cuántas operaciones aproximadamente se necesitarían para resolver el sistema si lo hacemos mediante su factorización de Cholesky de manera directa?

(e) En `Matlab`, el método Gradiente Conjugado se puede ejecutar mediante el comando `[x,flag,relres,iter,resvec] = pcg(T,b,tol,maxit)`, donde `tol` es una tolerancia dada y `maxit` el número máximo de iteraciones posibles. Utilice dicho comando para resolver el sistema $T\boldsymbol{x} = \boldsymbol{b}$ con `tol` $= 10^{-8}$, `maxit` $= 500$. Determine si el algoritmo converge, y en tal caso, indique el número de iteraciones realizadas. Grafique además el vector de residuos relativos `resvec` en una escala adecuada. ¿Dicho comportamiento es el esperado? (Sugerencia: vea Lema 9.13 en las notas).

(f) Estime el número de operaciones realizado en el inciso (e). Compare con el número de operaciones obtenido en el inciso (d), y concluya cuál método requiere un número menor de operaciones.

Ejercicio 9.28. Considere la matriz tridiagonal $T \in \mathbb{R}^{m\times m}$ dada por

$$T = \begin{bmatrix} a_1 & b_1 & & & & \\ c_1 & a_2 & b_2 & & & \\ & \ddots & \ddots & \ddots & & \\ & & c_{m-2} & a_{m-1} & b_{m-1} & \\ & & & c_{m-1} & a_m & \end{bmatrix},$$

donde $b_i c_i > 0$ para todo $i = 1,\ldots,m-1$.

(a) Determine de manera explícita una matriz diagonal $D \in \mathbb{R}^{m\times m}$ tal que $A := DTD^{-1}$ sea simétrica.

(b) Demuestre que todos los valores propios de T son reales.

(c) Demuestre que todos los valores propios de T son diferentes.

Para las siguientes preguntas, considere el caso particular

$$\begin{aligned}\boldsymbol{a} &:= [1, 1, \ldots, 1]^T \in \mathbb{R}^m,\\ \boldsymbol{b} &:= [-2, -2, \ldots, -2]^T \in \mathbb{R}^{m-1},\\ \boldsymbol{c} &:= [-3, -3, \ldots, -3]^T \in \mathbb{R}^{m-1};\end{aligned}$$

esto es, $T = \texttt{diag}(\boldsymbol{a}) + \texttt{diag}(\boldsymbol{b}, 1) + \texttt{diag}(\boldsymbol{c}, -1)$.

(d) Para $m = 10$, construya D y verifique que A es simétrica. Obtenga los valores propios de T y de A mediante el comando `eig()` y compare los resultados. ¿Qué relación se tiene entre los valores propios obtenidos de A y los de T?

(e) Para $m = 500$, construya D y verifique que A es simétrica. Obtenga los valores propios de T y de A mediante el comando `eig()`. Grafique los valores propios de T mediante el comando `plot(eig(T),'.')`. ¿Qué se observa en el gráfico? ¿Qué relación se tiene entre los valores propios obtenidos de A y los de T? Comente sus resultados.

(f) ¿Qué inconveniente tiene utilizar el método de Jacobi para calcular los valores propios de A del inciso anterior?

Ejercicio 9.29. Dado n entero positivo, sea $\mathbb{P}_n$ el espacio de polinomios de grado a lo sumo n con coeficientes reales definidos en el intervalo $[-1, 1]$. Defina el operador diferencial $\mathcal{L} : \mathbb{P}_n \to \mathbb{P}_n$ dado por

$$\mathcal{L}P(x) = (1 - x^2)\frac{d^2P(x)}{dx^2} - 2x\frac{dP(x)}{dx}.$$

En este ejercicio deseamos aproximar los valores propios y funciones propias del operador diferencia. Al ser $\mathbb{P}_n$ un espacio de dimensión finita, es posible escribir la acción del operador $\mathcal{L}$ como una matriz $A \in \mathbb{R}^{n\times n}$ que actúa en los coeficientes de cada polinomio.

(a) Escriba $p(x) = \sum_{k=0}^{n} b_k x^k$, donde $\boldsymbol{b} = [b_0, \ldots, b_n]^T$ representa el vector de coeficientes. Determine la matriz A tal que $\boldsymbol{c} = A\boldsymbol{b}$ da el vector con coeficientes de la imagen $\mathcal{L}P(x) = \sum_{k=0}^{n} c_k x^k$.

(b) Fije n = 7. Calcule la matriz A. Mediante el comando `[V,D] = eig(A)` aproxime los vectores y valores propios de A. Deduzca una fórmula para los valores propios, según los resultados encontrados. Grafique además las funciones propias obtenidas. Comente sus resultados.

Ejercicio 9.30. Considere el Método QR para encontrar valores propios de una matriz A:

Data: $A \in \mathbb{C}^{m\times m}$
Result: Aproximación $\widetilde{T}$ de la matriz triangular superior de la factorización de Schur de $A = QTQ^*$
for $k = 1, 2, 3, \ldots$ **do**
  Factorice $Q^{(k)}R^{(k)} = A^{(k-1)}$;
  Defina $A^{(k)} = R^{(k)}Q^{(k)}$;
end

Algoritmo 27: Método QR

(a) Escriba una función `T = eigQR(A,k)` que implemente el Algoritmo 1 para una matriz dada A y un número máximo de iteraciones k.

(b) Para `A=hilb(m)`, con $m \in \{20, 40, 60, 80\}$, valide el comportamiento de su código. Grafique la norma infinito del error entre los valores propios aproximados y los reales, en función de m. Utilice `eig(A)` como los valores propios exactos (verifique que ambos vectores están ordenados).

Capítulo 10

Ejercicios finales

En este capítulo presentamos preguntas cortas que han sido recopiladas de exámenes orale del curso, así como preguntas conceptuales, que no requieren mayores demostraciones matemáticas.

1. Considere una función f continua tal que $f(c) = 0$ para algún $c \in [a, b]$.

   a) Suponga que f es derivable en $[a, b]$. Dado un valor inicial $x_0 \in [a, b]$, escriba el método de iteración de Newton.

   b) Sabemos que si c es un cero simple, entonces el método converge cuadráticamente. Enuncie las hipótesis necesarias.

   c) Suponga ahora que c es un cero doble; i.e.,

   $$f(c) = f'(c) = 0, \ f''(c) \neq 0.$$

   ¿Cómo debe modificarse el método de Newton descrito en la parte (a) para restablecer convergencia cuadrática en este caso?

   d) En la práctica, es posible no tener conocimiento del orden del cero c a priori. ¿Cómo puede determinar numéricamente la multiplicidad del cero entonces?

   e) Mencione una ventaja de utilizar el método de la secante en lugar del método de Newton, si sabemos que f cumple las hipótesis para la convergencia a c en ambos casos.

2. Considere $f \in C^1([a, b])$ y sea $\{x_j\}_{j=1}^n$ un conjunto de n puntos en el intervalo $[a, b]$.

   a) Defina el polinomio de interpolación de Hermite h en los nodos $\{x_j\}_{j=1}^n$.

b) Escriba la fórmula para el error $f(x) - h(x)$ y las hipótesis necesarias que debe cumplir f.

c) Considere en particular el intervalo $[-1, 1]$ con nodos de interpolación

$$\{-1, -1/3, 1/3, 1\}.$$

Calcule el polinomio de interpolación de Hermite para $f(x) = x^6 - x + 1$ con dichos nodos de interpolación.

3. Considere una función f continua en $[a, b]$ y asuma que existe $c \in (a, b)$ tal que $f(c) = 0$.

a) Suponga que f es derivable en $[a, b]$. Dado un valor inicial $x_0 \in [a, b]$, escriba el método de iteración de Newton.

b) Considere la función $f(x) = x(1-x)^2$. Si tomamos x_0 suficientemente cercano a $c = 0$, ¿cuál es la velocidad de convergencia del método de iteración de Newton?

c) Considere la función $f(x) = x(1-x)^2$. Si tomamos x_0 suficientemente cercano a $c = 1$, ¿cuál es la velocidad de convergencia del método de iteración de Newton?

d) En los incisos (b) y (c), si la convergencia no es cuadrática, ¿cómo debe modificarse el método de Newton descrito en la parte (a) para restablecer convergencia cuadrática?

e) Mencione una ventaja de utilizar el método de la bisección en lugar del Método de Newton, si sabemos que f cumple las hipótesis para la convergencia a c en ambos casos.

4. Considere una función f continua en el intervalo $[a, b]$, y suponga que $f(a)f(b) < 0$.

a) Escriba explícitamente el algoritmo del Método de Bisección.

b) Demuestre formalmente que la sucesión del método de bisección converge a una solución c de la ecuación $f(x) = 0$.

c) ¿Qué ocurre si la solución c no es única?

5. Considere el problema de interpolación para un conjunto dado de n puntos $\{(x_i, y_i)\}_{i=1}^n$.

a) Escriba el polinomio de interpolación de Lagrange p para el conjunto de puntos dado. ¿Cuál es el grado de dicho polinomio?

b) Describa una manera eficiente de evaluar tal polinomio (puede indicar un algoritmo o una fórmula explícita).

c) ¿Existe algún problema si los nodos son escogidos como una partición uniforme del dominio $[a, b]$ al aproximar f por p? En caso afirmativo, indique un ejemplo.

d) Indique dos ventajas de utilizar nodos de Chebyshev al calcular el polinomio de interpolación de Lagrange.

6. Asuma que $f : [a, b] \to \mathbb{R}$ tiene al menos un cero en su dominio. ¿Cómo determinaría los valores iniciales para utilizar el Método de la Secante?

7. Se sabe que $f(c) = 0$. ¿Cómo puede determinar numéricamente la multiplicidad de dicho cero?

8. Considere el método de bisección con intervalo inicial $[1, 2]$. ¿Tiene sentido hacer más de 52 iteraciones en precisión doble?

9. Se requiere interpolar un conjunto de datos $(x_i, y_i)_{i=0}^n$ mediante una función que se requiere sea continua. ¿Cuál método utilizaría? ¿Por qué?

10. Para evaluar el polinomio de interpolación de Lagrange en un punto dado, ¿cuál algoritmo utilizaría para hacerlo de forma eficiente? ¿Por qué?

11. En una regla de cuadratura en el intervalo $[0, 1]$ con $n + 1$ nodos, se fijan 3 nodos internos. ¿Cuál es el mayor grado de un polinomio que se puede integrar de manera exacta?

12. ¿Cómo integra un polinomio dado en el intervalo $[a, b]$ de una manera eficiente?

13. ¿Cómo aproxima el valor de $\int_0^\infty f(x)\ dx$ para una función continua dada (asumiendo que sabemos la integral converge)?

14. ¿Cómo calcula los valores singulares de una matriz dada?

15. Dada una matriz A, ¿cómo calcula numéricamente su rango, núcleo, espacio generado por filas, espacio generado por columnas?

16. Defina la transformación de Householder. ¿Cuáles son sus valores propios, determinante y valores singulares?

17. ¿Cómo determina el determinante de una matriz a partir de sus diferentes factorizaciones?

18. ¿Qué es pivoteo parcial? ¿Por qué es importante realizar pivoteo en la factorización LU?

19. ¿Cómo determina numéricamente si una matriz es definida positiva?

20. Dada una matriz definida positiva, ¿cuál factorización es recomendable utilizar? ¿Por qué?

21. Dada una matriz definida positiva, ¿existe alguna factorización más (o menos) favorable para calcular? ¿Por qué?

22. Para resolver un sistema con solución única $A\boldsymbol{x} = \boldsymbol{b}$, ¿cuál factorización utilizaría y por qué?

23. Defina el número de condición de una función f. ¿Qué significado tiene?

24. ¿Cuál es la relación entre el número de condición de una matriz y sus valores singulares?

25. ¿Qué ocurre si resolvemos un sistema de ecuaciones con una matriz cuyo número de condición es alto?

26. ¿Qué significa en palabras sencillas que un algoritmo es estable hacia atrás?

27. ¿Qué es el factor de crecimiento de una matriz? ¿Por qué es relevante hablar del factor de crecimiento al resolver sistemas de ecuaciones?

28. ¿Qué inconveniente se puede presentar al resolver el problema de mínimos cuadrados con ecuaciones normales?

29. ¿Bajo qué condiciones se garantiza convergencia del Método de Newton para resolver sistemas de ecuaciones no lineales?

30. Con respecto a los métodos de Euler y el Trapecio para resolver ecuaciones diferenciales ordinarias, ¿qué ventajas y desventajas tienen entre sí?

31. Defina orden de un método para resolver una ecuación diferencial ordinaria.

32. ¿Que cuidado hay que tener al utilizar un método multipaso como el de Adams?

33. Con respecto a los métodos de Runge Kuta explícitos e implícitos, ¿qué ventajas y desventajas tienen entre sí?

34. Con respecto a los métodos del trapecio y predictor-corrector, ¿qué ventajas y desventajas tienen entre sí?

35. ¿Qué ocurre si deseamos resolver un sistema de ecuaciones con una matriz mal condicionada de gran tamaño? ¿Qué ventaja presentan los métodos iterativos?

36. ¿Bajo qué condiciones convergen los métodos de Jacobi y Gauss-Seidel?

37. Describa en términos generales el método de gradiente para resolver sistemas de ecuaciones lineales.

38. Asuma que $f : [a, b] \to \mathbb{R}$ tiene al menos un cero en su dominio. ¿Cómo determinaría los valores iniciales para utilizar el Método de la Secante?

39. Se sabe que $f(c) = 0$. ¿Cómo puede determinar numéricamente la multiplicidad de dicho cero?

40. Considere el método de bisección con intervalo inicial $[1, 2]$. ¿Tiene sentido hacer más de 52 iteraciones en precisión doble?

41. Se requiere interpolar un conjunto de datos $(x_i, y_i)_{i=0}^n$ mediante una función que se requiere sea continua. ¿Cuál método utilizaría? ¿Por qué?

42. Para evaluar el polinomio de interpolación de Lagrange en un punto dado, ¿cuál algoritmo utilizaría para hacerlo de forma eficiente? ¿Por qué?

43. En una regla de cuadratura en el intervalo $[0, 1]$ con $n + 1$ nodos, se fijan 3 nodos internos. ¿Cuál es el mayor grado de un polinomio que se puede integrar de manera exacta?

44. ¿Cómo integra un polinomio dado en el intervalo $[a, b]$ de una manera eficiente?

45. ¿Cómo aproxima el valor de $\int_0^\infty f(x)\ dx$ para una función continua dada (asumiendo que sabemos la integral converge)?

46. ¿Qué método utilizaría para aproximar numéricamente la solución de la ecuación $|x^2 - 1| = 0$? ¿Por qué?

47. ¿Qué espera que ocurra si utiliza el Método de Newton para resolver la ecuación $x^{100} = 0$ con un valor inicial distinto de cero? ¿Por qué?

48. Escriba la iteración de Newton para encontrar la solución de la ecuación $-x^3 + 4x^2 - 2x + 2 = 0$. Tome como valor inicial $x_0 = 0$. Utilice el valor exacto de la derivada para evaluar la iteración. ¿Qué ocurre en este caso?

49. Se sabe que $f(c) = 0$. ¿Cómo puede determinar numéricamente la multiplicidad de dicho cero?

50. Considere el método de bisección con intervalo inicial $[1, 2]$. ¿Tiene sentido hacer más de 52 iteraciones en precisión doble? ¿Por qué?

51. ¿Cuál es el polinomio de Lagrange de la función $f(x) = x^2 + 3x + 1$ que interpola a f en los nodos $x_0 = 0, x_1 = 1, x_2 = 2$?

52. Si conoce a priori que una función tiene un cero de multiplicidad dos, ¿qué método numérico utilizaría para encontrar dicho cero? Indique al menos una ventaja del método seleccionado.

53. Al resolver cierta ecuación, se obtiene la sucesión $a_k = 0.99a_{k-1}$, con a_0 un valor dado. Utilice la definición de orden de convergencia para determinar si converge lineal, sublineal o superlinealmente.

54. Suponga que tiene un conjunto de datos $(x_i, y_i, z_i, w_i)_{i=0}^n$, donde $\{x_i\}_{i=0}^n$ es una partición de un intervalo dado $[a, b]$. Se desea construir un polinomio p que satisfaga

$$p(x_i) = y_i, p'(x_i) = z_i, p''(x_i) = w_i.$$

¿Cuál es el menor n para el cual se garantiza que $p \in \mathbb{P}_n$, para cualquier conjunto de datos? (Recuerde que $\mathbb{P}_n$ es el conjunto de polinomios de grado a lo sumo n). Justifique.

55. Suponga que tiene un código que implementa la evaluación de cierta función f para cualquier $x \in [a, b]$ (esto es, no conocemos la función, pero podemos evaluarla en cualquier punto). Describa un método que utilizaría para construir un polinomio que aproxime f'. Indique ventajas de su escogencia.

56. Suponga que desea calcular numéricamente

$$\int_{-\infty}^{\infty} e^{-x^2}(x^5 + x^4 + x^3 + x^2 + x + 1)\, dx$$

de forma exacta mediante una cuadratura de Gauss. ¿Cuál es el mínimo de nodos que garantiza exactitud en la fórmula? Justifique.

57. Considere la función $h(x) = -1 + x\ln(x)$. Deseamos resolver la ecuación $h(c) = 0$, la cual sabemos que tiene una única solución positiva.

    a) Describa un método numérico visto en clase que utilizaría para resolver dicha ecuación. Justifique su escogencia (esto es, que se satisfacen las condiciones para garantizar convergencia). Escriba cómo obtiene la sucesión correspondiente (ya sea mediante un pseudocódigo o una fórmula para la sucesión). Indique cómo selecciona la(s) condición(es) inicial(es).

    b) Indique una forma de detener las iteraciones para garantizar una tolerancia dada.

    c) Indique el orden de convergencia del método seleccionado.

58. Sea A una matriz cuadrada no invertible. Describa una forma de obtener una solución numérica no trivial (distinta de cero) del sistema $Ax = 0$.

59. ¿Por qué es relevante conocer el número de condición de una matriz? ¿Y por qué el factor de crecimiento es importante?

60. Dada una matriz $A \in \mathbb{R}^{3\times 2}$, ¿cómo puede obtener a mano los valores singulares de A? ¿Y las matrices U, V ortogonales de la factorización en valores singulares de A?

61. Dados tres nodos $(x_0, y_0), (x_1, y_1), (x_2, y_2)$, con x_1, x_2, x_3 distintos, ¿cómo calcula una función lineal f tal que se minimice el error cuadrático $\sum_{i=1}^{3}[y_i - f(x_i)]^2$?

62. Considere $A \in \mathbb{C}^{m\times n}$.

    a) Escriba la definición de la factorización completa en valores singulares de una matriz A. Indique el tamaño de cada factor, así como sus características.

b) ¿Bajo qué condiciones dicha factorización existe?

c) Demuestre que $\|A\|_F = \sqrt{\sum \sigma_j^2}$, donde $\{\sigma_j\}$ son los valores singulares de A.

d) ¿Cómo puede obtener la imagen y el núcleo de A si conoce su factorización en valores singulares?

e) Describa cómo obtener una matriz B de rango r que minimiza la norma $\|A - B\|_2$. ¿Cuál es el valor de $\|A - B\|_2$?

f) (3 puntos) Considere $\boldsymbol{u} \in \mathbb{C}^m$, $\boldsymbol{v} \in \mathbb{C}^n$, con $\|\boldsymbol{u}\|_2 = \|\boldsymbol{v}\|_2 = 1$. Si $A = \boldsymbol{u}\boldsymbol{v}^T$, ¿cuáles son los valores singulares de A? ¿Cómo obtiene una factorización completa en valores singulares de A?

g) Considere $\boldsymbol{u} \in \mathbb{C}^m$, $\boldsymbol{v} \in \mathbb{C}^n$ vectores no necesariamente unitarios. Si $A = \boldsymbol{u}\boldsymbol{v}^T$, ¿cuáles son los valores singulares de A?

63. Dado $\boldsymbol{v} \in \mathbb{R}^n$ no nulo, considere el reflector de Householder $F \in \mathbb{R}^{n\times n}$ dado por

$$F = I - 2\frac{\boldsymbol{v}\boldsymbol{v}^T}{\boldsymbol{v}^T\boldsymbol{v}},$$

donde $I \in \mathbb{R}^{n\times n}$ es la matriz identidad.

a) Dado $\boldsymbol{x} \in \mathbb{R}^n$, explique el significado geométrico de $F\boldsymbol{x}$.

b) Demuestre que F es una matriz ortogonal.

c) Calcule los valores propios y los valores singulares de F.

d) Dados $\boldsymbol{a}, \boldsymbol{b} \in \mathbb{R}^n$, ¿qué condiciones deben cumplir para poder definir F tal que $F\boldsymbol{a} = \boldsymbol{b}$? En tal caso, determine un vector $\boldsymbol{v}$ que garantice que $F\boldsymbol{a} = \boldsymbol{b}$.

e) Describa de forma completa el algoritmo para obtener la factorización QR de una matriz $A \in \mathbb{C}^{m\times m}$ mediante reflectores de Householder. Explique el efecto de la acción del reflector de Householder en cada paso.

64. Considere una matriz simétrica cuadrada invertible $A \in \mathbb{R}^{m\times m}$.

a) ¿Cómo determinaría numéricamente si A es definida positiva?

b) Suponga que A es definida positiva. Defina la factorización de Cholesky de A. ¿Es única?

c) Describa un algoritmo para resolver el sistema de ecuaciones $A\boldsymbol{x} = \boldsymbol{b}$ si sabemos la factorización de Cholesky de A. Indique el número de operaciones necesarias para resolver el sistema.

d) ¿Qué se puede decir del error $\|\boldsymbol{x} - \tilde{\boldsymbol{x}}\|$ entre el valor exacto $\boldsymbol{x}$ y la aproximación numérica $\tilde{\boldsymbol{x}}$ al utilizar el algoritmo del punto anterior?

65. Dada $A \in \mathbb{C}^{m\times m}$ invertible, considere su factorización $PA = LU$ obtenida mediante eliminación Gaussiana con pivoteo parcial.

    a) Defina el número de condición $\kappa_2(A)$ de A en la 2-norma.

    b) ¿Cuál es el cuidado que debemos tener si utilizamos la factorización LU al resolver sistemas de ecuaciones, si sabemos que $\rho(A)$ es muy alto? Mencione una manera de evitar dicho problema.

    c) Suponga que $\kappa_2(A) = 10^{16}$. Si resolvemos el sistema $LU\boldsymbol{x} = \boldsymbol{b}$ mediante sustitución hacia adelante y hacia atrás (en aritmética de punto flotante con precisión doble), ¿qué se espera de la precisión de nuestra aproximación numérica?

66. Sea $A \in \mathbb{R}^{m\times m}$ invertible con $A = QR$ su factorización QR, donde las entradas de la diagonal de R satisfacen $r_{j,j} > 0$. Denote por $A^*A = U^*U$ la factorización de Cholesky de A^*A, donde las entradas de la diagonal de U satisfacen $u_{j,j} > 0$. ¿Es cierto que $R = U$? Demuestre formalmente o dé un contraejemplo.

Bibliografía

[1] Å. Björck, *Numerical Methods for Least Squares Problems*, Siam Philadelphia, 1996.

[2] R. Brent, *Algorithms for minimization without derivatives*, Prentice-Hall, 1973.

[3] J. Butcher, *Numerical Methods for Ordinary Differential Equations*, Wiley, 2003.

[4] J. C. Butcher, *The Numerical Analysis of Ordinary Differential Equations: Runge-Kutta and General Linear Methods*, Wiley-Interscience, New York, NY, USA, 1987.

[5] C. De Boor, *A practical guide to splines; rev. ed.*, Applied mathematical sciences, Springer, Berlin, 2001.

[6] P. Deuflhard, *A Short History of Newton's Method*, Documenta Mathematica, Extra Volume: Optimization Stories (2012), pp. 25–30.

[7] M. Dowell and P. Jarratt, *A modified regula falsi method for computing the root of an equation*, BIT Numerical Mathematics, 11 (1971), pp. 168–174.

[8] W. Gautschi, *Quadrature formulae on half-infinite intervals*, BIT Numerical Mathematics, 31 (1991), pp. 437–446.

[9] D. Gibb, *A Course in Interpolation and Numerical Integration for the Mathematical Laboratory*, BiblioBazaar, 1915.

[10] D. Goldberg, *What every computer scientist should know about floating-point arithmetic*, ACM Comput. Surv., 23 (1991), pp. 5–48.

[11] G. H. Golub and C. F. Van Loan, *Matrix Computations (3rd Ed.)*, Johns Hopkins University Press, Baltimore, MD, USA, 1996.

[12] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2nd ed., 2002.

[13] IEEE Task P754, *ANSI/IEEE 754-1985, Standard for Binary Floating-Point Arithmetic*, IEEE, New York, NY, USA, Aug. 1985.

[14] ——, *IEEE 754-2008, Standard for Floating-Point Arithmetic*, IEEE, New York, NY, USA, Aug. 2008.

[15] A. Iserles, *A First Course in the Numerical Analysis of Differential Equations*, Cambridge Texts in Applied Mathematics, Cambridge University Press, 1996.

[16] L. W. Johnson and D. R. Scholz, *On Steffensen's Method*, SIAM Journal on Numerical Analysis, 5 (1968), pp. 296–302.

[17] MathWorks, *Choose an ODE Solver*, 2019.

[18] E. Meijering, *A chronology of interpolation: From ancient astronomy to modern signal and image processing*, Proceedings of the IEEE, 90 (2002), pp. 319–342.

[19] D. E. Muller, *A method for solving algebraic equations using an automatic computer*, Mathematical Tables and Other Aids to Computation, 10 (1956), pp. 208–215.

[20] M. Overton, *Numerical Computing with IEEE Floating Point Arithmetic*, Society for Industrial and Applied Mathematics, 2001.

[21] A. Quarteroni and F. Saleri, *Scientific Computing with MATLAB and Octave (Texts in Computational Science and Engineering)*, Springer-Verlag, Berlin, Heidelberg, 2006.

[22] C. Ridders, *A new algorithm for computing a single root of a real continuous function*, Circuits and Systems, IEEE Transactions on, 26 (1979), pp. 979 – 980.

[23] R. L. Rivest, A. Shamir, and Y. Tauman, *How to share a secret*, Communications of the ACM, 22 (1979), pp. 612–613.

[24] T. R. Scavo and J. B. Thoo, *On the Geometry of Halley's Method*, The American Mathematical Monthly, 102 (1995), pp. 417–426.

[25] J. Schauder, *Der Fixpunktsatz in Funktionsräumen*, Studia. Math, 2 (1930), pp. 171–180.

[26] A. Sidi, *Generalization of the secant method for nonlinear equations*, Applied Mathematics E-Notes, 8 (2008).

[27] E. Süli and D. Mayers, *An Introduction to Numerical Analysis*, An Introduction to Numerical Analysis, Cambridge University Press, 2003.

[28] L. N. Trefethen, *Approximation Theory and Approximation Practice*, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2012.

[29] L. N. Trefethen and D. Bau, *Numerical Linear Algebra*, Society for Industrial and Applied Mathematics, 1997.

Índice alfabético

A-estabilidad, 225

Chebyshev,
  polinomios, 79
  puntos, 79
  recurrencia, 79
Contracción, 37
Coordenadas baricéntricas, 81
  fórmula, 82
  fórmula en puntos de Chebyshev, 83

Derivadas, 26, 72
  Fórmula de $n+1$ puntos, 73
  regla de cinco puntos, 73
  regla de tres puntos, 73

Ecuaciones diferenciales
  Tipo *Stiff*, 218
Ecuaciones diferenciales,
  A-estabilidad, 226
  de van der Pol, 219
  Método de diferencias finitas, 222
  Método de Euler, 208
  Método de Rung-Kuta implícito, 216
  Método de Runge-Kuta adaptivo, 229
  Método de Runge-Kuta explícito, 213
  Método del trapecio, 210
  Método multipaso de Adams, 213
  Método predictor-corrector, 212
  stiff, 218
Ecuaciones no lineales, 33
  Existencia y unicidad, 33
Error
  por cancelación, 28
  por redondeo, 26
  por truncamiento, 26
Estabilidad, 25, 175, 178
  Definición, 25
  Valores propios, 28
Estabilidad,
  hacia atrás, 175, 178

Fenómeno de Runge, 68

Integración,
  cuadratura de Gauss, 112
  cuadratura de Gauss-Lobatto, 115
  cuadratura de Radau, 119
  fórmula compuesta de Simpson, 103

fórmula compuesta del trapecio, 103
fórmula de Simpson, 101
fórmula de trapecio, 98, 100
método de Monte Carlo, 116
Newton-Cotes, 98
nodos y pesos, 98
Simpson adaptivo, 104
Interpolación
de Hermite, 69, 70
de Lagrange, 66, 67
diferencias divididas, 88
Iteración simple, 39
convergencia cuadrática, 44
número de iteraciones, 41
Iteración simultánea, 193

MATLAB
expm1, 30
format, 20
Inf, 29
int16,32,64, 21
interp1, 78
log1p, 30
NaN, 29
poly, 29
polyfit, 69
polyval, 69
randi, 22
roots, 29
single, 20
uint16,32,64, 21
Matrices,
∞-norma, 137
1-norma, 136
aproximación de bajo rango, 144
factor de crecimiento, 181
factorización de Cholesky, 161
factorización en valores singulares, 139
factorización LU, 152
factorización QR, 145
factorización QR reducida, 146
norma de Frobenius, 138
norma inducida, 134
resolución de sistemas de ecuaciones, 163
sustitución hacia adelante, 164
sustitución hacia atrás, 163
Matriz de Hessenberg, 240
Matriz de Hilbert, 27
Matriz de Vandermonde, 190
Método
de colocación, 216
de diferencias finitas, 222
de Euler, 208
de Nystrom, 215
de Runge-Kuta clásico, 215
de Runge-Kuta explícito, 213
del trapecio, 210
Multipaso de Adams, 212
Predictor-corrector, 211
RK4, 215
RKI orden 4, 217
Runge-Kuta implícito, 216
Método de
aproximaciones sucesivas, 37
convergencia, 40
ejemplo, 41
bisección, 34
algoritmo, 36
convergencia, 35
número de iteraciones, 36
Halley, 55
Illinois, 56
la Regla Falsa, 50, 55

Newton, 45, 50
algoritmo, 46
con multiplicidad $m > 1$, 52
convergencia, 46
intervalos de convergencia, 53
Newton en varias variables, 194
secante, 48, 52
algoritmo, 49
convergencia, 49
Steffensen, 57

Número de condición, 27, 175

Operaciones en punto flotante, 22
Orden de convergencia de una sucesión, 42

Polinomios ortogonales, 107
ceros, 110
de Chebyshev, 107
de Legendre, 108
ortogonalización, 108
recurrencia, 108
Polinomios ortogonales,
de Hermite, 122
de Laguerre, 119
Precisión, 178

Reflector de Householder, 149
Representación de números en computadoras
Ejemplo de juguete, 16
Enteros con signo, 21
Enteros sin signo, 21
Número en punto flotante, 16
Precisión, 16
Precisión doble, 19
Precisión simple, 18
Punto fijo, 15
Punto flotante, 16
Épsilon de la máquina, 16

Secuencia de Sturm, 241
Sucesión de Fibonacci, 31

Teorema de Punto Fijo
de Banach, 38
de Brouwer, 38
Teorema del valor medio para integrales, 99
Trazadores
cúbicos, 75
de Hermite, 76
naturales, 77
cúbicos,
naturales,, 196
lineales, 74
Trazadores,
cuadráticos, 202

Valores propios,
algoritmo QR, 251
Cociente de Rayleigh, 248
iteración de potencias, 247
Método de Jacobi, 244
secuencia de Sturm, 242

Épsilon de la máquina, ϵ_m, 16